Data Visualization with R

Base Graphics

Aravind Hebbali

2026-10-16

Contents

Preface 6
Software information 7
How to cite this book 8
Corrections 9
Why Base R Graphics in 2026? 9
Introduction 9
What "base graphics" means 9
Why the naming matters 10
Speed 11
The cost of a session 13
The dependency count 15
What it weighs in a container 17
What base graphics costs 18
A decision tree 19
When you have read this chapter 20
Introduction 21
Introduction 21
What is data visualization? 21
Why visualize data? 21
R Graphics System 21
Graphics 21
ggplot2 21
Lattice 21
Getting Help 22
mtcars 22
Variable Info 22
plot() 22
One continuous variable 22
One categorical variable 23

Two continuous variables ... 23
Two categorical variables ... 24
Continuous/Categorical variable ... 24
Categorical/Continuous variable ... 25
Try it live ... 25
Title & Axis Labels ... 26
Introduction ... 26
Syntax ... 26
Title ... 27
Subtitle ... 27
Axis Labels ... 28
title() ... 29
Axis Range ... 31
Try it live ... 32
Scatter Plots ... 32
Introduction ... 32
Basic Plot ... 33
Shape ... 34
Size ... 37
Color ... 39
Try it live ... 42
Line Graphs ... 42
Introduction ... 42
Basic Plot ... 42
Color ... 46
Line Type ... 48
Line Width ... 53
Enhance Points ... 54
Additional Lines ... 56
Uncertainty and Annotation ... 58
segments() ... 58
arrows() ... 60
polygon() ... 62
Putting it all together ... 63
Bar Plots ... 64
Introduction ... 64
Bar Plot ... 65
Using plot function ... 65
Using barplot function ... 66
Horizontal or Vertical ... 67
Bar Width ... 68
Equal Width ... 68
Different Widths ... 69

Labels ... 72
Color ... 73
Same color for all bars ... 73
Differnt color for the bars ... 74
Recycling colors ... 75
Same color for all bars ... 76
Differnt color for the bars ... 77
Recycling colors ... 78
Axes ... 79
Remove axes ... 79
Putting it all together ... 83
Title & Axis Labels ... 84
Bivariate Bar Plots ... 85
Stacked Bar Plot ... 86
Grouped Bar Plot ... 88
Box Plots ... 89
Introduction ... 89
Box Plot ... 90
Structure ... 90
Univariate Box Plot ... 90
Basic Plot ... 90
Horizontal Box Plot ... 91
Color ... 92
Border Color ... 93
Range ... 94
Outline ... 97
Varwidth ... 99
Bivariate/Multivariate Box Plot ... 99
Color ... 101
Compare Medians ... 103
Putting it all together ... 104
Histograms ... 105
Introduction ... 105
Distributions ... 106
Normal Distribution ... 106
Skewed Distributions ... 106
Basics ... 107
Bins ... 109
Intervals ... 111
Frequency Density ... 112
Frequency Distribution II ... 114
Color ... 116
Single Color ... 116

Different Colors 117
Recycled Colors 118
Border Color 119
Different Colors 120
Labels 121
Method 1 121
Method 2 122
Putting it all together 123
Legends 124
Introduction 124
Data 124
Line Graph 125
Legend Location 126
Lines 129
Points 130
Text Placement 131
Title 134
Title Color 135
Title Position 136
Box Appearance 138
Box Type 138
Background Color 139
Border 140
Justification 141
Horizontal Justification 142
Vertical Justification 142
Text Appearance 142
Text Annotations 143
Introduction 143
Syntax 143
Text Inside the Plot 144
Color 145
Font 146
Font Family 147
Font Size 148
Text on the Margins 149
Specify Margin 150
Line 152
Alignment 155
Position 158
Offset 159
Perpendicular Adjustment 160
Outer Margin 161

Exact Position 162
Faceting 163
Introduction 163
Row 164
Case Study 1 164
Case Study 2 165
Case Study 3 166
Case Study 4 167
Case Study 5 168
Column 170
Case Study 6 170
Special Cases 171
Case Study 7 172
Case Study 8 173
Layout 175
Case Study 1 176
Case Study 2 177
Case Study 3 178
Case Study 4 180
Case Study 5 181
Case Study 6 182
Case Study 7 184
Case Study 8 186
Case Study 9 187
Putting it all together 188
Production Export & Graphical Devices 190
Introduction 190
Screenshots Are Not Export 190
What a Graphical Device Is 190
Raster: png() 191
Vector: pdf() 191
Vector: svg() 192
Comparing the three 192
Batch Reports 193
Controlling the Page with par() 193
mar and oma 193
mgp 194
las 195
bty, xaxs, tck 196
Custom Axes 198
Colourblind-Safe Palettes 201
Checking for Greyscale 202
Exercises 203

Exercise 1 203
Exercise 2 204
Exercise 3 204
The Full Set 205
References 205
Base R graphics 205
The grammar of graphics, and the alternative 206
Colour and accessibility 206
Datasets used in this book 206
Reproducing the measurements in Chapter 1 207
Further reading in the R ecosystem 207
A note on the sources for this chapter 208
About the Author 208
Base R ↔ ggplot2: A Rosetta Stone 208
Introduction 208
The one-sentence version 208
Marks and coordinates 209
The scale trap 210
Titles, labels and ranges 210
`xlim` is not `coord_cartesian(xlim =)` 210
Grouping and the language difference 211
High-level functions 212
Devices, layout and everything else 213
When to switch 214
What does not translate 215
Where to go next 215

Preface



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

This book requires R 4.1 or later.

Software information

The R session information when compiling this book is shown below:

```
sessionInfo()
```

```
R version 4.5.3 (2026-03-11)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 24.04.5 LTS

Matrix products: default
```

```
BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblasp-r0.3.26.so;
LAPACK version 3.12.0

locale:
 [1] LC_CTYPE=C.UTF-8       LC_NUMERIC=C           LC_TIME=C.UTF-8
 [4] LC_COLLATE=C.UTF-8     LC_MONETARY=C.UTF-8    LC_MESSAGES=C.UTF-8
 [7] LC_PAPER=C.UTF-8       LC_NAME=C              LC_ADDRESS=C
[10] LC_TELEPHONE=C         LC_MEASUREMENT=C.UTF-8 LC_IDENTIFICATION=C

time zone: UTC
tzcode source: system (glibc)

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

loaded via a namespace (and not attached):
 [1] compiler_4.5.3  fastmap_1.2.0   cli_3.6.6       tools_4.5.3
 [5] htmltools_0.5.9 yaml_2.3.12     rmarkdown_2.32  knitr_1.52
 [9] jsonlite_2.0.0  xfun_0.62       digest_0.6.39   rlang_1.3.0
[13] evaluate_1.0.5
```

We do not add prompts (> and +) to R source code in this book, and we comment out the text output with two hashes ## by default, as you can see from the R session information above. This is for your convenience when you want to copy and run the code (the text output will be ignored since it is commented out). Package names are in bold text (e.g., **rmarkdown**), and function names are followed by parentheses (e.g., `knitr::knit()`). The double-colon operator `::` means accessing an object from a package.

How to cite this book

If this book is useful to your work, please cite it. A ready-made BibTeX entry is in `citation.bib` and is reproduced below.

```
@misc{vizbase2026,
  author    = {Aravind Hebbali},
  title     = {Data Visualization with R: Base Graphics},
  year      = {2026},
  version   = {1.0},
  publisher = {Rsquared Academy},
  doi       = {10.5281/zenodo.23138733},
  url       = {https://viz-base.rsquaredacademy.com},
  note      = {Version 1.0. Source: https://github.com/rsquaredacademy-
publications/viz-base}
}
```

The DOI is the Zenodo *version* DOI for v1.0.0, so this citation keeps resolving to the edition you are reading even after a later release. If you would rather your citation track the current

edition, use the concept DOI `10.5281/zenodo.23138732` instead — it always resolves to the latest version.

Corrections

This book is a living document and mistakes will be found. If you find one, please open an issue at rsquaredacademy-publications/viz-base.

Every measurement quoted in Chapter 1 is regenerable from the scripts in `scripts/`, so a disputed number is a reproducible question rather than an argument. See the References chapter for the full list.

Why Base R Graphics in 2026?

Introduction

Base graphics have a reputation problem. The usual first question about them is some version of *isn't this the old way?*, and the usual answer is a shrug. This chapter argues the opposite: in 2026, base graphics are the correct first tool for a specific and fairly large set of jobs, and this book is that tool's documentation.

In this chapter, we will

- establish what base graphics actually is, and what ships with R
- measure how fast they draw, and how that measurement is usually faked
- measure what a plotting session costs to start
- count the dependencies that `ggplot2` adds and base graphics does not
- walk a decision tree for choosing between the two

We will do this with numbers rather than adjectives. Every figure in this chapter is produced by a script in `scripts/`, stored as a CSV in `data/`, and committed. Nothing here runs when the book is built, which is deliberate: a book that argues for zero dependencies cannot quietly require two of them to prove the point. If a number in this chapter is ever wrong, you can re-run the script that produced it and find out which of us was wrong.

What "base graphics" means

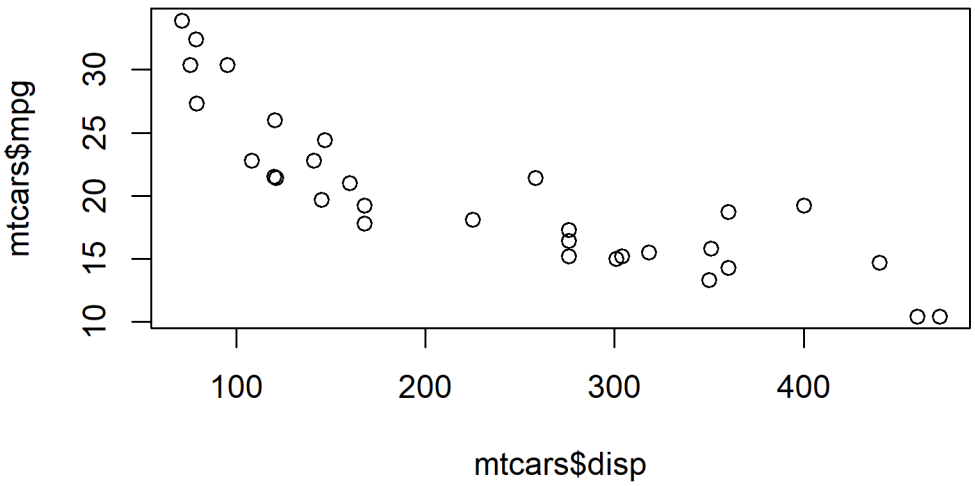
Base graphics is the set of plotting functions that ship inside R itself, in the `graphics` package. There is nothing to install.

```
?graphics
```

The package has been part of R since before R 1.0 and its core has barely changed, which is either its greatest strength or its most damning indictment depending on who is talking. The reason it survives is that it does one thing very well: it takes coordinates and draws them. Everything else in this book is an elaboration on that idea.

The key practical fact is this one:

```
plot(mtcars$disp, mtcars$mpg)
```



No `library()` call. `graphics` is one of the packages R attaches at startup, alongside `stats`, `grDevices`, `utils` and `methods`. It is not merely available, it is already loaded, which is a stronger guarantee than "cheap to install". We will quantify that in a moment.

The three systems you are likely to meet in R are:

- **base graphics** (`graphics`) — in R, always present, coordinates in, marks out
- **`ggplot2`** — a large, opinionated package built on the grammar of graphics, with a large dependency tree
- **`lattice`** — a Trellis-style system for multivariate data, part of R's recommended packages

This book is about the first one. Chapter 2 surveys all three.

Why the naming matters

Base graphics is frequently described as "low-level". That is accurate and slightly misleading in equal measure. The *drawing* primitives are low-level: `plot()`, `points()`, `lines()`, `text()`, `polygon()`, `arrows()` and `rect()` put marks at coordinates you choose. But the high-level functions you actually call most often — `barplot()`, `boxplot()`, `hist()`, `pairs()`, `plot()` as a scatterplot, `dotchart()`, `stripchart()`, `mosaicplot()`, `image()` — are convenience wrappers that pick sensible defaults and let you override any of them.

So base graphics is low-level in the sense that **you stay in control**. Nothing is computed behind your back, no scale is transformed without being named, no grouping is inferred. That control is the actual product here, and it is what the rest of the book teaches you to spend.

Speed

The claim to test is that base graphics are fast, and the interesting part is that the claim is usually tested wrong.

We drew 10,000 and 100,000 points three ways, timing a real render each time:

- **`base`** — `plot(x, y)`
- **`ggplot2_construct`** — `ggplot(df, aes(x, y))`, with no `print()`
- **`ggplot2_render`** — the same, plus `print()` so it actually draws

```
bench <- read.csv('data/bench-base-vs-ggplot2.csv')
bench
```

```
      n          approach median_ms peak_mem_mb n_iter
1 1e+04              base      10.9        19.7     15
2 1e+04 ggplot2_construct      13.4        19.9     15
3 1e+04    ggplot2_render     871.9        36.8     15
4 1e+05              base      87.4        30.7     14
5 1e+05 ggplot2_construct      13.2        24.9     15
6 1e+05    ggplot2_render    1244.9        77.3     15
```

Three approaches, two sizes, median of 15 runs. Now the part that matters:

> The middle row measures almost nothing. `ggplot()` *builds an object.* It does not draw. Until something calls `print()` on it, no marks are laid down and no pixels or vector output are produced. So `ggplot2_construct` is timing R's object system, not its graphics engine. If you have ever read that "ggplot2 is faster than base R", that is very likely the number you read.

Both sides here genuinely draw. Base renders to `pdf(NULL)`, a device that accepts and discards output, so the plotting work happens with no window and no file.

```
init <- par(no.readonly = TRUE)

sizes <- sort(unique(bench$n))
approaches <- c('base', 'ggplot2_construct', 'ggplot2_render')
cols <- c(base = '#0072B2', ggplot2_construct = '#999999',
          ggplot2_render = '#D55E00')
labs <- c(base = 'base plot()',
          ggplot2_construct = 'ggplot2\nconstruct only',
          ggplot2_render = 'ggplot2\nprint()')

# 1e+05 formats as "1e+05" unless scientific is switched off, which is not a
# caption anyone wants on a figure.
fmt_n <- function(x) format(x, big.mark = ',', scientific = FALSE, trim = TRUE)

draw_panel <- function(n) {
  v <- bench$median_ms[match(approaches, bench$approach[bench$n == n])]
  # barplot stacks bottom-up, so rev() puts base first and reads top-down in the
  # same order as the prose does. Titles are kept short because a long one
```

```
  # overflows the panel once the left margin has to clear two-line labels.
  barplot(rev(v), horiz = TRUE, log = 'x', names.arg = rev(labs), las = 1,
          cex.names = 0.8, col = rev(cols[approaches]), border = NA, space = 0.4,
          xlim = c(1, 3000), main = sprintf('%s points', fmt_n(n)),
          xlab = 'median time to draw, milliseconds (log scale)')
}

# Left margin has to clear two-line category labels plus the axis; the default
# silently clipped them.
par(mfrow = c(1, 2), mar = c(4.5, 6, 3, 1))
draw_panel(sizes[1])
draw_panel(sizes[2])

par(init)
```

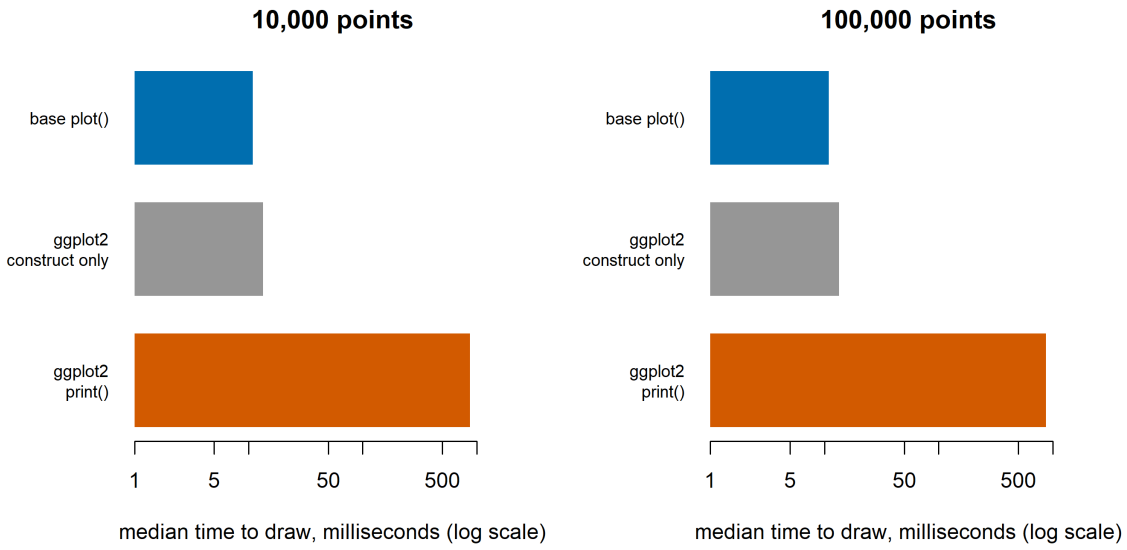


Figure 1: Median time to draw, by engine and problem size

Figure 1 shows the result. At 10,000 points, constructing the ggplot2 object is about as fast as drawing the whole thing in base graphics — and drawing it is roughly **80 times slower than drawing it in base**. At 100,000 points the gap is worse, and base graphics has not become slower so much as ggplot2 has failed to scale the way the underlying drawing work does.

Memory tells a similar story:

```
init <- par(no.readonly = TRUE)
n <- max(bench$n)
v <- bench$peak_mem_mb[match(approaches, bench$approach[bench$n == n])]
barplot(rev(v), horiz = TRUE, names.arg = rev(labs), las = 1, cex.names = 0.8,
        col = rev(cols[approaches]), border = NA, space = 0.4,
        main = sprintf('Peak memory to draw (%s points)', fmt_n(n)),
        xlab = 'MB')
par(init)
```

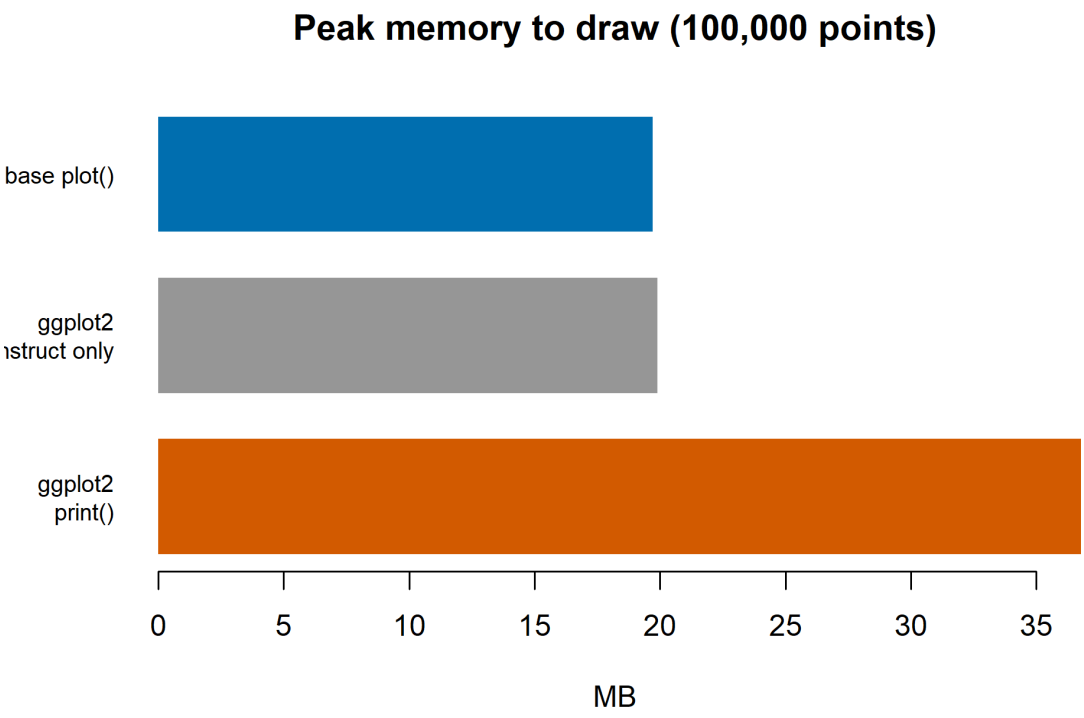


Figure 2: Peak memory to draw 100,000 points, by engine

Base peaks at 31 MB where ggplot2 peaks at 77 MB. Note the middle bar again: the object-only path uses *less* memory than base, which is a direct consequence of not drawing anything.

> These are absolute timings from one machine — Windows, R 4.5.2, `x86_64-w64-mingw32`, ggplot2 4.0.1 — and the script is committed at `scripts/bench.R`, so the numbers are reproducible rather than remembered. Treat them as an order of magnitude. What travels across machines is the *shape* of the result: one engine does the drawing work immediately, and the other builds a description of a plot that something else has to be asked to render. That extra step is the cost, and it is not a constant — it grows with what you ask for.

That last clause is the real argument. Base graphics is a direct-mode engine: your call draws. ggplot2 is deferred: your call records intent, and rendering is a separate step performed later, on a plot that may have grown additional layers in the meantime. Deferred is the right design for iteration and exploration. It is the wrong design when the answer is a fixed number in a fixed report.

The cost of a session

Speed per plot only matters if you pay to start plotting at all. Here is what it costs to bring a session up, measured as wall clock around a whole `Rscript` process.

```
startup <- read.csv('data/startup-latency.csv')
startup[, c('approach', 'wall_ms', 'spread_ms', 'overhead_ms', 'load_ms',
'namespaces')]
```

```
  approach wall_ms spread_ms overhead_ms load_ms namespaces
1  vanilla    1210       350           0       0          8
2 graphics    1220       450          10       0          8
3  ggplot2    7170      3340        5960    5910         29
```

Three rows, and they are not close.

```
init <- par(no.readonly = TRUE)
op <- par(mar = c(4.5, 5, 3, 1))
barplot(
  startup$wall_ms,
  horiz = TRUE, names.arg = startup$approach, las = 1, cex.names = 0.9,
  col = c('#999999', '#0072B2', '#D55E00'), border = NA, space = 0.4,
  xlim = c(0, max(startup$wall_ms) * 1.15),
  main = 'Wall clock to start a plotting session (ms)',
  xlab = 'milliseconds, median of 15 runs'
)
par(op)
par(init)
```

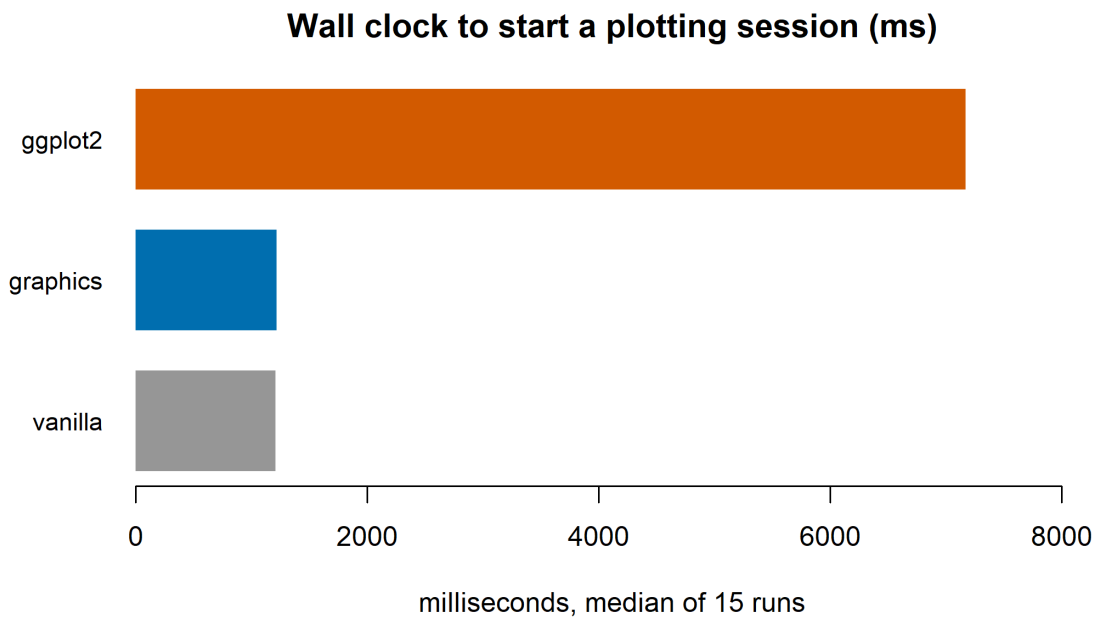


Figure 3: Wall clock to start a plotting session, by engine

`library(graphics)` costs nothing measurable, and the reason is visible in the last column: the namespace count does not move from 8 to 8. `graphics` is already attached at startup, so the call is a no-op. That is not "cheap" in the way a cached download is cheap — it is *free*, structurally.

`library(ggplot2)` takes about six seconds on this machine and pulls the loaded namespace count from 8 to 29. Twenty-one packages.

> The `overhead_ms` column for `graphics` is near zero and can land slightly negative; `spread_ms` is the run-to-run range, and it is larger than the difference. The honest reading is "below the noise floor", not "measurably fast". Again, one machine — but the namespace column is not machine-specific, and that is the number to remember.

The dependency count

The strongest form of the zero-dependency claim is not an adjective but a count. Base graphics needs zero packages that are not already in R. Packages that build on it inherit that property, because there is nothing to pull in.

We read this from CRAN's own metadata, counting each package's declared `Depends`, `Imports` and `LinkingTo` entries and discarding everything that ships with R:

```
deps <- read.csv('data/cran-deps.csv')
deps[, c('package', 'role', 'version', 'n_nonbase', 'imports_ggplot2')]
```

```
         package                 role version n_nonbase imports_ggplot2
1       graphics    base R (baseline)   4.5.2         0           FALSE
2        plotrix base-graphics add-on  3.8-14         0           FALSE
3  TeachingDemos base-graphics add-on    2.13         0           FALSE
4        faraway base-graphics add-on   1.0.9         1           FALSE
5         gplots base-graphics add-on   3.3.0         2           FALSE
6          psych base-graphics add-on   2.6.9         2           FALSE
7        effects          base + grid   4.2-5         6           FALSE
8          Hmisc               mixed   5.3-0        12            TRUE
9          olsrr      author package   0.7.0         6            TRUE
10           rfm      author package   0.4.0        10            TRUE
11         blorr      author package   0.3.1         5            TRUE
12     descriptr      author package   0.6.0         6            TRUE
```

Sort that by dependency count and a pattern appears (Figure 4):

```
init <- par(no.readonly = TRUE)

d <- deps[order(deps$n_nonbase, deps$package), ]
lab <- sprintf('%-16s %2d', d$package, d$n_nonbase)
# Axis runs to the observed maximum. An earlier version capped it at 12 and
# warned about capping, which is a confusing thing to say when the cap happens
# to equal the largest bar and nothing is actually cut.
op <- par(mar = c(5.5, 9, 3, 1))
bp <- barplot(
  d$n_nonbase, horiz = TRUE, names.arg = FALSE, border = NA,
  space = 0.35, col = ifelse(d$imports_ggplot2, '#D55E00', '#0072B2')
)
axis(2, at = bp, labels = lab, las = 1, tick = FALSE, cex.axis = 0.85)
title(main = 'Non-base CRAN dependencies by package',
```

```
      xlab = 'declared Depends / Imports / LinkingTo outside base + recommended')
mtext('orange = also depends on ggplot2', side = 1, line = 3.6,
      cex = 0.75, col = 'grey30')
par(op)
par(init)
```

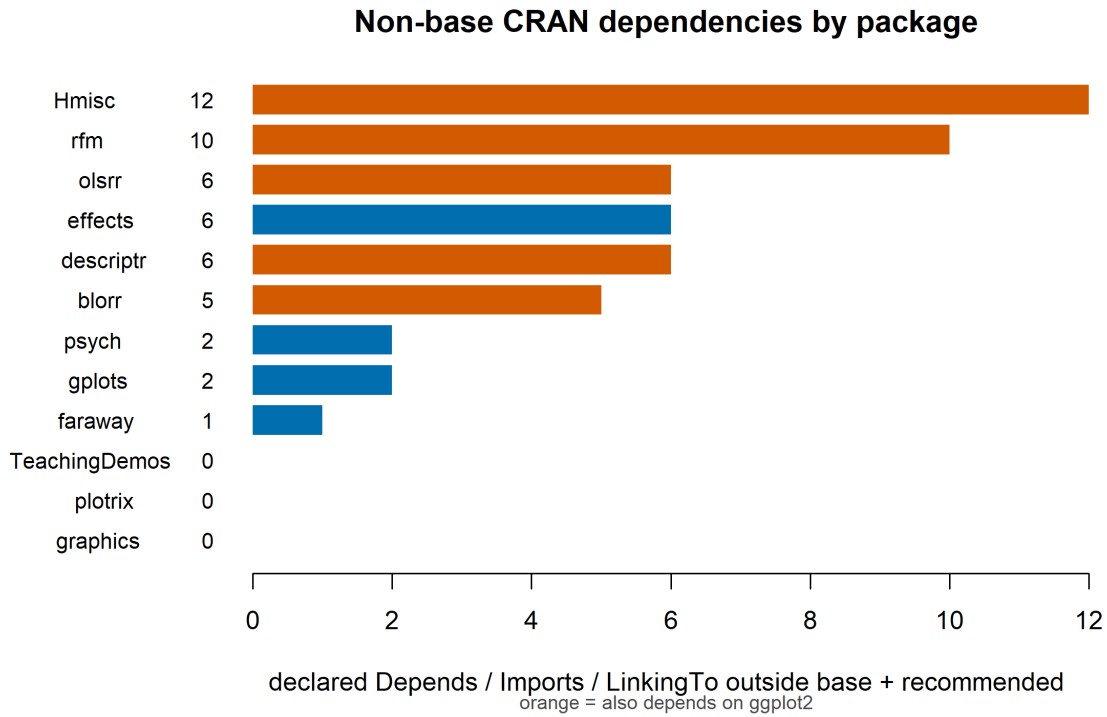


Figure 4: Declared non-base CRAN dependencies by package

Read the blue bars. `TeachingDemos`, `plotrix` and base R itself declare **zero** non-base dependencies, and they still produce publication figures — `plotrix` is a dedicated graphics add-on. `faraway` needs one. `psych`, which has been drawing its own plots with base graphics for a decade, needs two.

That is the practical shape of the argument. A base-graphics package on CRAN is cheap to install, cheap to check, cheap to run in a locked-down environment, and cheap to keep building in ten years, because its dependency graph is nearly empty.

And here is the part that would be dishonest to leave out:

> The orange bars include this author's own CRAN packages — `olsrr`, `rfm`, `blorr` and `descriptr` — and they all depend on `ggplot2`. They do not use base graphics. We checked their sources rather than inferring it from metadata: as of 2026-10-04, `rfm/R/rfm-plots.R` and `descriptr/R/ds-plots.R` are entirely ggplot2, and so are the diagnostic plots in `olsrr`. Metadata cannot tell you what a package draws with; you have to read the code.

So what does that say about base graphics? Read it as a trajectory rather than an endorsement. Those packages got onto CRAN drawing base graphics, then outgrew it, exactly as this book predicts they would. The base layer was the on-ramp; ggplot2 is where they landed once the plotting became the product rather than a by-product. Chapter 2 covers the systems side by side, and Appendix A maps every argument in this chapter onto its ggplot2 equivalent so you can translate in either direction.

What it weighs in a container

Dependency counts are an abstraction. If you ship R in a container — and if you ship R anywhere locked down, you probably do — the question stops being "how many packages" and becomes "how many bytes".

Two images from the same publisher make that measurable, because `rocker/tidyverse` is built *on top of* `rocker/r-ver`: the same R, the same base system, plus the package layer. The difference between them is therefore the cost of the tidyverse layer and nothing else, which is a cleaner comparison than diffing two unrelated images.

```
docker <- read.csv('data/docker-sizes.csv')
docker[, c('image', 'tag', 'layer', 'size_mb', 'extra_layer_mb')]
```

```
             image    tag         layer size_mb extra_layer_mb
1     rocker/r-ver  4.5.2        R only   349.4          574.5
2     rocker/r-ver latest        R only   355.1          574.5
3 rocker/tidyverse  4.5.2 R + tidyverse   923.9          574.5
4 rocker/tidyverse latest R + tidyverse  1269.2          574.5
```

```
init <- par(no.readonly = TRUE)
d <- docker[docker$tag == '4.5.2', ]
# Bottom margin has to clear the x label AND the delta caption below it.
par(mar = c(5.5, 9, 3, 1))
bp <- barplot(d$size_mb, horiz = TRUE, names.arg = FALSE, border = NA,
              space = 0.7, col = c('#0072B2', '#D55E00'))
axis(2, at = bp, labels = d$image, las = 1, tick = FALSE, cex.axis = 0.9)
# Title kept short: a long one overflows the panel once the left margin has to
# clear these labels.
title(main = 'Container image size, R 4.5.2', xlab = 'MB, compressed download')
delta <- d$size_mb[2] - d$size_mb[1]
mtext(sprintf('the tidyverse layer adds %s MB on top of R, a %.2fx increase',
              format(delta, big.mark = ',', scientific = FALSE),
              d$size_mb[2] / d$size_mb[1]),
      side = 1, line = 4.2, cex = 0.75, col = 'grey30')
par(init)
```

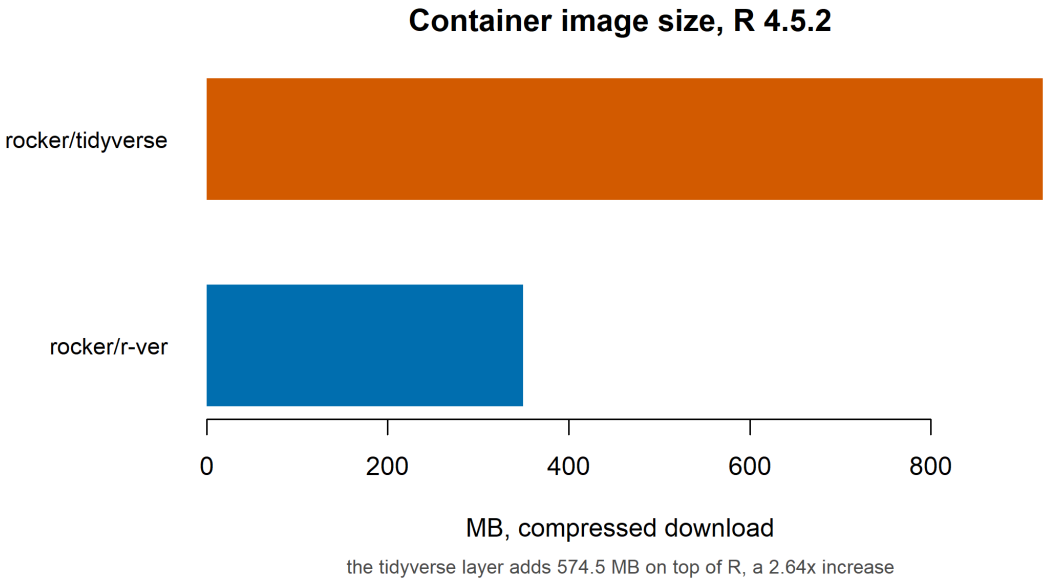


Figure 5: Published container image size for R 4.5.2, base R against base R plus the tidyverse layer

A base-R image is 349 MB; adding the tidyverse layer takes it to 924 MB. That 574 MB is not a mistake or a packaging artefact — it is the price of the dependency tree, paid on every pull, every CI run and every cold start, whether or not the code being run touches a single one of those packages.

> Two caveats, both recorded in the CSV. These are Docker Hub's reported sizes for the whole tag, which is the sum of its **compressed** layers — a real number a real user downloads, but not the unpacked size on disk, and the ratio is only as fair as the fact that both tags were built at the same time by the same publisher. And they are published metadata read over HTTP, not a local build: no daemon was involved, which is why this is reproducible on any machine, and also why it cannot control the comparison.

Read next to the CRAN table above, the pattern is consistent. Base graphics is not lighter by a little; it is lighter by an order of magnitude, because it inherits the empty part of R's dependency graph rather than extending it.

What base graphics costs

An honest chapter has to state the other side.

- **More typing.** Base graphics is verbose by design. The same plot is longer in base than in ggplot2, and it stays that way.
- **No automatic grouping or faceting.** You build panels yourself with `par()` or `layout()` and a loop, where ggplot2 has `facet_wrap()`. Chapter 11 covers this and it is the single biggest readability jump in the book.

- **No automatic scale transformation.** Nothing is transformed unless you name it. This is a feature when you want control and a cost when you do not.
- **A smaller ecosystem.** The reference grid, the grammar extensions, the themes, the colour scales: all of that ecosystem is ggplot2's, not base's.
- **Older idiom.** Base graphics code from 2001 still runs. That cuts both ways.

A decision tree

Most of the above reduces to one question: who decides what the plot looks like?

```
init <- par(no.readonly = TRUE)
# A flowchart is a picture, not a data plot: no axes, and the coordinates are
# laid out by hand. ps is set explicitly because text size otherwise depends on
# the device knitr happened to open, which is how a hand-placed diagram ends up
# with labels larger than its boxes.
par(xpd = NA, mar = c(0, 0, 0, 0), ps = 9)
plot.new()
plot.window(xlim = c(0, 100), ylim = c(12, 104))

box <- function(x, y, w, h, label, sub, fill) {
  rect(x, y - h / 2, x + w, y + h / 2, col = fill, border = 'grey25', lwd = 1.5)
  text(x + w / 2, y + (if (nzchar(sub)) 3.2 else 0), label, font = 2, cex = 0.92)
  if (nzchar(sub)) {
    text(x + w / 2, y - 4.6, sub, cex = 0.66, col = 'grey20')
  }
}
yes_no <- function(x, y, lab) text(x, y, lab, cex = 0.7, col = 'grey25')

Q <- '#DCE9F5'   # question
G <- '#F7E2D5'   # ggplot2 outcome
B <- '#DFF0E3'   # base outcome

box(4, 90, 52, 15, 'Is the plot final?',
    'One answer, written once, no iteration', Q)
box(4, 57, 52, 19, 'Does it have to run somewhere\nyou do not control?',
    'Locked-down server, CRAN check, minimal container', Q)
box(64, 90, 34, 15, 'ggplot2', 'Explore, iterate, facet, publish', G)
box(64, 57, 34, 15, 'ggplot2', 'Communicate, share, present', G)
box(22, 22, 52, 17, 'Base graphics',
    'Package internals, batch reports,\nexact control over the marks', B)

arrows(56, 90, 63.5, 90, length = 0.07, lwd = 1.6)
arrows(30, 82.4, 30, 66.7, length = 0.07, lwd = 1.6)
arrows(56, 57, 63.5, 57, length = 0.07, lwd = 1.6)
arrows(30, 47.4, 40, 30.7, length = 0.07, lwd = 1.6)

yes_no(59.7, 93, 'yes')
yes_no(32.4, 74.5, 'no')
yes_no(59.7, 60, 'yes')
yes_no(24, 39, 'no')
```

```
par(init)
```

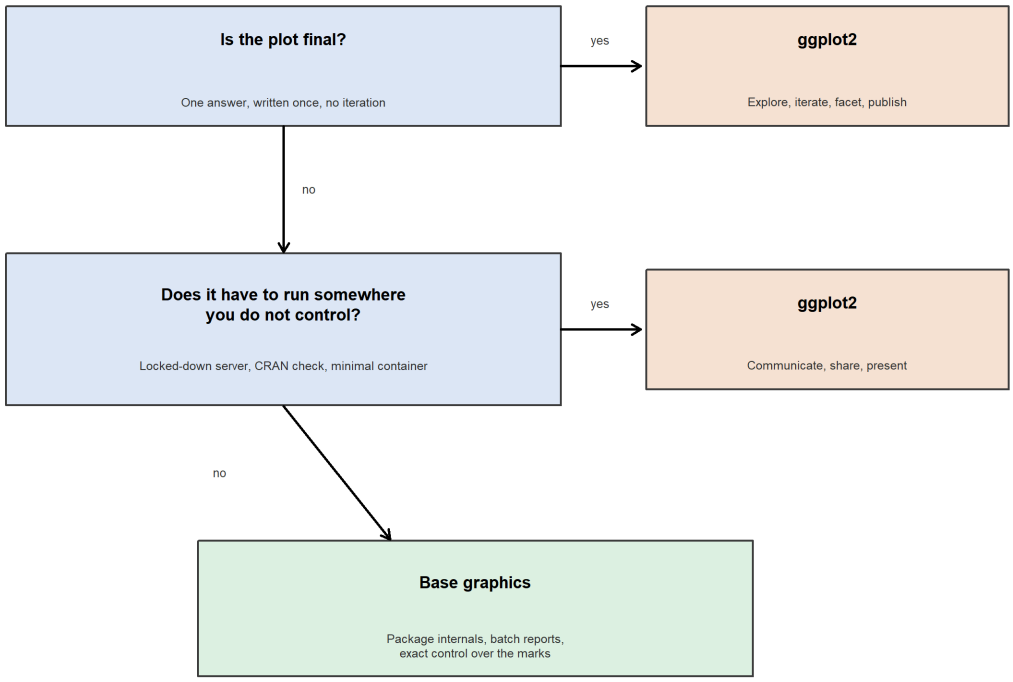


Figure 6: Choosing between base graphics and ggplot2

Two questions, and note that "final" is doing more work than "simple". A plot you will iterate on twenty times belongs to ggplot2 even if it is a single scatterplot, because the cost of base graphics is paid on every edit. A plot you will write once and run ten thousand times belongs to base graphics even if it is complicated, because that is where the per-plot cost actually lands.

Note also that the second question overrides the first. A plot that must run on a server you do not control is a base-graphics plot even if you would rather be iterating, because "install ggplot2 on this locked-down box" is not a decision you get to make.

That tree is a starting point, not a rulebook. Plenty of working code mixes the two: base graphics for the bulk of a report, one ggplot2 plot where the faceting carries the argument. The point is that the choice should be made deliberately, per plot, rather than by default.

When you have read this chapter

You now have four measured numbers and a decision tree. In the rest of the book we stop arguing and start drawing, and every chapter is written to the assumption that you want the marks exactly where you put them.

Chapter 2 surveys the three R graphics systems and walks the six dispatch cases of `plot()`. From Chapter 3 onward the argument is one argument at a time: here is the bare call, here is

what changing `pch` does, here is everything combined. Nothing in this book requires installing anything.

Introduction

Introduction

This is the first chapter of **Data Visualization With R**. The objective of the book is to provide a gentle introduction to working with base graphics in R.

- what is data visualization
- why visualize data
- understand R graphics system
  - graphics
  - ggplot2
  - lattice
- build some simple plots

What is data visualization?

In simple words, data visualization is the representation of data in graphical format.

Why visualize data?

- Explore: Visualization helps in exploring and explaining patterns and trends
- Detect: Patterns or anomalies in data can be detected by looking at graphs
- Make sense: Possible to make sense of large amount of data efficiently and in time
- Communicate: Easy to communicate and share the insights from data

R Graphics System

- graphics
- ggplot2
- lattice

Graphics

- It is part of base R and is the fundamental package for visualizing data.
- It has a lot of good features and we can create all the basic plots using it.

ggplot2

ggplot2, created by Hadley Wickham, is based on the Grammar of Graphics written by Leland Wilkinson. It has a structured approach to data visualization and builds upon the features available in the Graphics and Lattice packages.

Lattice

The lattice package is inspired by Trellis Graphics and created by Deepayan Sarkar. It is a very powerful data visualization system with an emphasis on multivariate data.

Getting Help

Use the `help()` to learn more about `plot()` function and `mtcars` data set.

```
help(plot)
help(mtcars)
```

mtcars

```
head(mtcars)
```

```
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

Variable Info

```
str(mtcars)
```

```
'data.frame':   32 obs. of  11 variables:
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num  160 160 108 258 360 ...
 $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num  16.5 17 18.6 19.4 17 ...
 $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
 $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

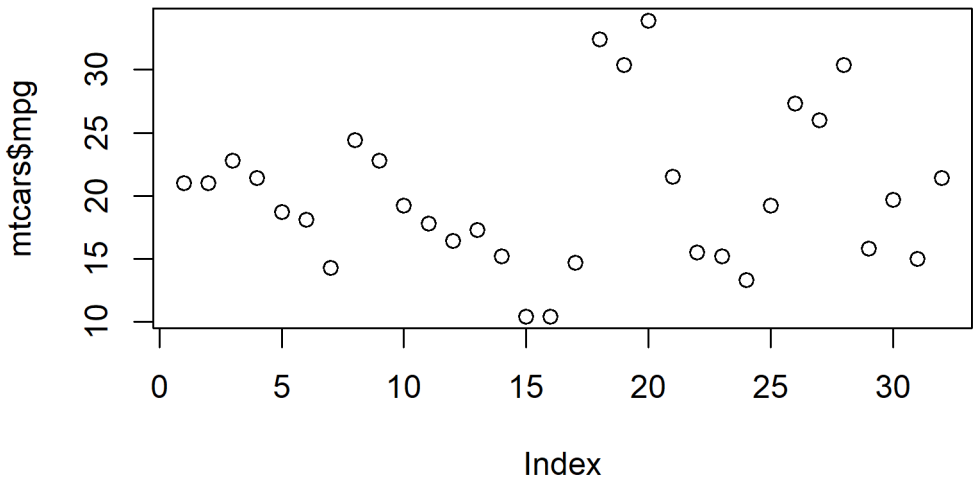
plot()

Now that we have some idea about the data set, let us explore the `plot()` function. We will use the following different data inputs and observe the kind of plots that are generated:

- Case 1: 1 continuous variable
- Case 2: 1 categorical variable
- Case 3: 2 continuous variables
- Case 4: 2 categorical variables
- Case 5: 1 continuous and 1 categorical variable
- Case 6: 1 categorical and 1 continuous variable

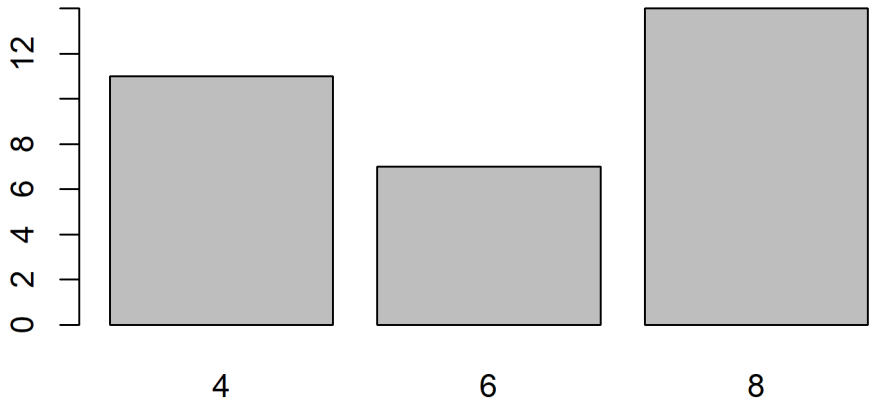
One continuous variable

```
plot(mtcars$mpg)
```



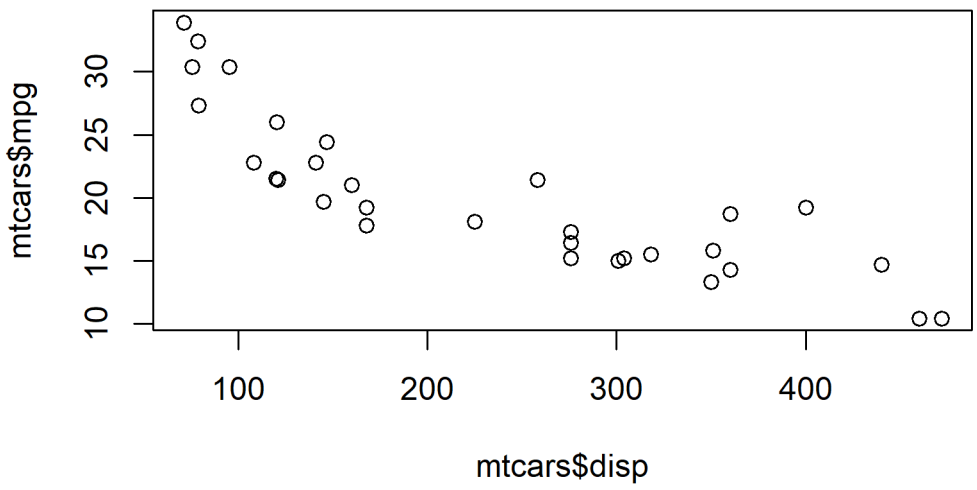
One categorical variable

```
plot(as.factor(mtcars$cyl))
```



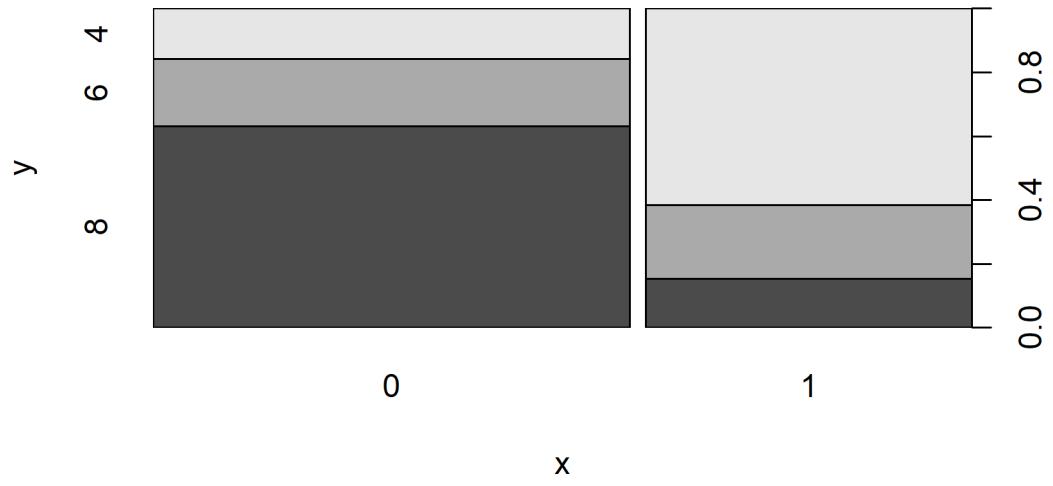
Two continuous variables

```
plot(mtcars$disp, mtcars$mpg)
```



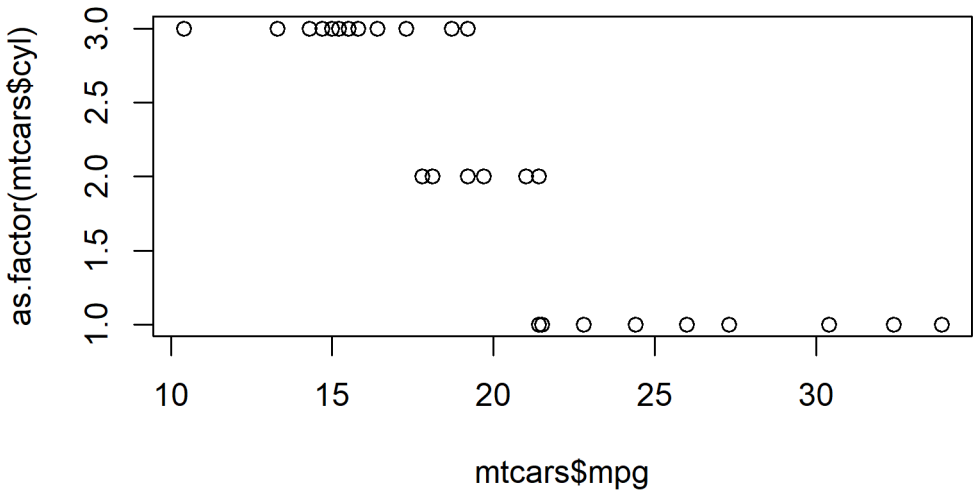
Two categorical variables

```
plot(as.factor(mtcars$am), as.factor(mtcars$cyl))
```



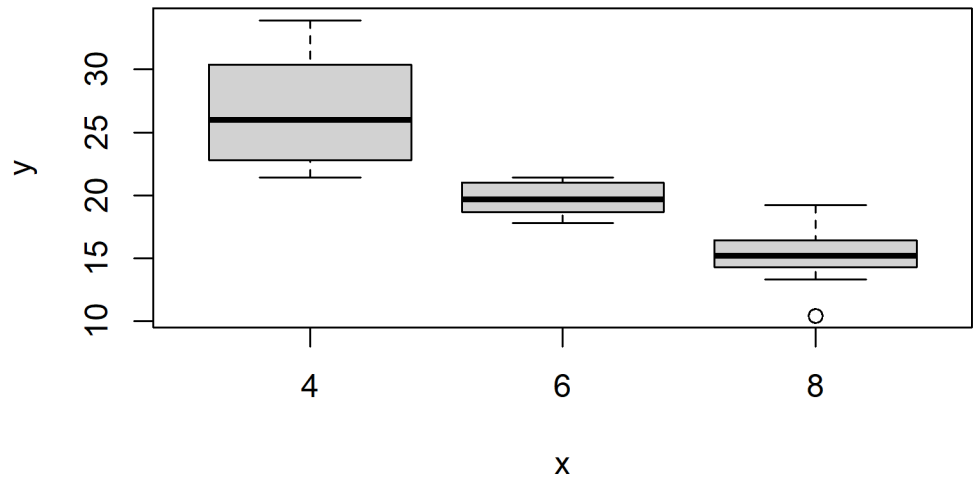
Continuous/Categorical variable

```
plot(mtcars$mpg, as.factor(mtcars$cyl))
```



Categorical/Continuous variable

```
plot(as.factor(mtcars$cyl), mtcars$mpg)
```



Try it live

Try it in your browser. Edit the code below and press **Run** – it executes entirely in your browser via WebR, so no R installation is needed. The runtime downloads once, on this page only. The printed plots are produced in your browser and are not part of the PDF or ePub editions.

```
{webr-r}
# walk the dispatch cases from this chapter
init <- par(no.readonly = TRUE)
par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))
plot(as.factor(mtcars$cyl))
plot(as.factor(mtcars$am), as.factor(mtcars$cyl))
plot(mtcars$disp, mtcars$mpg)
plot(mtcars$mpg, as.factor(mtcars$cyl))
plot(mtcars$mpg, mtcars$disp, main = 'displacement vs mpg',
     xlab = 'Miles Per Gallon', ylab = 'Displacement')
plot(as.factor(mtcars$cyl), mtcars$mpg)
par(init)
```

Title & Axis Labels

Introduction

In this chapter, we will learn how to add:

- Title
- Subtitle
- Axis Labels

to a plot and to modify:

- the range of an axis

In the previous chapter, we created plots which did not have any title or labels. Such plots are of no use to any one as they do not indicate what the X and Y axis represent or the primary information being communicated by the plot. The `title` and `labels` play an important part in making the plot holistic. There are two ways to add them to a plot:

- use the relevant arguments within the `plot()` function
- use the `title()` function

We will explore both the above methods one by one, and you can choose the method most convenient to you. Let us begin with the `plot()` function:

Syntax

Feature	Argument	Value	Example
Title	main	string	“Scatter Plot”
Subtitle	sub	string	“Displacement vs Miles Per Gallon”

Feature	Argument	Value	Example
X Axis Label	xlab	string	“Displacement”
Y Axis Label	ylab	string	“Miles Per Gallon”

Title

You can add a title to the plot using the `main` argument in the `plot()` function. Ensure that the title is enclosed in single/double quotes as it is a string. Let us create a scatter plot of `disp` and `mpg` from `mtcars` data set, and add a title to it.

```
plot(mtcars$disp, mtcars$mpg,
     main = 'Displacement vs Miles Per Gallon')
```

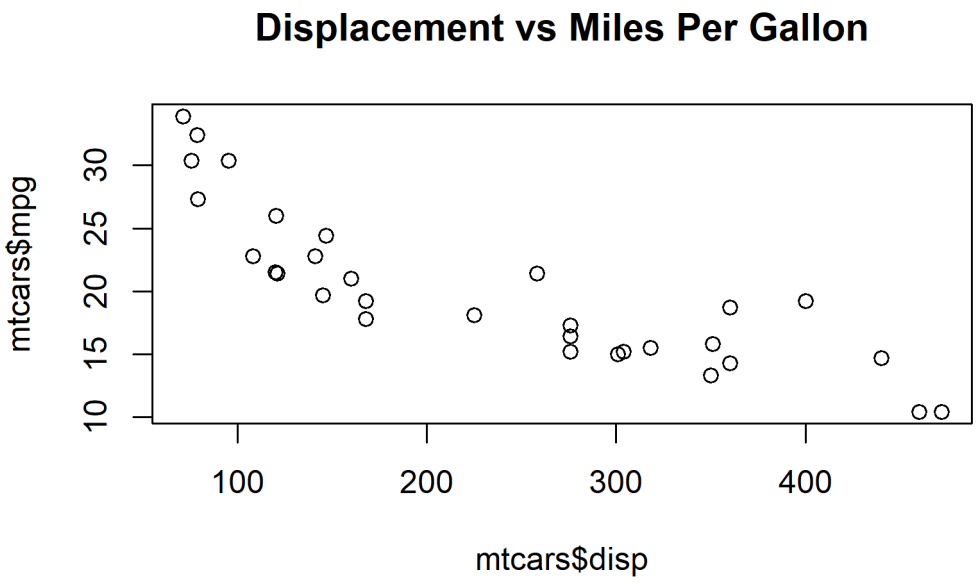
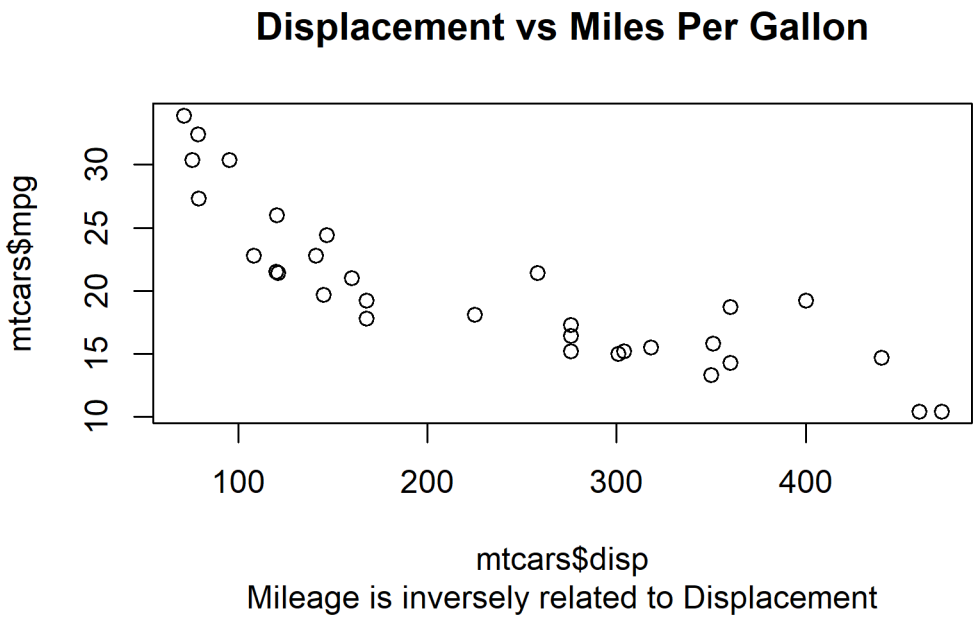


Figure 7: Adding a title and axis labels directly in the plot() call

Subtitle

You can add a subtitle to the plot using the `sub` argument in the `plot()` function. The subtitle will appear below the X axis label. Ensure that the subtitle is enclosed in single/double quotes as it is a string. Let us add a subtitle to the plot we created in the previous example:

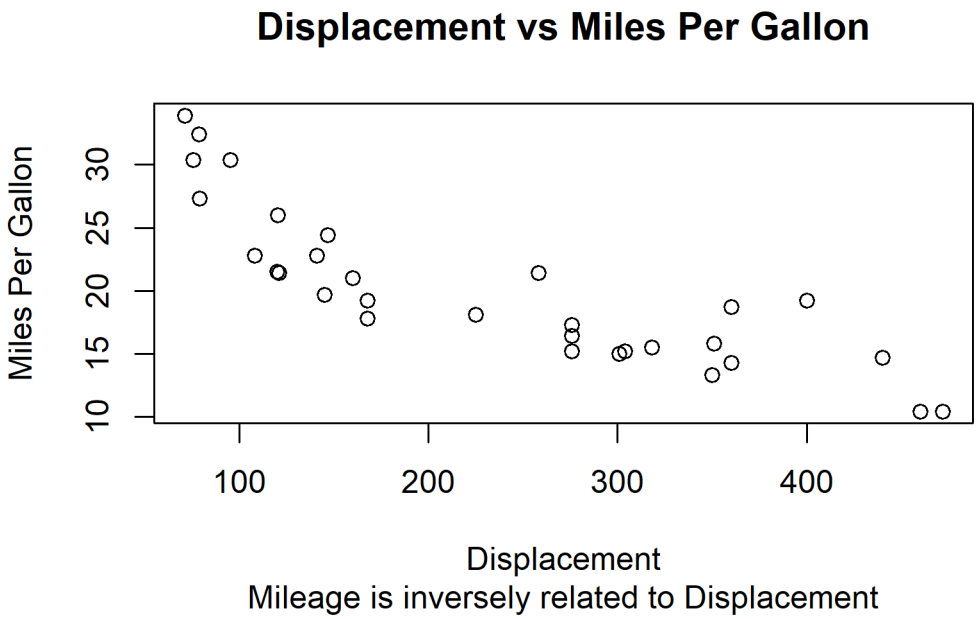
```
plot(mtcars$disp, mtcars$mpg,
     main = 'Displacement vs Miles Per Gallon',
     sub = 'Mileage is inversely related to Displacement')
```



Axis Labels

In the plots created in the previous examples, the axis labels appear as `mtcars$mpg` and `mtcars$disp`. It is not the best way to name the axis and it will make more sense to use names that describe the data. Let us modify the axis labels using the `xlab` and `ylab` arguments in the `plot()` function:

```
plot(mtcars$disp, mtcars$mpg,
     main = 'Displacement vs Miles Per Gallon',
     sub = 'Mileage is inversely related to Displacement',
     xlab = 'Displacement', ylab = 'Miles Per Gallon')
```

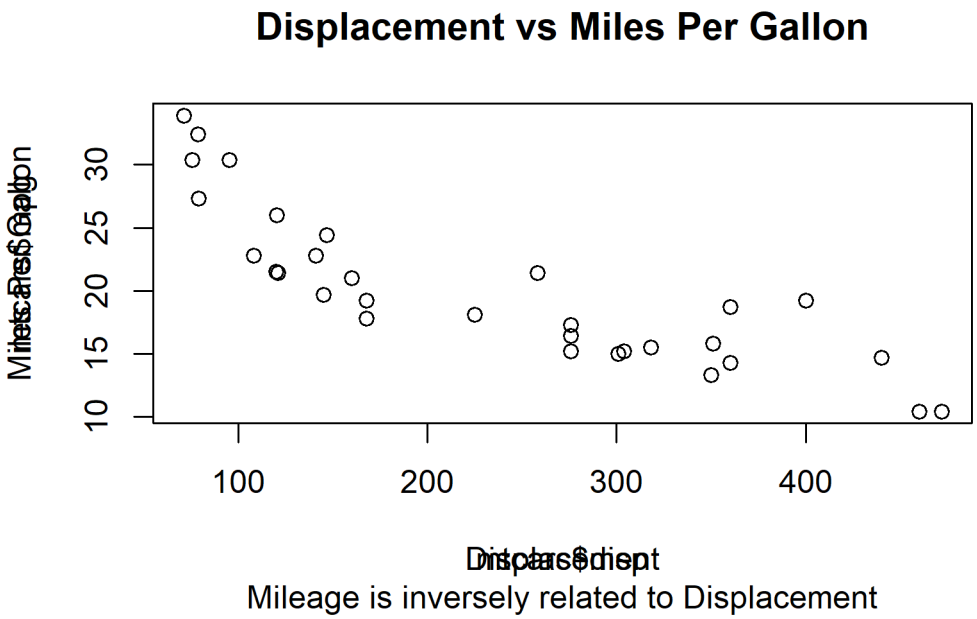


title()

We can add title, subtitle and axis labels using the `title()` function as well. Let us recreate the plots from the previous examples but this time we will use the `title()` instead of the `plot()` function. We will continue to use the `plot()` function to create the plot.

```
# create scatter plot
plot(mtcars$disp, mtcars$mpg)

# add title, subtitle and axis labels
title(main = 'Displacement vs Miles Per Gallon',
      sub = 'Mileage is inversely related to Displacement',
      xlab = 'Displacement', ylab = 'Miles Per Gallon')
```



Do you notice that the axis labels are overwritten? This happens because the `plot()` function adds the default labels and we add a new set of labels without modifying the existing ones. The solution is to instruct the `plot()` function not to add any labels to the X and Y axis. This can be achieved using the `ann` (annotate) argument in the `plot()` function and set it to `FALSE`. Let us try it:

```
# create scatter plot
plot(mtcars$disp, mtcars$mpg, ann = FALSE)

# add title, subtitle and axis labels
title(main = 'Displacement vs Miles Per Gallon',
      sub = 'Mileage is inversely related to Displacement',
      xlab = 'Displacement', ylab = 'Miles Per Gallon')
```

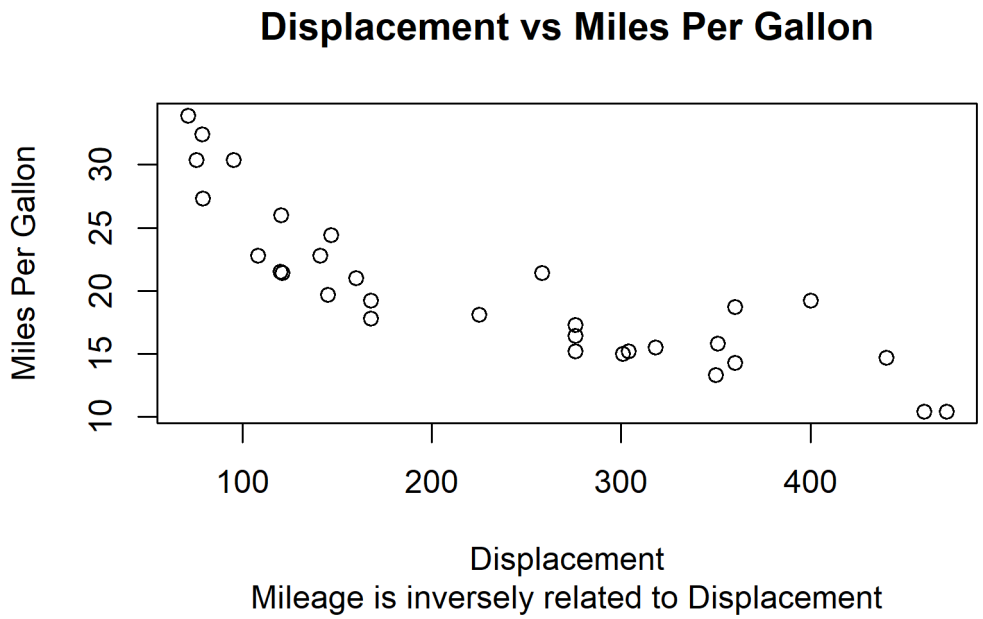


Figure 8: The two-step method: plot with ann = FALSE, then add text with title()

The axis labels are legible and not overwritten. You can use either the `plot()` function or the `title()` function to add title, subtitle and axis labels but ensure that in case you use the `title()` function, set `ann` argument to `FALSE` in the `plot()` function.

Axis Range

In certain cases, you would want to modify the range of the axis of the plots. By default, the `plot()` function will take into account the `min` and `max` values of the variable(s) and set the range for the axis. We can modify the range by using the `xlim` and `ylim` arguments in the `plot()` function. Both the `xlim` and `ylim` arguments take 2 values as inputes. The first value is the minimum value for the axis and the second value is the maximum value for the axis. The `plot()` function will return an error if we do not specify two values for both `xlim` and `ylim` arguments. Let us recreate the plot from the previous examples but change the range of both the X and Y axis:

```
plot(mtcars$disp, mtcars$mpg,
     xlim = c(0, 600), ylim = c(0, 50))
```

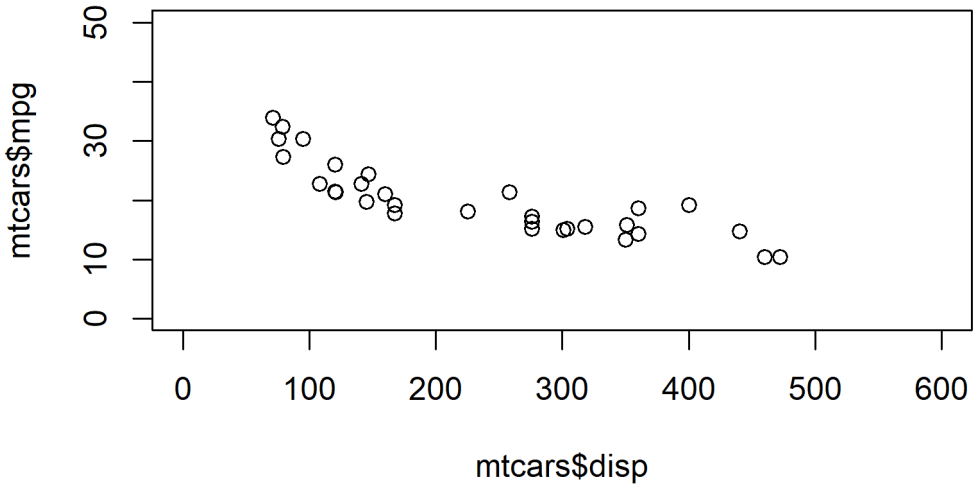


Figure 9: Title, subtitle and enlarged title text combined

Keep in mind that the axis ranges cannot be modified using the `title()` function.

Try it live

> **Try it in your browser.** Edit the code below and press **Run** – it executes entirely in your browser via WebR, so no R installation is needed. The runtime downloads once, on this page only. The printed plots are produced in your browser and are not part of the PDF or ePub editions.

```
{webr-r}
# every label argument from this chapter at once
plot(mtcars$disp, mtcars$mpg,
     main = 'Displacement vs Miles Per Gallon',
     sub = 'Mileage is inversely related to Displacement',
     xlab = 'Displacement', ylab = 'Miles Per Gallon',
     xlim = c(0, 600), ylim = c(0, 50))
title(main = 'Retitled with title()', cex.main = 1.4)
```

Scatter Plots

Introduction

In this chapter, we will learn how to create scatter plots.

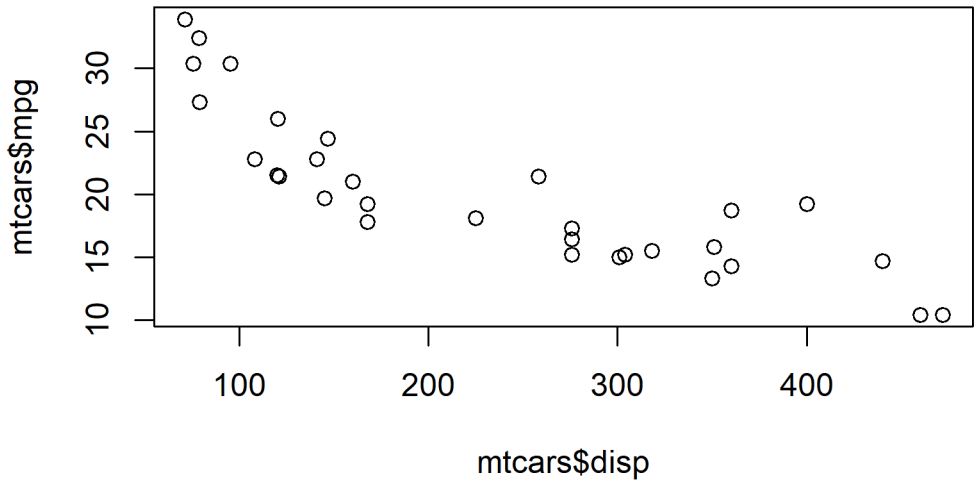
- adding color to the points
- modify shape of the points

- modify size of the points

Basic Plot

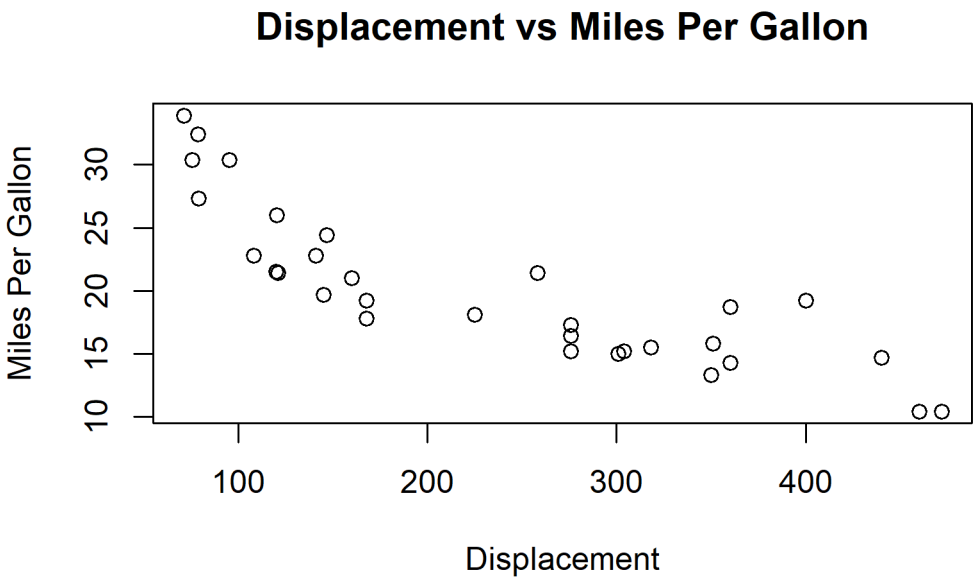
Let us recreate the plot that we had created in Chapter 2 by using the `mtcars` data set. We will use the `disp` (displacement) and `mpg` (miles per gallon) variables. `disp` will be on the X axis and `mpg` will be on the Y axis.

```
plot(mtcars$disp, mtcars$mpg)
```



We have created a very basic plot and any one looking at it for the first time will get confused with the axis labels `mtcars$disp` and `mtcars$mpg`. Let us put into practice what we learnt in Chapter 3, and add a title to the plot, and make the axis labels more meaningful.

```
plot(mtcars$disp, mtcars$mpg,
     main = 'Displacement vs Miles Per Gallon',
     xlab = 'Displacement', ylab = 'Miles Per Gallon')
```

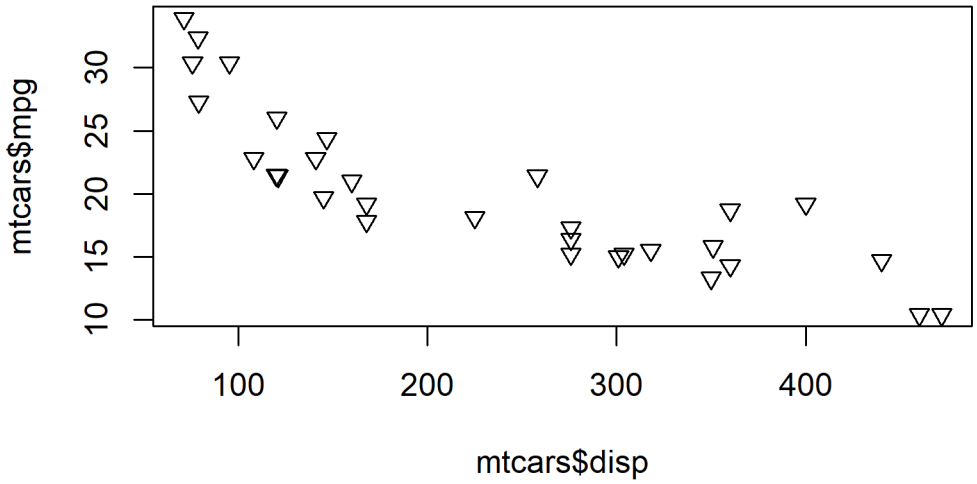


Now the plot clearly communicates that it represents the relationship between the displacement and mileage of cars. Now the color of the points in the plot is black by default. Some of us may agree that black is beautiful but not all of us will like it. As a first step in enhancing the way our plot looks, let us change the shape of the points.

Shape

The shape of the point can be specified using the `pch` argument. It will take values between **0** and **25**. Below is an example:

```
# point shape
plot(mtcars$disp, mtcars$mpg, pch = 6)
```



Let us check out a few of the other shapes:

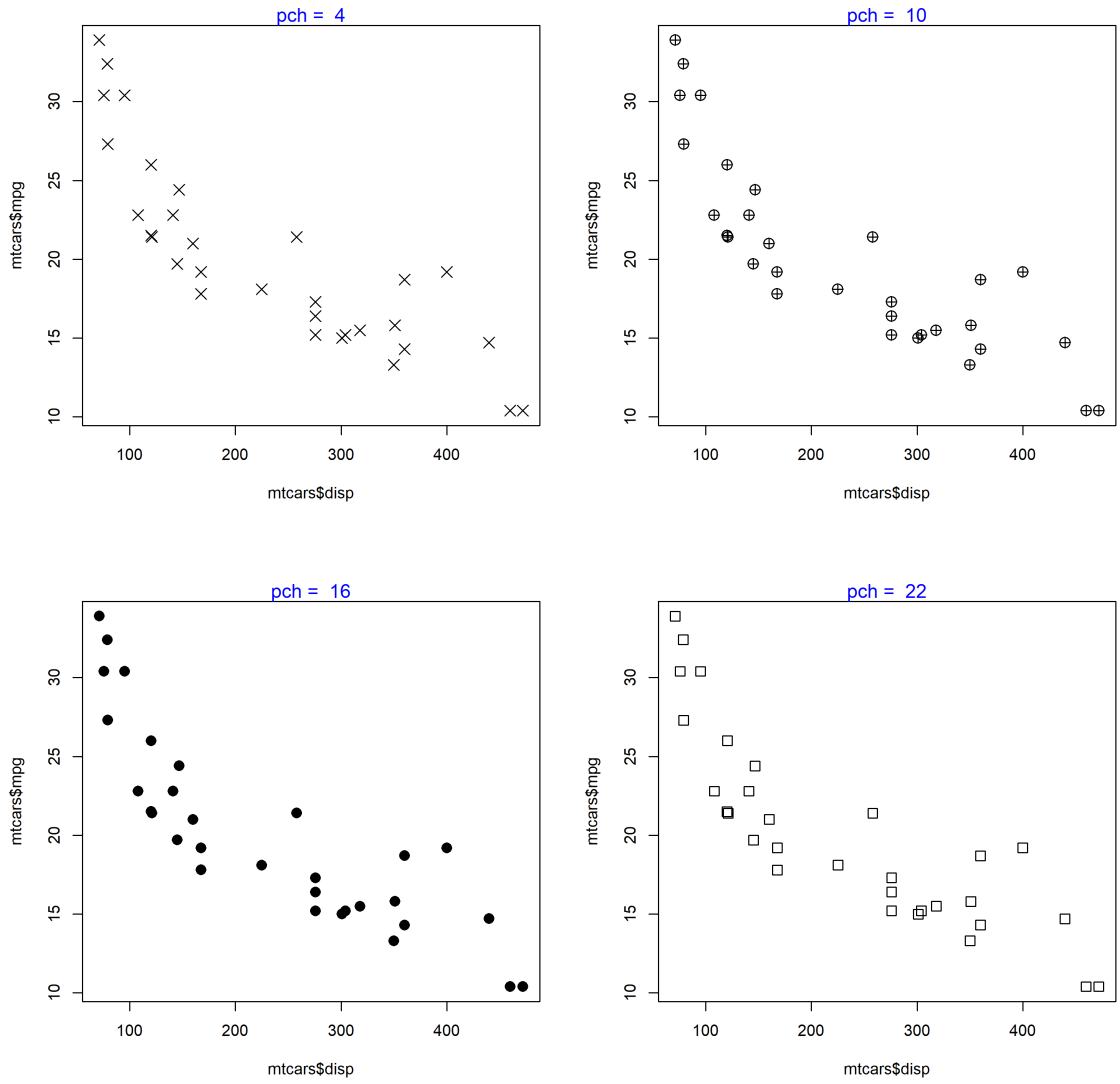


Figure 10: The pch argument across the plotting symbols

We can specify the shape based on a third (categorical variable as well). In the below plot, the shape is based on the levels of the categorical variable `cyl` (number of cylinders) from the `mtcars` data set. Note that `nlevels(factor(mtcars$cyl))` returns a single value (3), so it would give every point the same shape — instead, map each observation to its own level with `as.numeric(factor(...))`:

```
# shape based on a third categorical variable
plot(mtcars$disp, mtcars$mpg, pch = as.numeric(factor(mtcars$cyl)))
legend('topright', legend = c('4 cyl', '6 cyl', '8 cyl'), pch = 1:3)
```

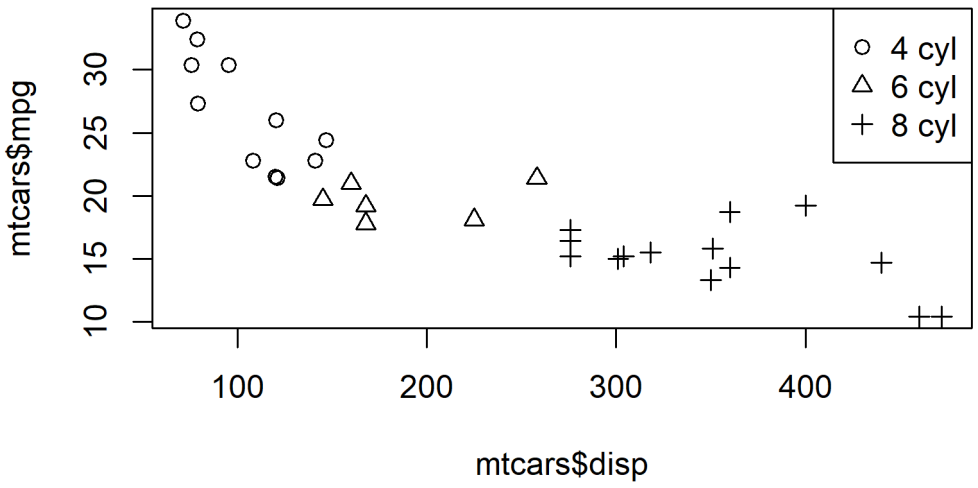
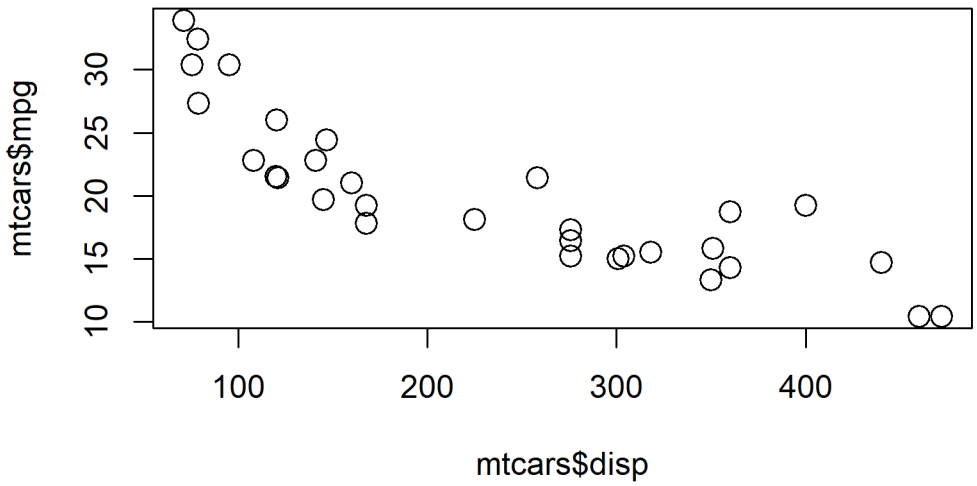


Figure 11: Mapping a third variable to point shape using as.numeric(factor())

Size

The size of the points in the scatter plot can be specified using the `cex` argument in the `plot()` function. The default value for `cex` is
1.

```
# point size
plot(mtcars$disp, mtcars$mpg, cex = 1.5)
```



The below plots show the size of the points for values relative to 1.

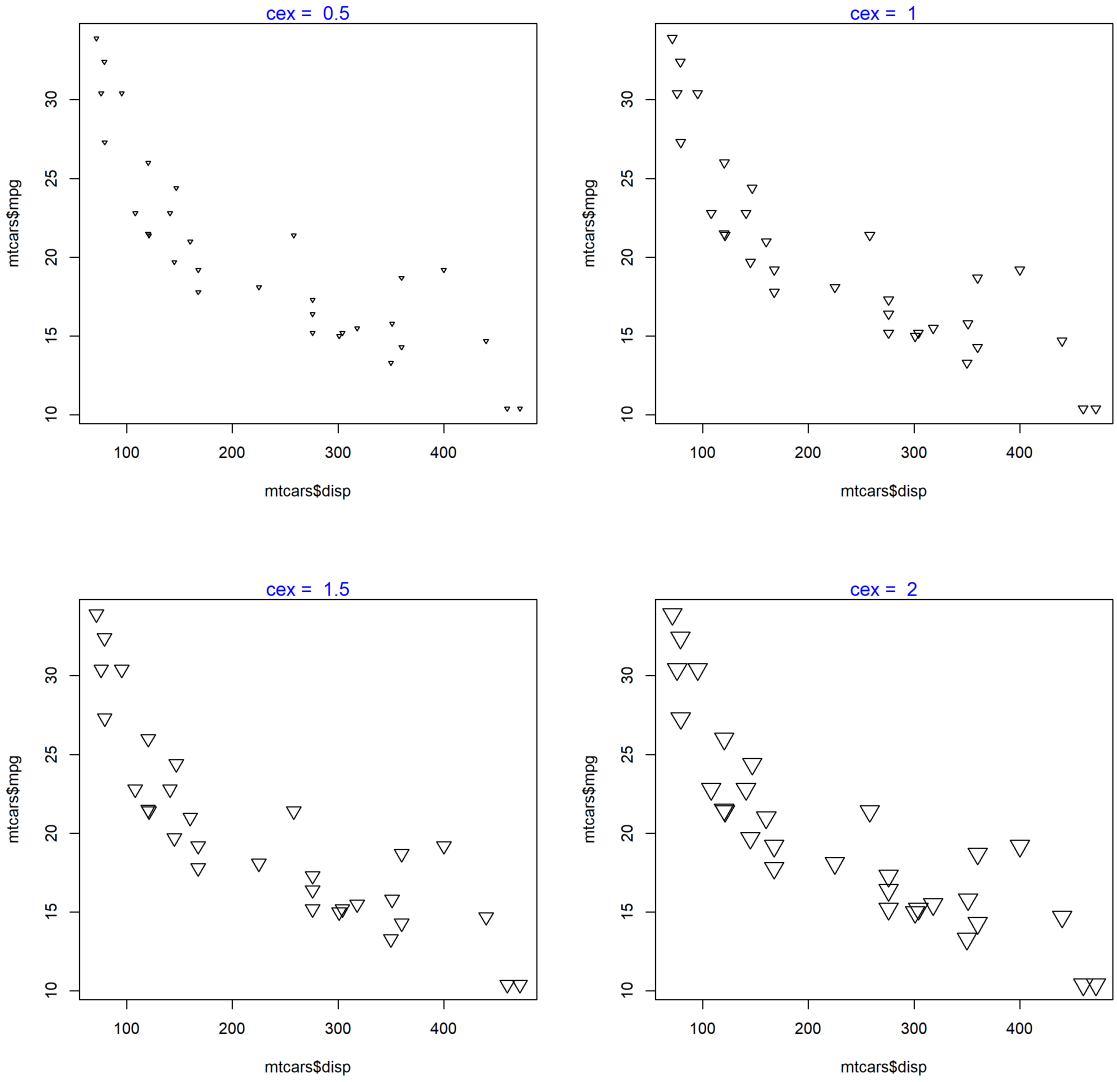
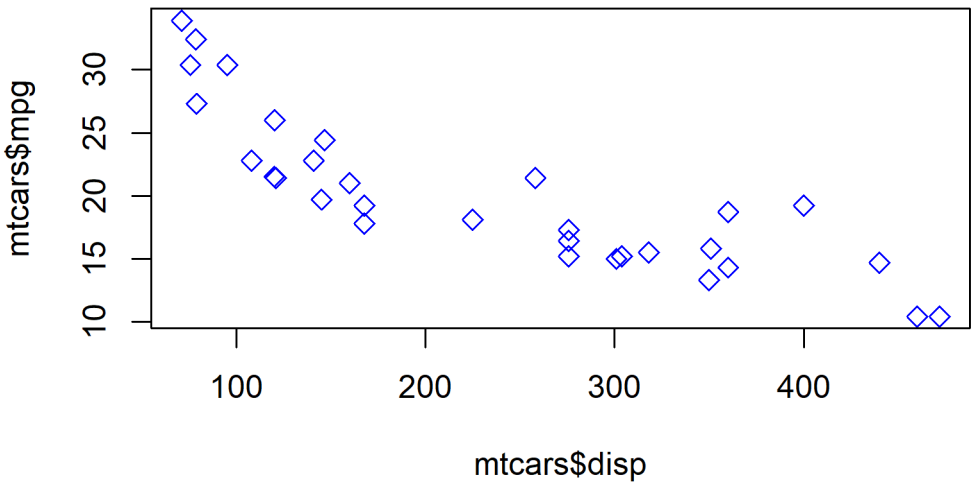


Figure 12: The cex argument across a range of point sizes

Color

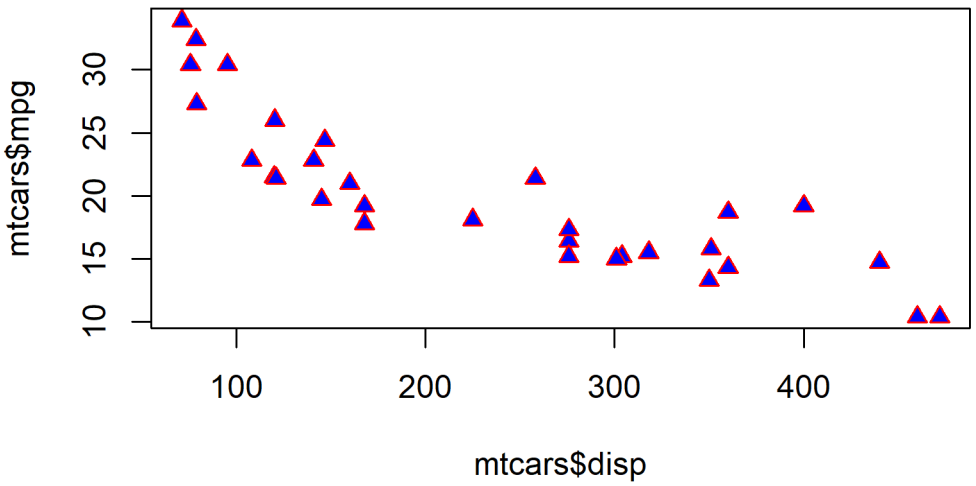
We can specify a border color for the points using the `col` argument and a background color using the `bg` argument. The background color can be specified only for points whose `pch` argument takes values between **21** and **25**. Let us look at some examples to understand this distinction between border and background color.

```
# shape between 0 and 21
plot(mtcars$disp, mtcars$mpg, pch = 5, col = 'blue', bg = 'red')
```



You can observe that although we have specified a background color using the `bg` argument, we do not see the red background color as the value specified for the `pch` (shape) argument is not between 21 and 25. In the next example, we will use a value between 21 and 25 so that the `pch` argument is effective.

```
# shape between 22 and 25
plot(mtcars$disp, mtcars$mpg, pch = 24, col = 'red', bg = 'blue')
```



The color of the points can be specified using (levels) of a categorical variable as well. In the next example, we will use the `cyl` variable to specify the color of the points.

```
# color based on a third variable
plot(mtcars$disp, mtcars$mpg, pch = 5, col = as.numeric(factor(mtcars$cyl)))
```

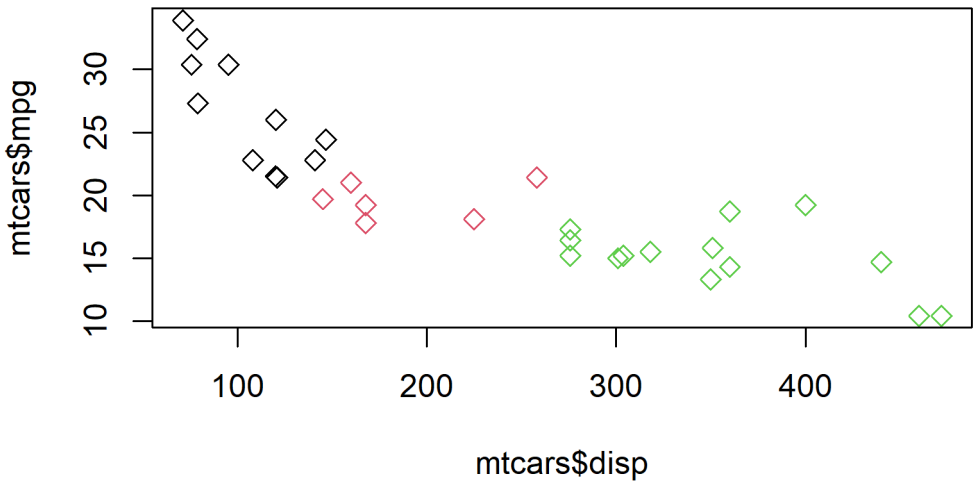


Figure 13: Two variables mapped at once: shape to cylinders, colour to transmission

Since `cyl` is a categorical variable with 3 levels, we can see that the points now have 3 different colors. The above method is useful when you want to segregate the points in a scatter plot based on a third variable.

Try it live

> **Try it in your browser.** Edit the code below and press **Run** – it executes entirely in your browser via WebR, so no R installation is needed. The runtime downloads once, on this page only. The printed plots are produced in your browser and are not part of the PDF or ePub editions.

```
{webr-r}
# combine shape, size and color from this chapter
plot(mtcars$disp, mtcars$mpg,
     pch = as.numeric(factor(mtcars$cyl)),
     cex = mtcars$wt * 4,
     col = as.numeric(factor(mtcars$am)))
legend("topright", legend = c("4 cyl", "6 cyl", "8 cyl"), pch = 1:3)
```

Line Graphs

Introduction

In this chapter, we will build line graphs. To be more specific we will learn to

- create line plots
- add color to lines
- modify line type/style
- modify line width
- add points to the lines
- modify axis range
- add additional lines to the plot
- add error bars with `segments()`
- annotate the plot with `arrows()`
- shade a confidence band with `polygon()`

Basic Plot

To build a line graph, we will learn a new argument in the `plot()` function called `type`. It allows us to specify the symbol that must be used to represent the data. Let us begin by building a simple line graph. We will use the `AirPassengers` data set in this chapter. Before we begin to build the plot, let us take a quick look at the data in order to understand what we are plotting.

```
head(AirPassengers)
```

```
     Jan Feb Mar Apr May Jun
1949 112 118 132 129 121 135
```

In order to build a line plot, we will set the `type` argument in the `plot()` function to **l** (line). There are other values which `type` takes but we will explore them later.

```
data <- head(AirPassengers)
plot(data, type = 'l')
```



If you do not like plain lines, you can represent the data using lines interspersed with points by setting the `type` argument to **b** (both lines and points).

```
plot(data, type = 'b')
```



Another option is to have the points and lines overplotted. It can be achieved by setting the `type` argument to **o** (overplotted).

```
plot(data, type = 'o')
```



You can also create lines without points but with breaks instead by setting the `type` argument to **c**.

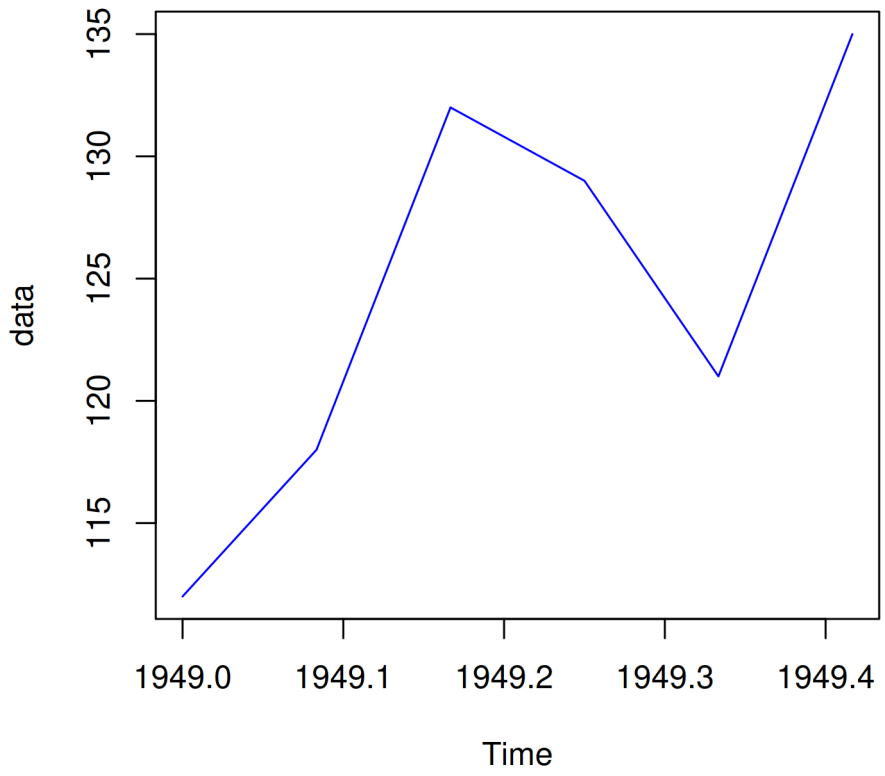
```
plot(data, type = 'c')
```



Color

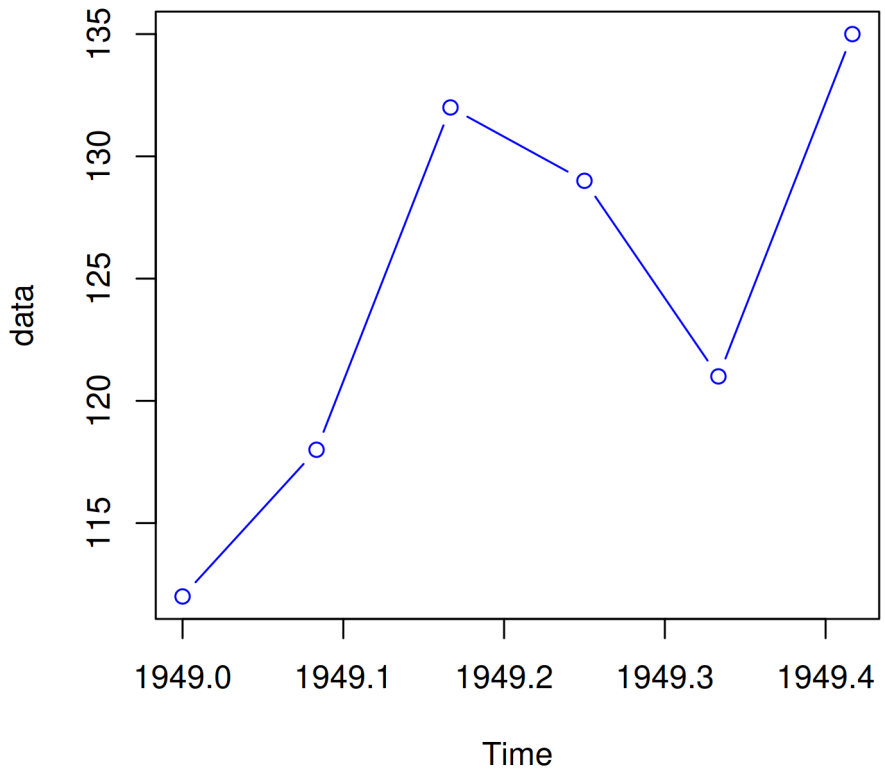
So now we know how to build a simple line graph. Let us now make this plot more elegant by modifying its appearance. Let us begin by adding some color to the line using the `col` argument in the `plot()` function.

```
plot(data, type = 'l', col = 'blue')
```



If you have points along with the line, they will have the same color as well.

```
plot(data, type = 'b', col = 'blue')
```



Line Type

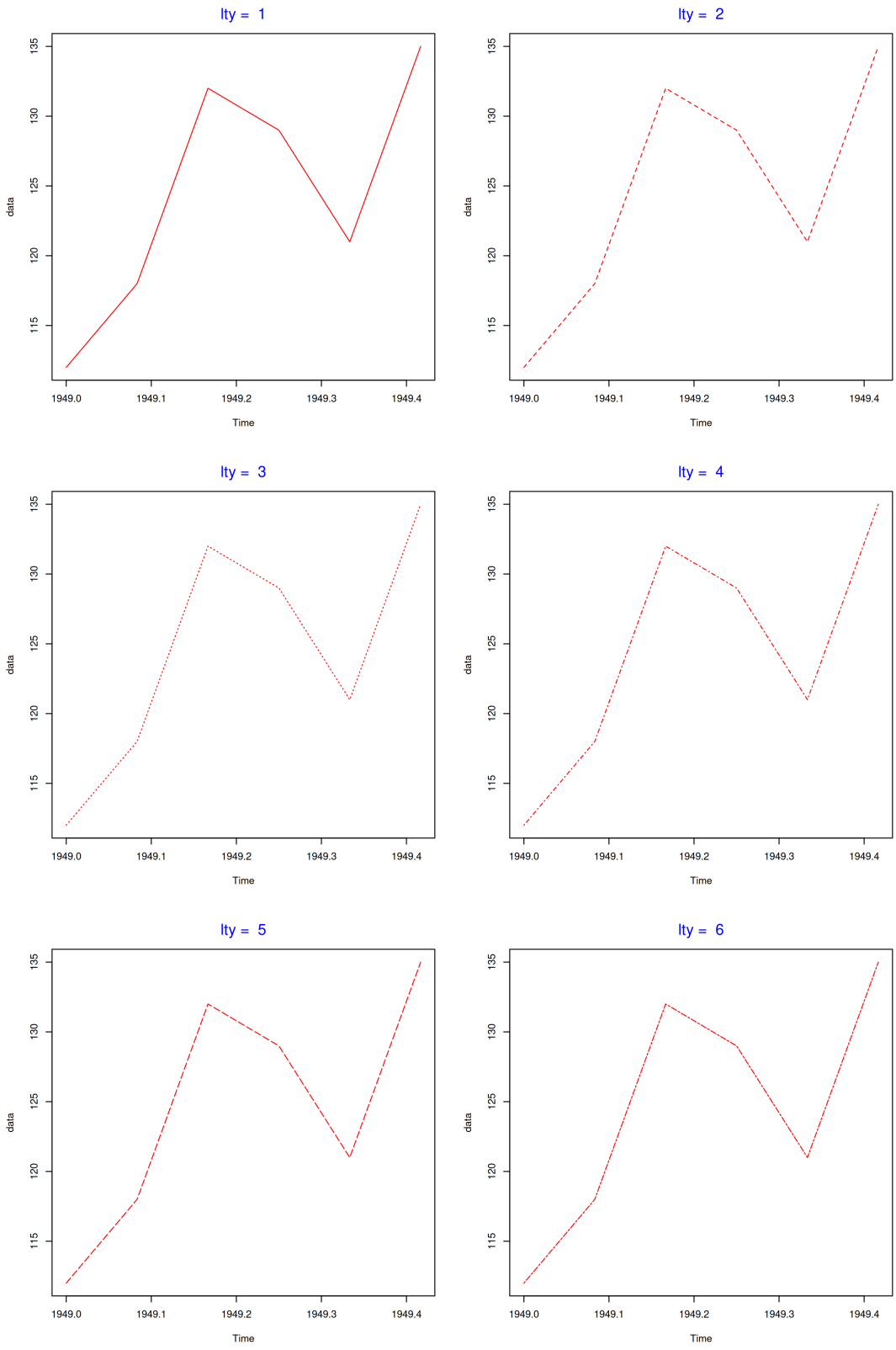
The line type can be modified using the `lty` argument. It takes values from 1 to 6 and the default value is 1. Below is an example:

- 1:solid
- 2:dashed
- 3:dotted
- 4:dotdash
- 5:longdash
- 6:twodash

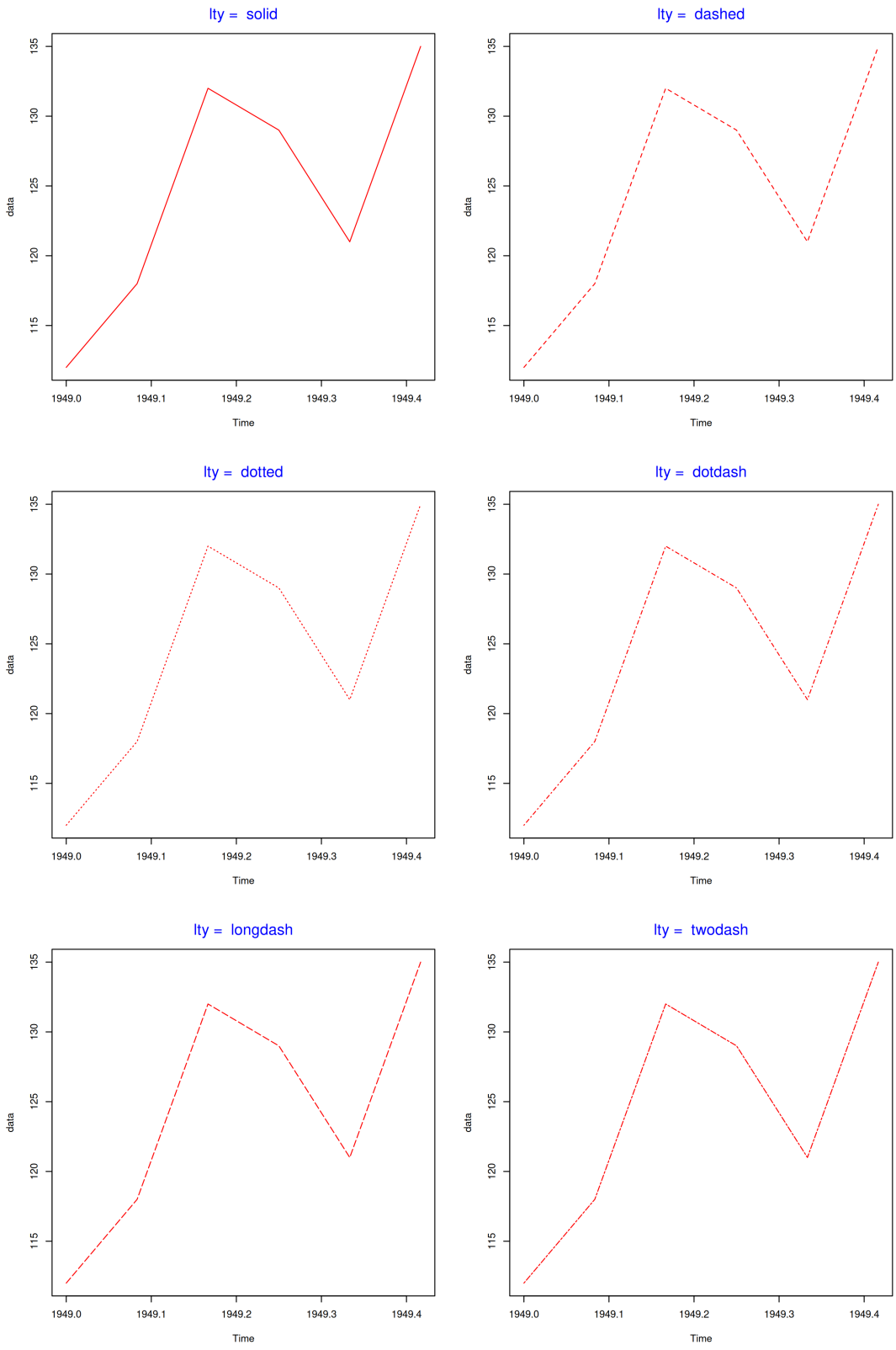
```
plot(data, type = 'l', lty = 3)
```



Let us look at all the line types in the below example:


lty = 1
lty = 2
lty = 3
lty = 4
lty = 5
lty = 6
data
Time
115
120
125
130
135
1949.0
1949.1
1949.2
1949.3
1949.4

Instead of specifying the numbers 1:6, you can use their description as well.


lty = solid
lty = dashed
lty = dotted
lty = dotdash
lty = longdash
lty = twodash
data
Time
115
120
125
130
135
1949.0
1949.1
1949.2
1949.3
1949.4

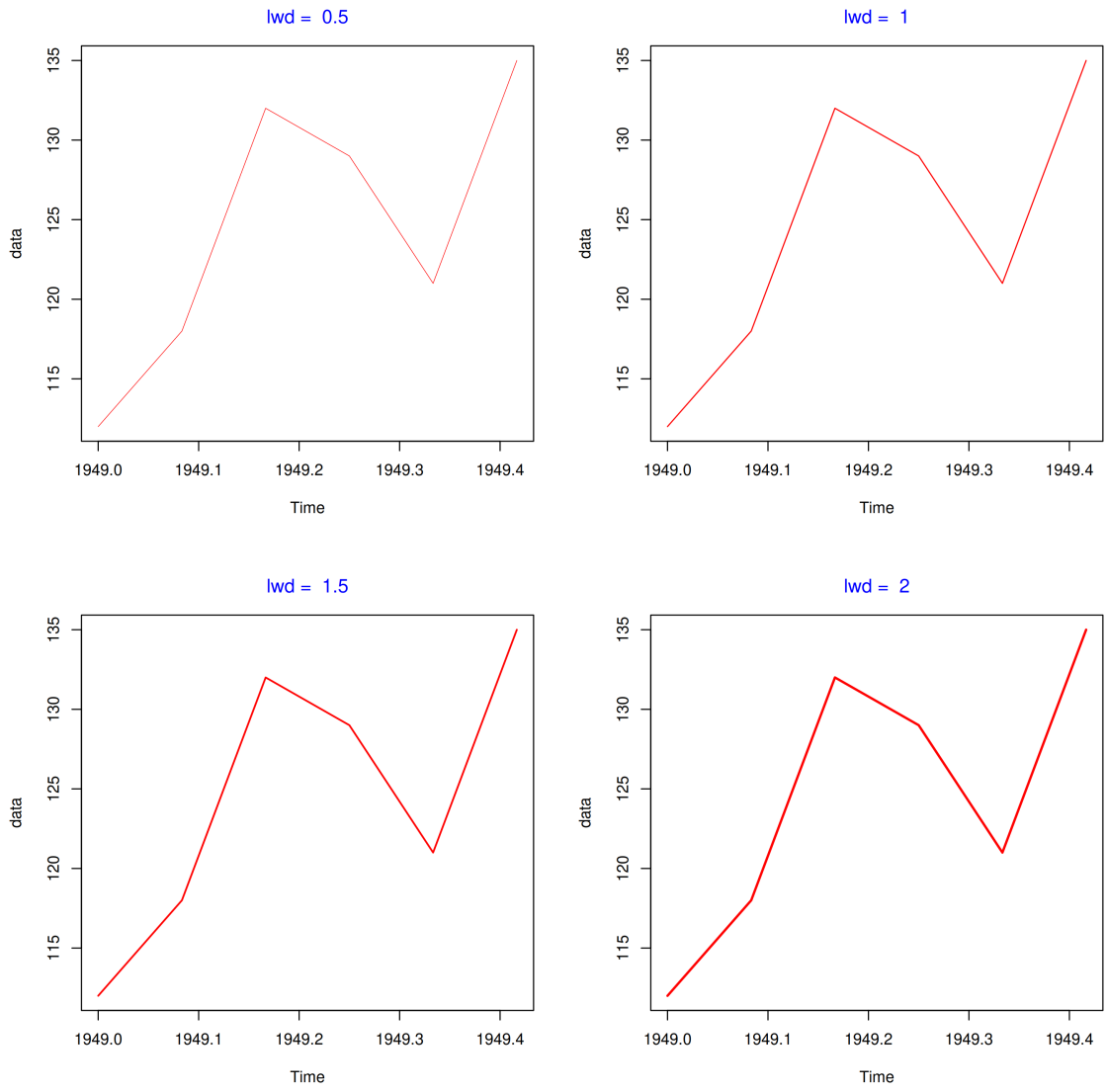
Line Width

The width of the lines can be modified using the `lwd` argument in the `plot()` function. The default value for width is 1.

```
plot(data, type = 'l', lwd = 2.5)
```



In the below example, we look at the width of the lines relative to the default value.



Enhance Points

We can enhance the points in the line plot in the same way as we enhanced the points in the scatter plot in this previous chapter. Let us look at an example:

```
plot(data, type = 'b', pch = 23, col = 'red', cex = 1.5)
```

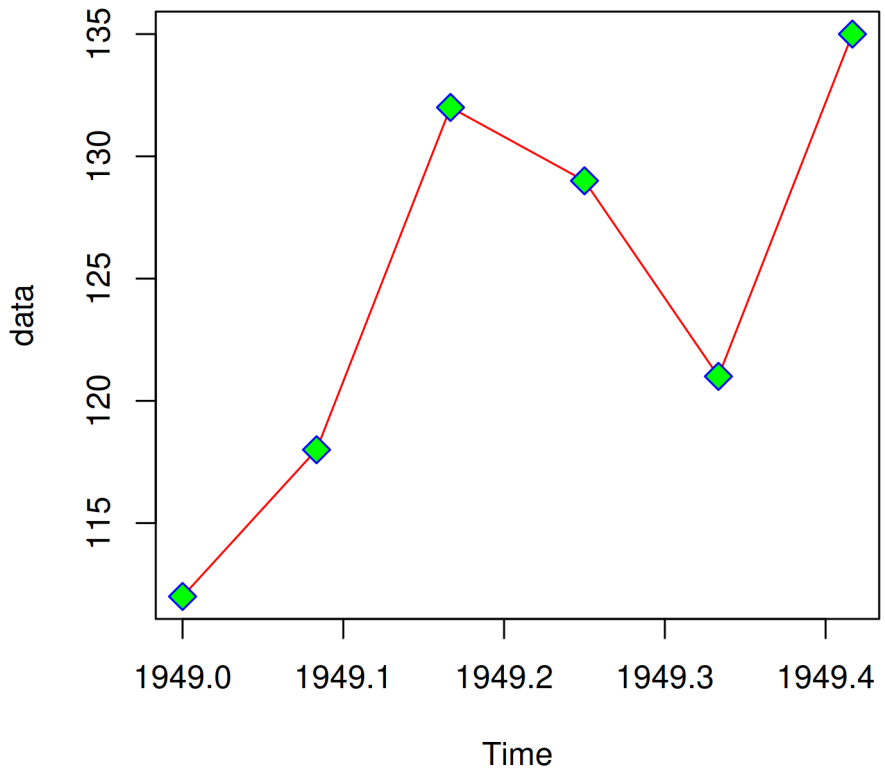


We have used the `pch`, `col` and `cex` arguments to modify the shape, color and size of the points. One drawback of the above method is that the color of the line and the points will be the same. What if we want them to have different colors? The solution is as follows:

- build the line graph using the `plot()` function
- add the points to the above plot using the `points()` function

In the next example, let us first build the line plot, add points using the `points()` function and then specify separate colors to the line and the points.

```
plot(data, type = 'l', col = 'red')
points(data, pch = 23, col = 'blue', bg = 'green', cex = 1.5)
```



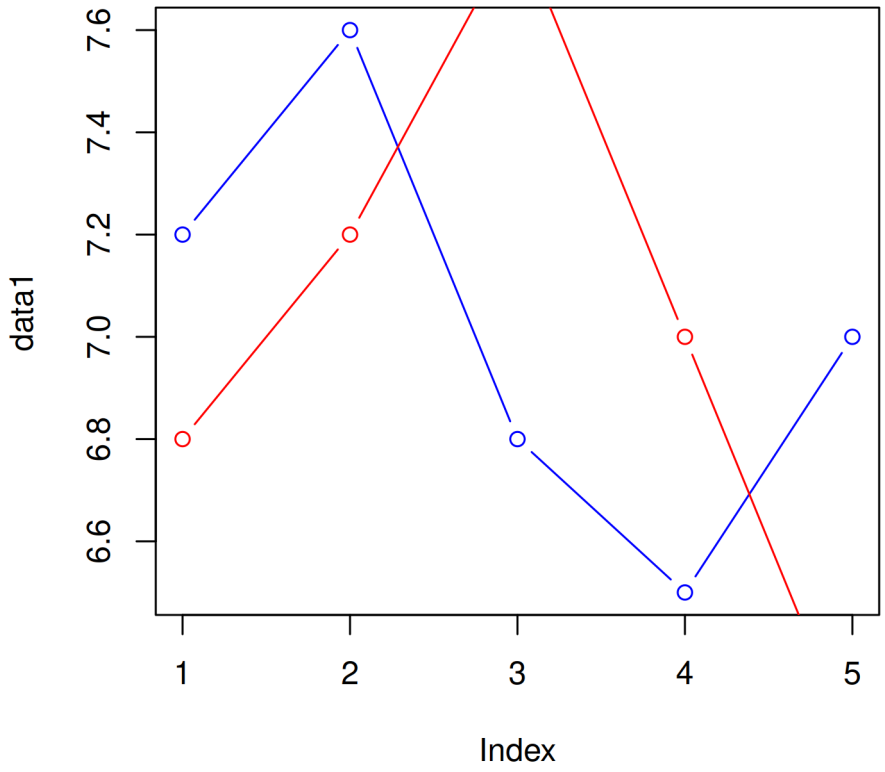
Additional Lines

If you want to compare variables, you would want to add additional lines to the line graph. In R, this can be achieved using the `lines()` function. First we create the line plot using the base variable and then we can add as many lines as we want using the `lines()` function.

Before you add additional lines, it is important to ensure that the range of both the axis are modified to accommodate the data of the additional lines. If we do not modify the axis range, some of the lines will be outside the plot.

Let us now create a line plot and add an additional line using the `lines()` function.We will use some dummy data for this example:

```
data1 <- c(7.2, 7.6, 6.8, 6.5, 7)
data2 <- c(6.8, 7.2, 7.8, 7, 6.2)
plot(data1, type = "b", col = "blue")
lines(data2, type = "b", col = "red")
```



As you can see the second line is outside the plot. Let us recreate the plot but this time we will modify the range of the axis to accommodate the second line (data2).

```
plot(data1, type = "b", col = "blue", ylim = c(5, 9))
lines(data2, type = "b", col = "red")
```

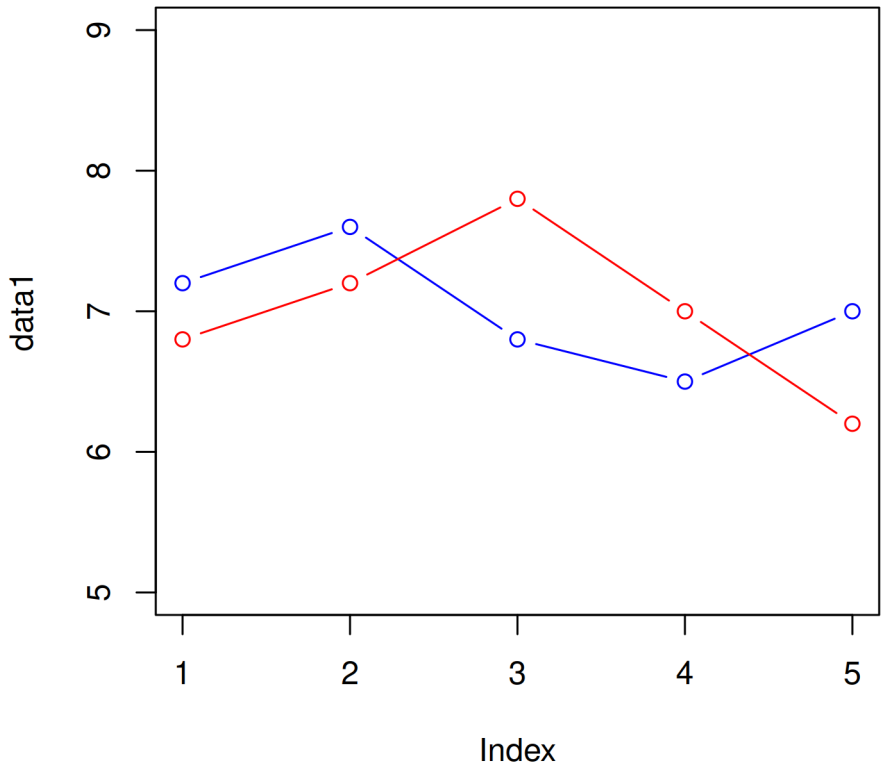


Figure 14: Widening the y axis so a second series fits inside the plot region

Uncertainty and Annotation

A line through data points claims more precision than the data usually supports. Three functions close that gap, and they are the ones you will reach for the moment a plot leaves the screen and someone asks what it means.

segments()

`segments()` draws straight line segments between coordinate pairs. Its commonest use on a line graph is an error bar: a vertical segment centred on each point.

```
data1 <- c(7.2, 7.6, 6.8, 6.5, 7)
sd1 <- c(0.4, 0.6, 0.5, 0.7, 0.3)
plot(data1, type = 'b', col = 'blue', ylim = c(5, 9),
     xlab = 'Index', ylab = 'Data')
segments(1:5, data1 - sd1, 1:5, data1 + sd1, col = 'red', lwd = 2)
```

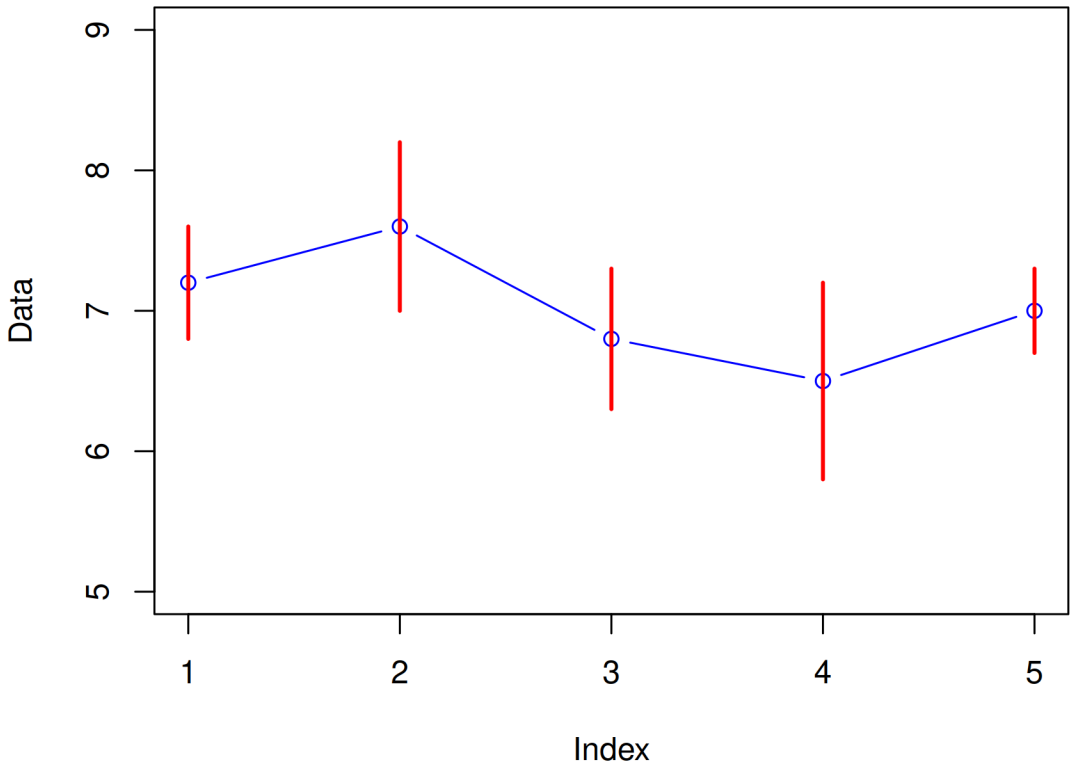


Figure 15: Vertical error bars drawn with segments()

`segments()` takes four vectors: the x of each segment's start, the y of the start, the x of the end, and the y of the end. Because all four are vectorised, one call covers every point. It also draws caps if you want them, via the horizontal segments underneath:

```
plot(data1, type = 'b', col = 'blue', ylim = c(5, 9),
     xlab = 'Index', ylab = 'Data')
segments(1:5, data1 - sd1, 1:5, data1 + sd1, col = 'red', lwd = 2)
# caps: two short horizontal segments per error bar
segments(1:5 - 0.1, data1 - sd1, 1:5 + 0.1, data1 - sd1, col = 'red')
segments(1:5 - 0.1, data1 + sd1, 1:5 + 0.1, data1 + sd1, col = 'red')
```

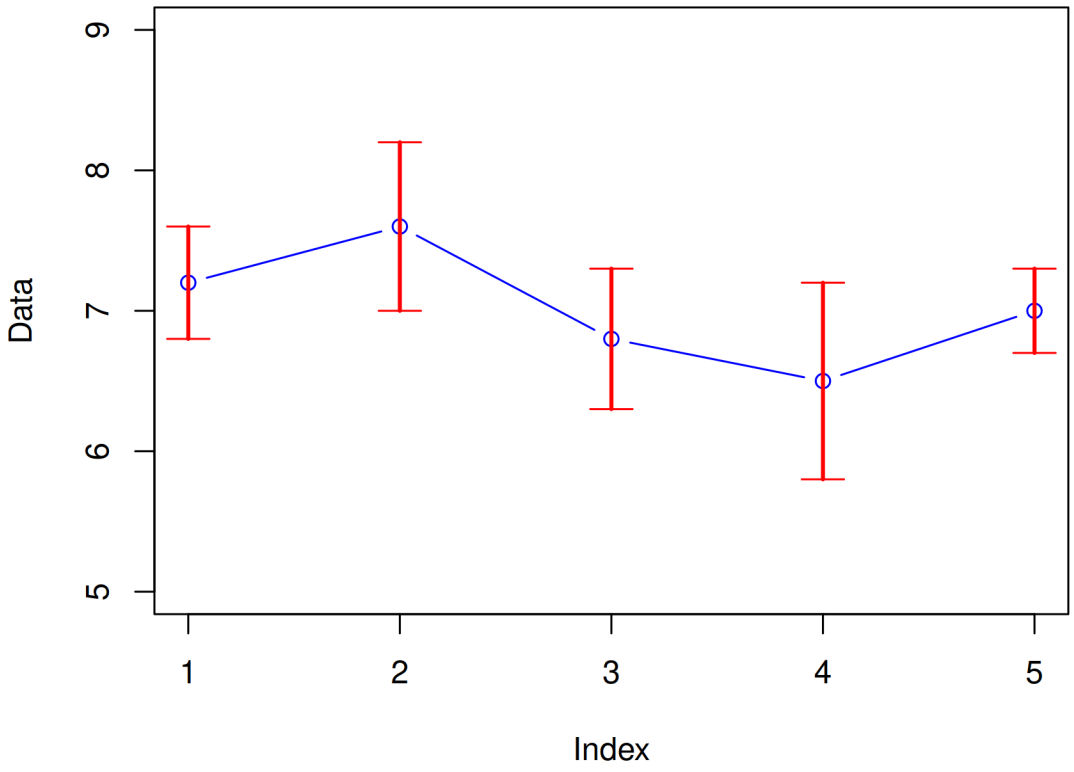


Figure 16: Adding caps to the error bars with two more segments() calls

arrows()

`arrows()` draws a line with a head at one end. Use it when the reader's eye needs to be sent somewhere specific.

```
data2 <- c(6.8, 7.2, 7.8, 7, 6.2)
plot(data1, type = 'b', col = 'blue', ylim = c(5, 9),
     xlab = 'Index', ylab = 'Data', main = 'Arrow points at the gap')
arrows(4.4, 6.1, 3, 7.6, col = 'red', lwd = 2)
```

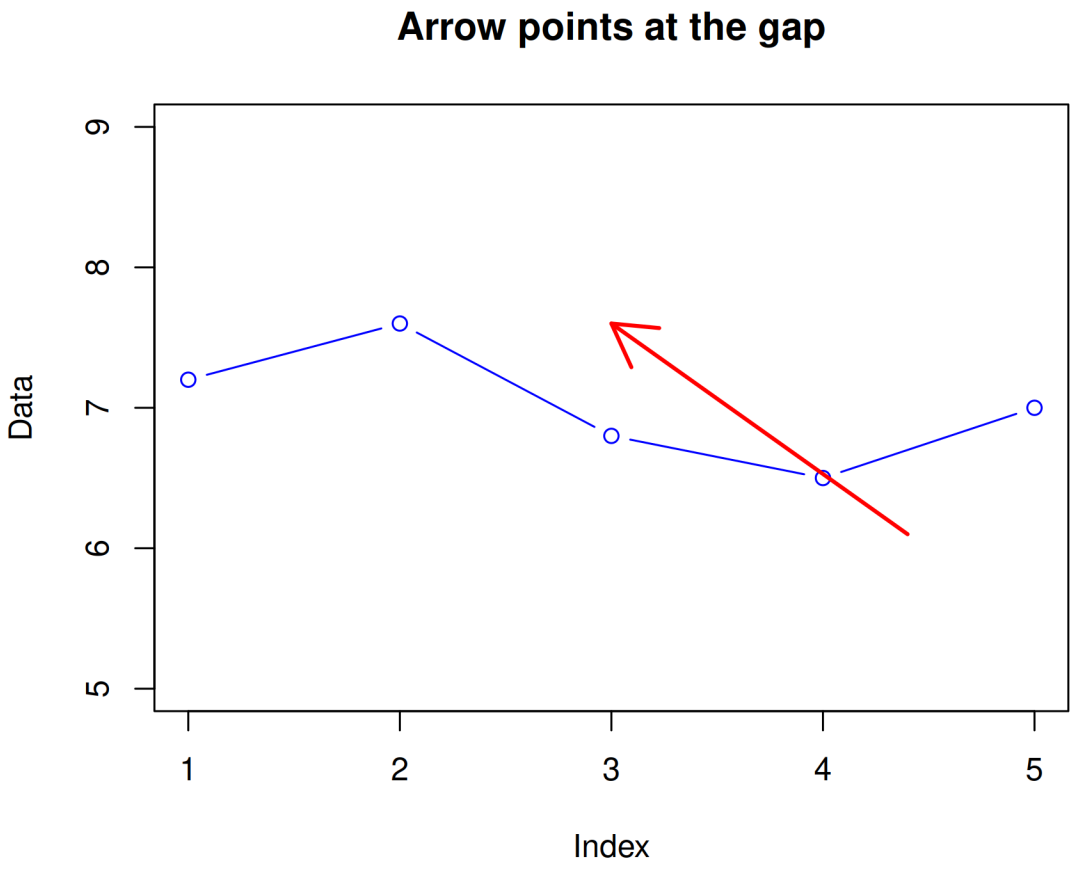


Figure 17: Using arrows() to draw attention to a feature

The first two arguments are where the arrow **starts**, the next two where it **ends**. That is the opposite of the order you would guess, and it is worth remembering: `arrows(x0, y0, x1, y1)` reads "from here, to there".

Useful arguments: `length` sets the head size (0.1 is the default; 0 leaves a plain line), `angle` opens the head, and `code` picks the head shape — 1 for a closed arrow, 2 for an open one.

```
plot(data1, type = 'b', col = 'blue', ylim = c(5, 9),
     xlab = 'Index', ylab = 'Data', main = 'Open head, thinner line')
arrows(4.4, 6.1, 3, 7.6, col = 'red', lwd = 1.5, length = 0.15,
       angle = 30, code = 2)
```

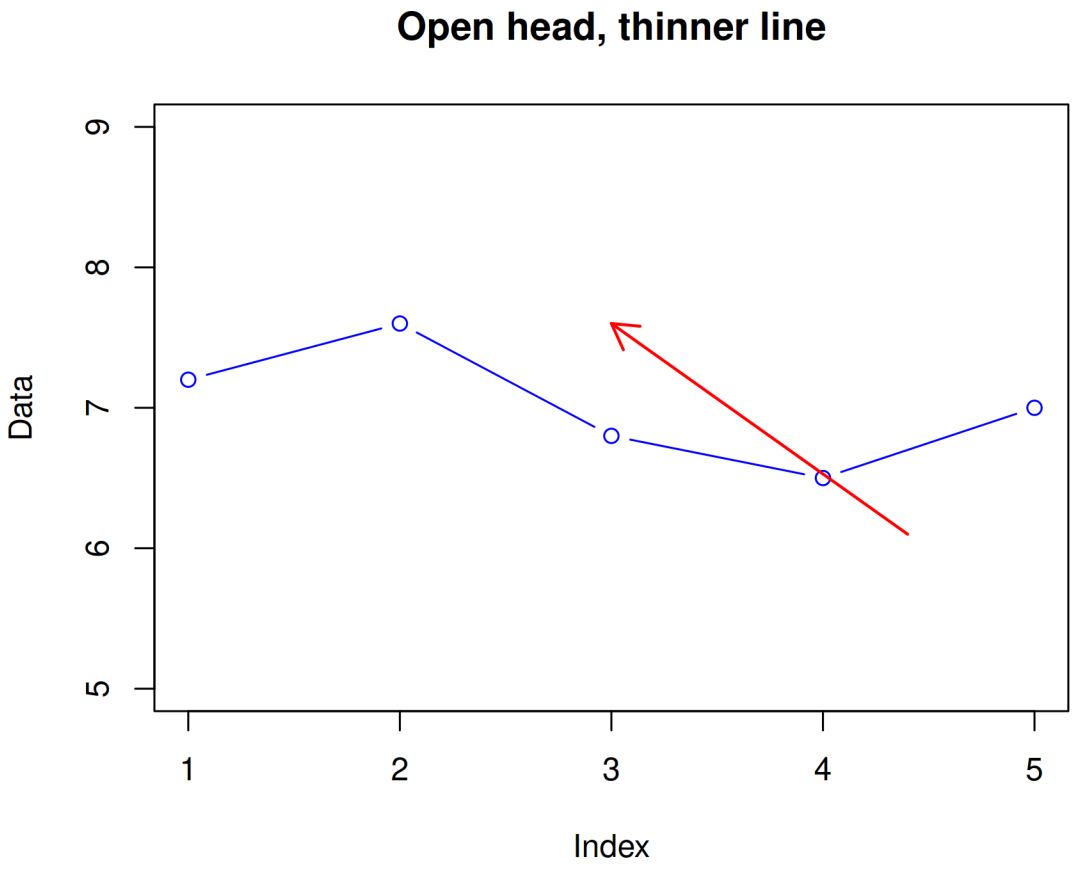


Figure 18: The same annotation with an open arrow head and a longer head length

polygon()

`polygon()` draws a closed shape and fills it. On a line graph its use is a confidence band: shade the area between an upper and a lower bound, then draw the line on top.

The order of the points matters, because that is the order they get joined. For a band you want to walk along the top from left to right, then back along the bottom from right to left, so the shape closes on itself instead of crossing.

```
x <- 1:12
mid <- c(4.5, 5.2, 5.0, 6.1, 6.8, 7.4, 7.1, 8.0, 8.6, 9.1, 9.4, 10.2)
lo <- mid - 0.9
hi <- mid + 0.9

plot(x, mid, type = 'n', ylim = c(2, 12), col = 'blue',
     xlab = 'Month', ylab = 'Value')
polygon(c(x, rev(x)), c(hi, rev(lo)), col = '#DCE9F5', border = NA)
lines(x, mid, col = 'blue', lwd = 2)
points(x, mid, pch = 19, col = 'blue')
```

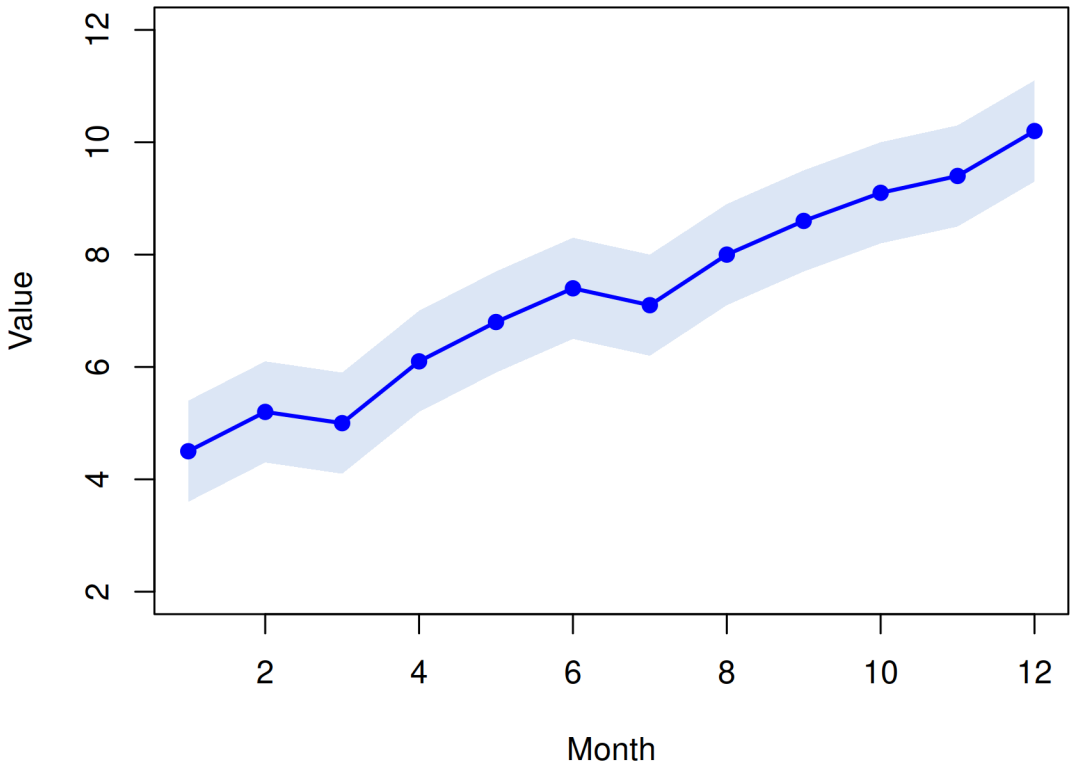


Figure 19: A confidence band shaded with polygon() and the line drawn on top

Two details carry that figure. The first is `type = 'n'`, which sets up the axes and the plot region without drawing anything — the standard way to draw a plot in layers when the first layer is not the data itself. The second is `rev()`, which reverses a vector so the polygon can be closed in one call instead of building the point list by hand.

Note that `polygon()` is drawn first and `lines()` second. Anything you fill will cover anything already plotted, so background bands belong underneath.

Putting it all together

Finally let us enhance the plot by adding a title and modifying the axis labels.

```
plot(data1, type = "b", col = "blue",
     ylim = c(5, 9), main = 'Line Graph',
     xlab = 'Index', ylab = 'Data')
lines(data2, type = "b", col = "red")
segments(1:5, data1 - 0.5, 1:5, data1 + 0.5, col = "blue", lwd = 2)
# Approach the peak from above-right so the arrow crosses neither series.
arrows(4.1, 8.5, 3.15, 7.85, col = "grey30", length = 0.1)
```

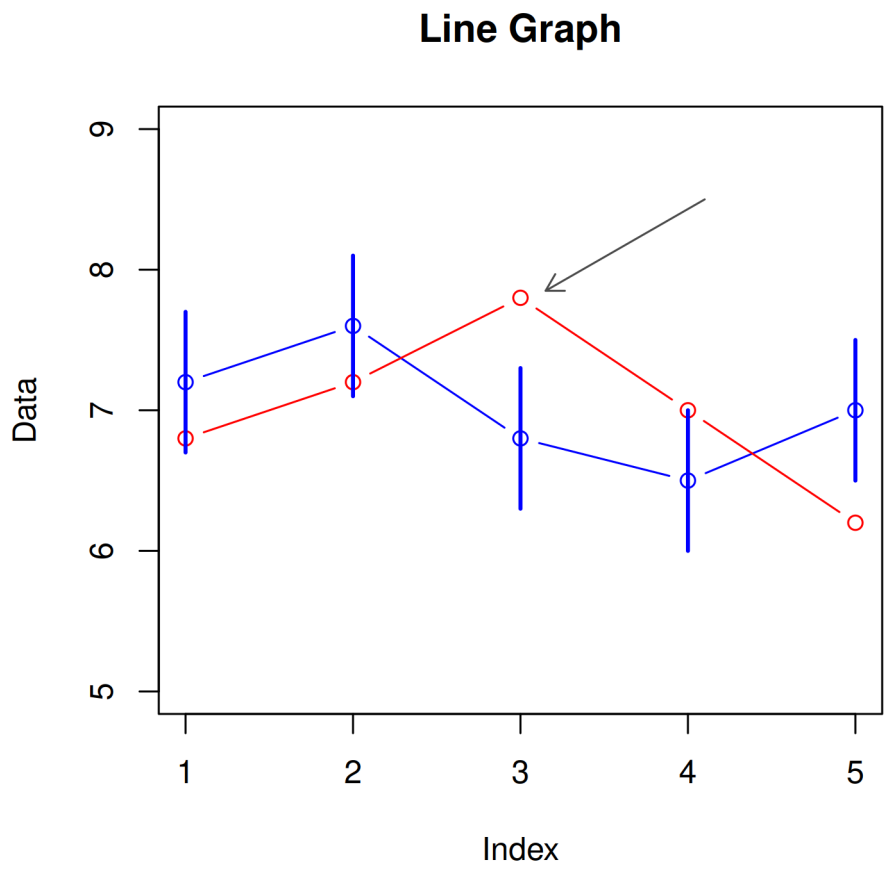


Figure 20: Layering error bars and an annotation onto a two-series line chart

Bar Plots

Introduction

In this chapter, we will visualize categorical data using univariate and bivariate bar plots. More specifically, we will learn to:

- create
  - simple bar plot
  - stacked bar plot
  - grouped bar plot
- modify bar
  - direction
  - color
  - line color
  - width
  - labels

- modify axis range
- remove axes from the plot
- specify the line type of the X axes
- offset the Y axes
- modify legend

Bar Plot

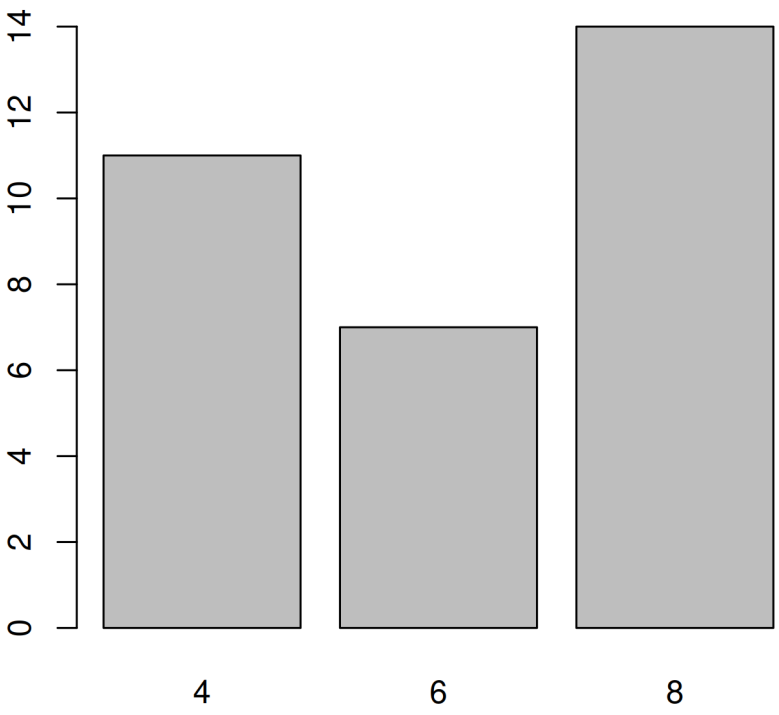
A bar plot represents data in rectangular bars. The length of the bars are proportional to the values they represent. Bar plots can be either horizontal or vertical. The X axis of the plot represents the levels or the categories and the Y axis represents the frequency/count of the variable.

A univariate bar plot represents a single categorical variable. The categories in the variable are represented on the X axis and their frequencies on the Y axis. In the below example, the `cyl` variable from the `mtcars` data set is visualized using a bar plot. The categories or levels are `4`, `6` and `8` which represent the number of cylinders in the automobile and are represented on the X axis. The frequency for each type of cylinder is represented by the Y axis.

In R, bar plots can be created using either the `plot()` or `barplot()` function. The input to both the functions are different. In case of the `plot()` function, we can specify the variable but it must be converted to a factor variable. In case of the `barplot()` function, the input must be the count or frequency of the variable. The `table()` function can be used to generate the counts/frequency for a variable. Let us use both the functions to create the bar plot:

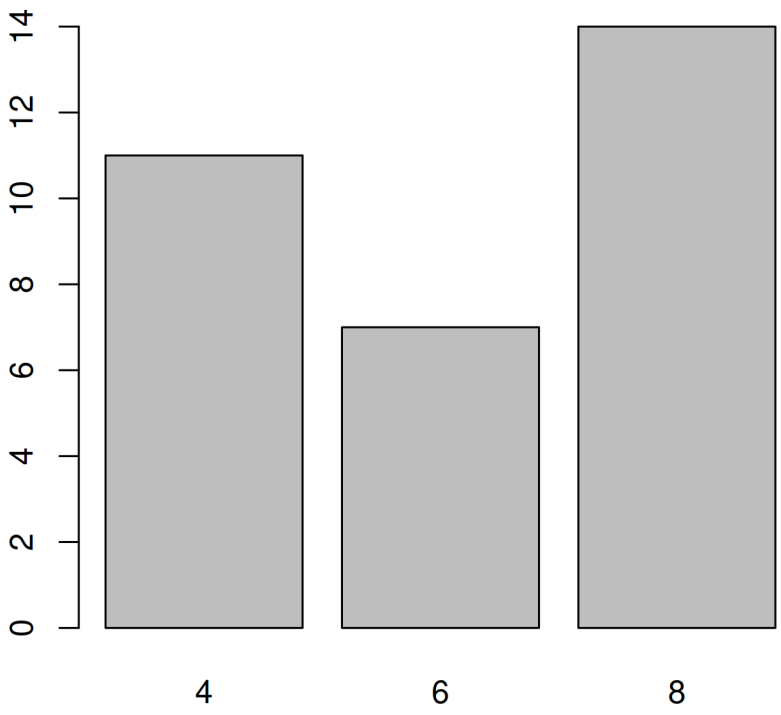
Using plot function

```
plot(as.factor(mtcars$cyl))
```



Using barplot function

```
barplot(table(mtcars$cyl))
```



If you observe carefully, the same plot is generated by both the functions. Before we explore the bar plots further, let us store the data in a new variable instead of using the `table()` function in every example:

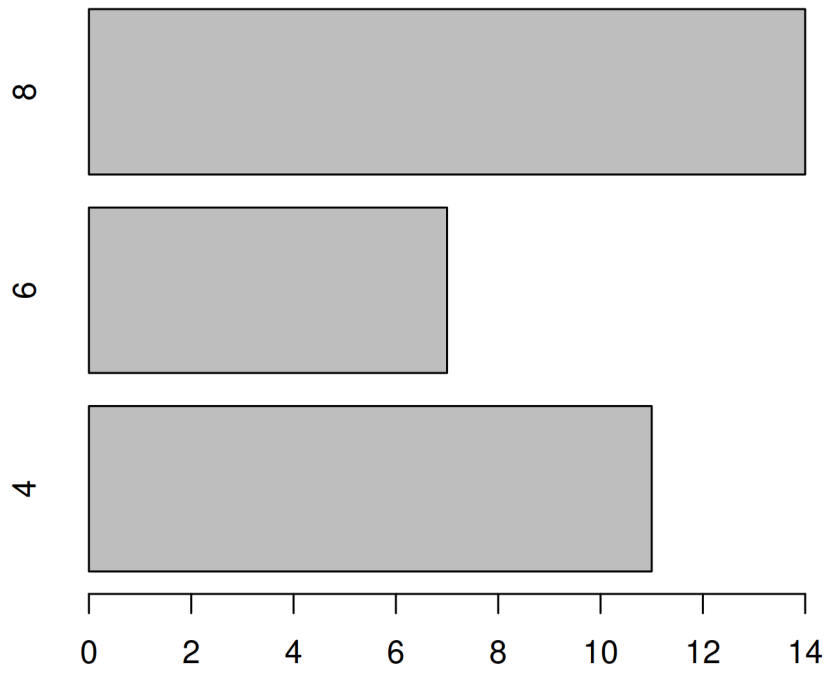
```
cyl_freq <- table(mtcars$cyl)
cyl_freq
```

```
 4  6  8
11  7 14
```

Horizontal or Vertical

Bar plots can be horizontal or veritcal (which is the default). Use the `horiz` argument in the `barplot()` function to build a horizontal bar plot. As you can see, the axis have been flipped. The Y axis represents the categories and the X axis represents their counts/frequencies.

```
barplot(cyl_freq, horiz = TRUE)
```

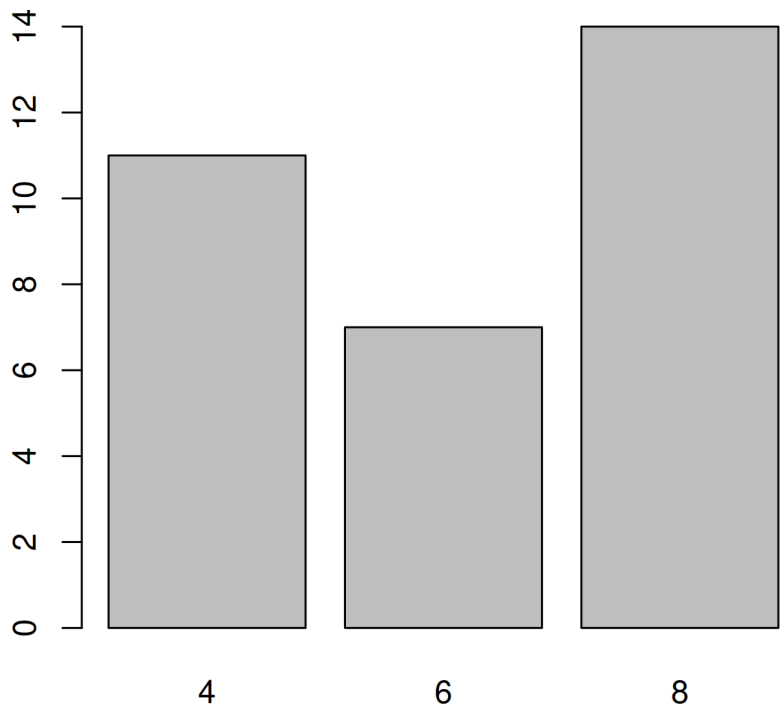


Bar Width

In the bar plot, the width of the bars and the space between them are same. A specific category of the variable can be highlighted by increasing/decreasing the width of the bar representing it. In our example, we will increase the width of the bar that represents automobiles with 8 cylinders. The `width` argument is used to specify the width of the bars. The width must be specified for all the bars in the plot. It must be a vector the length of which must be equal to the number of categories of the variable.

Equal Width

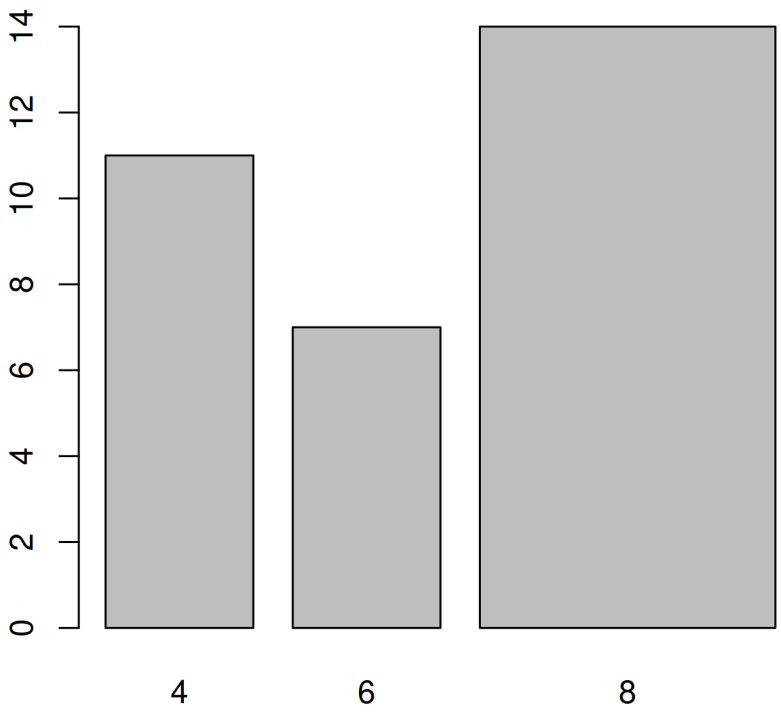
```
barplot(cyl_freq, width = 2)
```



Different Widths

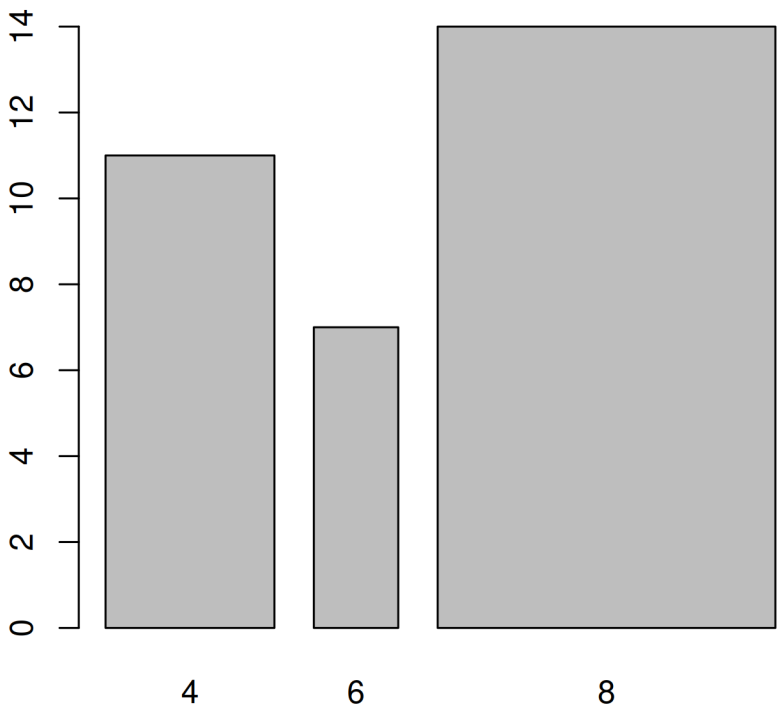
In the below example, the width of the third bar is twice the width of the other two bars

```
barplot(cyl_freq, width = c(1, 1, 2))
```



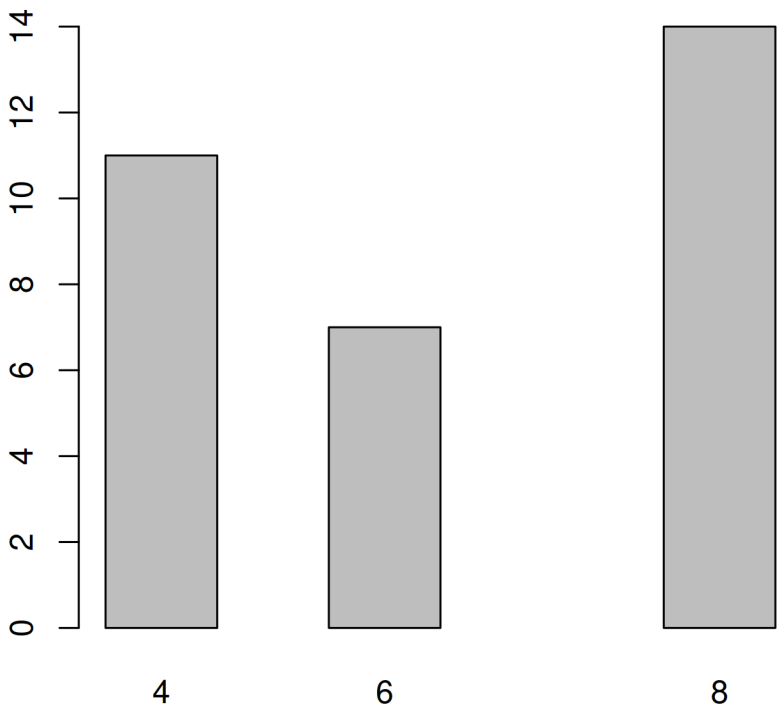
In the below example, the width of the second bar is half the width of the other first bar and the third bar is twice the width of the first bar.

```
barplot(cyl_freq, width = c(1, 0.5, 2))
```



The space between the bars can be specified in a similar manner but using the `space` argument in the `barplot()` function: In the below example, the space between the third bar and the second bar is twice the space between first and second bar.

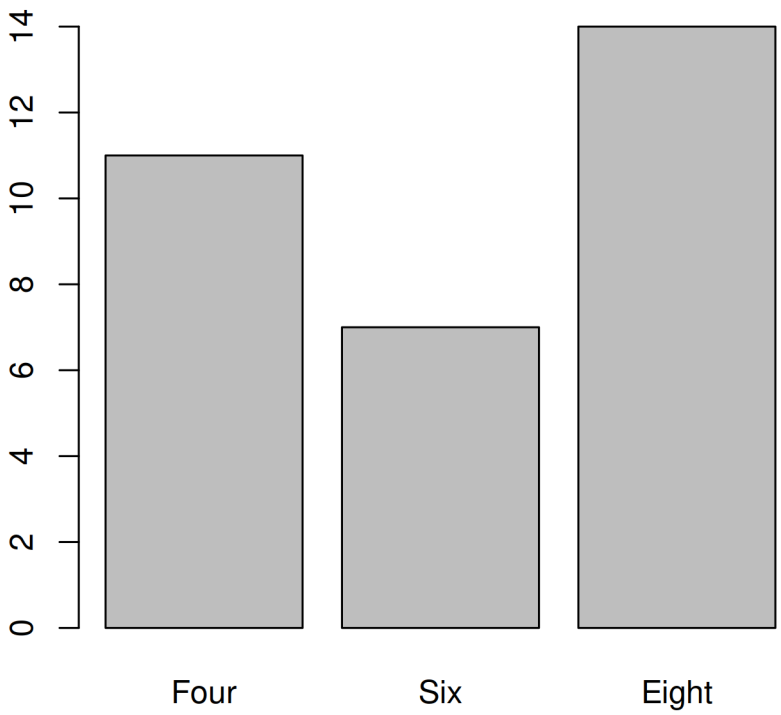
```
barplot(cyl_freq, space = c(1, 1, 2))
```



Labels

It is important to add appropriate labels to the bars in order to communicate properly. In our example, the bars represent automobiles with different number of cylinders. The labels likewise indicate the number of cylinders represented by the bars. In order to demonstrate how to add labels, we will change the labels from numbers to their corresponding words. The `names.arg` argument is used to add labels to the bars in a plot. Below is our example:

```
barplot(cyl_freq, names.arg = c('Four', 'Six', 'Eight'))
```



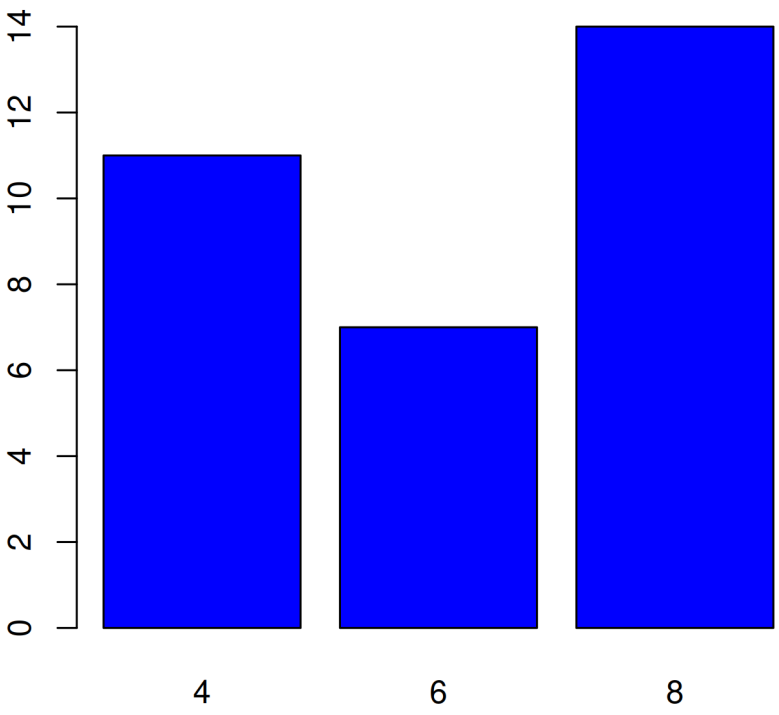
It is important to specify labels for all the bars in the plot else R will return an error.

Color

Let us add some color to the plots. In a bar plot, we can specify different colors for the bars and their borders. Use the `col` argument to add color to the bars.

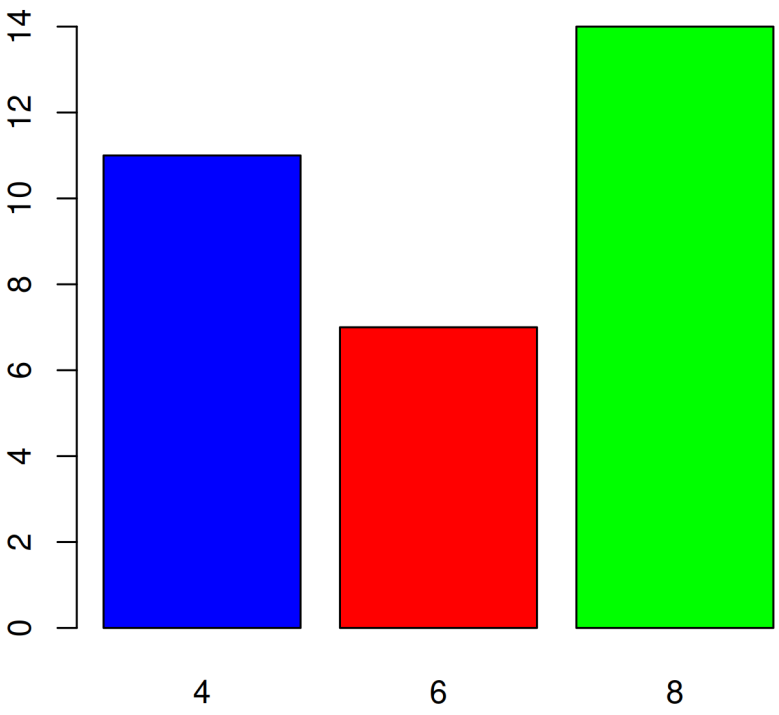
Same color for all bars

```
barplot(cyl_freq, col = 'blue')
```



Differnt color for the bars

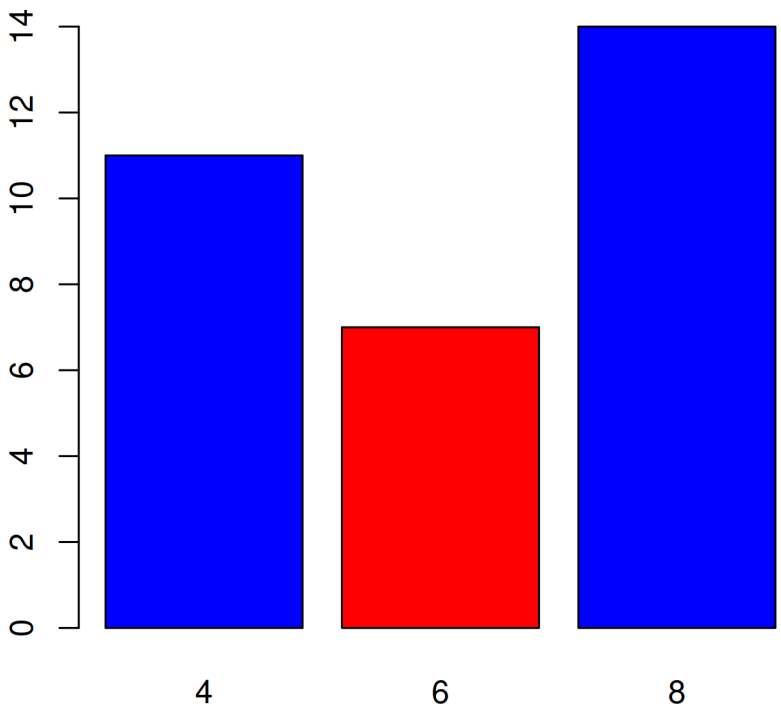
```
barplot(cyl_freq, col = c('blue', 'red', 'green'))
```



What happens if we do not specify color for all the bars? The colors you specify are recycled.

Recycling colors

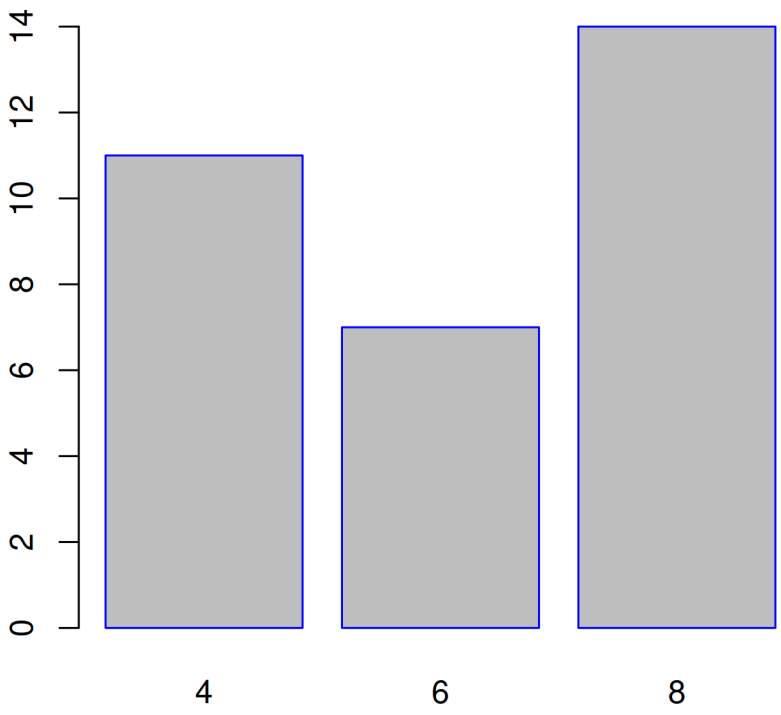
```
barplot(cyl_freq, col = c('blue', 'red'))
```



The `border` argument specifies the color of the border of the bars. The rules that apply to `col` argument apply here also. Below are the examples:

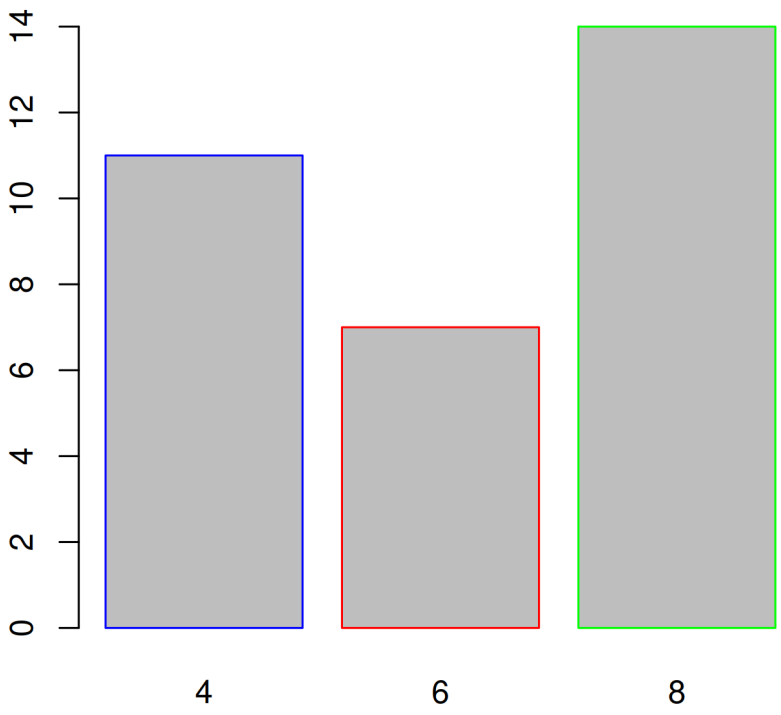
Same color for all bars

```
barplot(cyl_freq, border = 'blue')
```



Differnt color for the bars

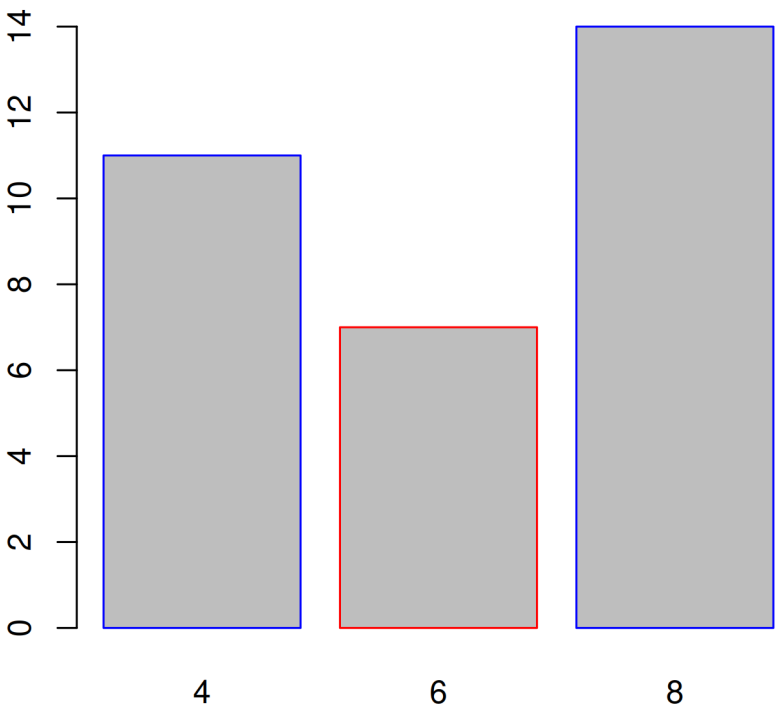
```
barplot(cyl_freq, border = c('blue', 'red', 'green'))
```



What happens if we do not specify color for all the bars? The colors you specify are recycled.

Recycling colors

```
barplot(cyl_freq, border = c('blue', 'red'))
```



Axes

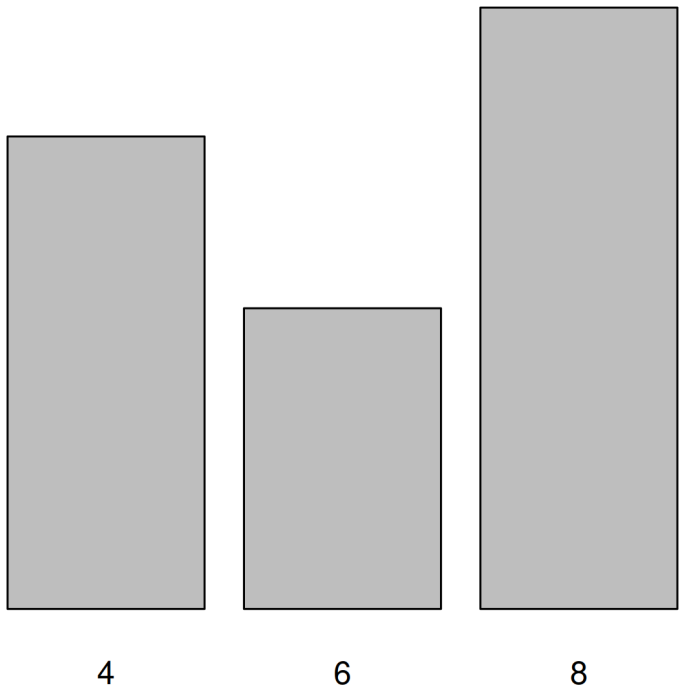
In this section, we will learn to

- remove axes from the plot
- specify the line type of the X axes
- offset the Y axes

Remove axes

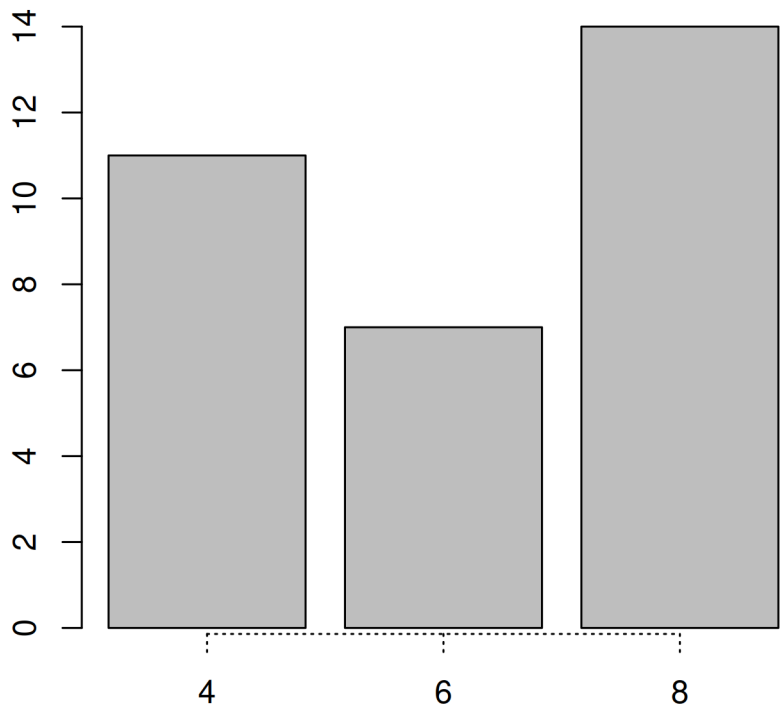
The `axes` argument can be used to retain/remove the axes from the plot. It takes logical values as input and the default is `TRUE`. Set it to `FALSE` to remove the axes from the plot:

```
barplot(cyl_freq, axes = FALSE)
```



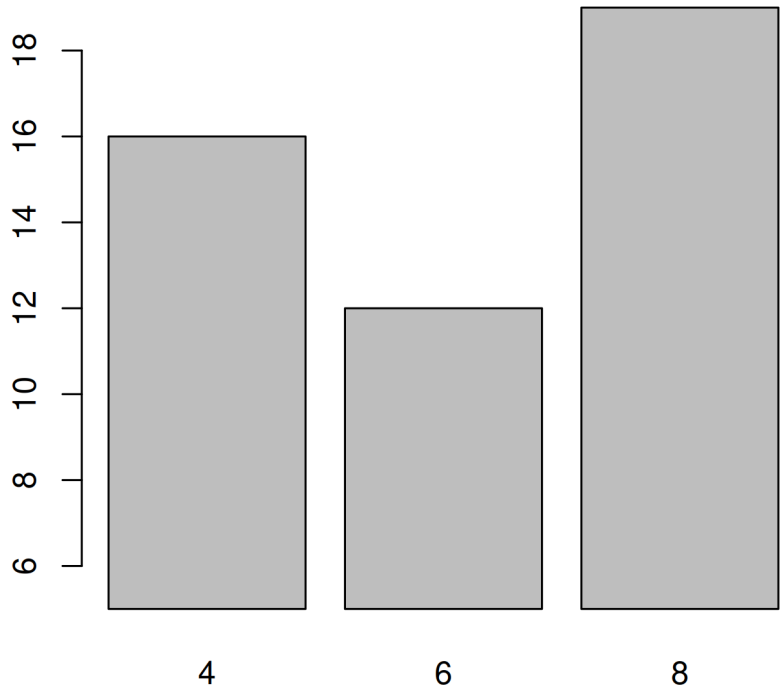
If we decide to retain the axes, the line type of the X axes can be specified using the `axis.lty` argument. It does not modify the line type of the Y axes and it will not work if the `axes` argument is set to `FALSE`.

```
barplot(cyl_freq, axis.lty = 3)
```



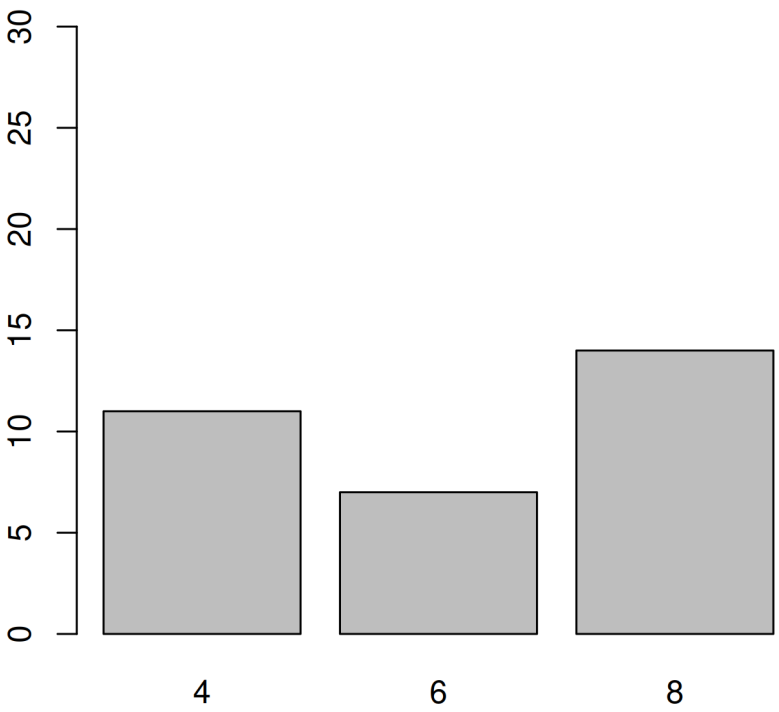
Though we cannot modify the line type of the Y axes, we can offset it using the `offset` argument. In the below example, we will offset the Y axes and you can observe that the minimum value of the Y axes is now 5 instead of 0.

```
barplot(cyl_freq, offset = 5)
```



You can similarly modify the range of the Y axes using the `ylim` argument. Although in case of bar plots, modifying the range of the plot may not be very useful.

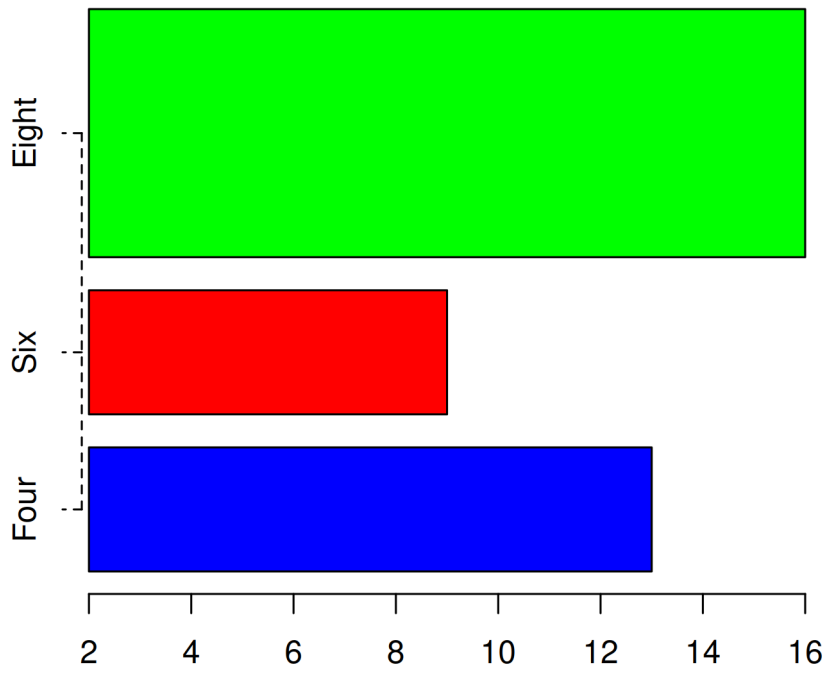
```
barplot(cyl_freq, ylim = c(0, 30))
```



Putting it all together

Let us quickly revise what we have learnt so far and build a bar plot for visualizing the `cyl` variable in the `mtcars` data set:

```
barplot(cyl_freq, col = c('blue', 'red', 'green'),
        horiz = TRUE, width = c(1, 1, 2),
        names.arg = c('Four', 'Six', 'Eight'),
        axis.lty = 2, offset = 2)
```



Title & Axis Labels

Well the plot looks good but for someone who does not know the underlying data, it will diffficult to understand what is being communicated. Let us add a title and labels for the axes.

```
barplot(cyl_freq, col = c('blue', 'red', 'green'),
        horiz = TRUE, width = c(1, 1, 2),
        names.arg = c('Four', 'Six', 'Eight'),
        axis.lty = 2, offset = 2)
title(main = 'Distribution of Cylinders',
      xlab = 'Frequency', ylab = 'Number of Cylinders')
```

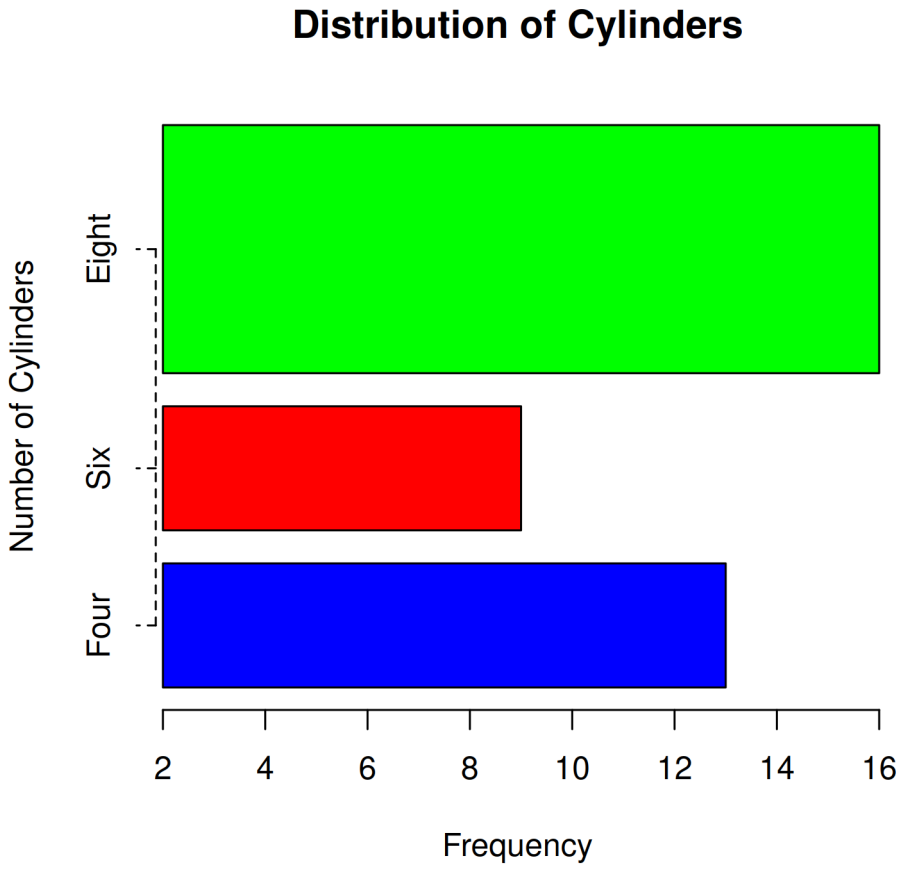


Figure 21: A complete bar chart with labels, axis limits and a colour palette

Bivariate Bar Plots

A bivariate bar plot represents the cross table or two way table of categorical variables. They are of two types:

- Stacked Bar Plots
- Grouped Bar Plots

Before we look at bivariate bar plots, let us create a two way table of `cyl` (number of cylinders) and `gear` (number of gears) using the `table()` function:

```
table(mtcars$gear)
```

```
 3  4  5
15 12  5
```

```
cyl_gear <- table(mtcars$cyl, mtcars$gear)
cyl_gear
```

```
   3  4  5
4  1  8  2
6  2  4  1
8 12  0  2
```

The number of gears is represented by the columns in the table and the number of cylinders is represented by the rows.

Stacked Bar Plot

```
barplot(cyl_gear)
```

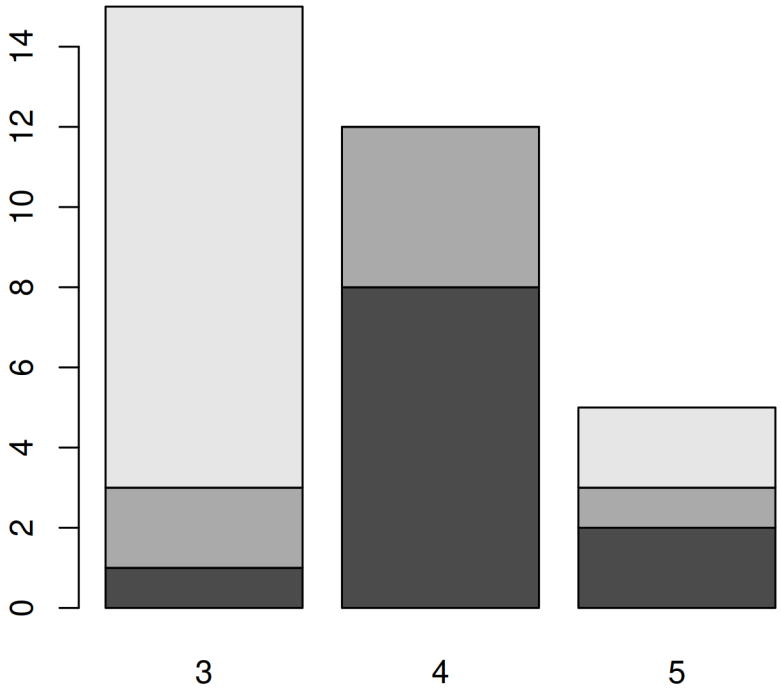


Figure 22: A two-way table of counts by gear and cylinder

The bars in the plot represent the distribution of `cyl` for each level of category of the `gear` variable. The first bar represents the distribution of cylinders for automobiles with 3 gears. From the two way table we saw earlier, the columns are the bars. The rows are represented by different sections of the bar. Let us add some colors to the plot as the default colors of the plot are not very intuitive. It will also allow us to clearly examine the distribution of `cyl` for the different levels of `gear`.

```
barplot(cyl_gear, col = c('blue', 'red', 'green'))
```

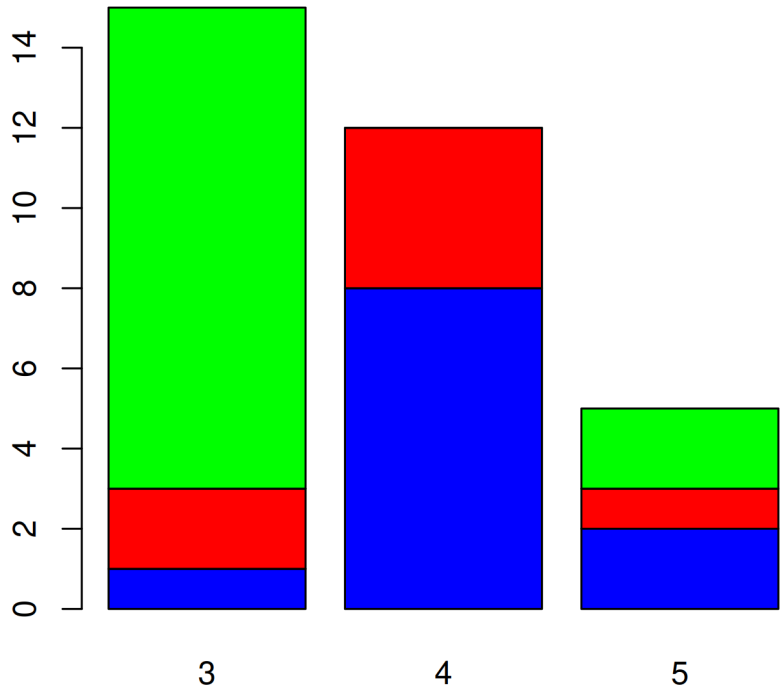


Figure 23: Stacked bars: each bar is one gear group, segmented by cylinder count

If you carefully observe the table and the plot:

- the blue sections of the bars represent the number of automobiles with 4 cylinders (in each gear group)
- the red sections represent the number of automobiles with 6 cylinders (in each gear group)
- the green sections represent the number of automobiles with 8 cylinders (in each gear group)

We need to convey the above information in some way and will do that using the `legend.text` argument. It takes a character vector as input and the default value is `NULL`, in which case no legend text is added. In the next example, we add a legend as well as other relevant information such as title and axis labels.

```
barplot(cyl_gear, col = c('blue', 'red', 'green'),
        main = 'Gears vs Cylinders', legend.text = c('4 cyl', '6 cyl', '8 cyl'),
        xlab = 'Number of Gears', ylab = 'Frequency')
```

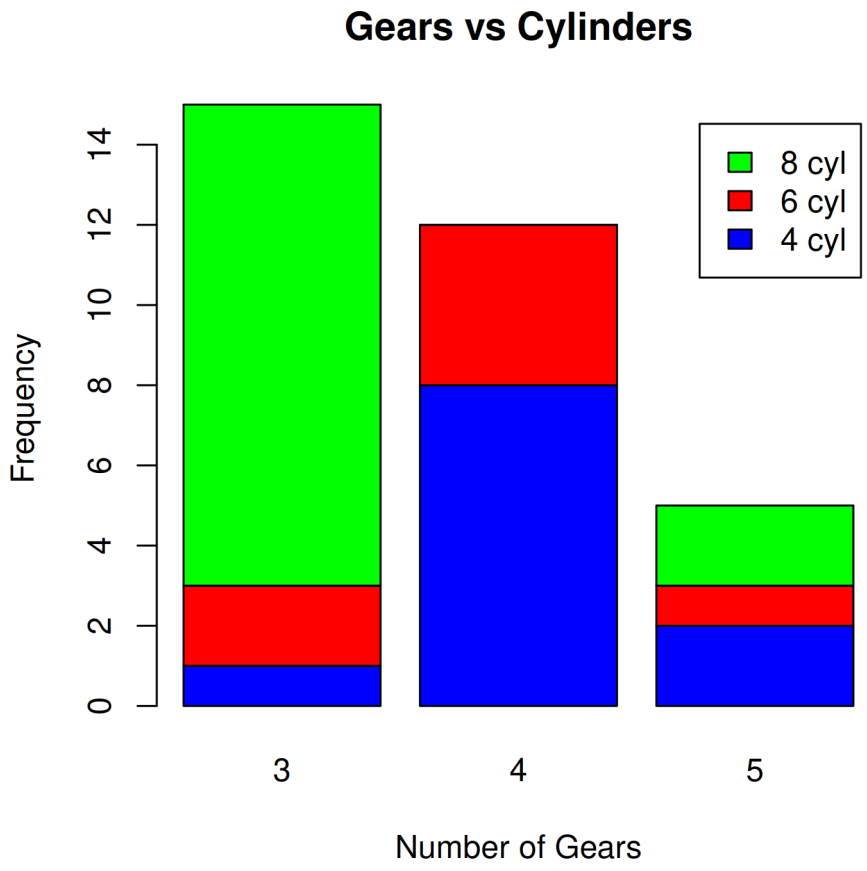
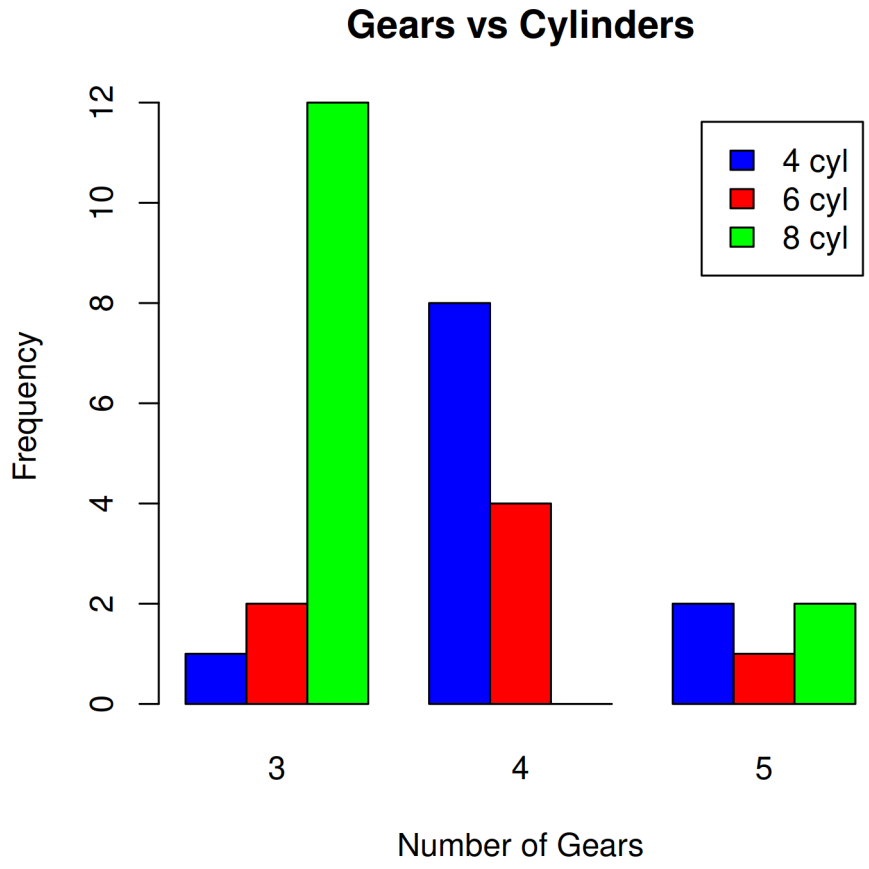


Figure 24: The same counts drawn beside each other rather than stacked

Grouped Bar Plot

A grouped bar plot represents the same data as the stacked bar plot but instead of being stacked, the bars are now grouped and placed besides each other.

```
barplot(cyl_gear, col = c('blue', 'red', 'green'),
        beside = TRUE, legend.text = c('4 cyl', '6 cyl', '8 cyl'),
        main = 'Gears vs Cylinders',
        xlab = 'Number of Gears', ylab = 'Frequency')
```



The `beside` argument in `barplot()` function is set to `TRUE` to build grouped bar plots. It takes logical values as inputs and the default values is FALSE. As you can observe from the plot, the bars are placed besides each other instead of being stacked.

Box Plots

Introduction

In this chapter, we will learn to

- create univariate/multivariate box plots
- interpret box plots
- create horizontal box plots
- detect outliers
- modify box color
- use formula to compare distributions of different variables
- use notches to compare medians

Box Plot

The box plot is a standardized way of displaying the distribution of data based on the five number summary: minimum, first quartile, median, third quartile, and maximum. Box plots are useful for detecting outliers and for comparing distributions. It shows the shape, central tendency and variability of the data.

Structure

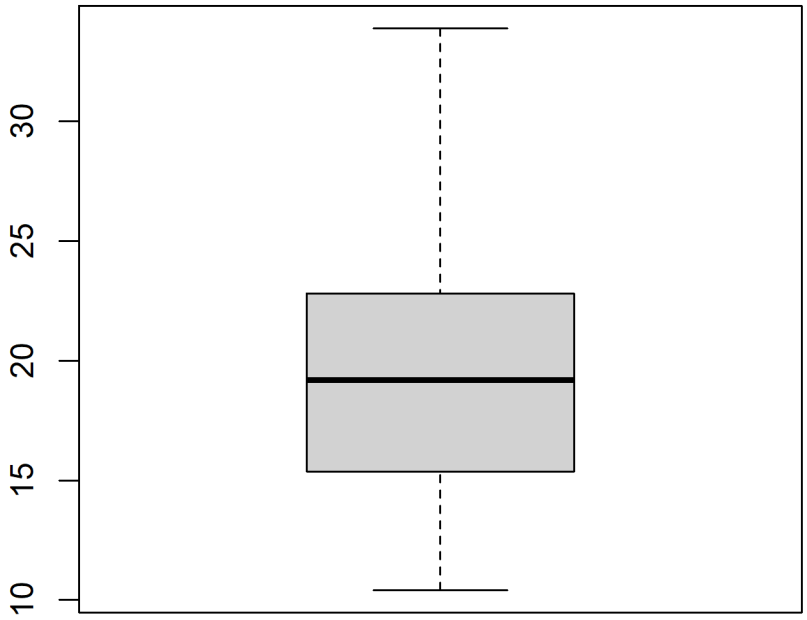
A boxplot splits the data set into quartiles. The body of the boxplot consists of a "box" (hence, the name), which goes from the first quartile (Q1) to the third quartile (Q3). Within the box, a vertical line is drawn at the Q2, the median of the data set. Two horizontal lines, called whiskers, extend from the front and back of the box. The front whisker goes from Q1 to the smallest non-outlier in the data set, and the back whisker goes from Q3 to the largest non-outlier. If the data set includes one or more outliers, they are plotted separately as points on the chart.

Univariate Box Plot

Basic Plot

Let us begin by creating a basic box plot. We will use the `boxplot()` function and specify the data.

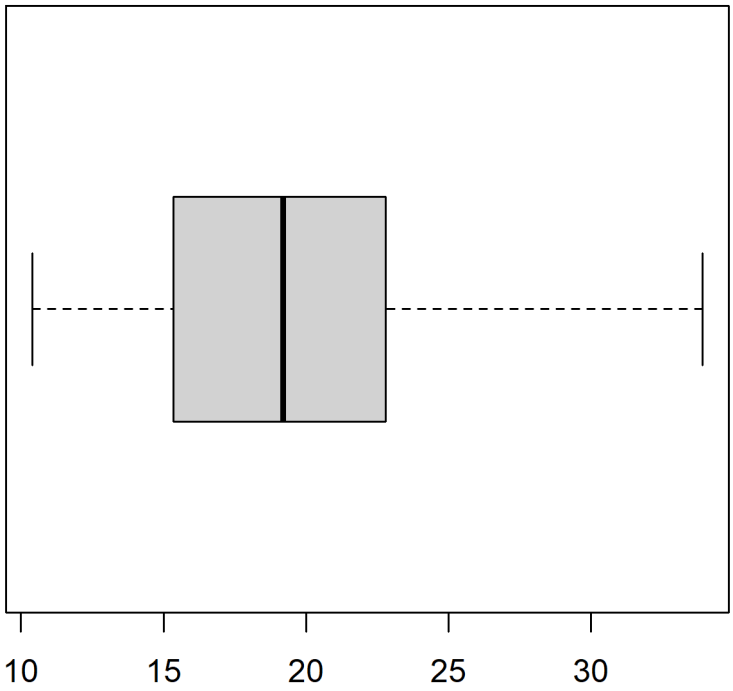
```
boxplot(mtcars$mpg)
```



Horizontal Box Plot

Use the `horizontal` argument in the `boxplot()` function to create a horizontal box plot.

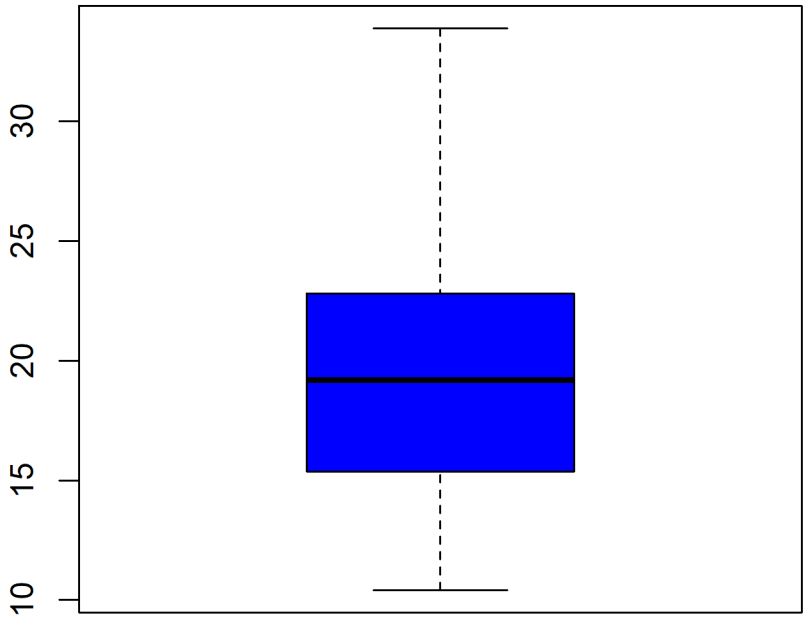
```
boxplot(mtcars$mpg, horizontal = TRUE)
```



Color

Let us add some color to the boxplot. Use the `col` argument to specify a color for the plot.

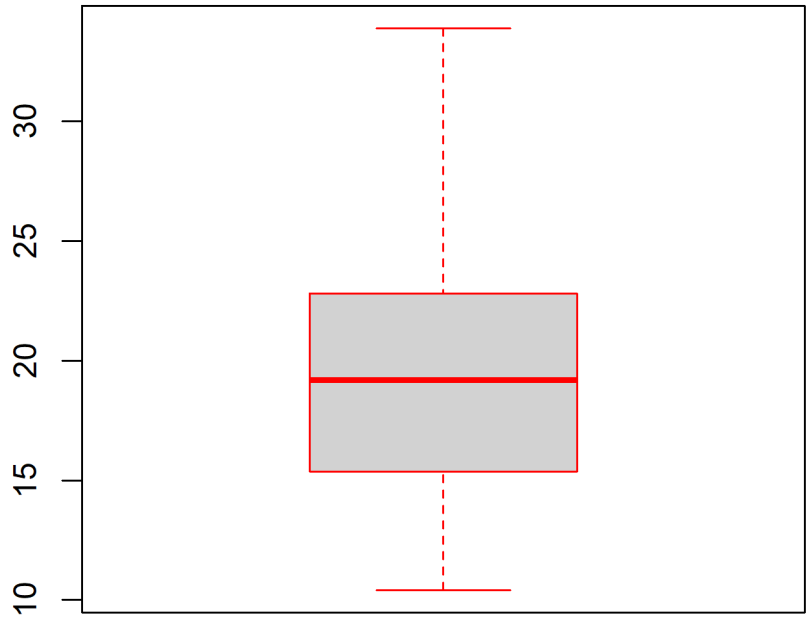
```
boxplot(mtcars$mpg, col = 'blue')
```



Border Color

We can specify a separate color for the border of the box in the boxplot. To modify the border color, use the `border` argument.

```
boxplot(mtcars$mpg, border = 'red')
```

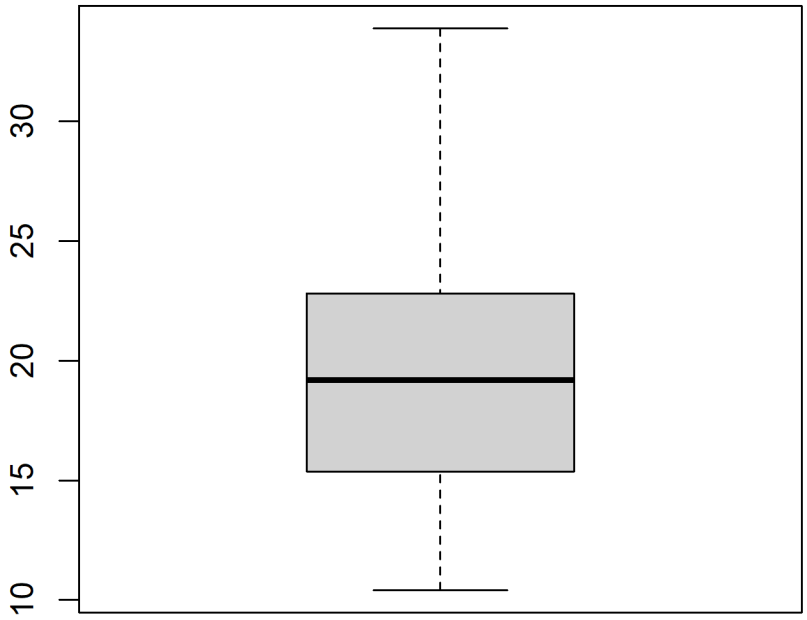


Range

The `range` argument determines how far the plot whiskers extend out from the box. If range is positive, the whiskers extend to the most extreme data point which is no more than range times the interquartile range from the box. A value of zero causes the whiskers to extend to the data extremes.

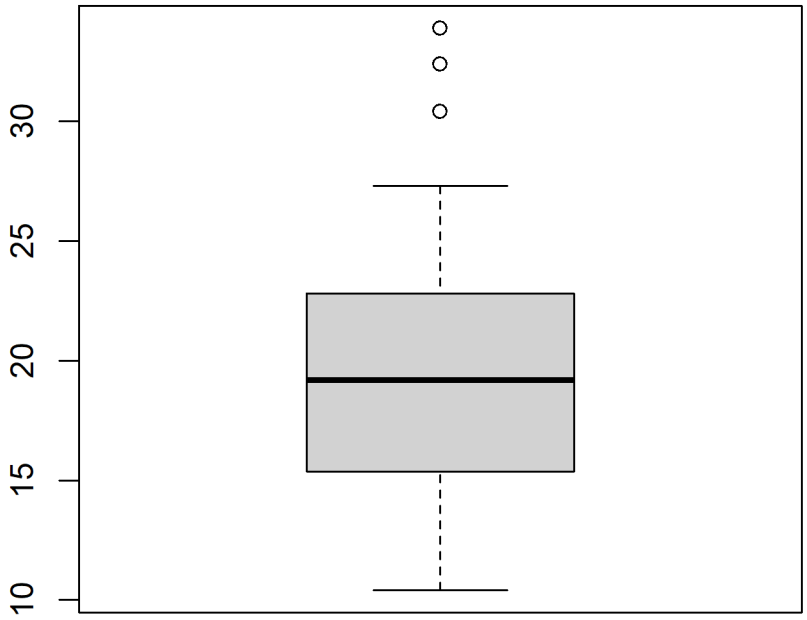
Let us set the value of `range` to 0 and observe the plot.

```
boxplot(mtcars$mpg, range = 0)
```

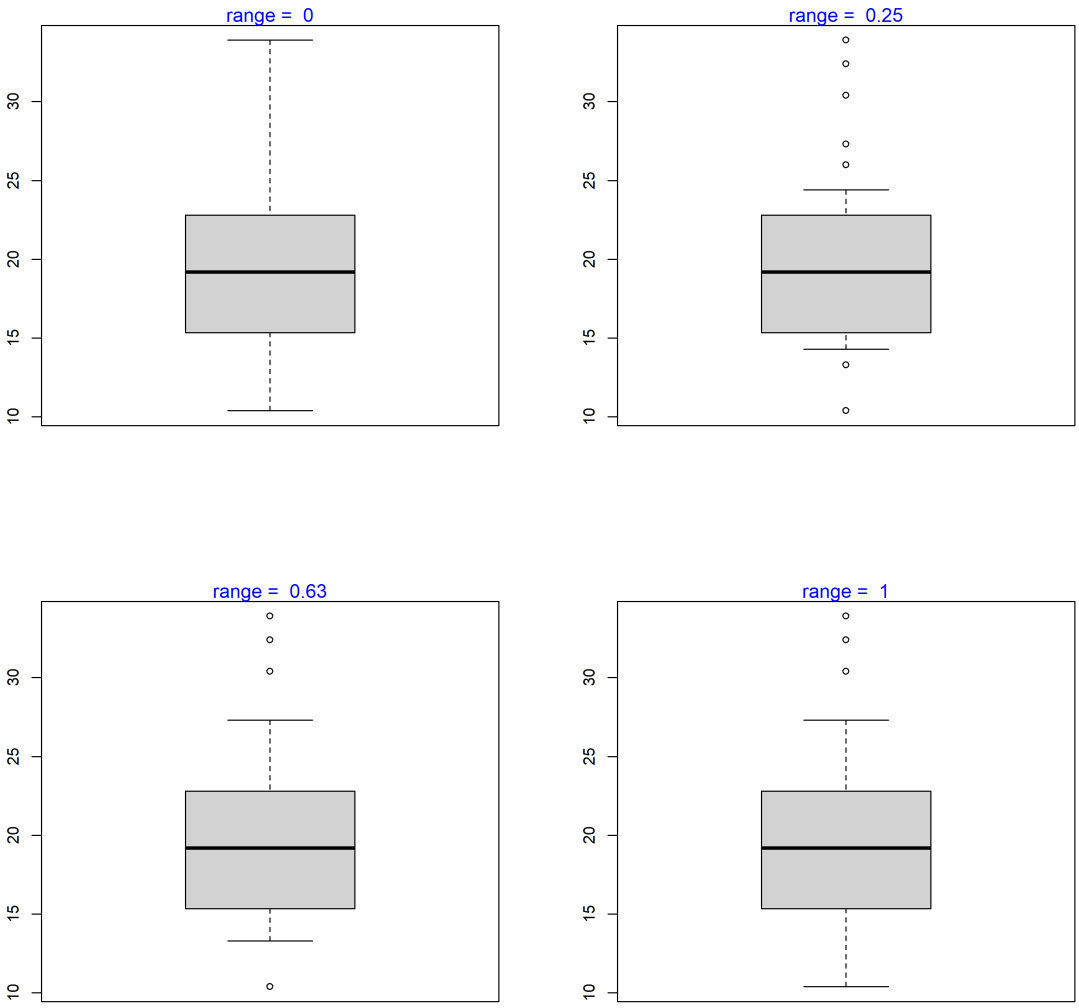


In the below plot, we set the value of `range` to 1.

```
boxplot(mtcars$mpg, range = 1)
```



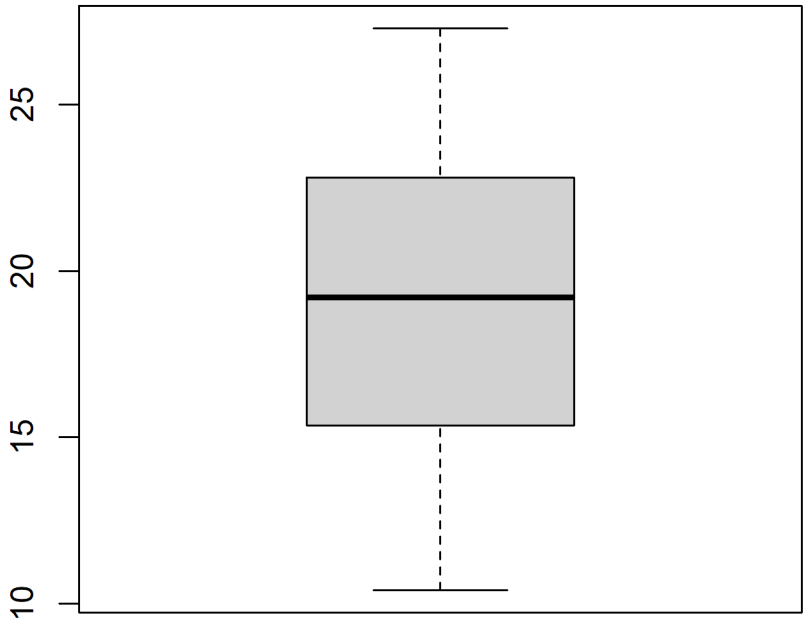
Let us observe how the plot appears as we change the value of `range` from 0 to 1.



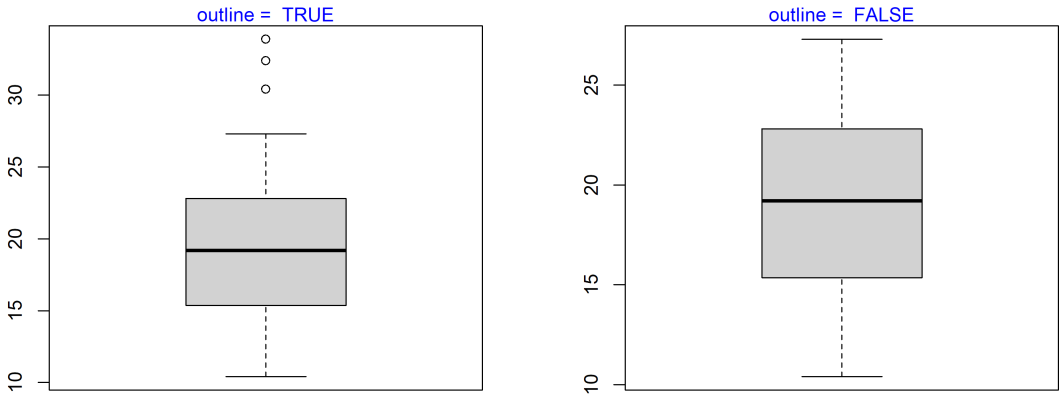
Outline

The outliers in the plot are not drawn if the `outline` argument is set to `FALSE`. The default value is `TRUE`.

```
boxplot(mtcars$mpg, range = 1, outline = FALSE)
```



The below plot displays how the plot changes with the values set for `outline`:



Varwidth

If the `varwidth` argument is set to `TRUE`, the boxes are drawn with widths proportional to the square-roots of the number of observations in the groups.

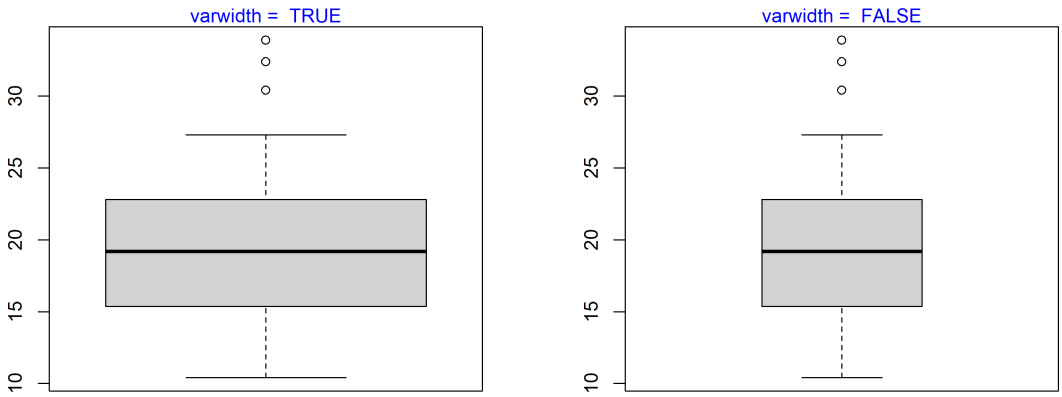
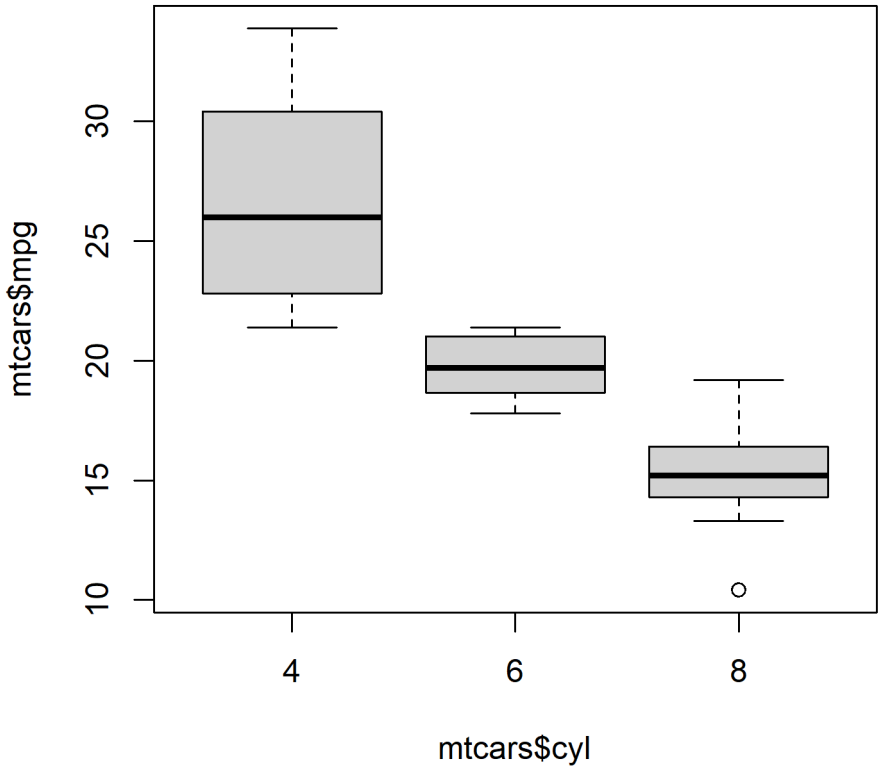


Figure 25: The five-number summary drawn on the same scale as the box plot

Bivariate/Multivariate Box Plot

As we said in the introduction, box plots can be used to compare distributions of several variables. Let us use the `mtcars` data set and compare the distribution of Miles Per Gallon (`mpg`) for automobiles with different number of cylinders (`cyl`). We will do this by specifying a formula as shown in the below example.

```
boxplot(mtcars$mpg ~ mtcars$cyl)
```



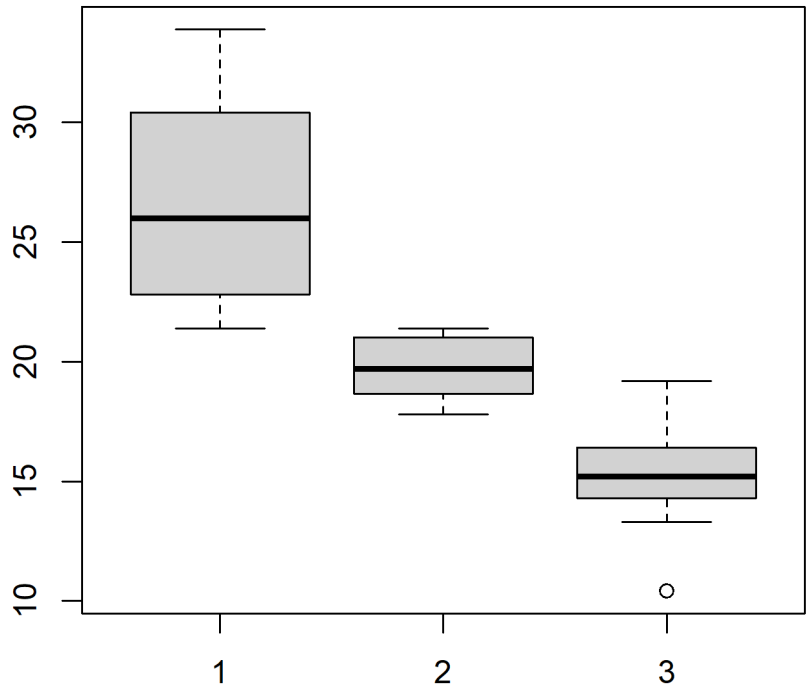
We use the formula when we are comparing the distribution of a continuous variable across different levels of a categorical variable. If we want to compare the distributions without using a categorical variable, we need to specify the variable separately in the `boxplot()` function. Below is an illustration of this method. We will split the `mpg` data using the `split()` function and plot them separately. The `split()` function splits a continuous variable based on the levels of a categorical variable.

```
mpg_split <- split(mtcars$mpg, mtcars$cyl)

mpg_split
## $`4`
##  [1] 22.8 24.4 22.8 32.4 30.4 33.9 21.5 27.3 26.0 30.4 21.4
##
## $`6`
## [1] 21.0 21.0 21.4 18.1 19.2 17.8 19.7
##
## $`8`
##  [1] 18.7 14.3 16.4 17.3 15.2 10.4 10.4 14.7 15.5 15.2 13.3 19.2 15.8 15.0
```

```r
mpg_4 <- mpg_split$`4`
mpg_6 <- mpg_split$`6`
mpg_8 <- mpg_split$`8`

boxplot(mpg_4, mpg_6, mpg_8)
```



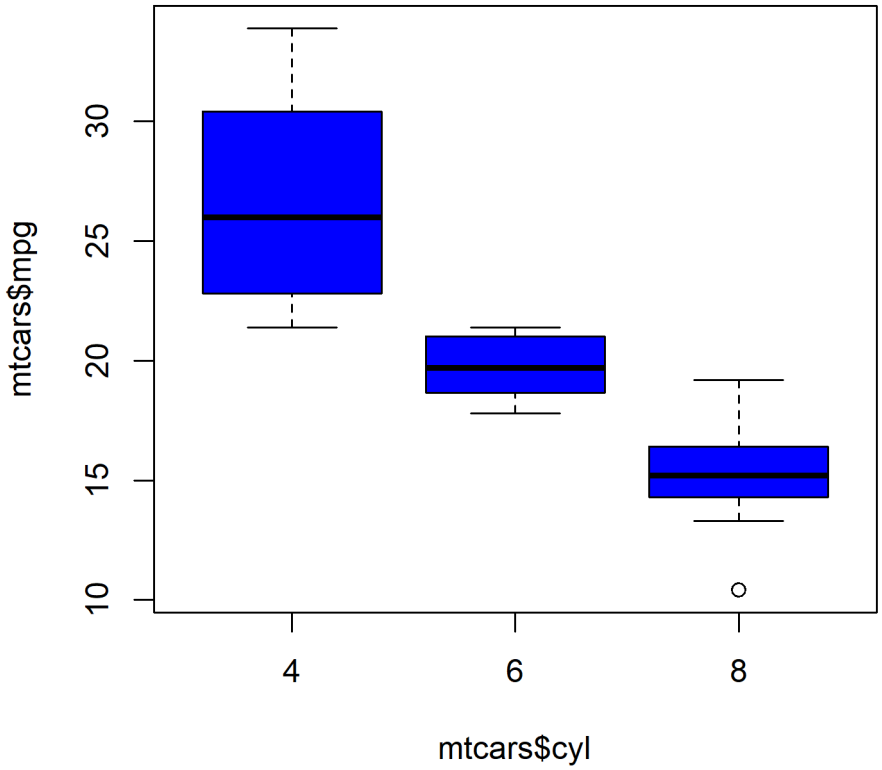
The same plot can be created in two ways. If you are comparing the distribution of a continuous variable for the different levels of a categorical variable, use the formula. If you are comparing distribution of independent variables, specify all the variables in the `boxplot()` function.

Color

Let us add some color to the plot. We can specify as many colors as the boxes or we can specify the same color for all of them. Below are two examples where we specify the same color in the first one and different colors in the second one.

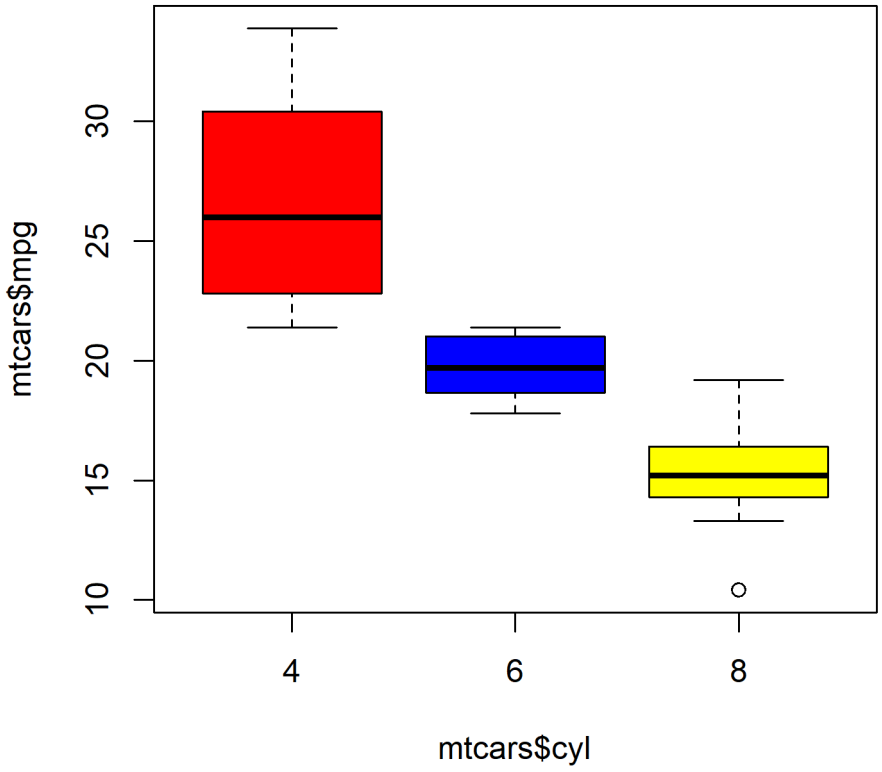
Single Color

```r
boxplot(mtcars$mpg ~ mtcars$cyl, col = 'blue')
```



Different Colors

```
boxplot(mtcars$mpg ~ mtcars$cyl,
        col = c('red', 'blue', 'yellow'))
```

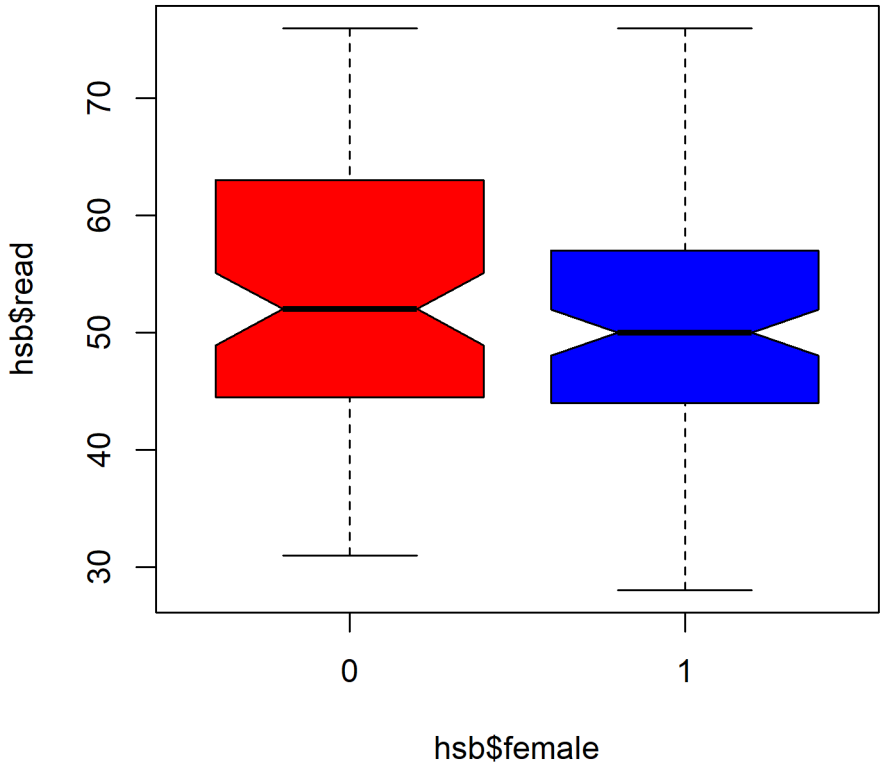


Compare Medians

If we want to test whether the medians of the different groups differ, we can use the `notch` argument and set it to `TRUE`. A notch is drawn in each side of the boxes and if the notches of the plots do not overlap, it is strong evidence that the medians differ.

We will use a different data set for this example. We use the `hsb2` data (High School and Beyond survey, 200 observations; vendored as `data/hsb2.csv`, originally from the UCLA Institute for Digital Research and Education) and compare the distribution of reading score (`read`) for males and females (`female`).

```
hsb <- read.csv('data/hsb2.csv')
boxplot(hsb$read ~ hsb$female, notch = TRUE,
        col = c('red', 'blue'))
```

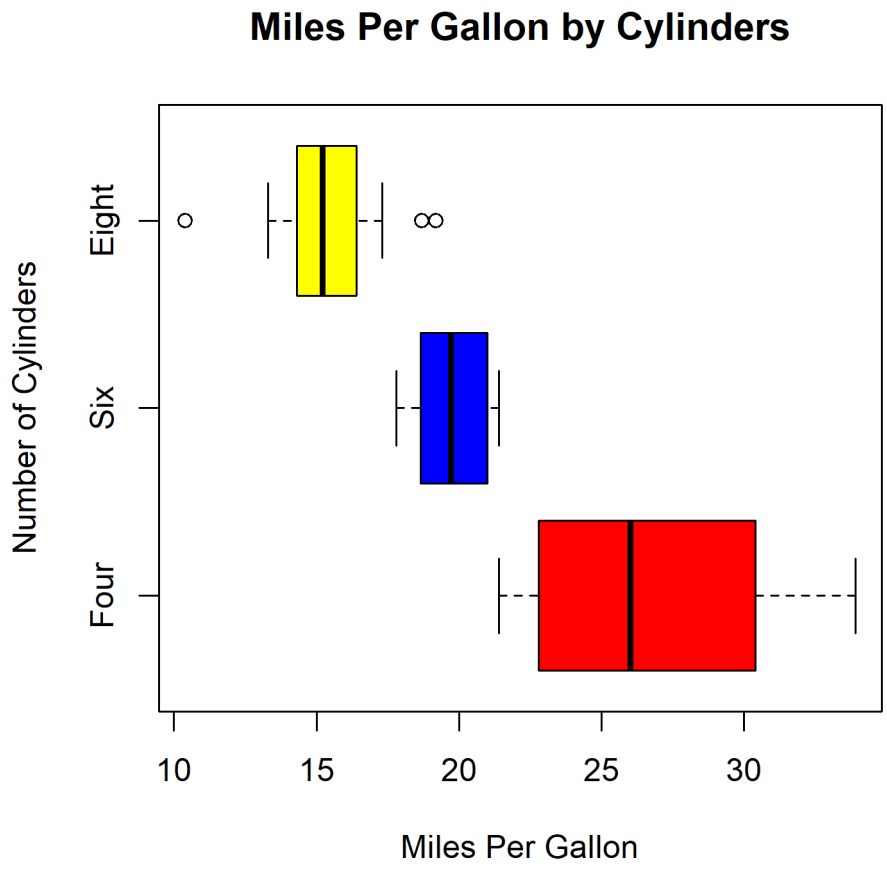


Since the notches overlap, there is no strong evidence that the medians differ.

Putting it all together

Let us conclude by adding a title and axis labels to the box plot.

```
boxplot(mtcars$mpg ~ mtcars$cyl, range = 1, outline = TRUE,
        horizontal = TRUE, col = c('red', 'blue', 'yellow'),
        main = 'Miles Per Gallon by Cylinders',
        ylab = 'Number of Cylinders', xlab = 'Miles Per Gallon',
        names = c('Four', 'Six', 'Eight'))
```



Histograms

Introduction

In this chapter, we will learn to:

- create a bare bones histogram
- specify the number of bins/intervals
- represent frequency density on the Y axis
- add colors to the bars and the border
- add labels to the bars

A histogram is a plot that can be used to examine the shape and spread of continuous data. It looks very similar to a bar graph and can be used to detect outliers and skewness in data. The histogram graphically shows the following:

- center (location) of the data
- spread (dispersion) of the data
- skewness

- outliers
- presence of multiple modes

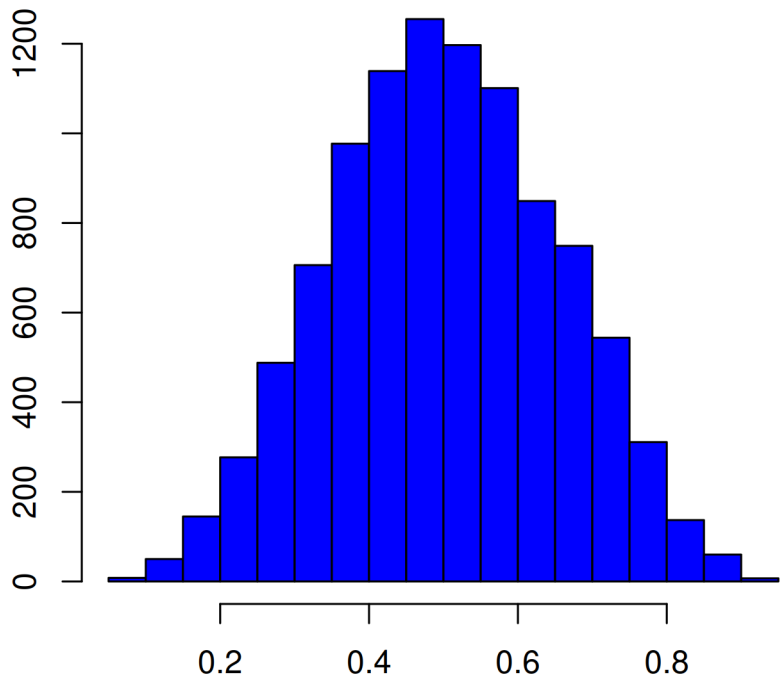
To construct a histogram, the data is split into intervals called bins. The intervals may or may not be equal sized. For each bin, the number of data points that fall into it are counted (frequency). The Y axis of the histogram represents the frequency and the X axis represents the variable.

Distributions

Before we learn how to create histograms, let us see how normal and skewed distributions look when represented by a histogram.

Normal Distribution

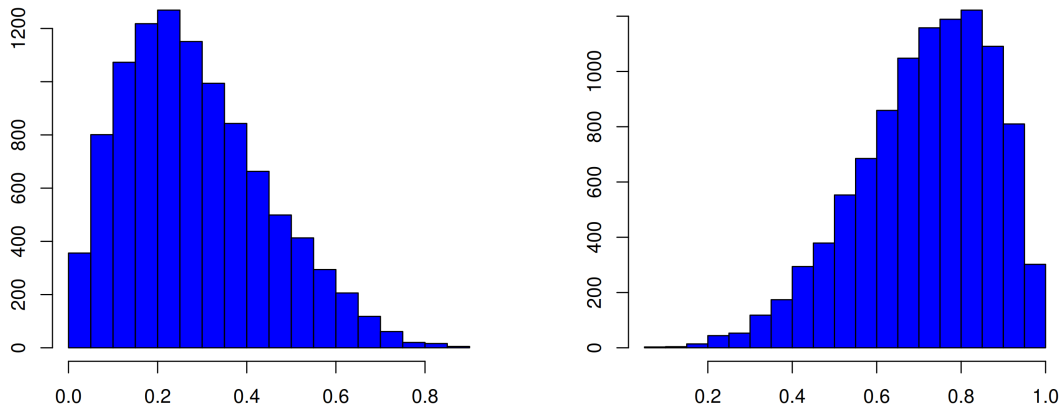
```
set.seed(1)
hist(rbeta(10000, 5, 5), ann = FALSE, col = 'blue')
```



Skewed Distributions

```
set.seed(1)
init <- par(no.readonly = TRUE)
par(mfrow = c(1, 2))
```

```r
hist(rbeta(10000, 2, 5), ann = FALSE, col = 'blue')
hist(rbeta(10000, 5, 2), ann = FALSE, col = 'blue')
```

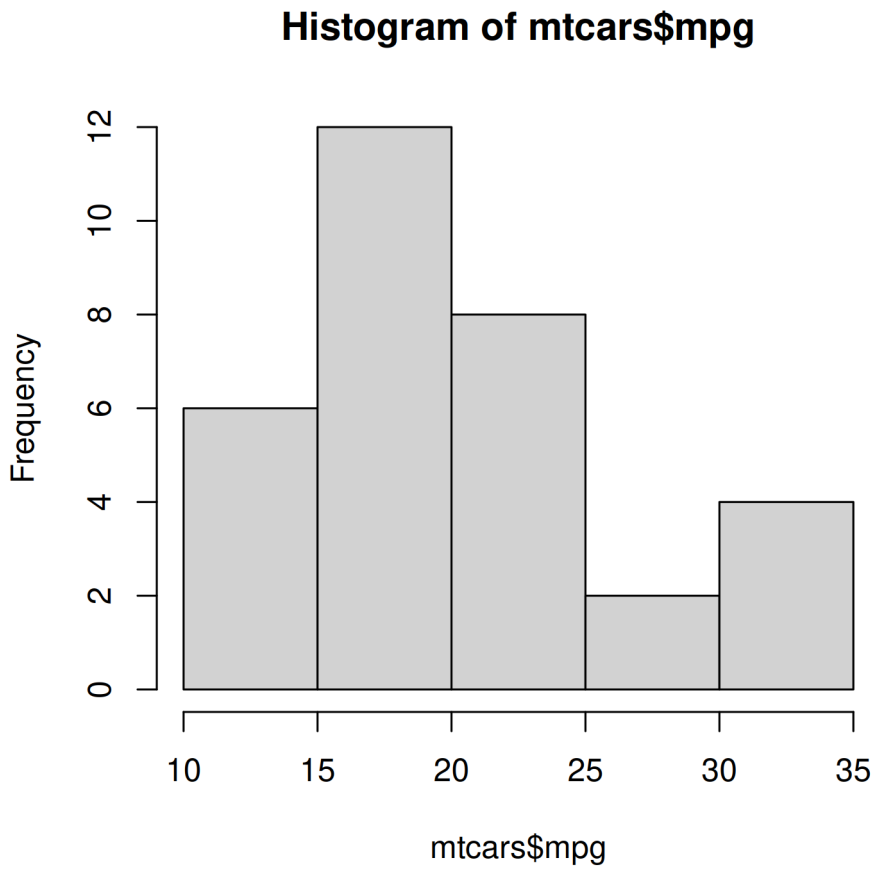



```r
par(init)
```

Basics

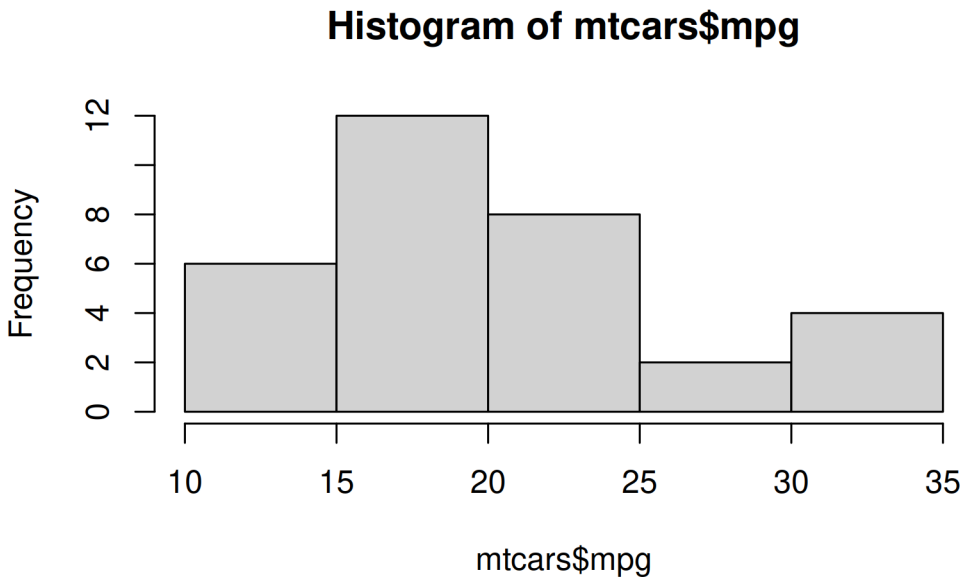
Histograms are created using the `hist()` function in R. The minimum input required to create a bare bones histogram is a continuous variable. Below is an example:

```r
hist(mtcars$mpg)
```



The `hist()` functions returns details of the histogram which can be accessed by assigning the histogram to a variable. Let us assign the above histogram to a variable `h` and use the `$` symbol to access the details stored in the variable.

```
# store the results of hist function
h <- hist(mtcars$mpg)
```




```
# display number of breaks
h$breaks
## [1] 10 15 20 25 30 35

# frequency of the intervals
h$counts
## [1]  6 12  8  2  4

# frequency density
h$density
## [1] 0.0375 0.0750 0.0500 0.0125 0.0250

# mid points of the intervals
h$mids
## [1] 12.5 17.5 22.5 27.5 32.5

# varible name
h$xname
## [1] "mtcars$mpg"

# whether intervals are of equal size
h$equidist
## [1] TRUE
```

Bins

The `hist()` function creates equidistant intervals by default. We can specify the number of bins using the `breaks` argument.

```r
hist(mtcars$mpg, breaks = 10)
```

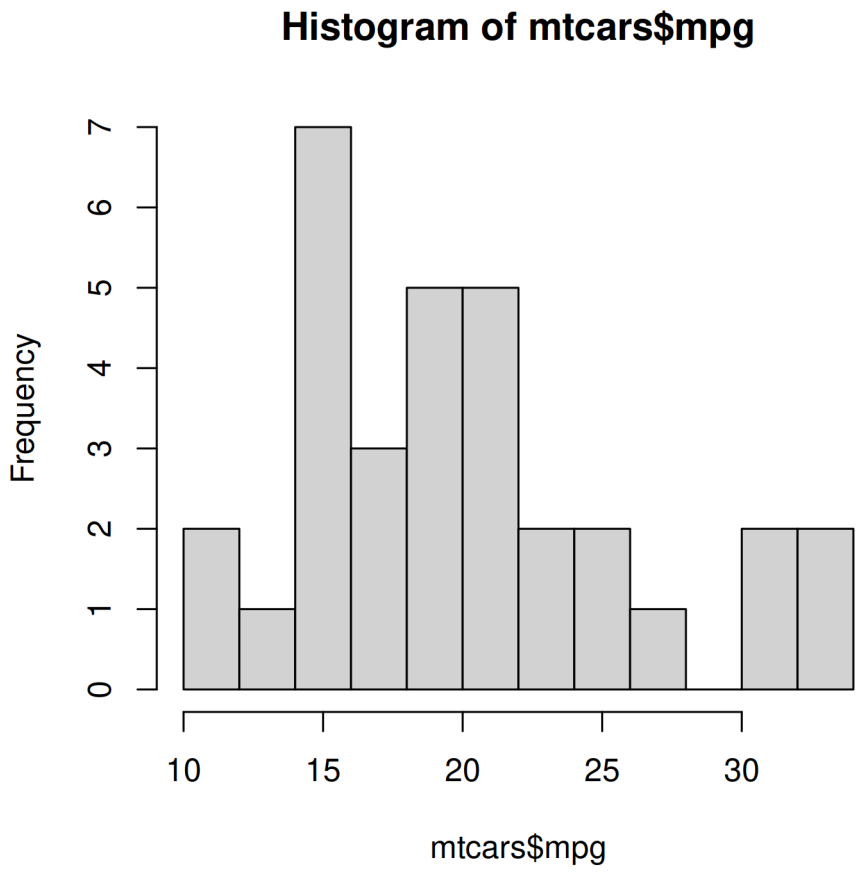


Figure 26: The same data with an explicit number of breaks

The below plot displays histograms with different number of bins:

```r
init <- par(no.readonly = TRUE)
par(mfrow = c(2, 2))
values <- c(5, 10, 15, 20)
for (i in values) {
    hist(mtcars$mpg, breaks = i)
    mtext(paste("breaks = ", i), side = 3, col = "blue")
}
par(init)
```

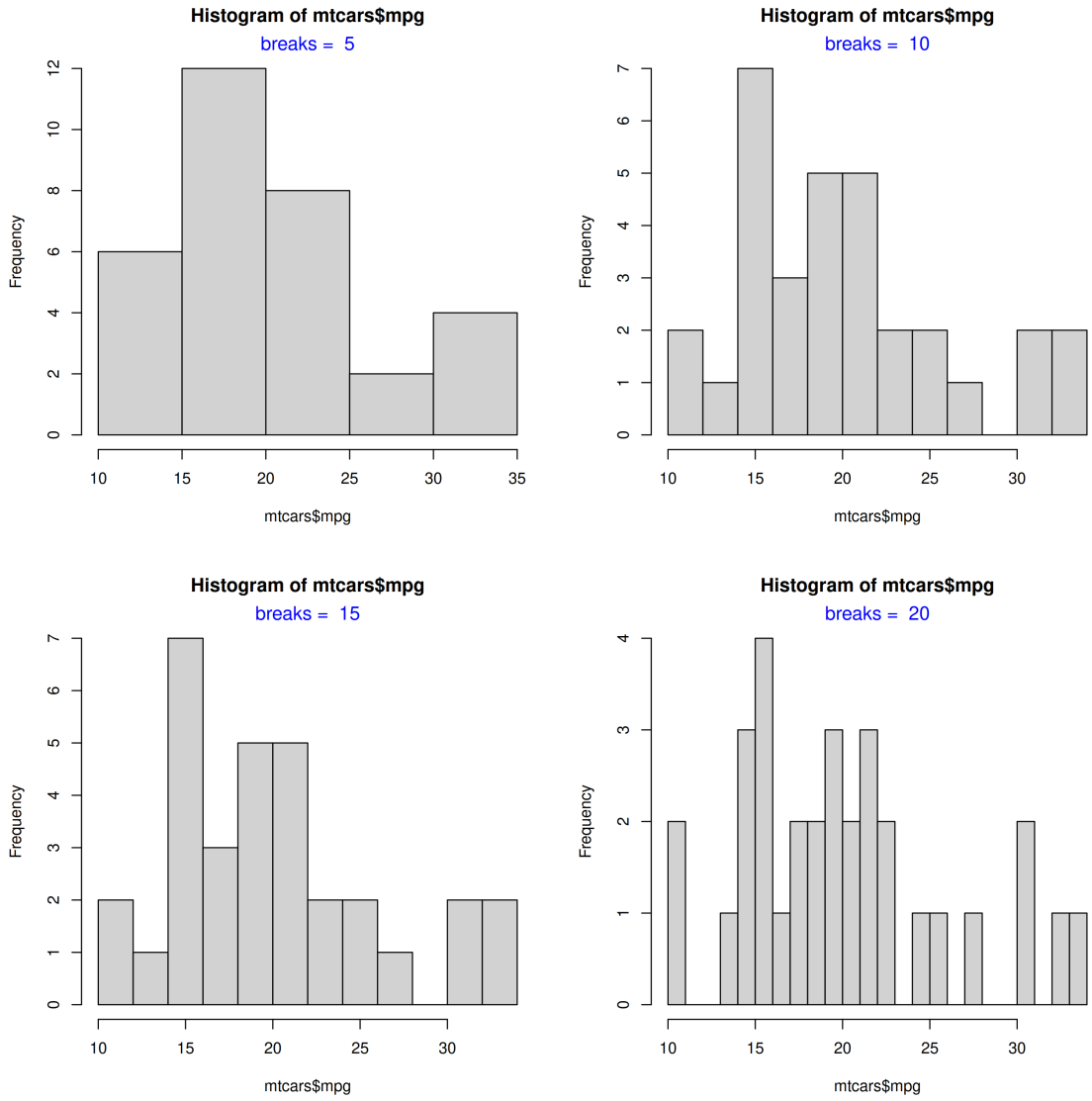
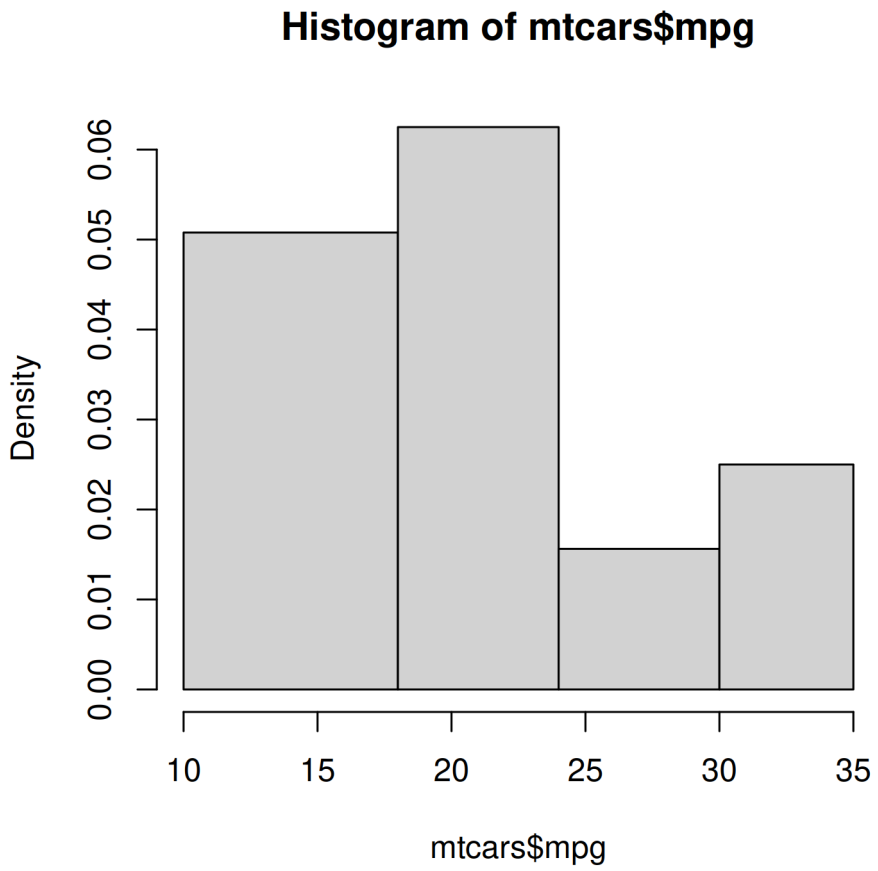


Figure 27: A gallery of histograms of the same data at different interval widths

Intervals

If we want to create histograms with specific intervals, the `breaks` argument can be supplied with the intervals.

```
hist(mtcars$mpg, breaks = c(10, 18, 24, 30, 35))
```



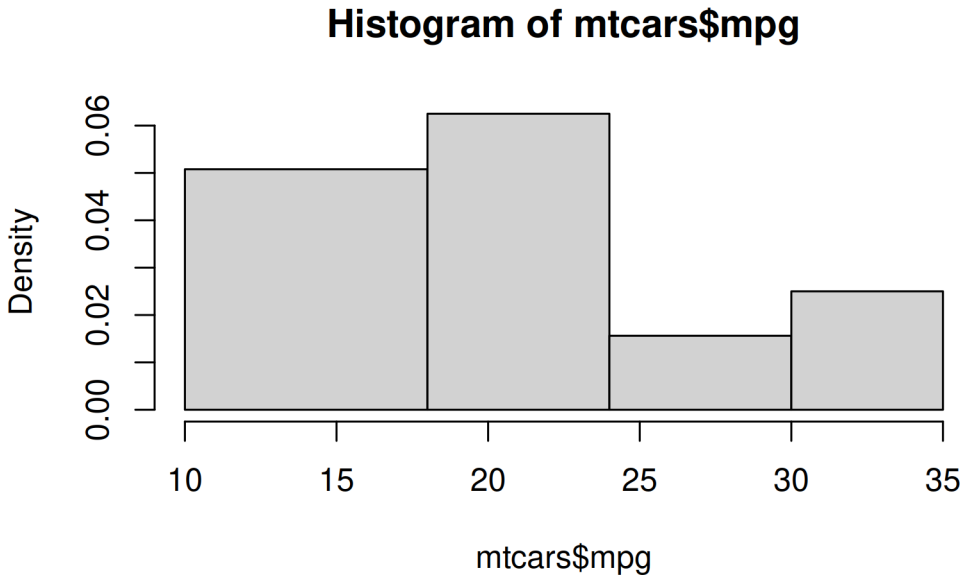
If you observe the Y axis, it does not represent frequency any more. Instead, it represents the frequency density. What is frequency density?

Frequency Density

Frequency Density = Relative Frequency / Class Width

Relative Frequency = Frequency / Total Observations

```
h <- hist(mtcars$mpg, breaks = c(10, 18, 24, 30, 35))
```




```
frequency <- h$counts
class_width <- c(8, 6, 6, 5)
rel_freq <- frequency / length(mtcars$mpg)
freq_density <- rel_freq / class_width
d <- data.frame(frequency = frequency, class_width = class_width,
relative_frequency = rel_freq, frequency_density = freq_density)
d
```

```
  frequency class_width relative_frequency frequency_density
1        13           8            0.40625        0.05078125
2        12           6            0.37500        0.06250000
3         3           6            0.09375        0.01562500
4         4           5            0.12500        0.02500000
```

When multiplied by the class width, the product will always sum upto 1.

```
sum(d$frequency_density * d$class_width)
```

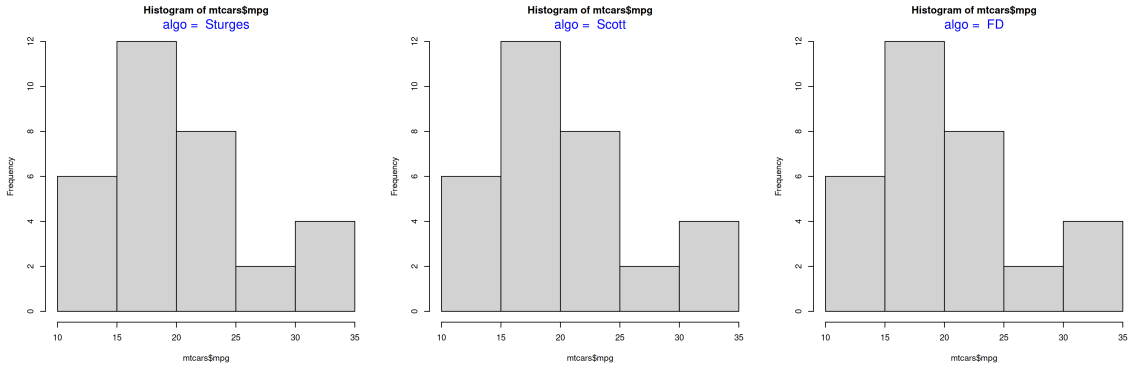
```
[1] 1
```

We will learn more about frequency density in a bit. Before we end this section, we need to learn about one more way to specify the intervals of the histogram, algorithms. The `hist()` function allows us to specify the following algorithms:

- Sturges (default)
- Scott
- Freedman-Diaconis (FD)

In the below plot, we examine how th algorithms work:

```
init <- par(no.readonly = TRUE)
par(mfrow = c(1, 3))
values <- c("Sturges", "Scott", "FD")
for (i in values) {
    hist(mtcars$mpg, breaks = i)
    mtext(paste("algo = ", i), side = 3, col = "blue")
}
```

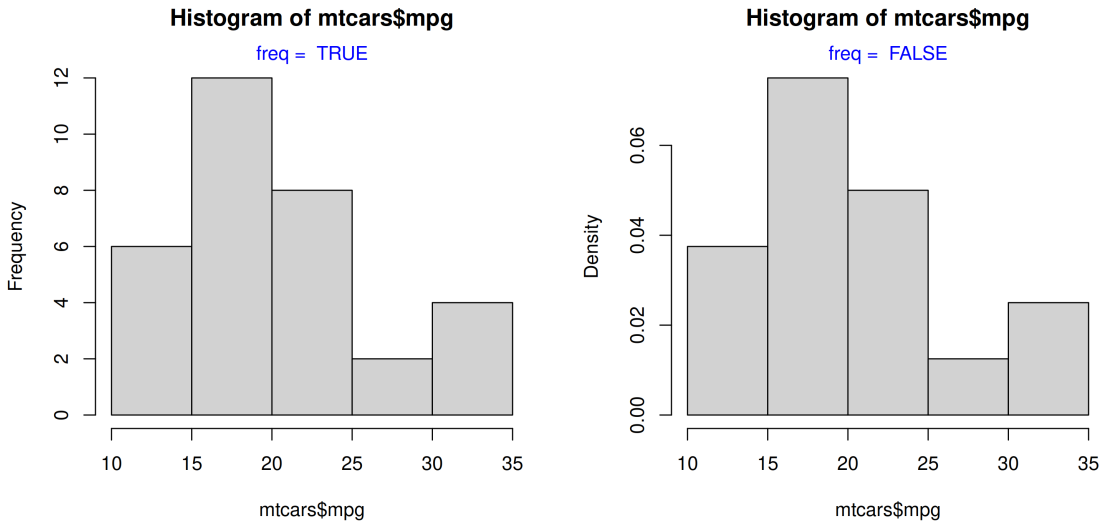



```
par(init)
```

Frequency Distribution II

Let us come back to frequency density. If you want the Y axis of the histogram to represent frequency density instead of counts, set the `freq` argument to `FALSE`.

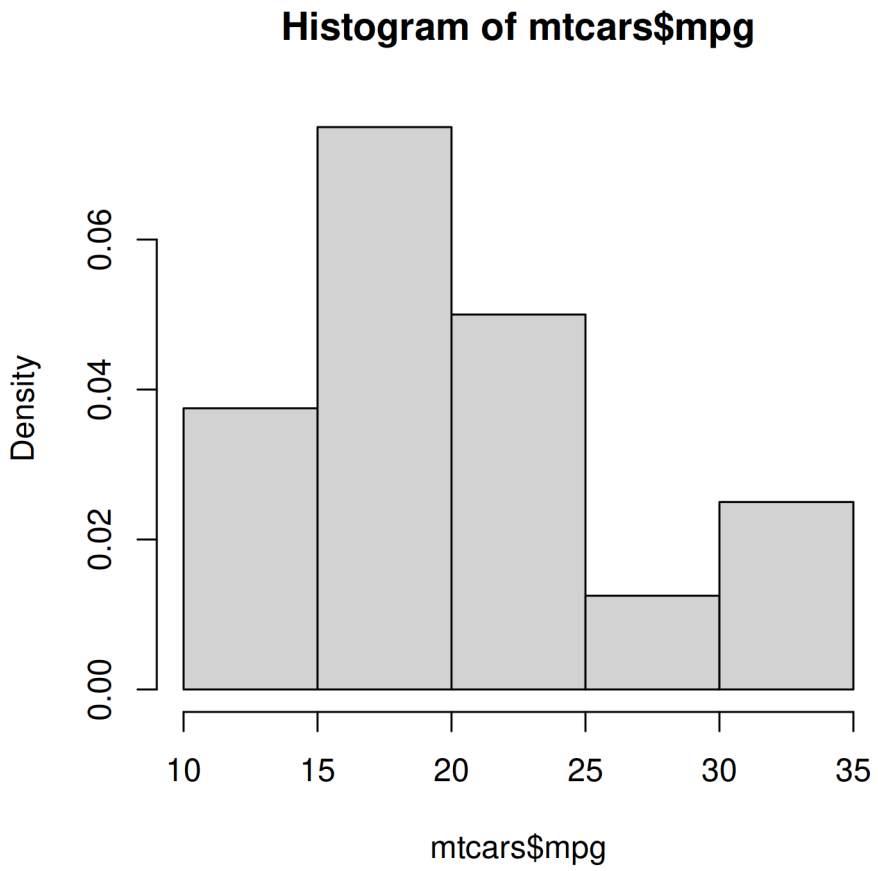
```
init <- par(no.readonly = TRUE)
par(mfrow = c(1, 2))
values <- c(TRUE, FALSE)
for (i in values) {
    hist(mtcars$mpg, freq = i)
    mtext(paste("freq = ", i), side = 3, col = "blue")
}
```




```
par(init)
```

The same result can be achieved by using the `probability` argument as well. It takes only logical values as inputs and the default is `FALSE`. If set to `TRUE`, the Y axis will represent the frequency density instead of counts.

```
hist(mtcars$mpg, probability = TRUE)
```



Color

To add colors to the bars of the histogram, use the `col` argument. If the number of colors specified is less than the number of bars, the colors are recycled. Below are a few examples:

Single Color

```
hist(mtcars$mpg, col = 'blue')
```

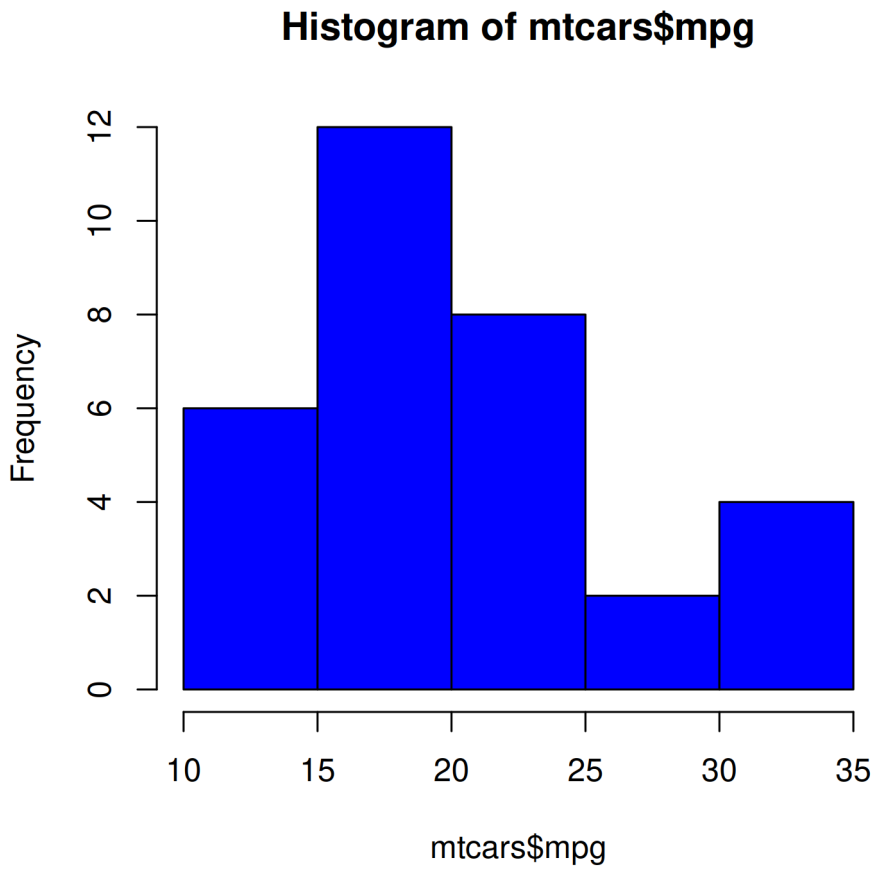
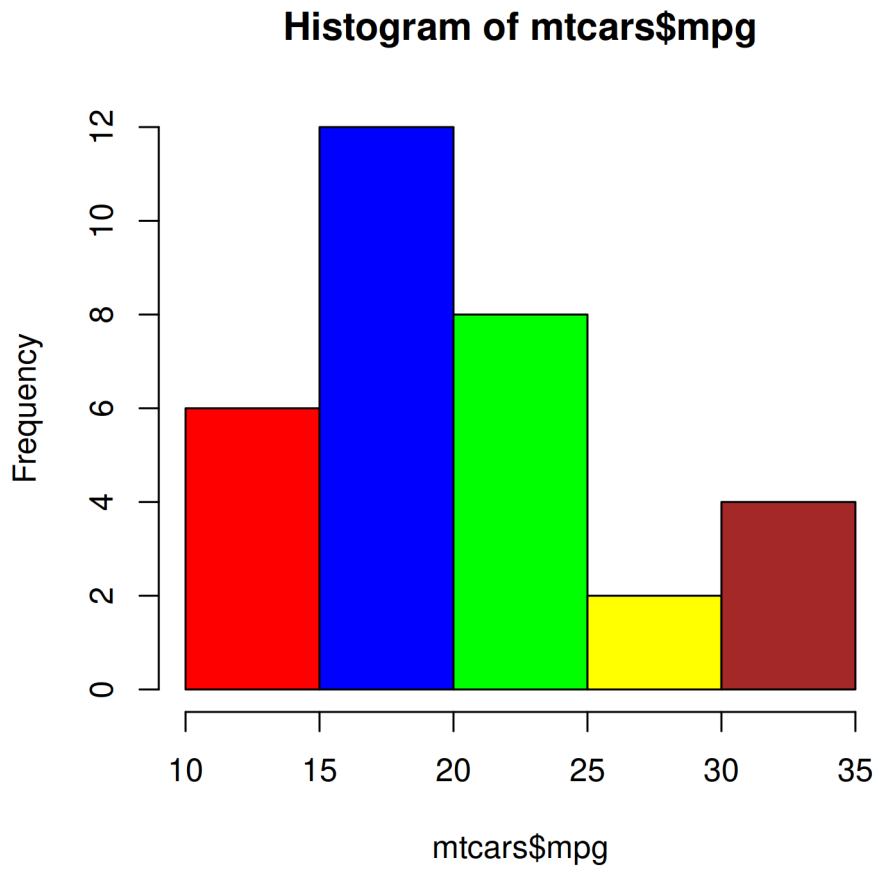


Figure 28: Frequency labels drawn on top of the histogram bars

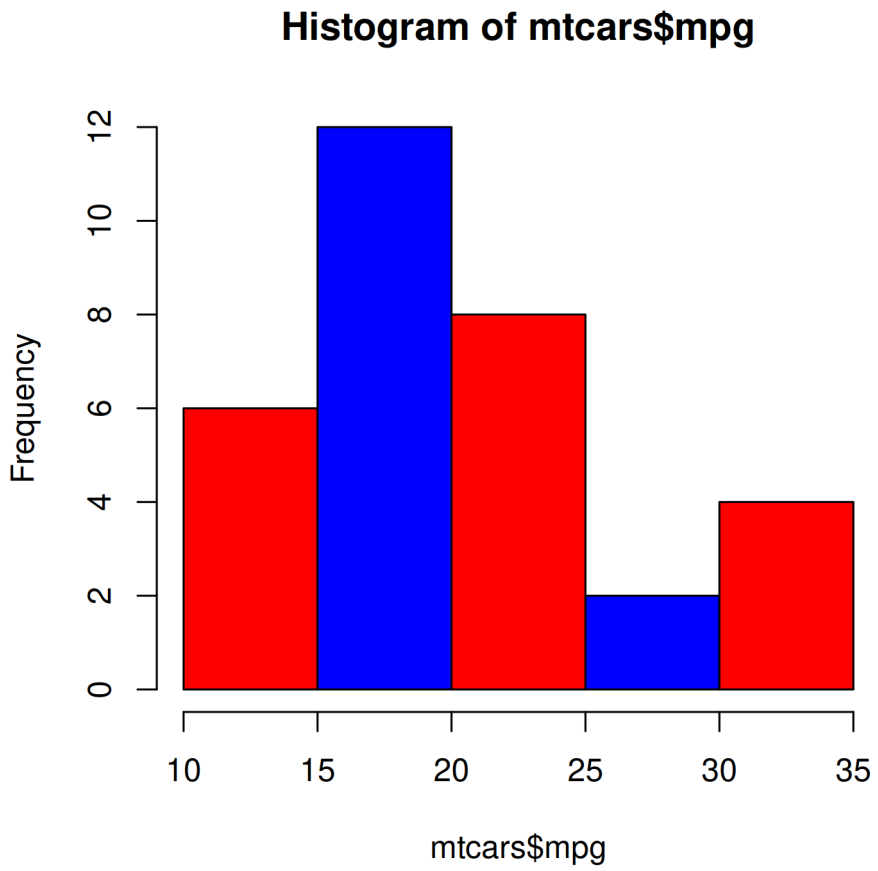
Different Colors

```
hist(mtcars$mpg, col = c('red', 'blue', 'green', 'yellow', 'brown'))
```



Recycled Colors

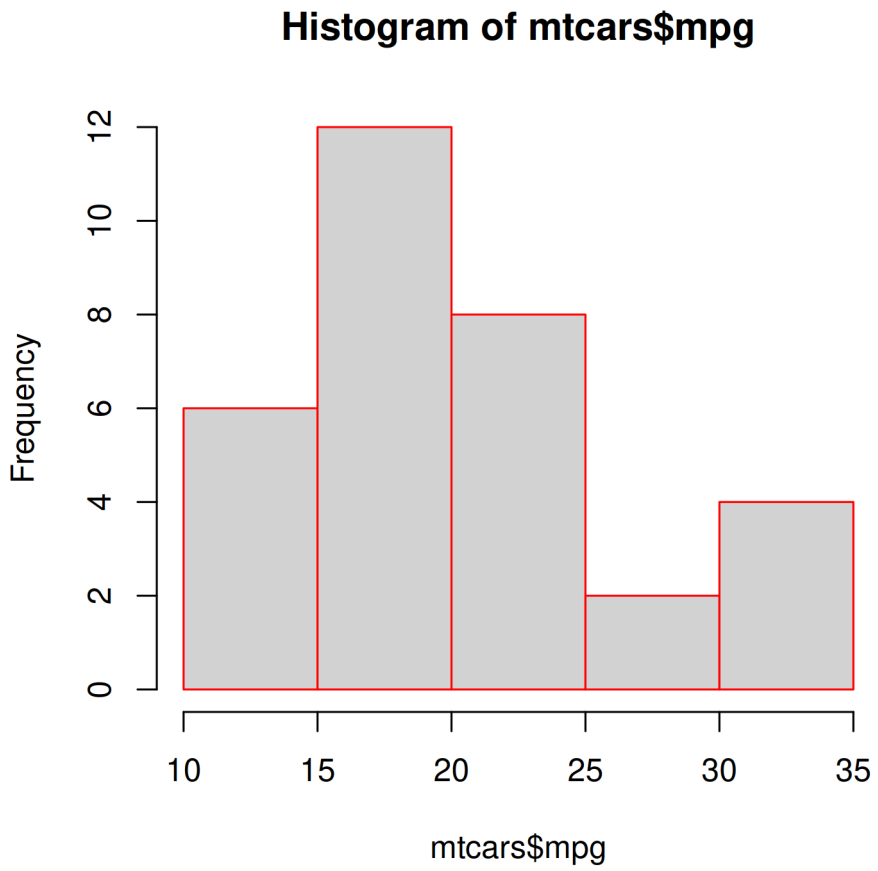
```
hist(mtcars$mpg, col = c('red', 'blue'))
```



Border Color

Colors can be specified for the borders of the histogrambars using the `border` argument.

```
hist(mtcars$mpg, border = 'red')
```



Different Colors

```
hist(mtcars$mpg, border = c('red', 'blue', 'green', 'yellow', 'brown'))
```

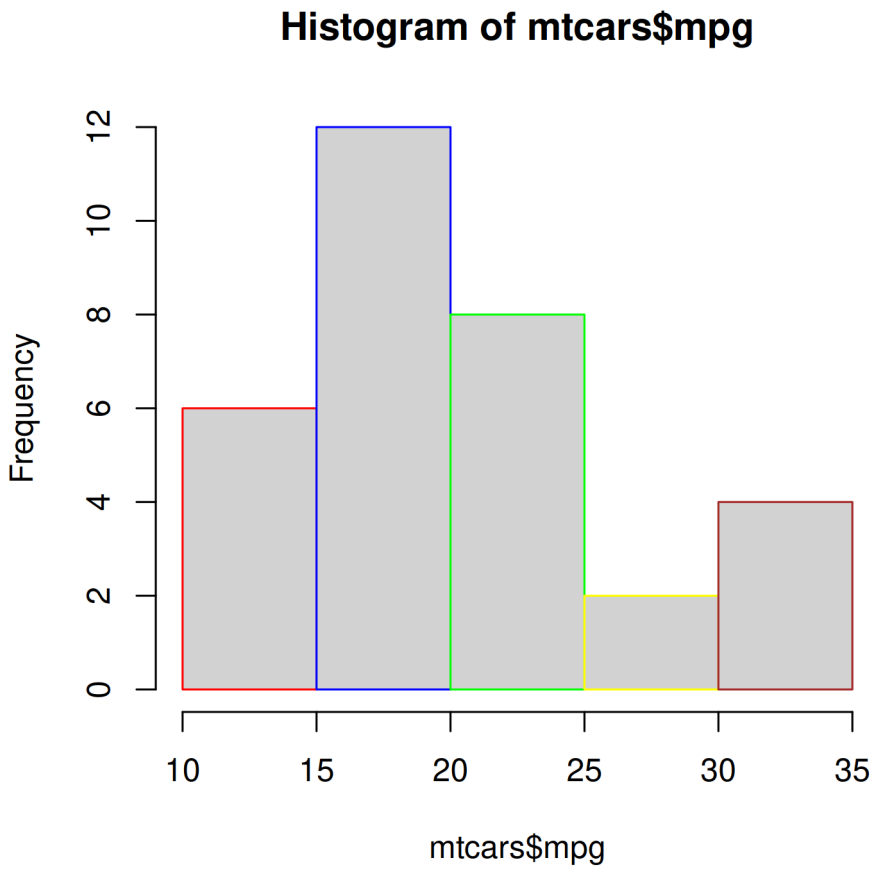


Figure 29: A right-skewed sample drawn with rbeta()

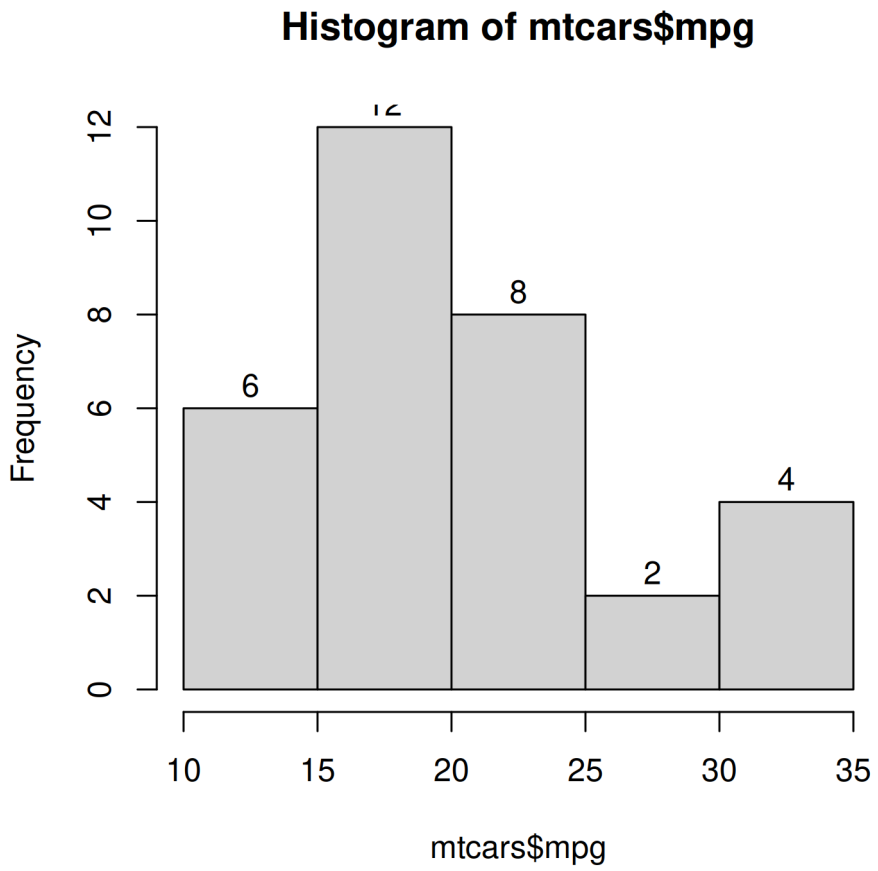
Labels

In certain cases, we might want to add the frequency counts on the histogram bars. It is easier for the user to know the frequencies of each bin when they are present on top of the bars. Let us add the frequency counts on top of the bars using the `labels` argument. We can either set it to `TRUE` or a character vector containing the label values. Let us look at both the methods.

Method 1

Set `labels` to `TRUE`.

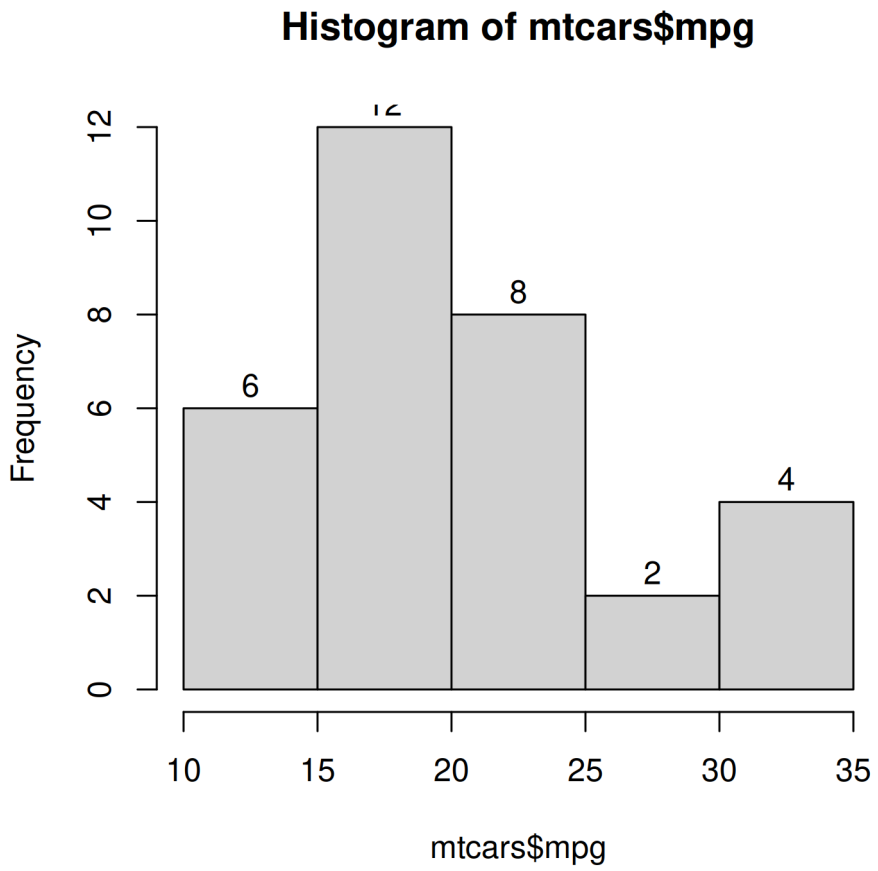
```
hist(mtcars$mpg, labels = TRUE)
```



Method 2

Specify the label values in a character vector.

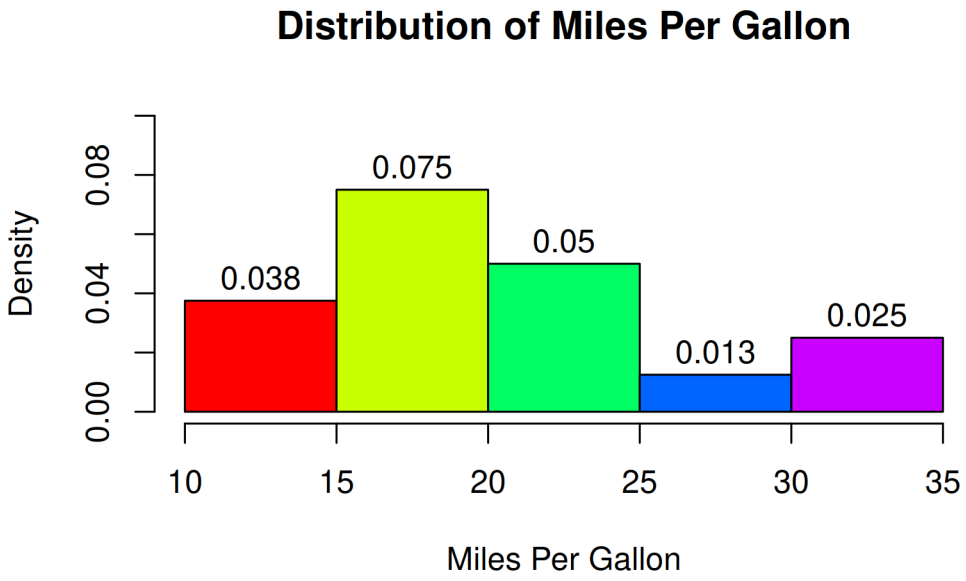
```
hist(mtcars$mpg, labels = c("6", "12", "8", "2", "4"))
```



Putting it all together

Let us add a title and axis labels to the histogram.

```
hist(mtcars$mpg, labels = TRUE, prob = TRUE,
     ylim = c(0, 0.1), xlab = 'Miles Per Gallon',
     main = 'Distribution of Miles Per Gallon',
     col = rainbow(5))
```



Legends

Introduction

In this chapter, we will learn how to:

- position the legend within the plot
- modify the layout using `ncol` and `horiz` arguments
- add title using the `title` set of arguments
- modify the appearance and position of the legend box
- modify the appearance of the text in the legend box

Legends are used to convey information about the data being represented by a plot. To understand the importance of legends, let us look at the two plots below. In the first plot, would you be able to understand what the lines represent in the absence of a legend? May be yes but only if the author provides information in a textual form outside the plot. While such information will be useful, it will also be very inconvenient. Now look at the second plot, from the legend at the top right we can easily interpret what the lines represent. Would you agree that a legend is integral to plot representing multiple data? If yes, let us go ahead and learn how to add a legend to different plots.

Since we have looked at a line graph in the above example, we will learn how to add a legend to a line graph. After that, we will generalise the steps to different plots.

Data

Let us build a line graph that represents annual economic growth (GDP) data of the BRICS nations for the years 2010-14.

The values below are a frozen teaching vintage, also shipped as `data/brics-gdp-2010-14.csv`. They differ slightly from the World Bank's current revised series, so treat them as illustrative, not citable statistics.

```
year <- seq(2010, 2014, 1)
india <- c(10.3, 6.6, 5.6, 6.6, 7.2)
china <- c(10.6, 9.5, 7.8, 7.7, 7.3)
russia <- c(4.5, 4.3, 3.5, 1.3, 0.7)
brazil <- c(7.5, 3.9, 1.9, 3.0, 0.1)
s_africa <- c(3.2, 3.2, 2.2, 2.2, 1.5)
gdp <- data.frame(year, india, china, russia, brazil, s_africa, stringsAsFactors
= FALSE)
gdp
```

```
  year india china russia brazil s_africa
1 2010  10.3  10.6    4.5    7.5      3.2
2 2011   6.6   9.5    4.3    3.9      3.2
3 2012   5.6   7.8    3.5    1.9      2.2
4 2013   6.6   7.7    1.3    3.0      2.2
5 2014   7.2   7.3    0.7    0.1      1.5
```

Line Graph

Below is the line graph that represents the above GDP data set:

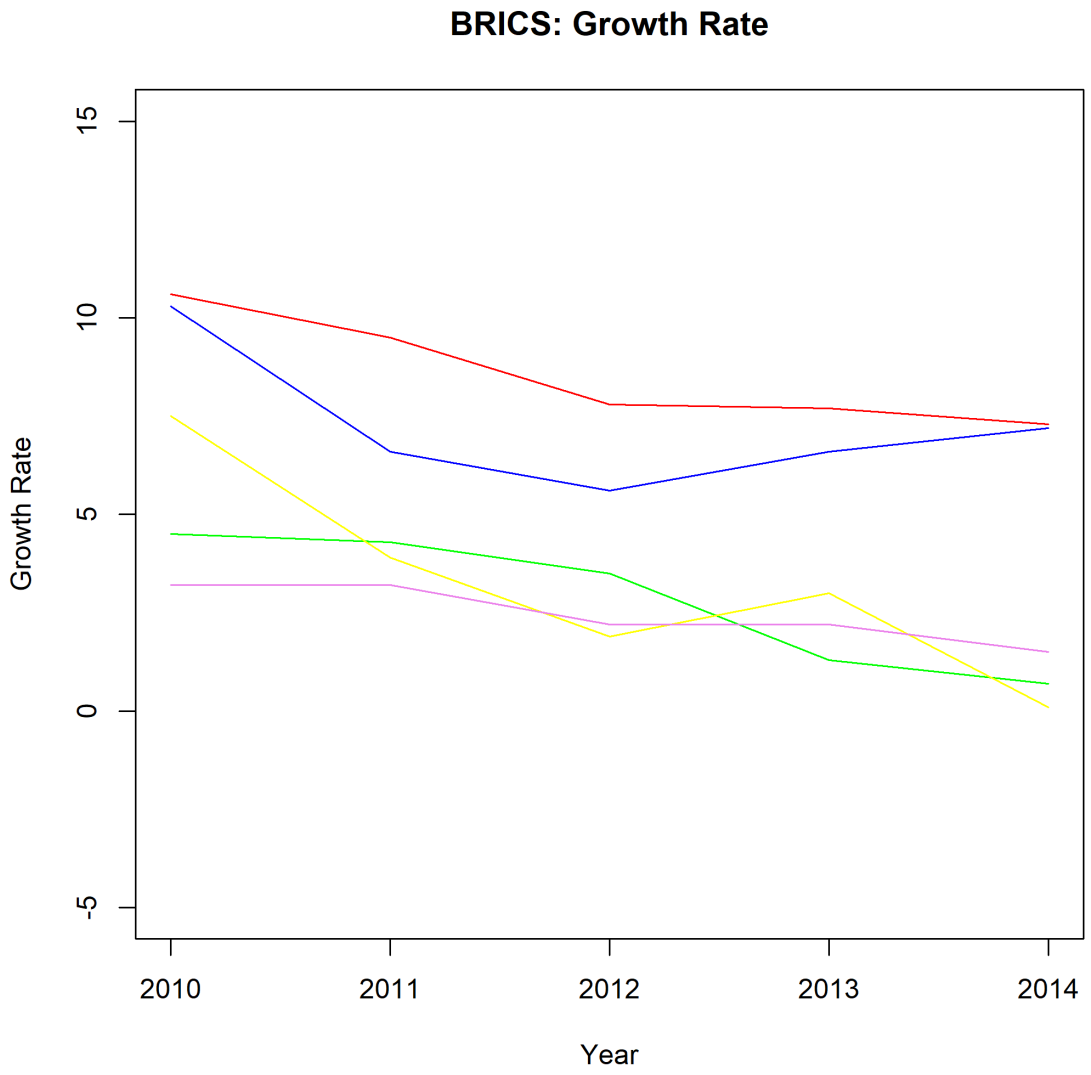


Figure 30: The problem a legend solves: five series with nothing to tell them apart

Without a legend, it will be very difficult to map the lines to the BRICS nations. We will add a legend to the above plot using the `legend()` function and do so one step at a time.

Legend Location

In order to add a legend to the plot, the first thing we must specify is the location of the legend in the plot. There are 2 ways to do this:

- use x and y axis coordinates
- use keywords

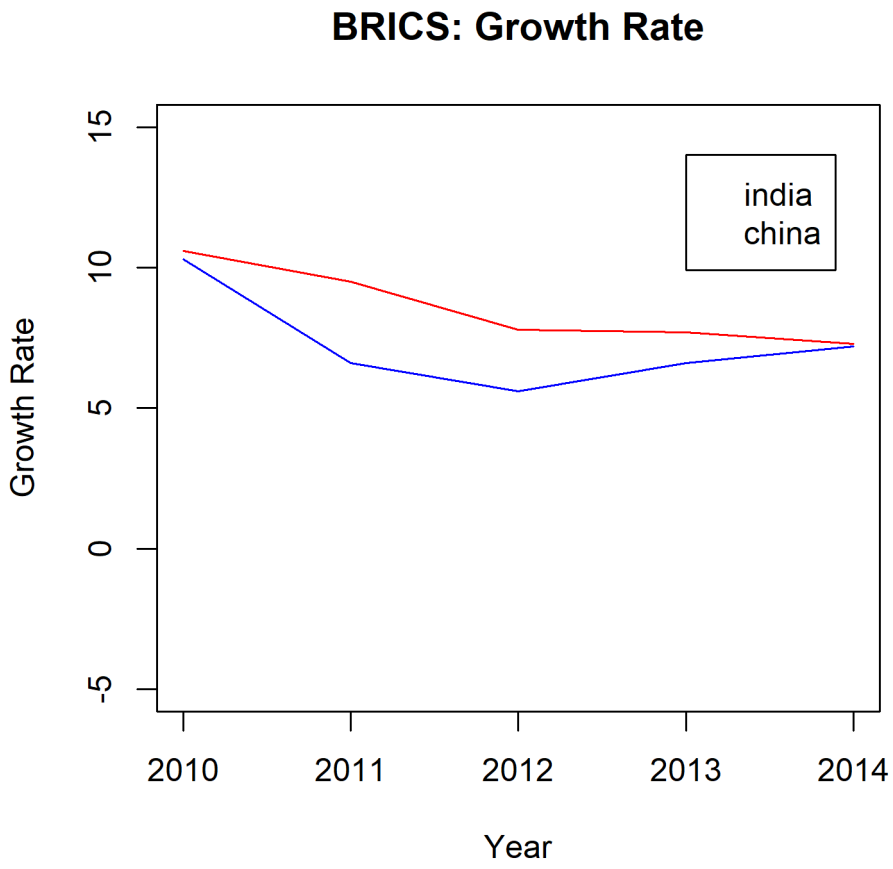
The list of keywords include:

- top

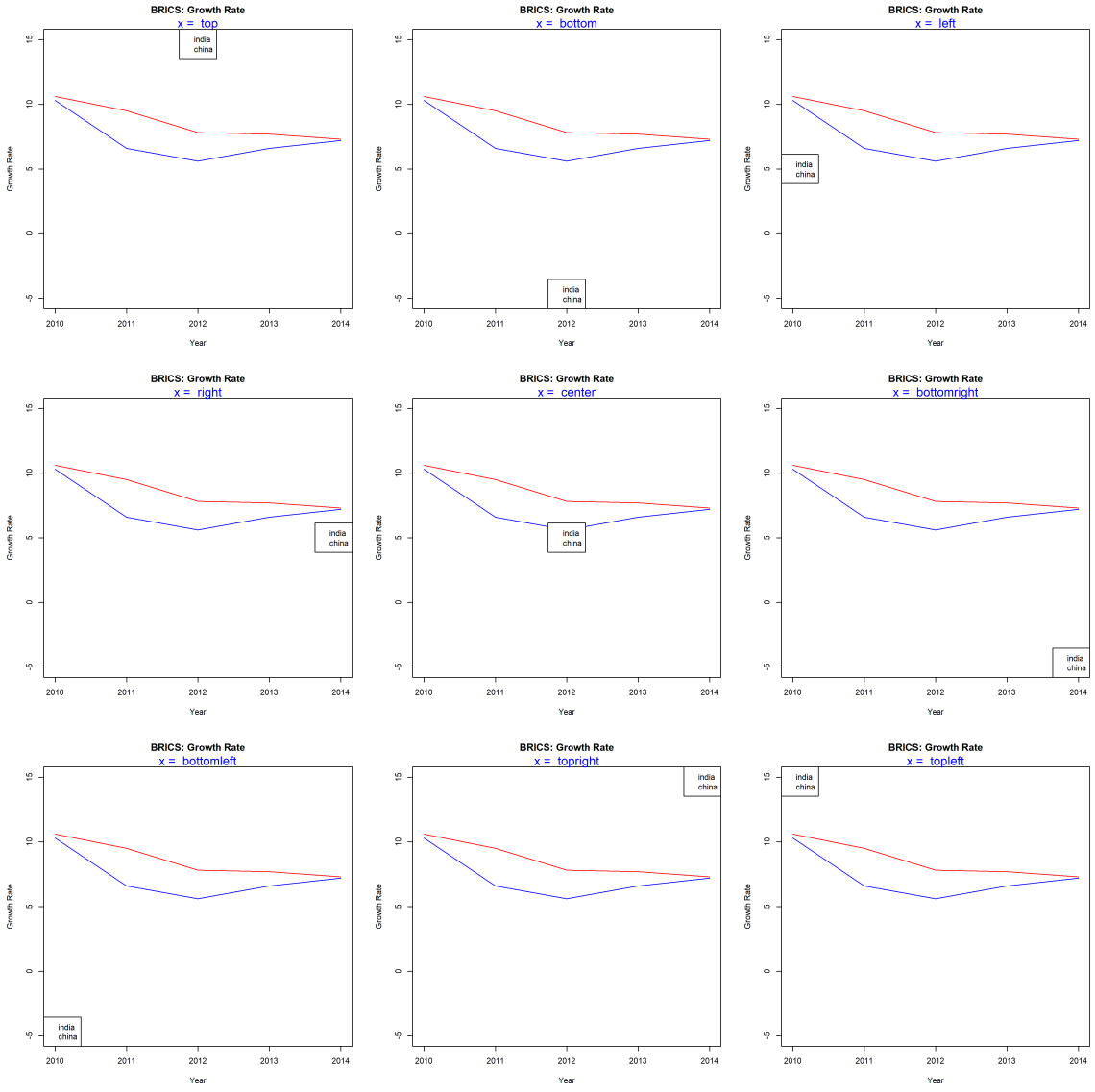
- bottom
- left
- right
- center
- bottomright
- bottomleft
- topright
- topleft

But there is a glitch. If we do not specify what goes into the legend, the `legend()` function will return an error. Before we experiment with the location of the legend inside the plot, we need to learn about another argument used to specify the content of the legend. The argument is also named `legend`. It takes a vector as input. In the next example, we will plot the GDP data for India and China and add a basic legend.

```
{plot(gdp$year, gdp$india, type = 'l',
      ylim = c(-5, 15), xlab = 'Year',
      ylab = 'Growth Rate', col = 'blue',
      main = 'BRICS: Growth Rate')
lines(gdp$year, gdp$china, col = 'red')
legend(x = 2013, y = 14, legend = c('india', 'china'))}
```



You can see that a legend has been added bases on the X and Y axis coordinates we specified in the `legend()` function. But the legend is incomplete and a user still cannot map the lines to the countries using the legend. We will learn how to add lines inside the legend shortly but before that let us use keywords to position the legend inside the plot.



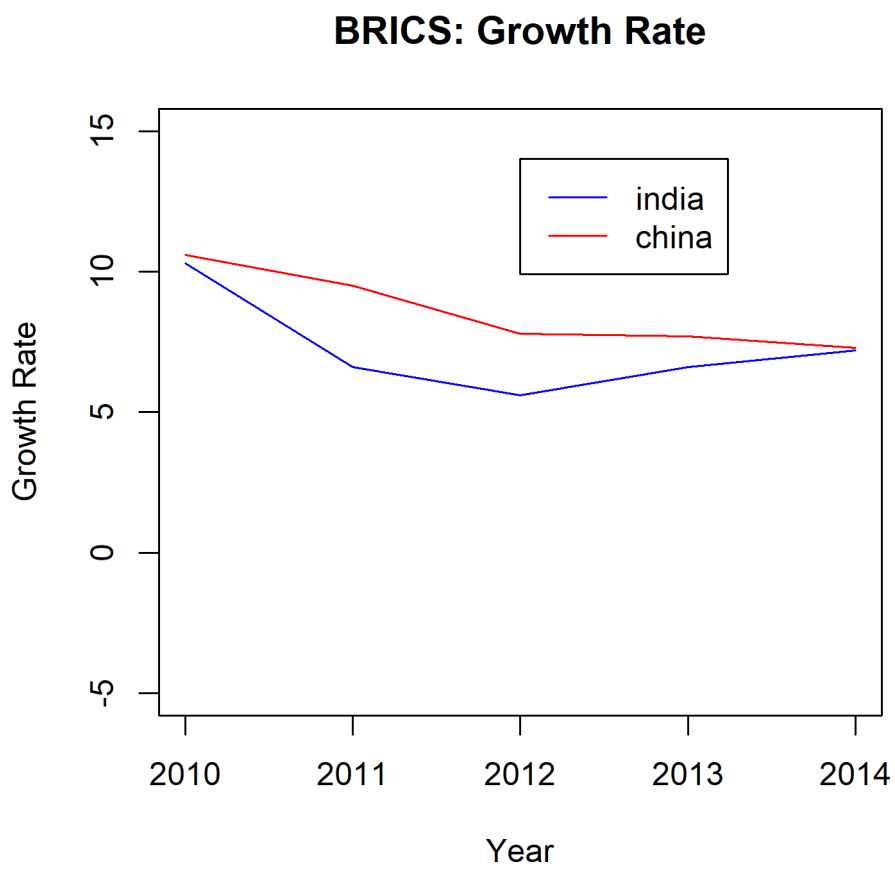
You can either use the keywords or the axis coordinates to position the legend inside the plot. Use the coordinates method if you want greater control over the position of the legend. Next step is to add lines inside the legend so that a user can map the lines in the plots to the countries.

Lines

Adding a line in the legend is very simple. Use the `lty` argument to specify the line type and the `col` argument to add color to the line.

```
{plot(gdp$year, gdp$india, type = 'l',
      ylim = c(-5, 15), xlab = 'Year',
      ylab = 'Growth Rate', col = 'blue',
      main = 'BRICS: Growth Rate')
lines(gdp$year, gdp$china, col = 'red')
```

```
legend(x = 2012, y = 14, legend = c('india', 'china'),
       lty = 1, col = c('blue', 'red'))}
```



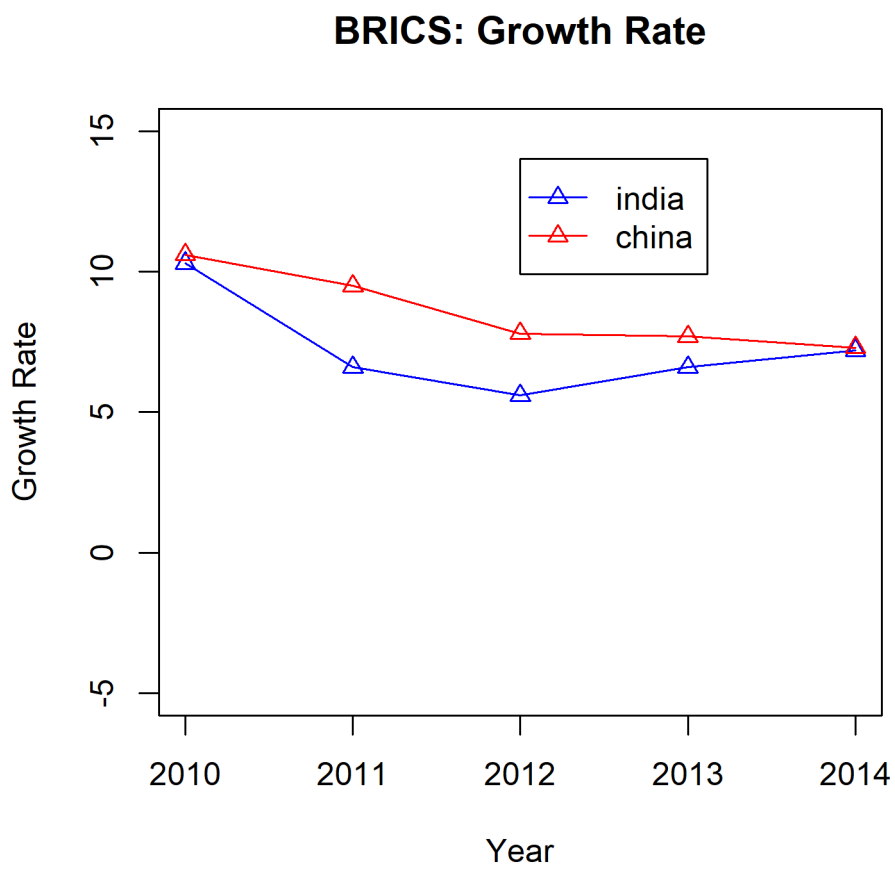
Now we can map the lines to the respective countries using the legend. But our legend looks very simple right. Let us explore the options available to modify and enhance the appearance of the legend.

Points

If the plot has both lines and points, we can use the `pch` argument in the `legend()` function to specify the shape of the point.

```
{plot(gdp$year, gdp$india, type = 'l',
      ylim = c(-5, 15), xlab = 'Year',
      ylab = 'Growth Rate', col = 'blue',
      main = 'BRICS: Growth Rate')
points(gdp$year, gdp$india, pch = 2, col = 'blue')
lines(gdp$year, gdp$china, col = 'red')
points(gdp$year, gdp$china, pch = 2, col = 'red')
```

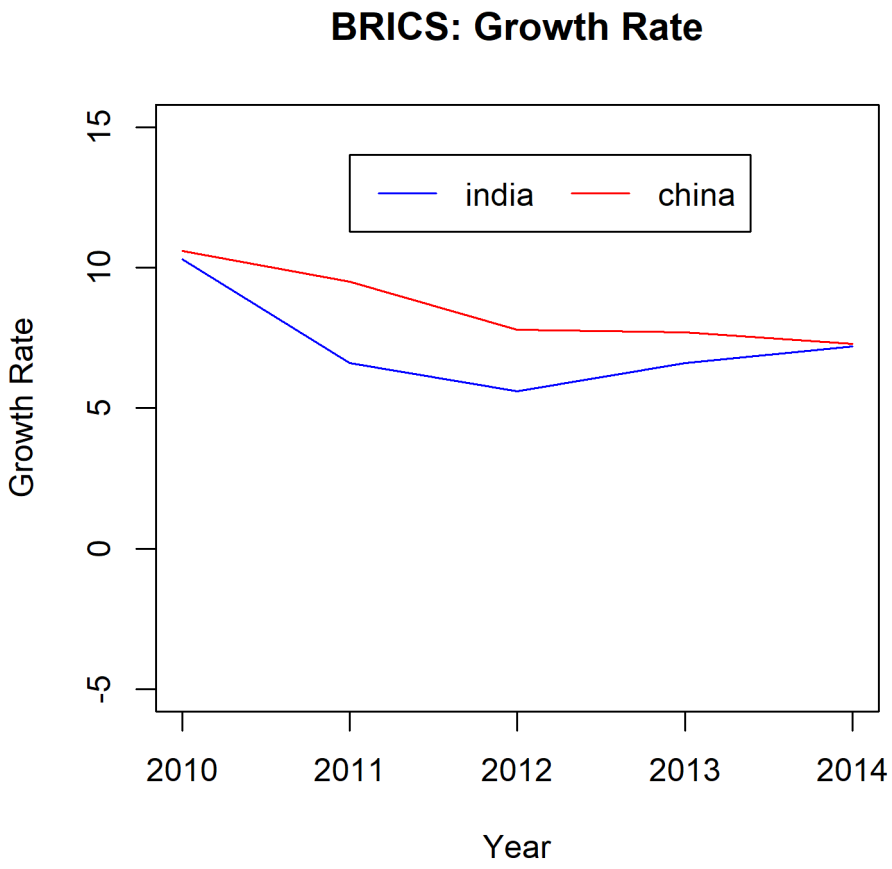
```
legend(x = 2012, y = 14, legend = c('india', 'china'),
       lty = 1, pch = 2, col = c('blue', 'red'))}
```



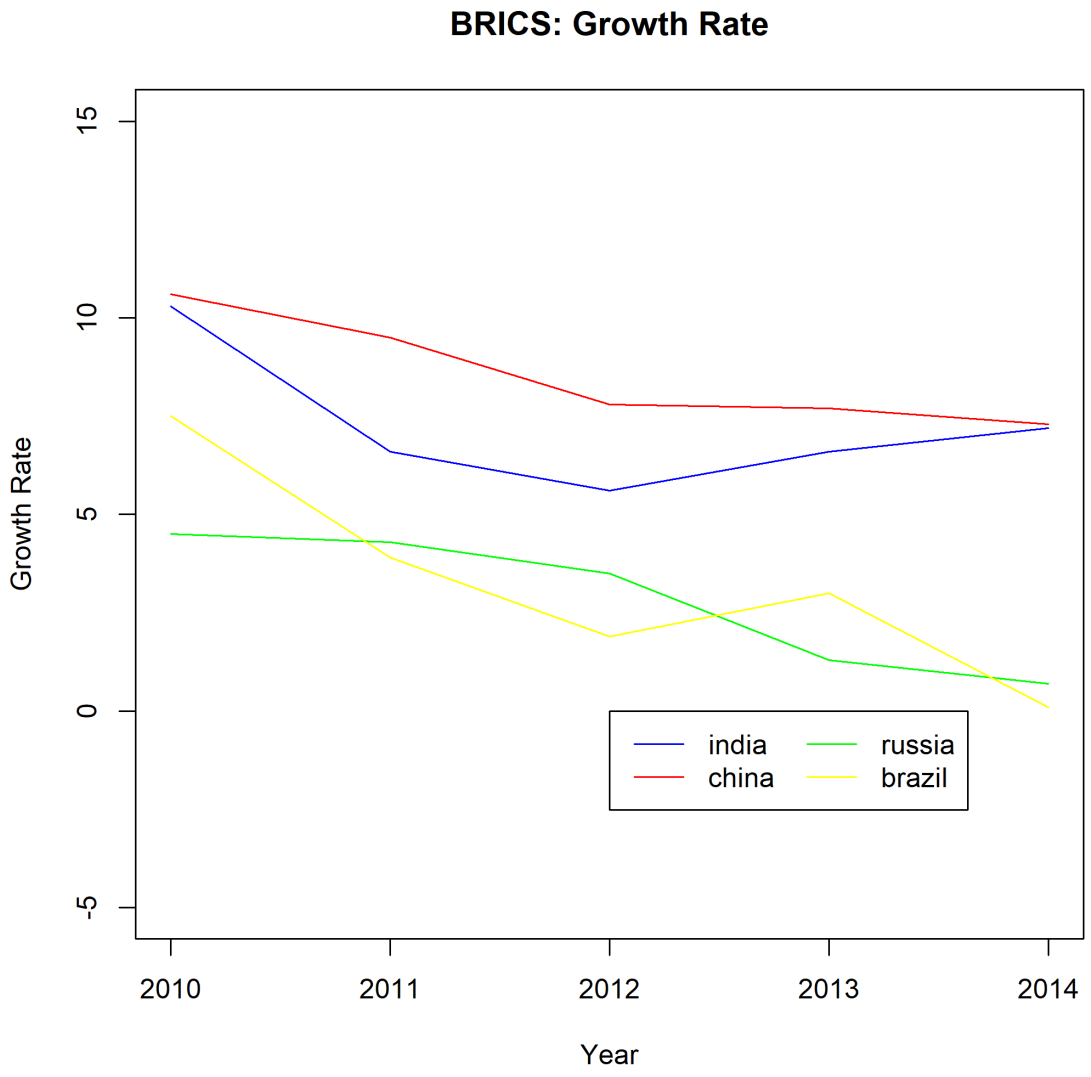
Text Placement

The contents of the legend can be positioned horizontally using the `horiz` argument. It takes logical values as inputs and the default is `FALSE`. Set it to `TRUE` to position the contents horizontally:

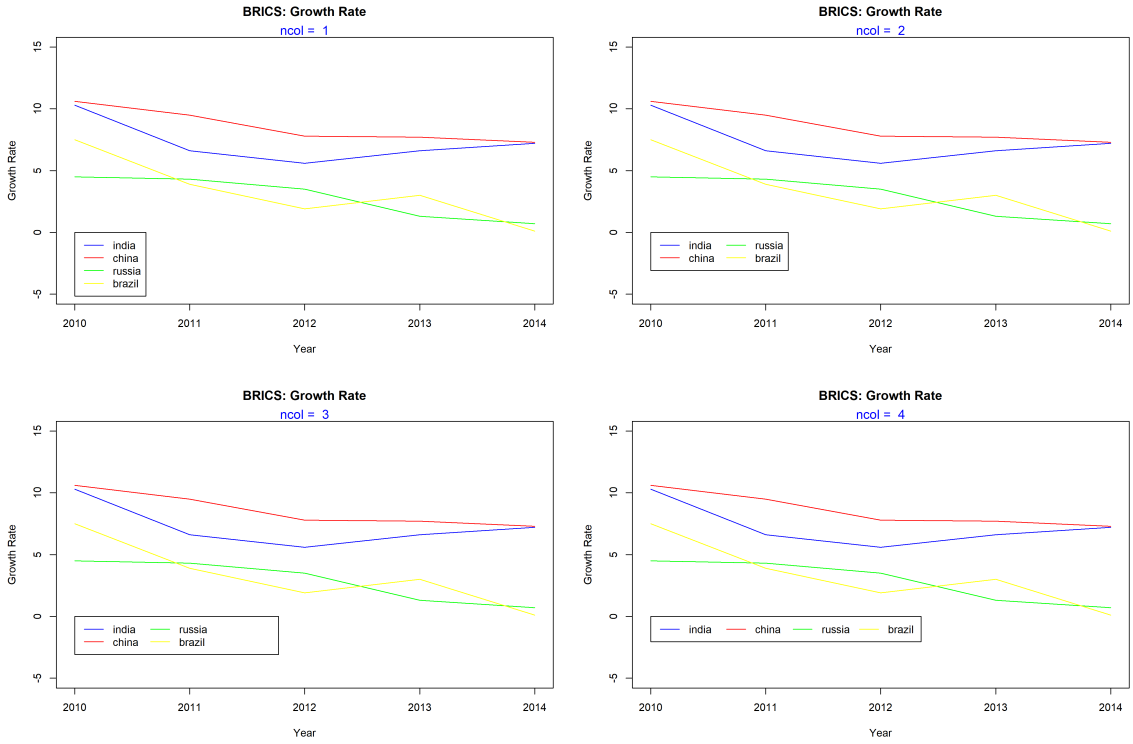
```
{plot(gdp$year, gdp$india, type = 'l',
      ylim = c(-5, 15), xlab = 'Year',
      ylab = 'Growth Rate', col = 'blue',
      main = 'BRICS: Growth Rate')
lines(gdp$year, gdp$china, col = 'red')
legend(x = 2011, y = 14, legend = c('india', 'china'),
       lty = 1, col = c('blue', 'red'),
       horiz = TRUE)}
```



Another way to position the content inside the legend is to use columns. In the below example, we use the `ncol` argument to split the contents of the legend into two columns instead of the default one.



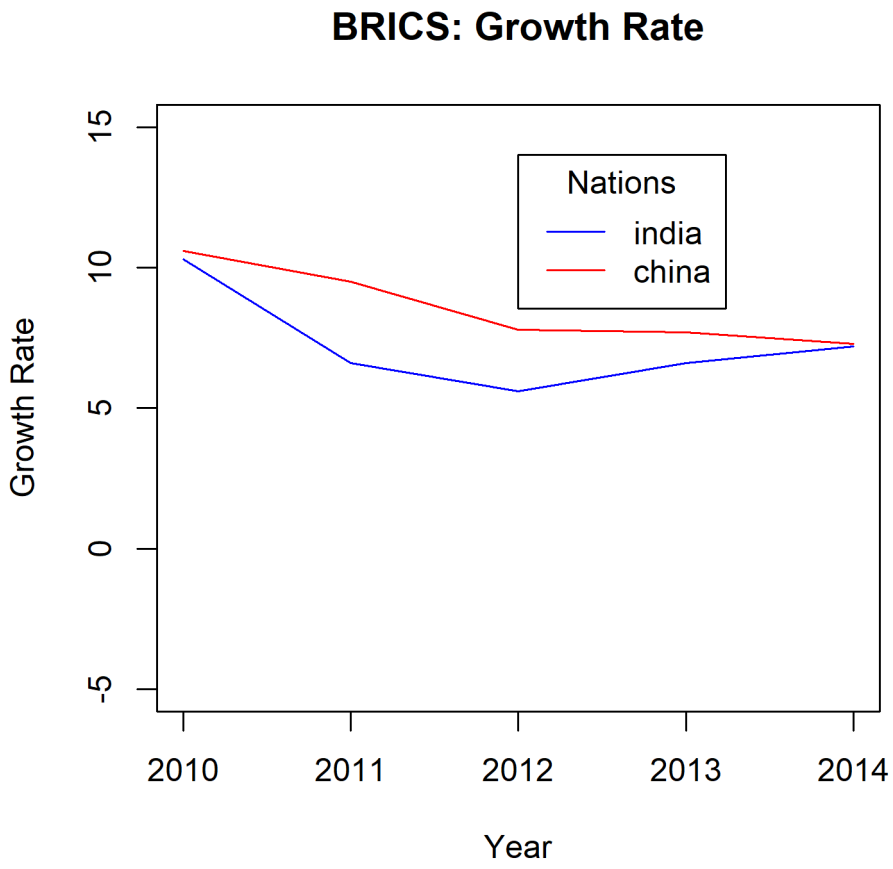
The below plots show the difference in appearance:



Title

Let us add a title to the legend using the `title` argument:

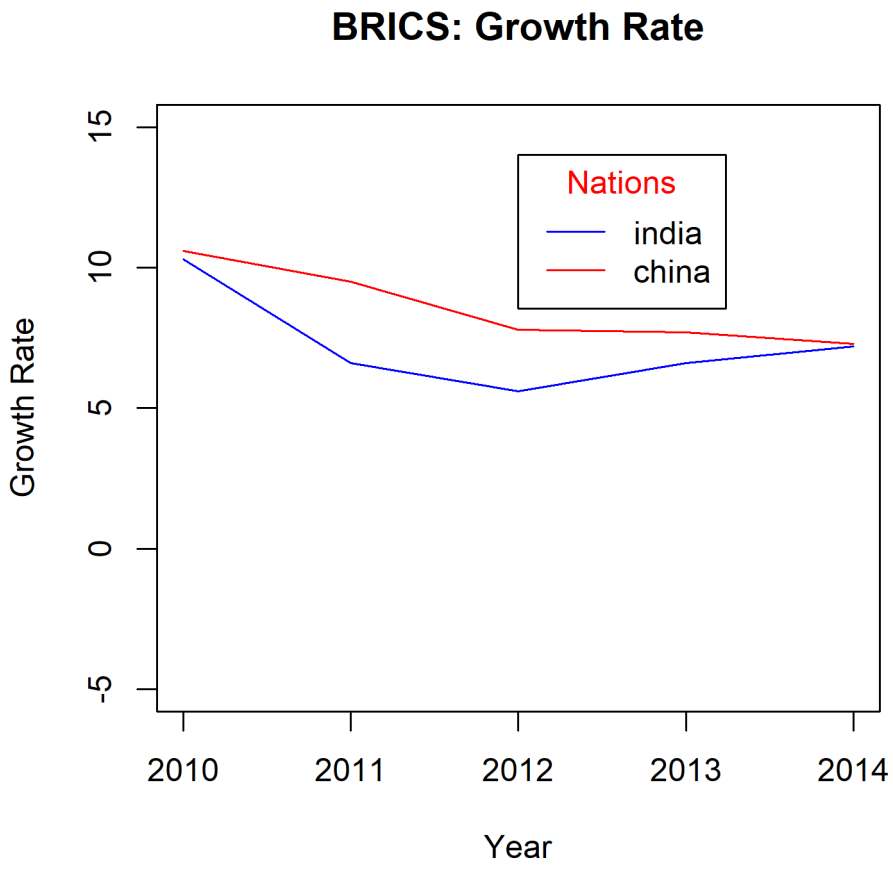
```
{plot(gdp$year, gdp$india, type = 'l',
      ylim = c(-5, 15), xlab = 'Year',
      ylab = 'Growth Rate', col = 'blue',
      main = 'BRICS: Growth Rate')
lines(gdp$year, gdp$china, col = 'red')
legend(x = 2012, y = 14, legend = c('india', 'china'),
       lty = 1, col = c('blue', 'red'),
       title = 'Nations')}
```



Title Color

The color of the title can be modified using the `title.col` argument:

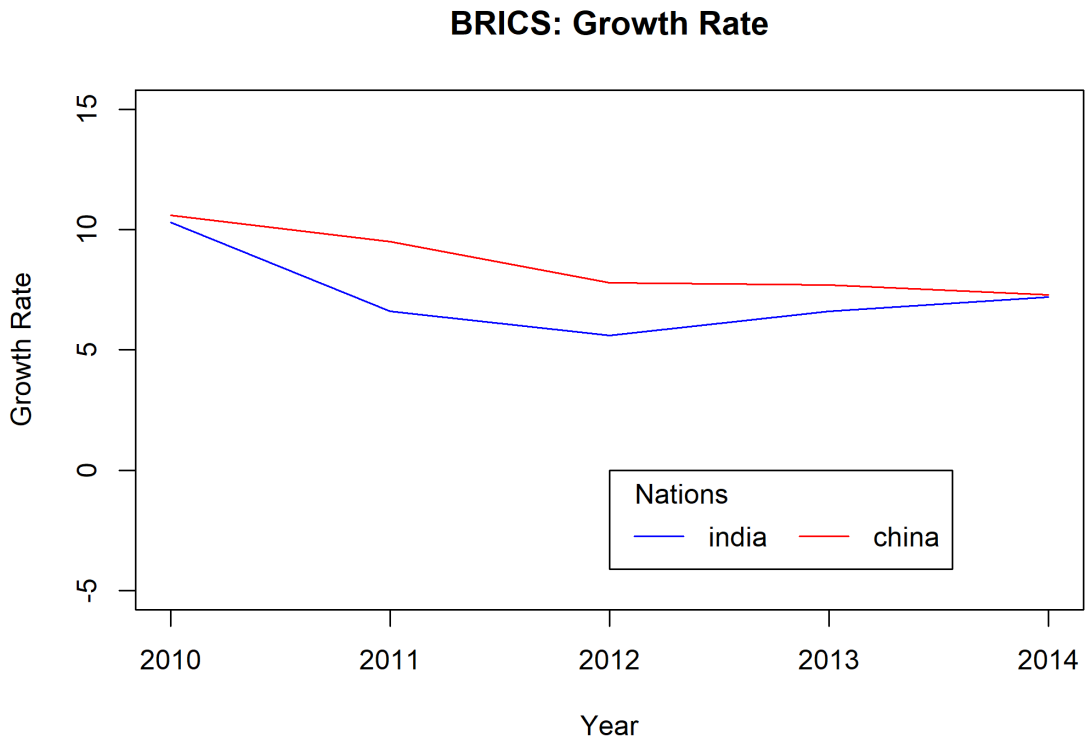
```
{plot(gdp$year, gdp$india, type = 'l',
      ylim = c(-5, 15), xlab = 'Year',
      ylab = 'Growth Rate', col = 'blue',
      main = 'BRICS: Growth Rate')
lines(gdp$year, gdp$china, col = 'red')
legend(x = 2012, y = 14, legend = c('india', 'china'),
       lty = 1, col = c('blue', 'red'),
       title = 'Nations', title.col = 'red')}
```



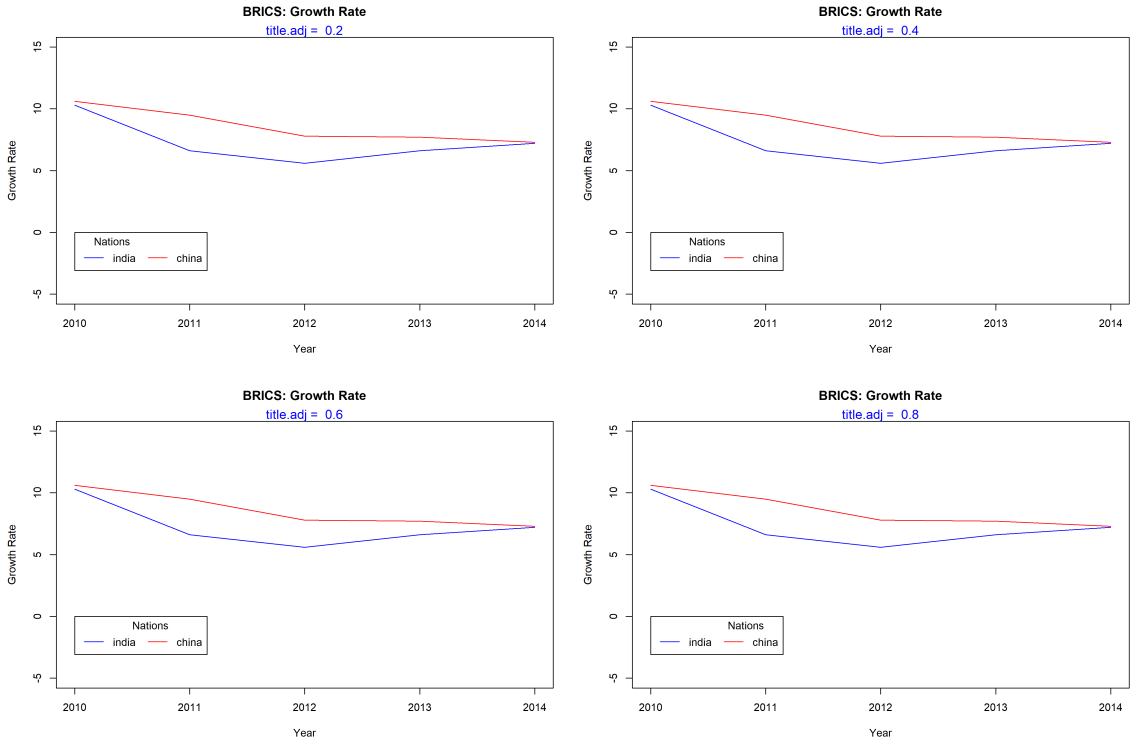
Title Position

The title can be positioned within the legend box using the `title.adj` argument. It will take values between 0 and 1. The default is 0.5 and the title is positioned in the middle of the box. As the value moves away from 0.5, the position of the title moves to the left or right respectively.

```
{plot(gdp$year, gdp$india, type = 'l',
      ylim = c(-5, 15), xlab = 'Year',
      ylab = 'Growth Rate', col = 'blue',
      main = 'BRICS: Growth Rate')
lines(gdp$year, gdp$china, col = 'red')
legend(x = 2012, y = 0, legend = c('india', 'china'),
       lty = 1, col = c('blue', 'red'), horiz = TRUE,
       title = 'Nations', title.adj = 0.1)}
```



The below plots show the relative position of the title within the legend box for different values between 0 and 1.



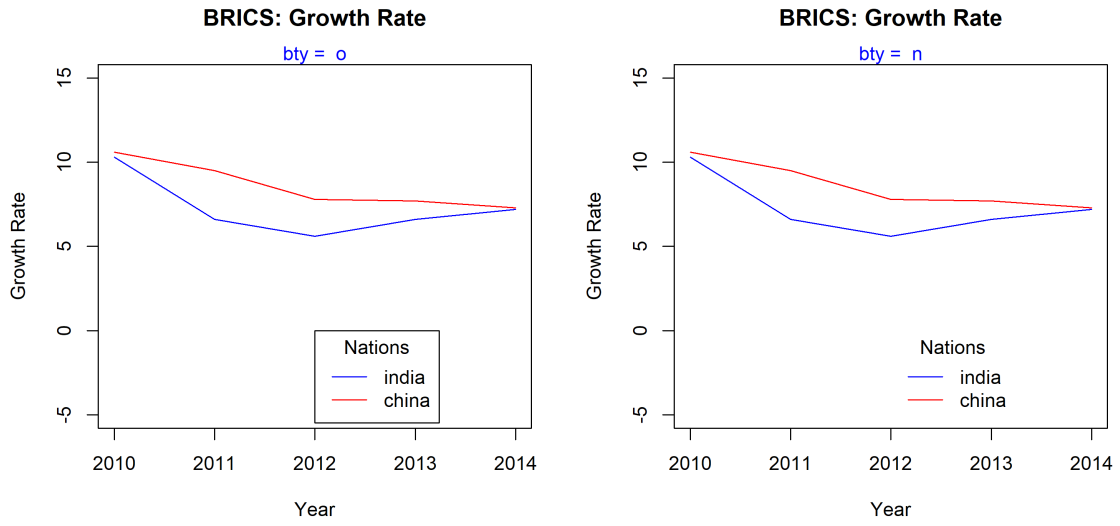
Box Appearance

There are a lot of options to modify the appearance of the legend box. The below table displays the arguments and their descriptions. Let us look at them one by one:

option	argument	values
Box Type	bty	o, n
Background Color	bg	blue, #0000ff
Border Line Type	box.lty	1:5
Border Line Width	box.lwd	0.5, 1, 1.5
Border Line Color	box.col	blue, #0000ff
Horizontal Justification	xjust	0:1
Vertical Justification	yjust	0:1
Text Color	text.col	blue, #0000ff
Text Font	text.font	1:5

Box Type

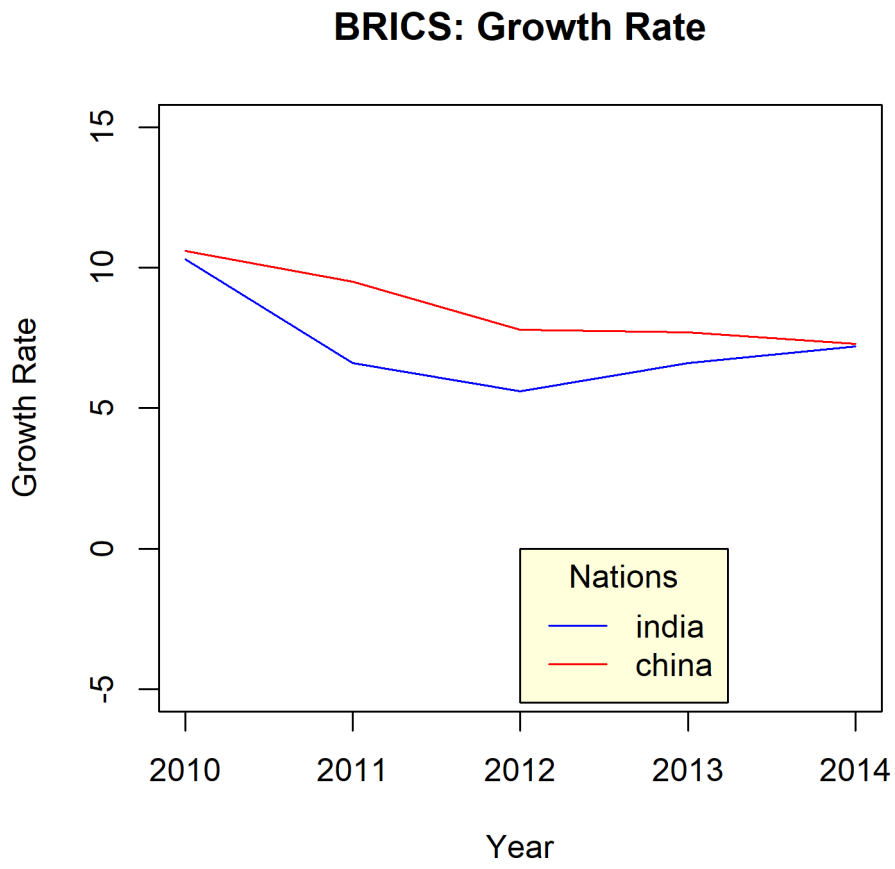
The `bty` argument takes two values, `o` and `n`. If set to `n`, there will be no box around the legend.



Background Color

A background color can be added to the legend box using the `bg` argument. Below is an example:

```
plot(gdp$year, gdp$india, type = 'l',
      ylim = c(-5, 15), xlab = 'Year',
      ylab = 'Growth Rate', col = 'blue',
      main = 'BRICS: Growth Rate')
lines(gdp$year, gdp$china, col = 'red')
legend(x = 2012, y = 0, legend = c('india', 'china'),
       lty = 1, col = c('blue', 'red'), title = 'Nations',
       bg = 'lightyellow')
```

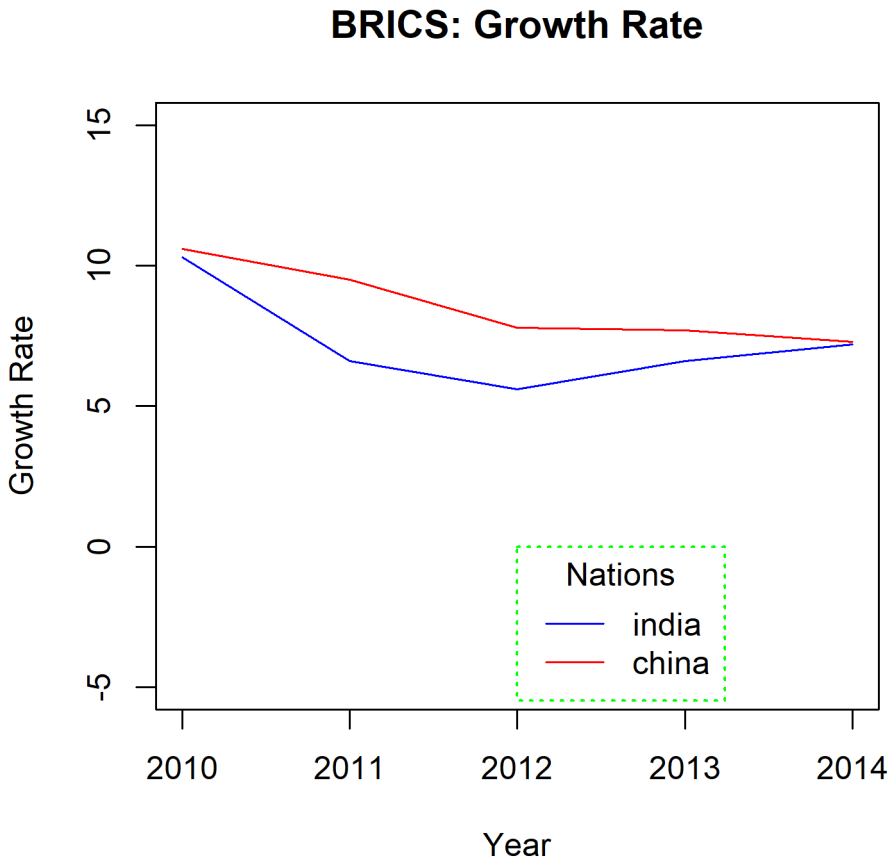


Border

The following arguments can be used to modify the border of the legend box:

- `box.lty`: line type
- `box.lwd`: line width
- `box.col`: color

```
plot(gdp$year, gdp$india, type = 'l',
      ylim = c(-5, 15), xlab = 'Year',
      ylab = 'Growth Rate', col = 'blue',
      main = 'BRICS: Growth Rate')
lines(gdp$year, gdp$china, col = 'red')
legend(x = 2012, y = 0, legend = c('india', 'china'),
       lty = 1, col = c('blue', 'red'), title = 'Nations',
       box.lty = 3, box.lwd = 1.5, box.col = 'green')
```



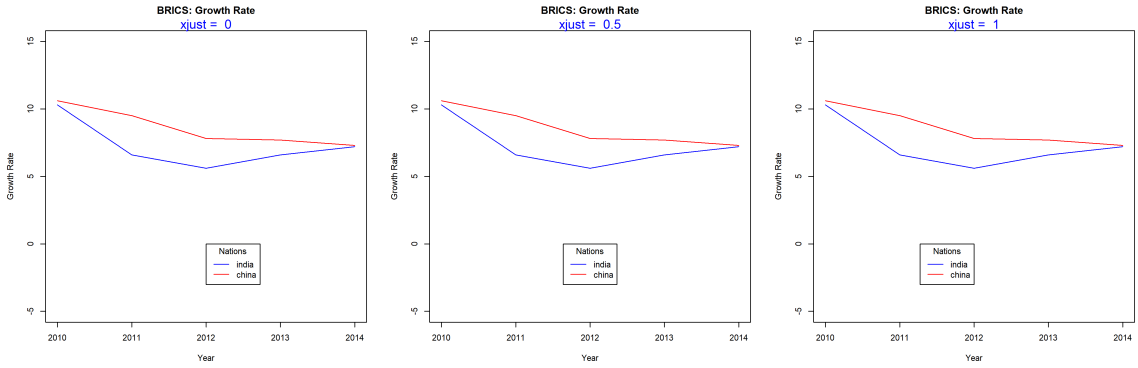
Justification

The `xjust` and `yjust` arguments can be used to position the legend relative to the **X** and **Y** axis respectively. Listed below is the value and the respective justification:

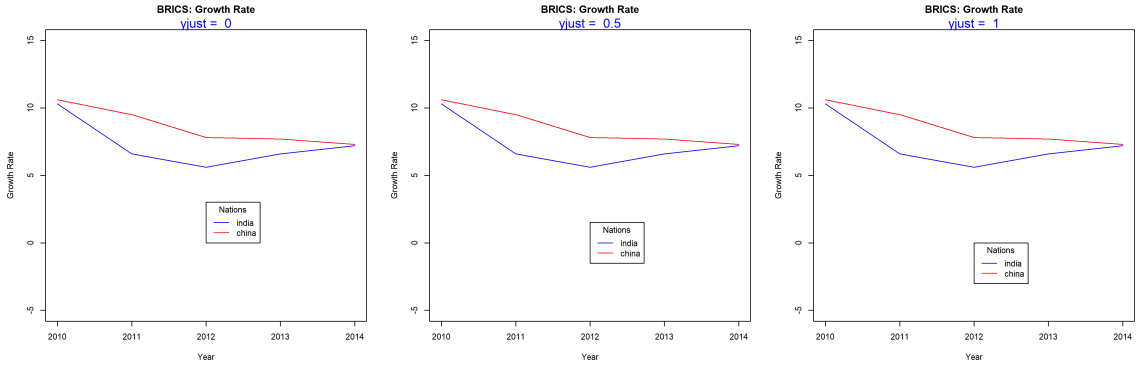
- `0`: left justified
- `0.5`: centered
- `1`: right justified

Let us look at a few examples to understand how it works.

Horizontal Justification



Vertical Justification



Text Appearance

The last topic we will explore is the appearance of the text inside the legend box. We will modify the color and font of text using the `text.col` and `text.font` arguments.

```
{plot(gdp$year, gdp$india, type = 'l',
      ylim = c(-5, 15), xlab = 'Year',
      ylab = 'Growth Rate', col = 'blue',
      main = 'BRICS: Growth Rate')
lines(gdp$year, gdp$china, col = 'red')
legend(x = 2012, y = 0, legend = c('india', 'china'),
       lty = 1, col = c('blue', 'red'), title = 'Nations',
       text.col = 'green', text.font = 3)}
```

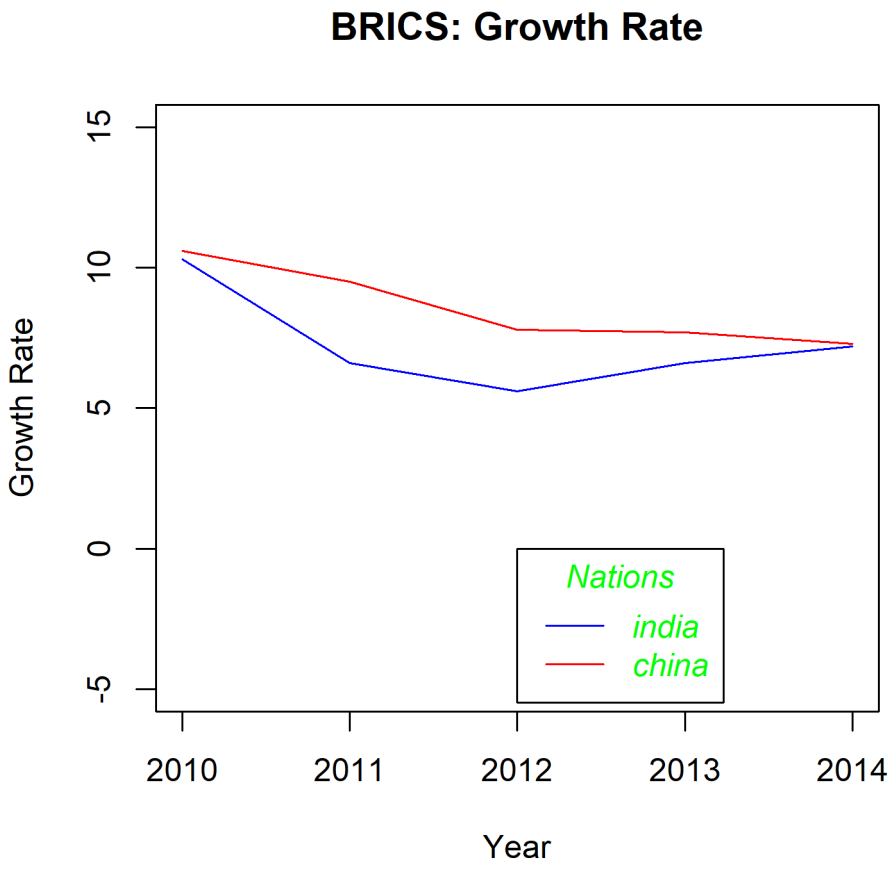


Figure 31: A complete legend with a title, styled keys and horizontal layout

Text Annotations

Introduction

In this chapter, we will learn to add text annotations. There are occasions when you want to display additional information in a plot. This is usually achieved by adding text either inside the plot or on the margins. For example, you might want to label a line/bar or add formulas to better communicate what is shown in the plot. The idea is to use the available space within/outside the plot to provide additional information that can be useful to the end users. We will learn to add text inside as well as on the margins of the plot. This is accomplished using the following two functions:

- `text()` : add text inside the plot
- `mtext()` : add text on the margins of the plot

Syntax

Let us take a quick look at the syntax of both the functions:

```
text(x, y = NULL, labels = seq_along(x$x), adj = NULL,
     pos = NULL, offset = 0.5, vfont = NULL,
     cex = 1, col = NULL, font = NULL, ...)

mtext(text, side = 3, line = 0, outer = FALSE, at = NA,
      adj = NA, padj = NA, cex = NA, col = NA, font = NA, ...)
```

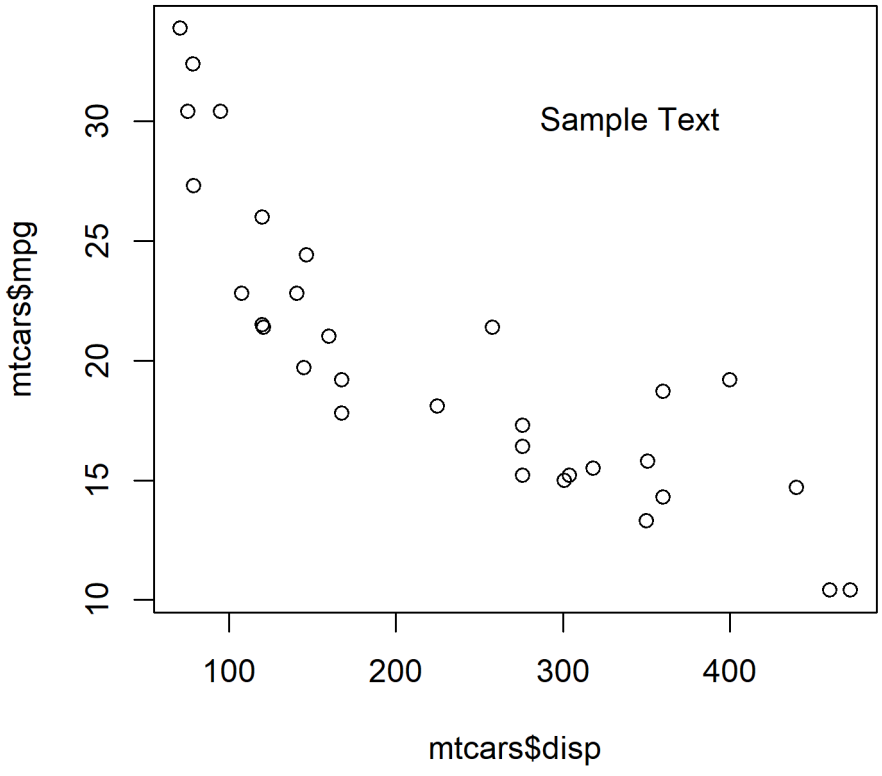
Text Inside the Plot

To add text inside a plot, the following arguments must be supplied to the `text()` function:

- `labels` : the text to be displayed
- `x` : x axis coordinate
- `y` : y axis coordinate

Below is a simple example:

```
plot(mtcars$disp, mtcars$mpg)
text(x = 340, y = 30, labels = 'Sample Text')
```

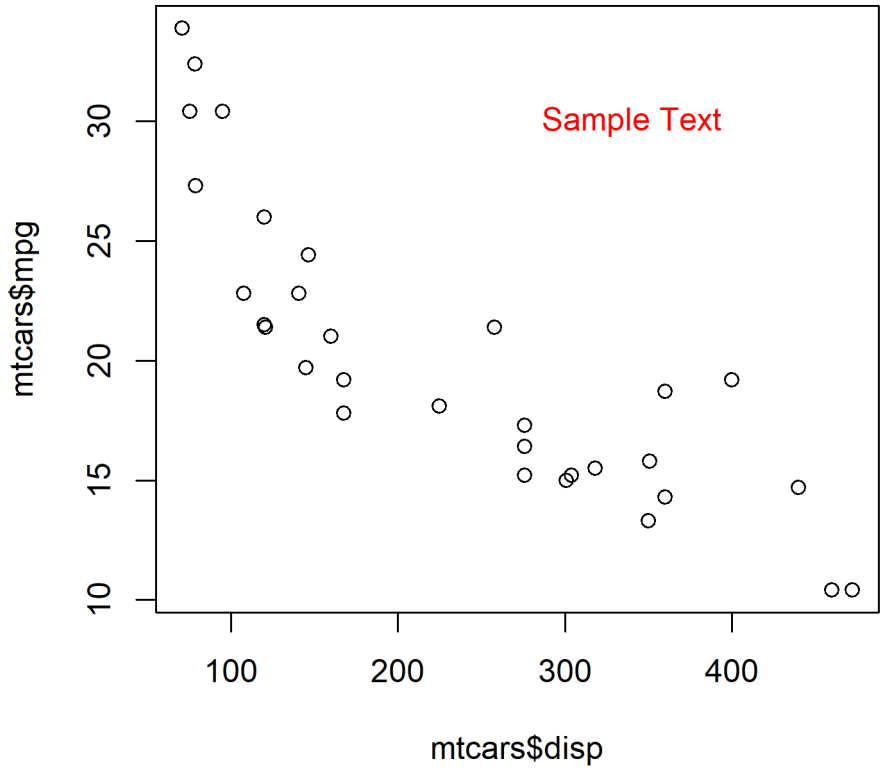


The text appears at the coordinates (340, 30). Ensure that the text is enclosed in single/double quotes and the coordinates provided are within the range of the **X** and **Y** axis variables.

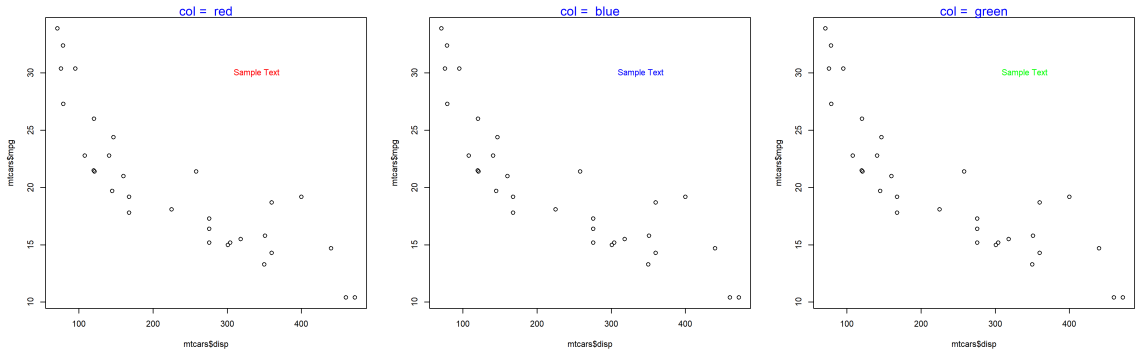
Color

The color of the text can be modified using the `col` argument in the `text()` function.

```
plot(mtcars$disp, mtcars$mpg)
text(x = 340, y = 30, labels = 'Sample Text', col = 'red')
```



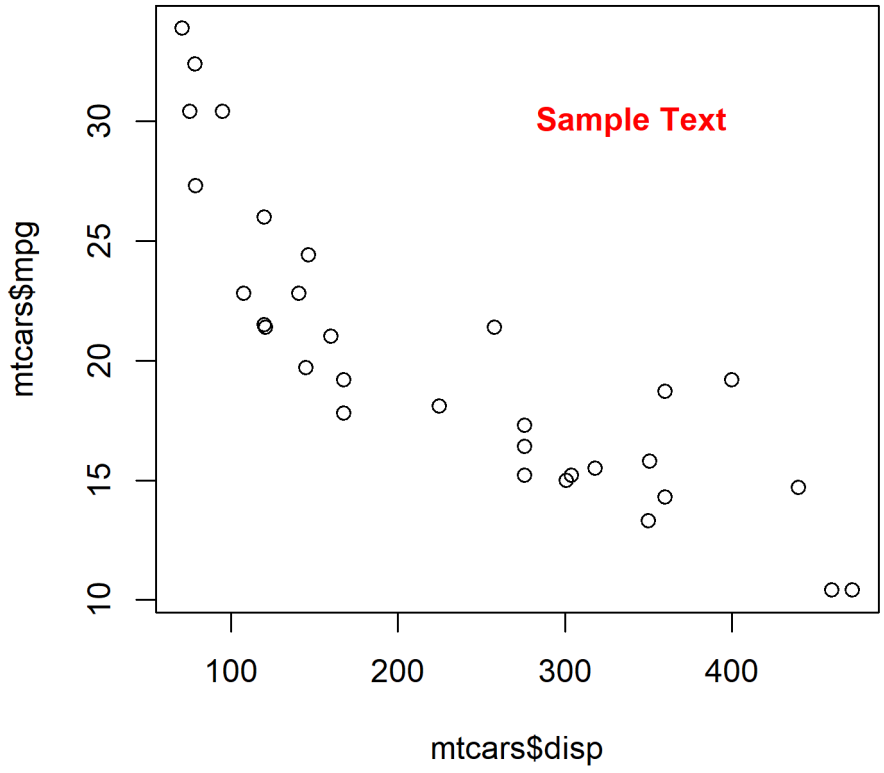
The below plot depicts the appearance of the text for different values of the `col` argument:



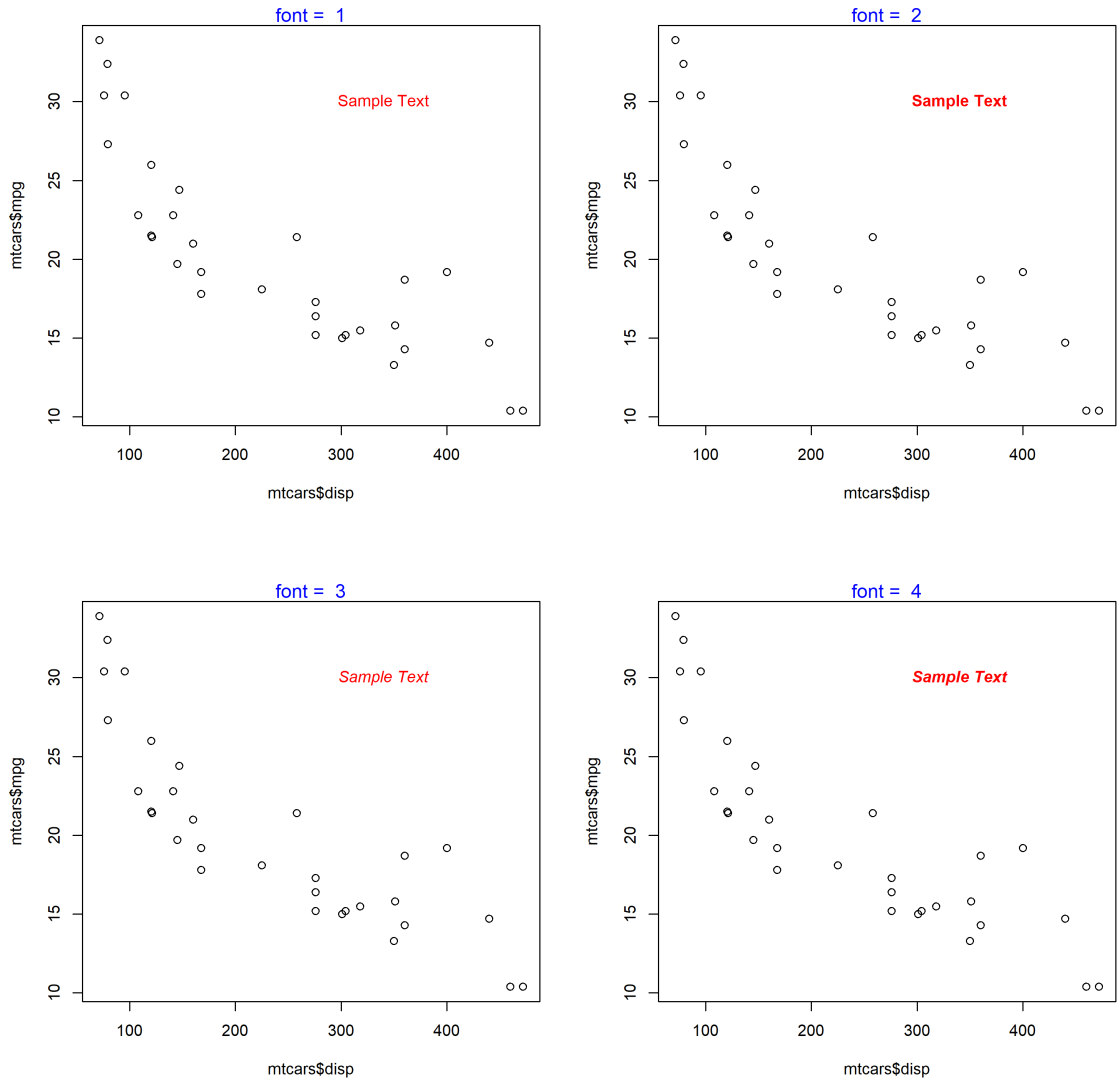
Font

The font of the text can be modified using the `font` argument in the `text()` function.

```
plot(mtcars$disp, mtcars$mpg)
text(x = 340, y = 30, labels = 'Sample Text', col = 'red', font = 2)
```



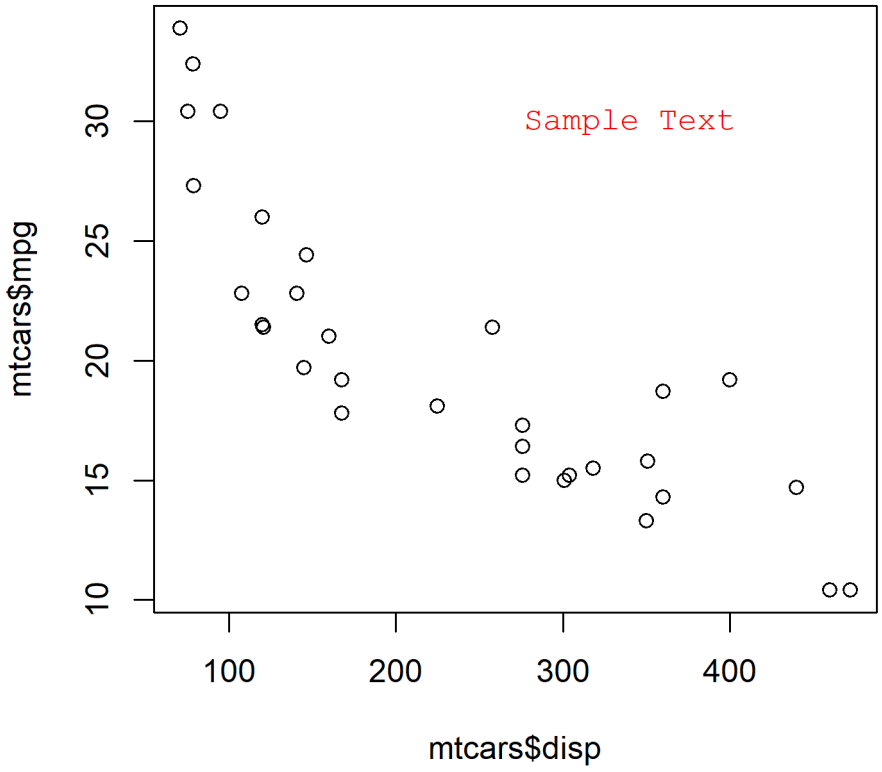
The below plot depicts the appearance of the text for different values of the `font` argument:



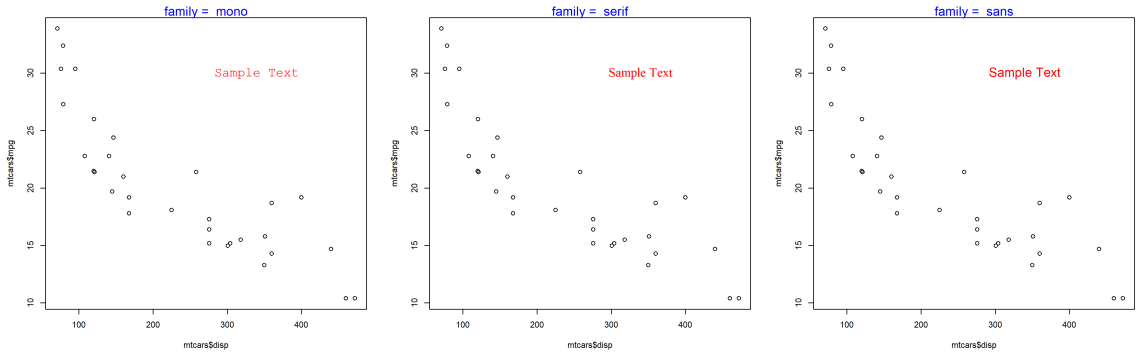
Font Family

The font family of the text can be modified using the `family` argument in the `text()` function.

```
plot(mtcars$disp, mtcars$mpg)
text(x = 340, y = 30, labels = 'Sample Text', col = 'red', family = 'mono')
```



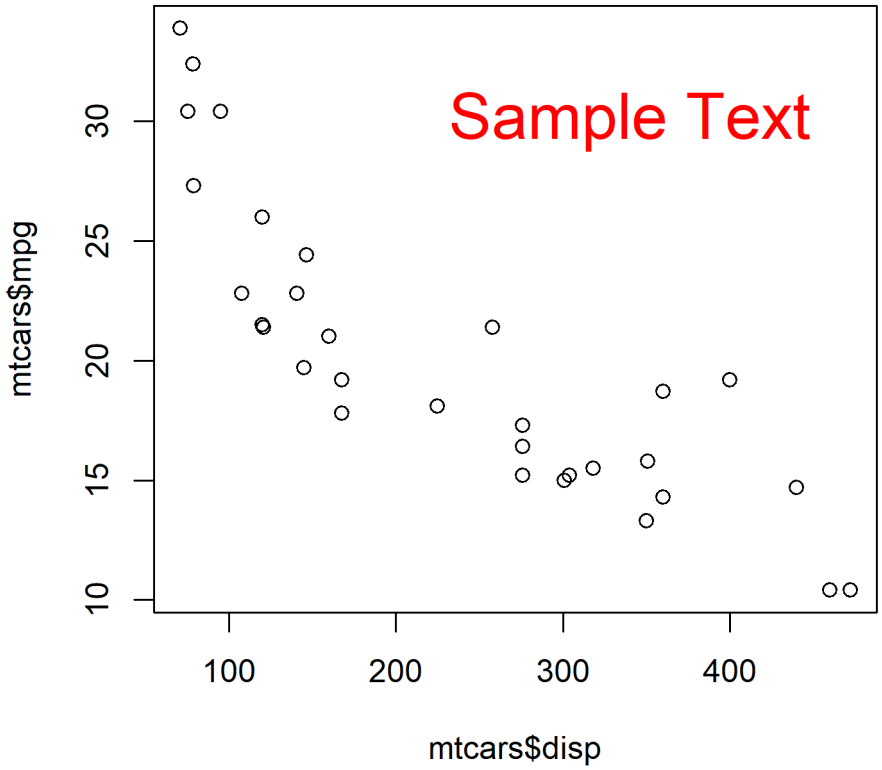
The below plot depicts the appearance of the text for different values of the `family` argument:



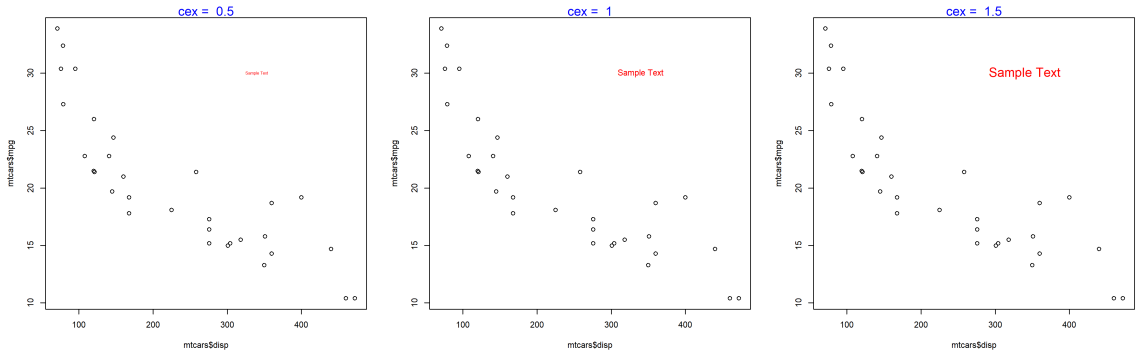
Font Size

The font size of the text can be modified using the `cex` argument in the `text()` function.

```
plot(mtcars$disp, mtcars$mpg)
text(x = 340, y = 30, labels = 'Sample Text', col = 'red', cex = 2)
```



The below plot depicts the appearance of the text for different values of the `cex` argument:

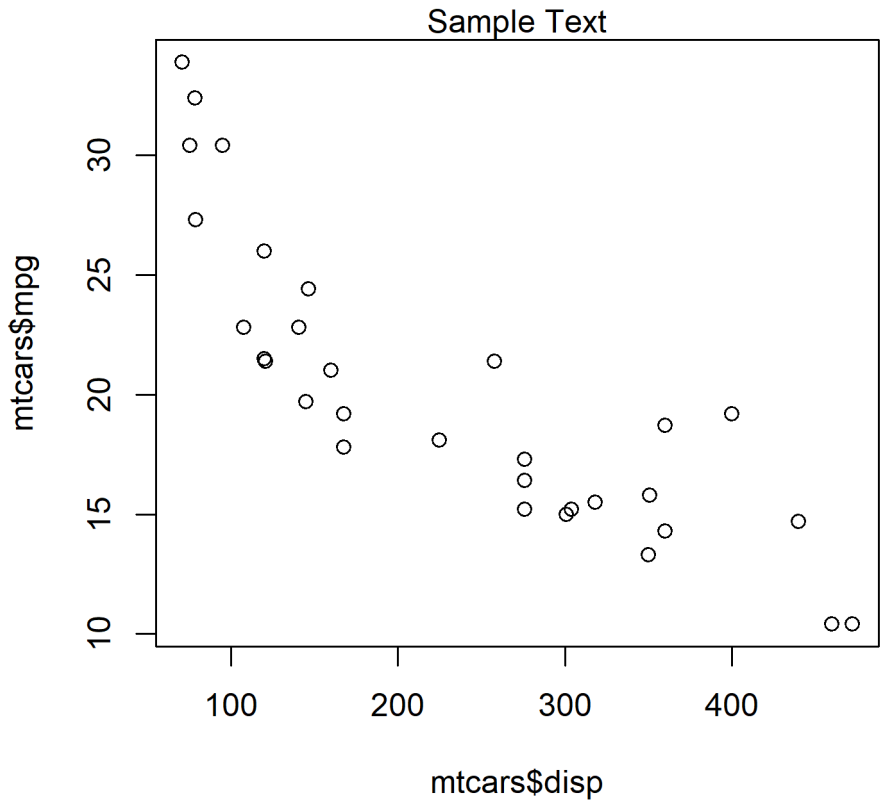


Text on the Margins

The `mtext()` function allows the user to place the text on the margins of the plot. It allows the user to modify the location of the text in multiple ways and we will explore them one by one. To

begin with, let us add text to the plot using the `mtext()` function. The minimum input you need to provide is the text itself. Below is a simple example:

```
plot(mtcars$disp, mtcars$mpg)
mtext('Sample Text')
```

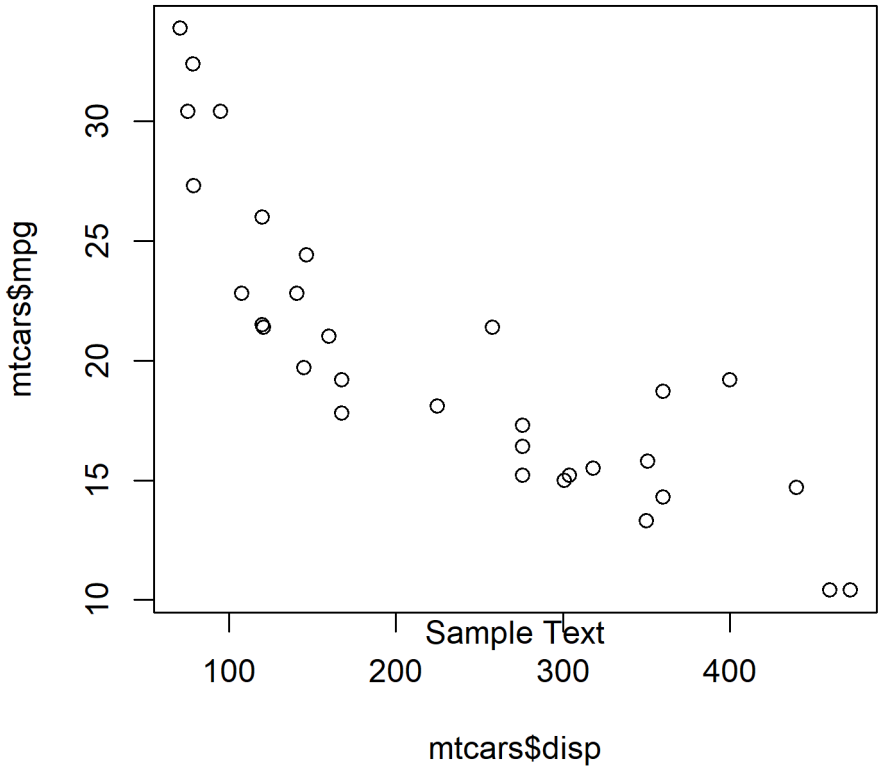


As you can see, the text is placed on the margin of the plot and not inside the plot. Next, we will specify the margin on which to place the text.

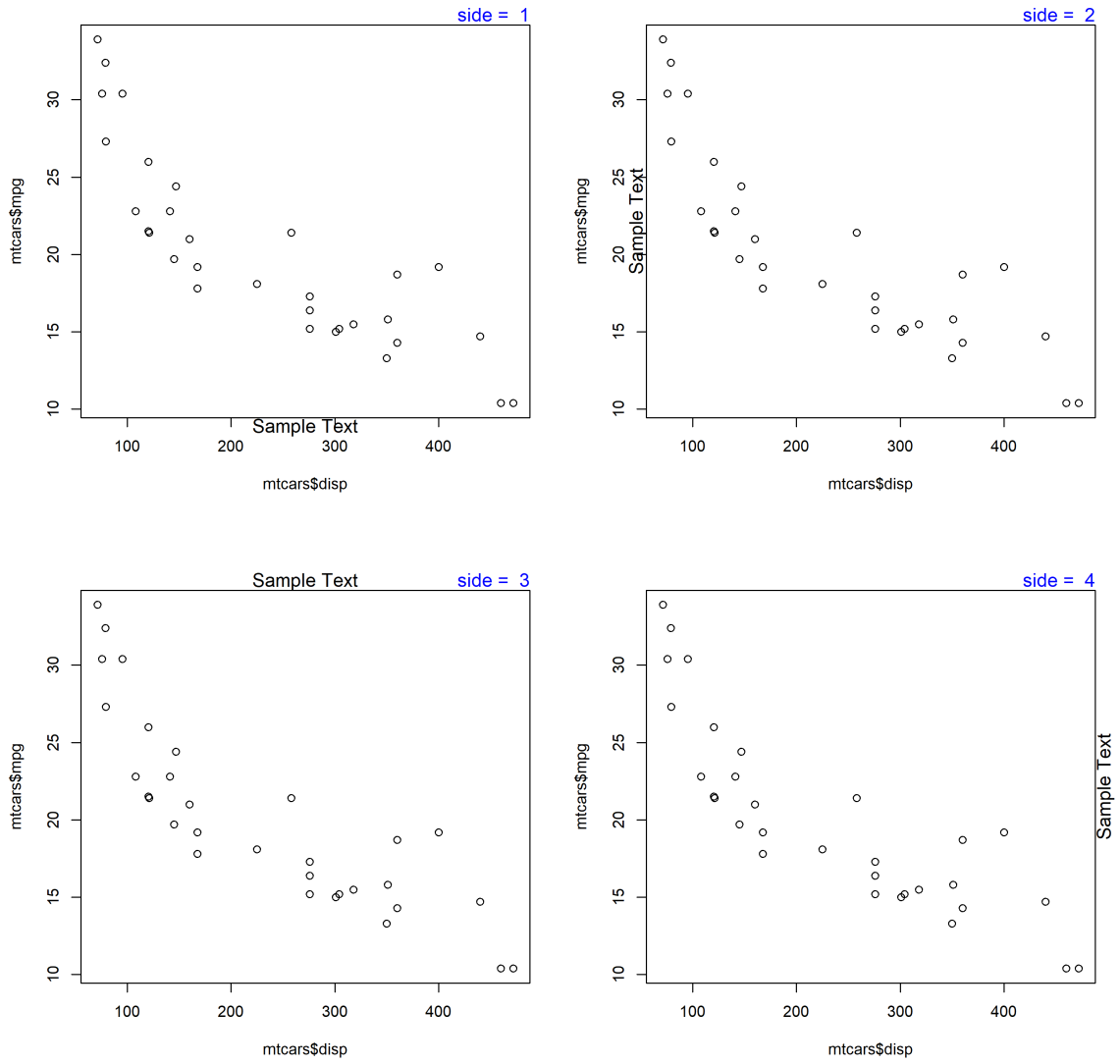
Specify Margin

Use the `side` argument to specify the margin on which you want to place the text. If takes values 1 to 4, each representing one side of the plot.

```
plot(mtcars$disp, mtcars$mpg)
mtext('Sample Text', side = 1)
```



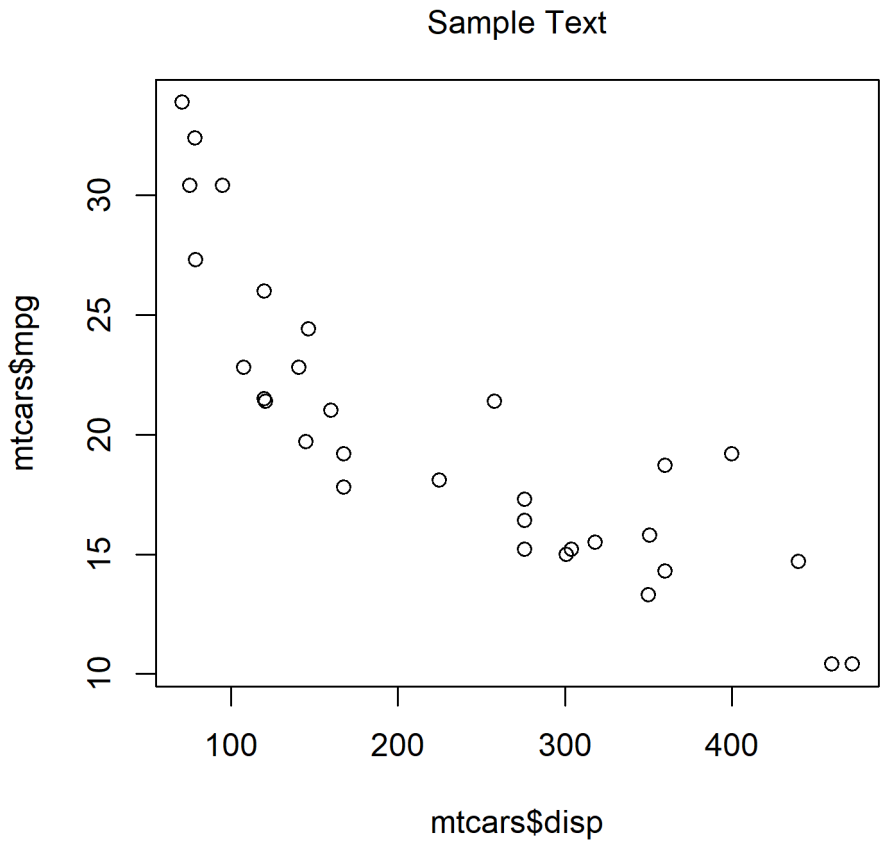
The below plot displays the appearance of the text when differnt options for `side` argument are supplied:



Line

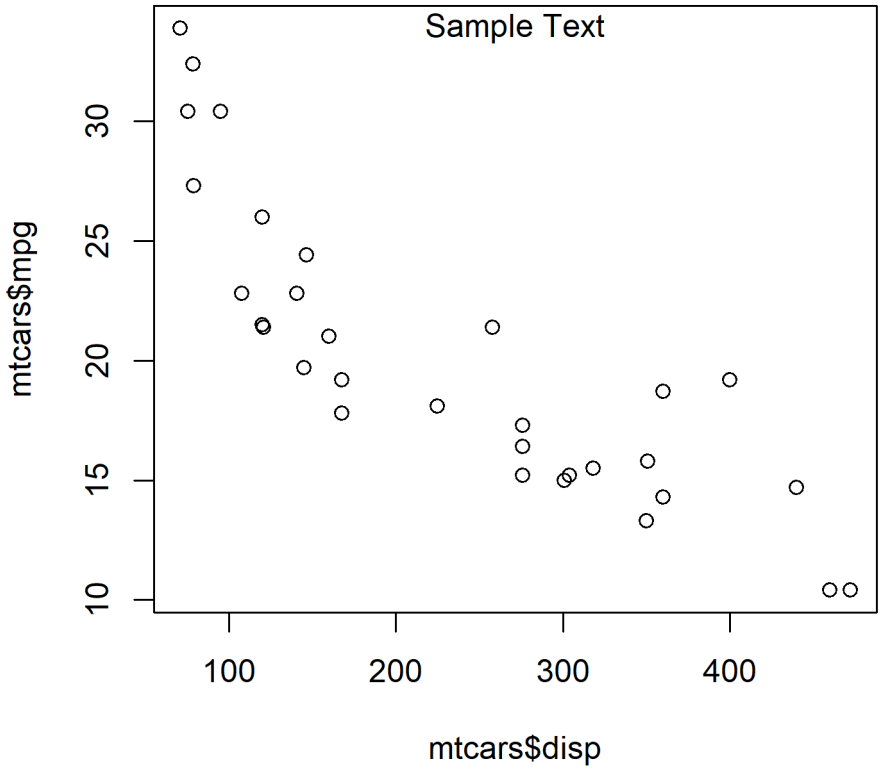
The `line` argument places the text at a specified distance from the margin. The default value is `0`. As the value increases, the text is placed farther from the margin and outside the plot. As the value decreases, the text is placed inside the plot and farther from the margin. Below is an example where the text is placed outside the plot as the value is greater than 1.

```
plot(mtcars$disp, mtcars$mpg)
mtext('Sample Text', line = 1)
```

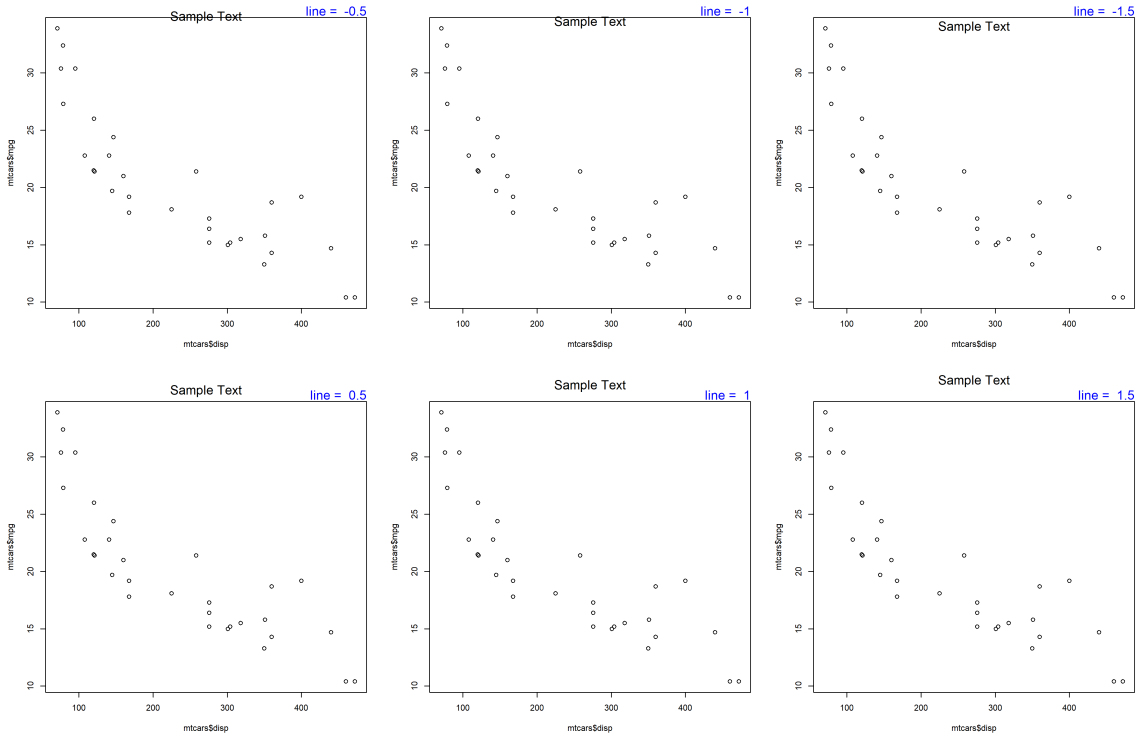


When the value is less than `0`, the line argument places the text inside the plot.

```
plot(mtcars$disp, mtcars$mpg)
mtext('Sample Text', line = -1)
```



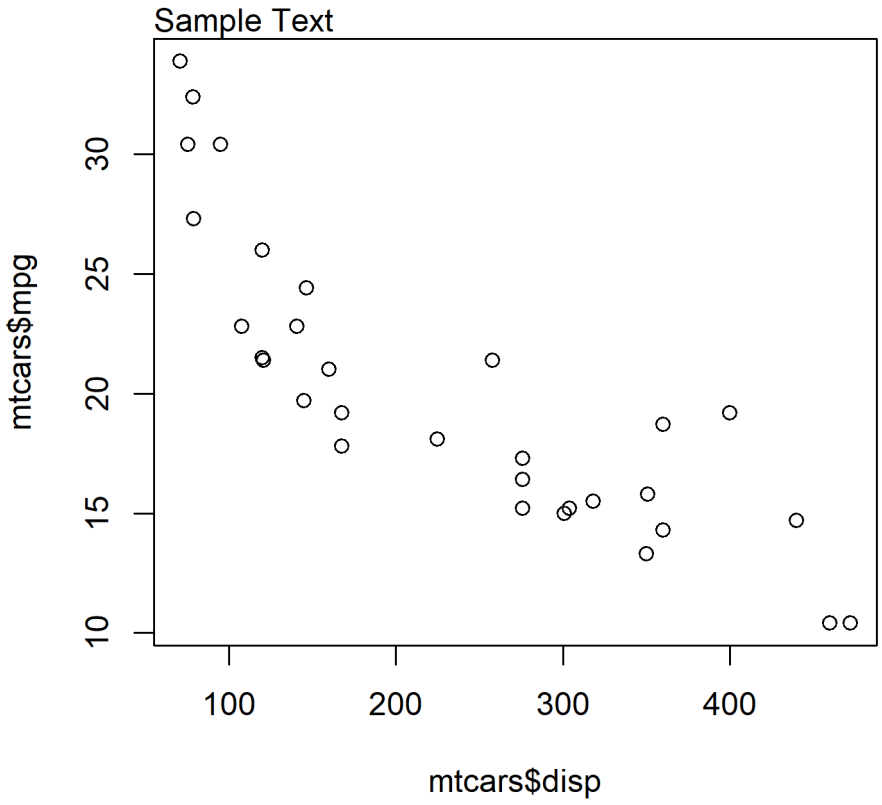
The below plot displays the appearance of the text when different values are supplied to the `line` argument:



Alignment

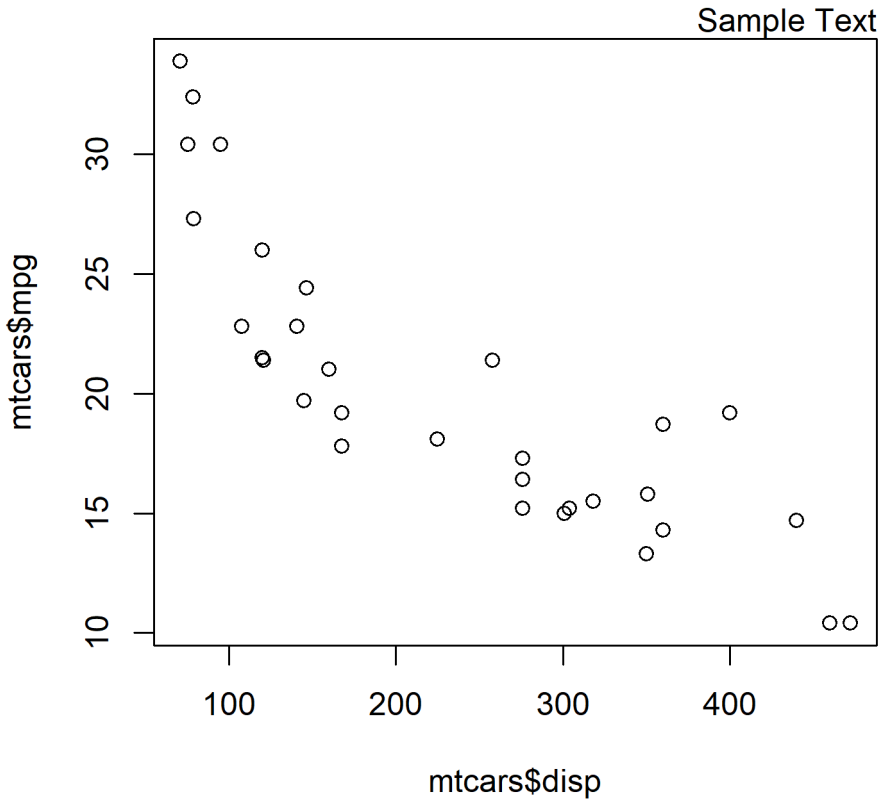
The `adj` argument is used for horizontal alignment of the text. It takes values between 0 and 1. If set to `0`, the text will be left aligned and at `1`, it will be right aligned. Below is an example where the text is left aligned as `adj` is set to `0`.

```
plot(mtcars$disp, mtcars$mpg)
mtext('Sample Text', adj = 0)
```

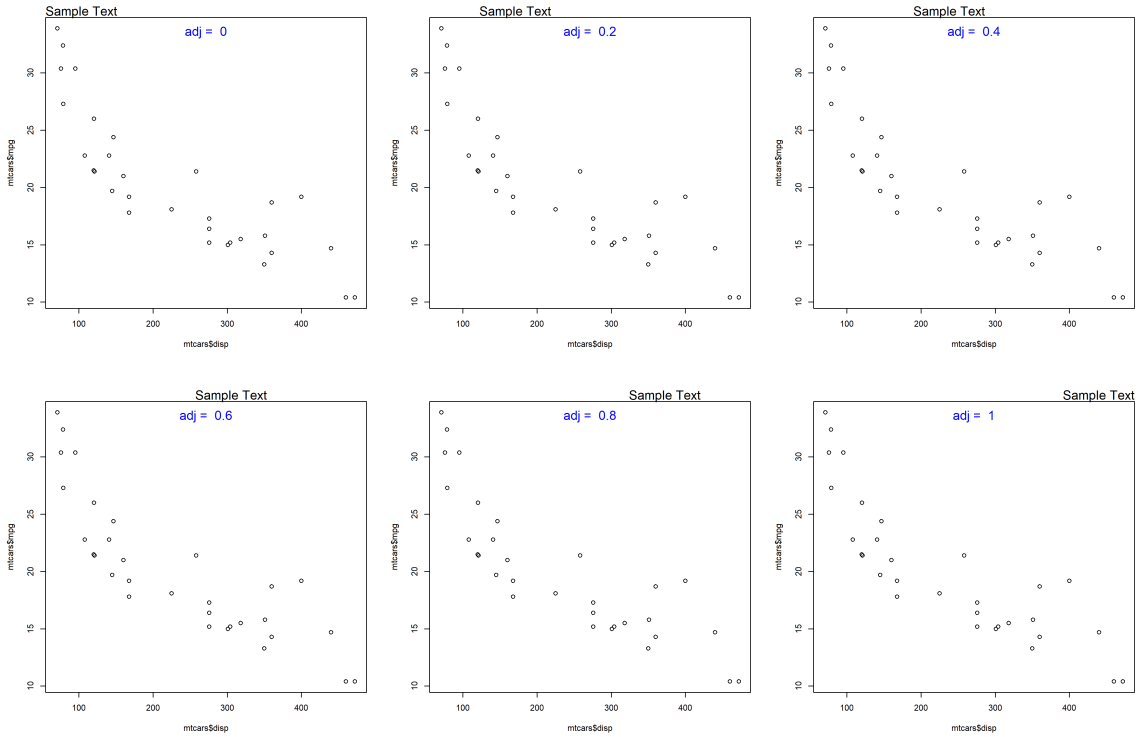


When the value is set to 1, the text is right aligned.

```
plot(mtcars$disp, mtcars$mpg)
mtext('Sample Text', adj = 1)
```



The below plot displays the appearance of the text when different values are supplied to the `adj` argument:



Position

The `pos` argument of the `text()` function places the text relative to the specified coordinates without needing `adj`. It takes values 1 to 4, each representing a position around the point:

- 1: below
- 2: to the left
- 3: above
- 4: to the right

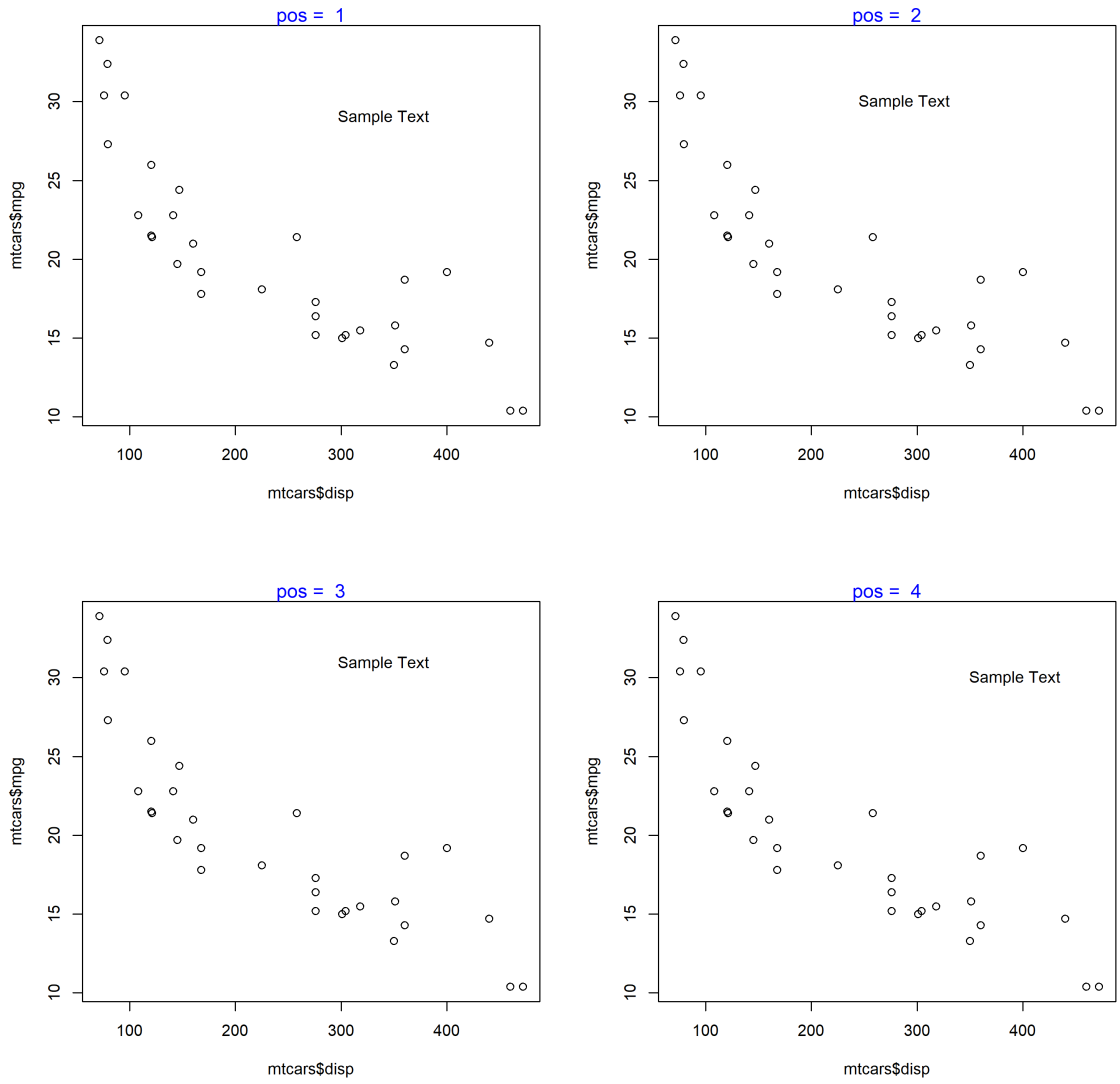


Figure 32: The pos argument placing text relative to a point

Offset

The `offset` argument of the `text()` function controls how far the text is placed from the coordinates when `pos` is used. The distance is measured in character widths and the default value is `0.5`.

```
plot(mtcars$disp, mtcars$mpg)
text(x = 340, y = 30, labels = 'Sample Text', pos = 3, offset = 1.5)
```

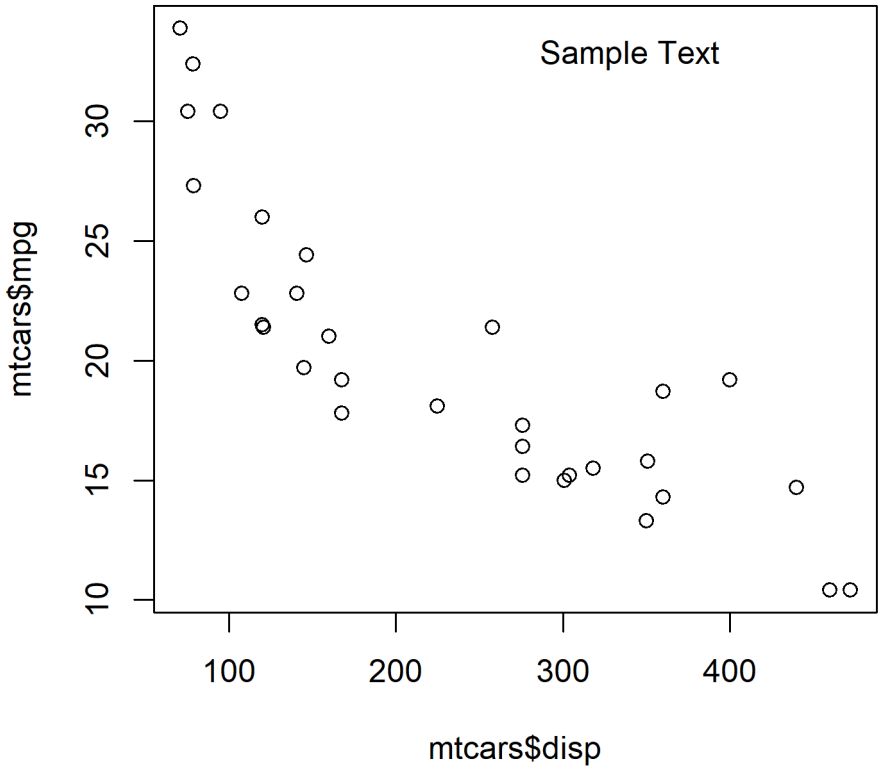
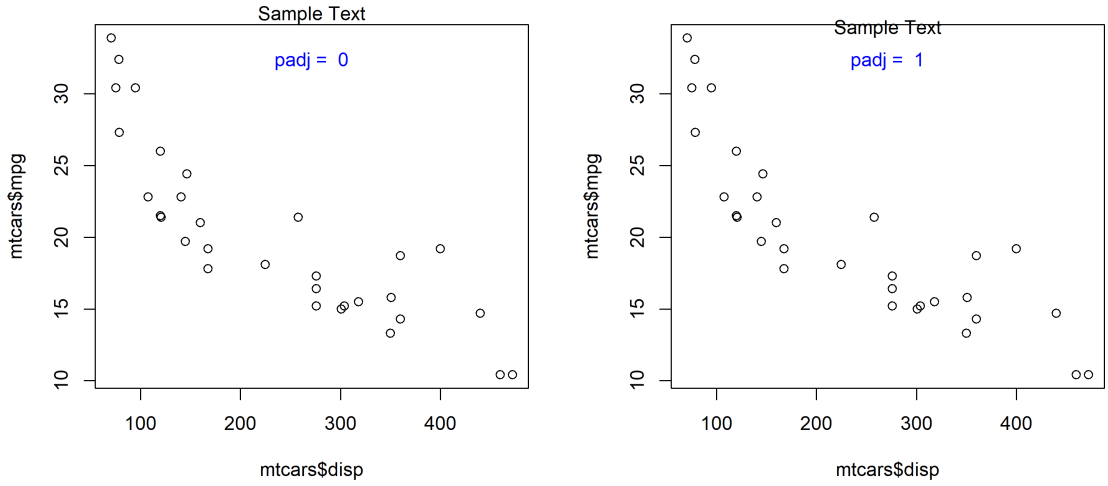


Figure 33: The offset argument pushing a label away from its point

Perpendicular Adjustment

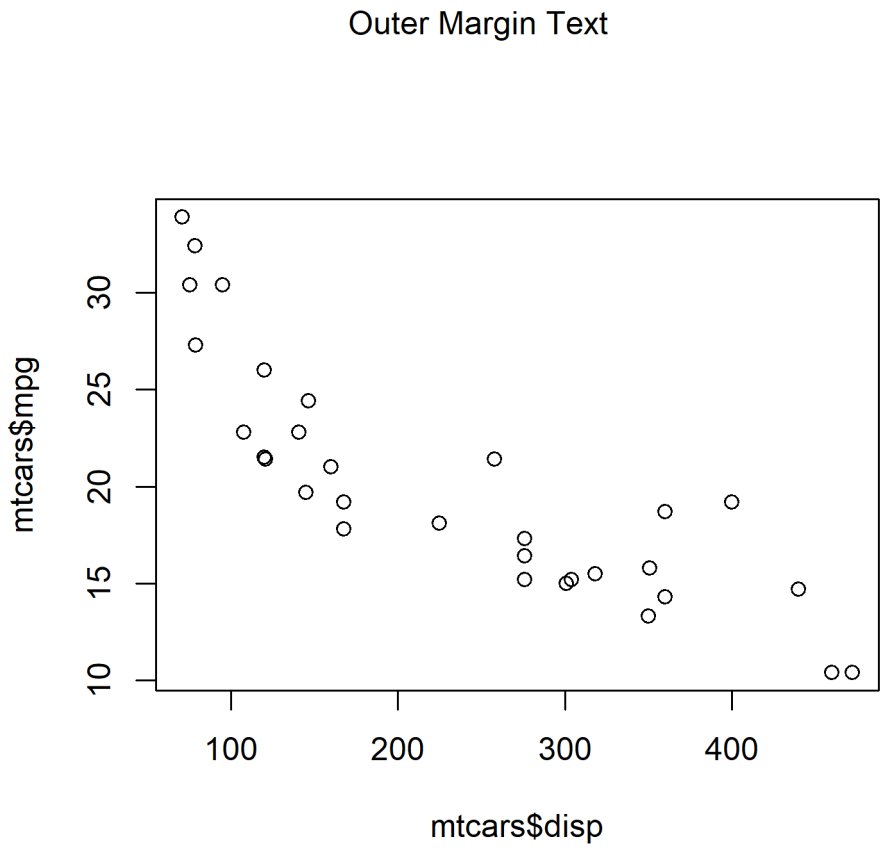
The `padj` argument of the `mtext()` function fine-tunes the placement of the text in the direction perpendicular to `adj`. It takes values between 0 and 1. Compare the two plots below and observe the nudge:



Outer Margin

By default, `mtext()` places text on the inner margin of the current plot. Set `outer = TRUE` to place the text on the outer margin of the whole figure region instead. This is useful with multi-panel layouts, but first reserve outer space with `par(oma = ...)`:

```
init <- par(no.readonly = TRUE)
par(oma = c(0, 0, 3, 0))
plot(mtcars$disp, mtcars$mpg)
mtext('Outer Margin Text', outer = TRUE)
```




```
par(init)
```

Exact Position

The `at` argument of the `mtext()` function places the text at an exact position along the margin, expressed in the user coordinates of the corresponding axis. Below, the text is anchored at `disp = 200` on the bottom margin instead of the default center:

```
plot(mtcars$disp, mtcars$mpg)
mtext('Sample Text', side = 1, at = 200)
```

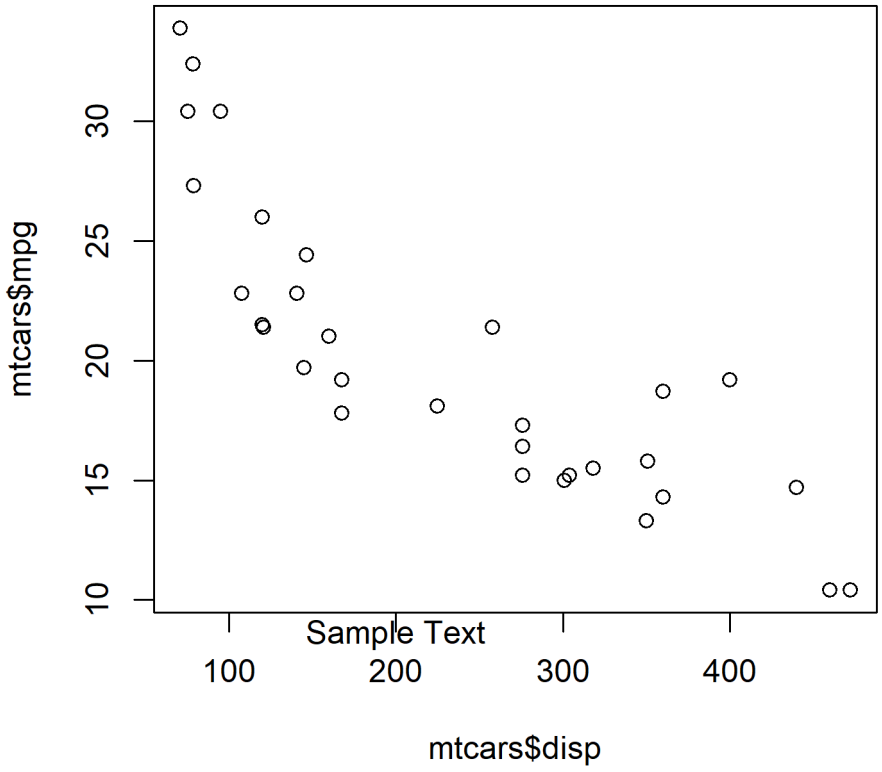


Figure 34: The at argument placing margin text at specific coordinates

Faceting

Introduction

In this chapter, we will learn how to combine multiple plots. Often, it is useful to have multiple plots in the same frame as it allows us to get a comprehensive view of a particular variable or compare among different variables. The Graphics package offers two methods to combine multiple plots. `par()` can be used to set graphical parameters regarding plot layout using the mfcol and mfrow arguments. `layout()` serves the same purpose but offers more flexibility by allowing us to modify the height and width of rows and columns.

`par()` allows us to customize the graphical parameters(title, axis, font, color, size) for a particular session. For combining multiple plots, we can use the graphical parameters mfrow and mfcol. These two parameters create a matrix of plots filled by rows and columns respectively. Let us combine plots using both the above parameters.

Option	Description	Arguments
mfrow	Fill by rows	Number of rows and columns
mfcol	Fill by columns	Number of rows and columns

Row

The `mfrow` parameter combines plots filled by rows i.e it takes two values, the number of rows and number of columns and then starts filling the plots by row. Below is the syntax for mfrow.

```
# mfrow syntax
par(mfrow = c(number of rows, number of columns))
```

Let us begin by combining 4 plots in 2 rows and 2 columns:

Case Study 1

Let us begin by combining 4 plots in 2 rows and 2 columns. The plots will be filled by rows as we are using the mfrow parameter:

```
# store the current parameter settings in init
init <- par(no.readonly=TRUE)

# specify that 4 graphs to be combined and filled by rows
par(mfrow = c(2, 2))

# specify the graphs to be combined
plot(mtcars$mpg)
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)

# restore the setting stored in init
par(init)
```

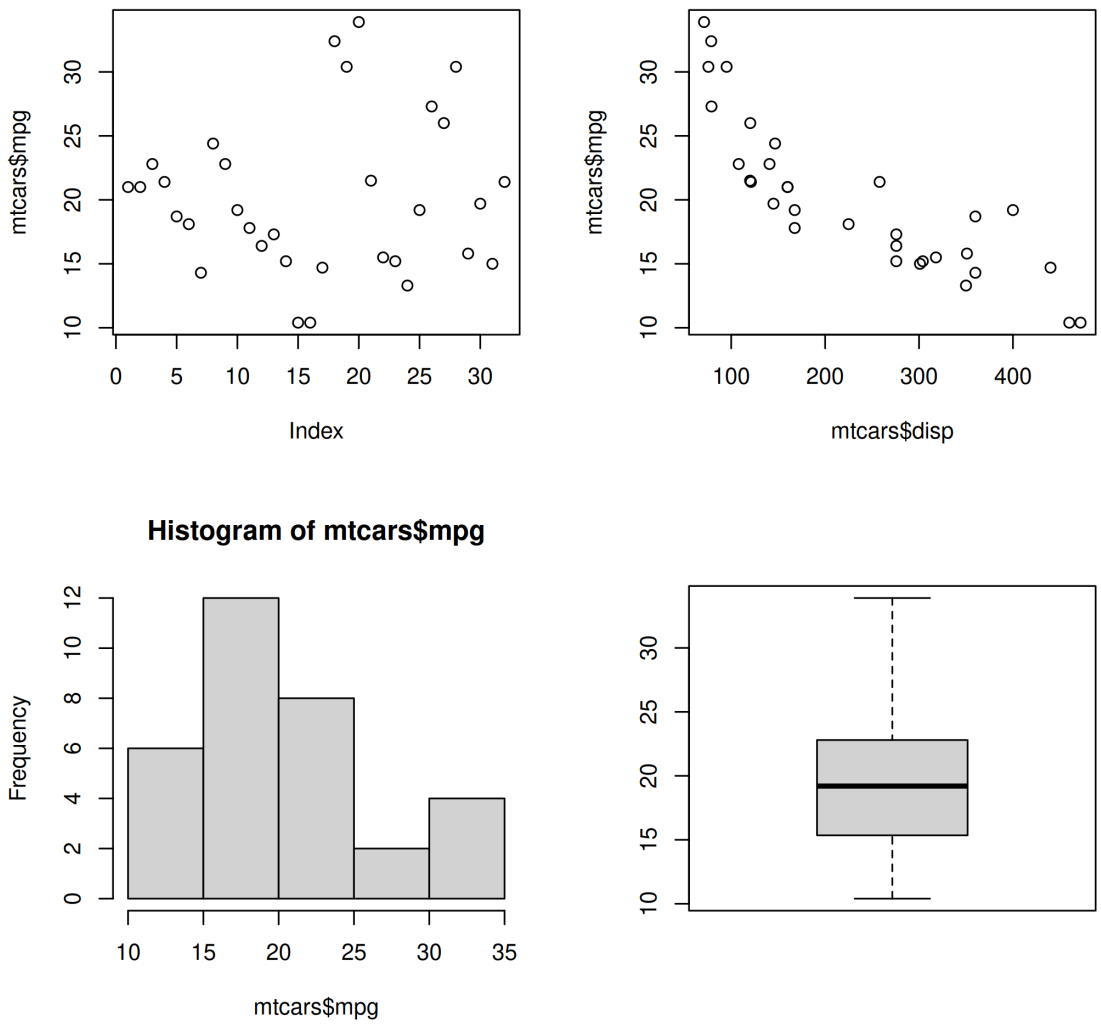


Figure 35: A 2 by 2 panel grid drawn with par(mfrow)

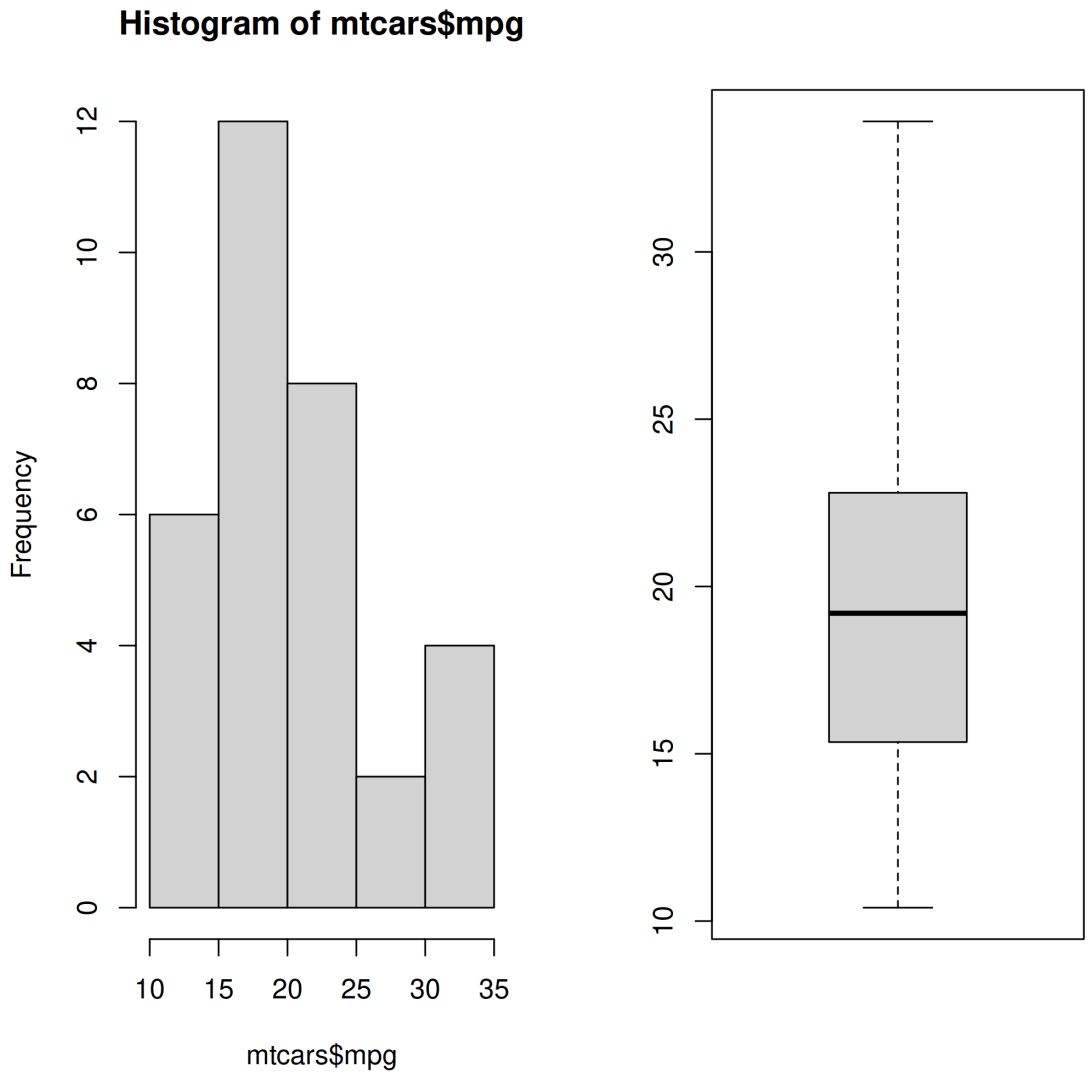
Case Study 2

Combine 2 plots in 1 row and 2 columns.

```
# store the current parameter settings in init
init <- par(no.readonly=TRUE)

# specify that 2 graphs to be combined and filled by rows
par(mfrow = c(1, 2))

# specify the graphs to be combined
hist(mtcars$mpg)
boxplot(mtcars$mpg)
```




```
# restore the setting stored in init
par(init)
```

Case Study 3

Combine 2 plots in 2 rows and 1 column.

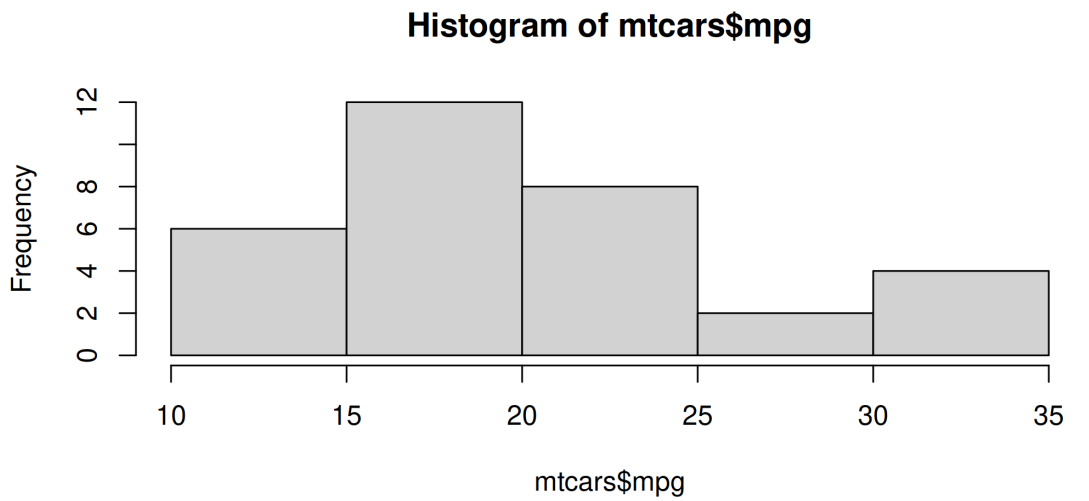
```
# store the current parameter settings in init
init <- par(no.readonly=TRUE)

# specify that 2 graphs to be combined and filled by rows
par(mfrow = c(2, 1))

# specify the graphs to be combined
```

```r
hist(mtcars$mpg)
boxplot(mtcars$mpg)

# restore the setting stored in init
par(init)
```



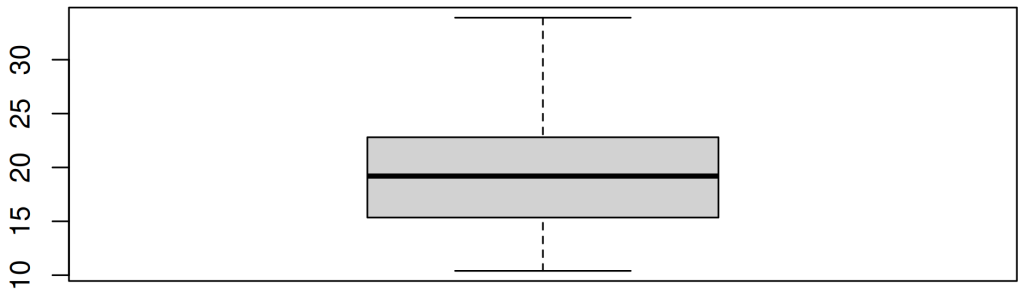


Figure 36: The same grid drawn with par(mfcol), filling by column

Case Study 4

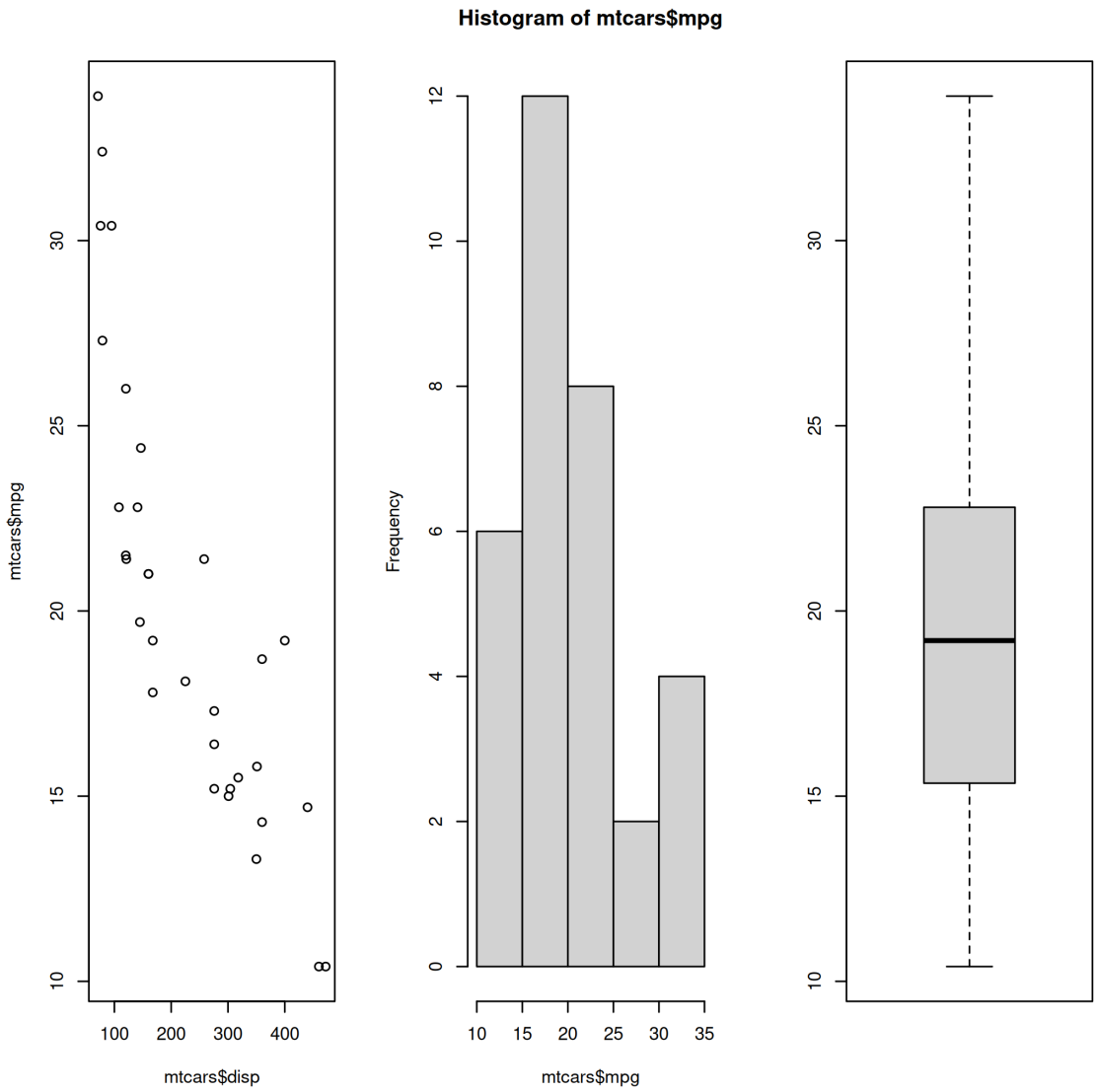
Combine 3 plots in 1 row and 3 columns.

```r
# store the current parameter settings in init
init <- par(no.readonly=TRUE)

# specify that 3 graphs to be combined and filled by rows
```

```r
par(mfrow = c(1, 3))

# specify the graphs to be combined
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)
```




```r
# restore the setting stored in init
par(init)
```

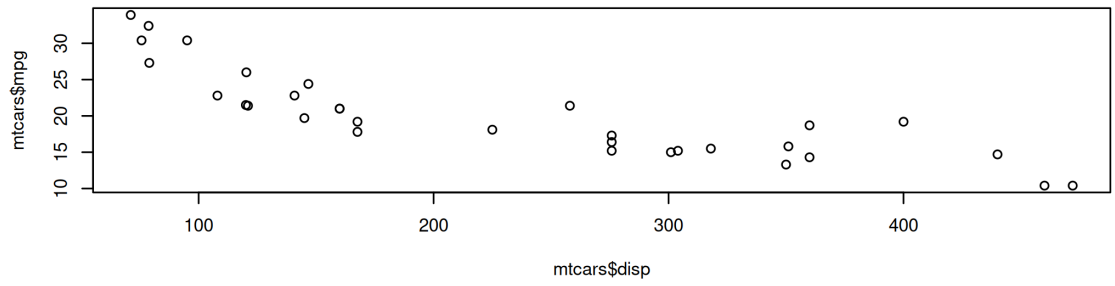
Case Study 5

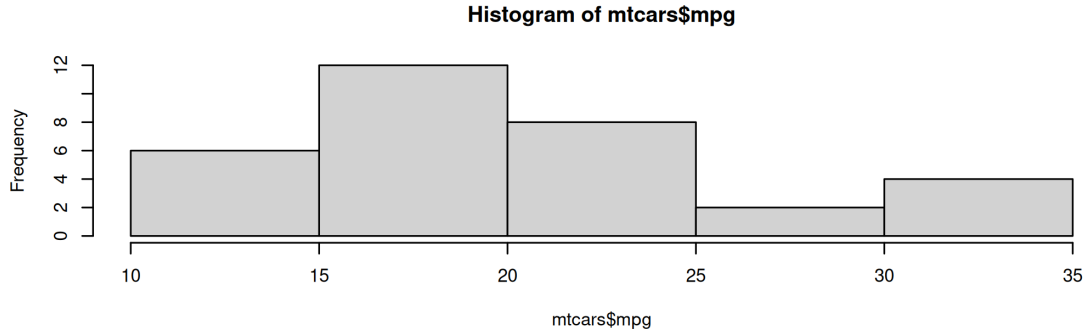
Combine 3 plots in 3 rows and 1 column.

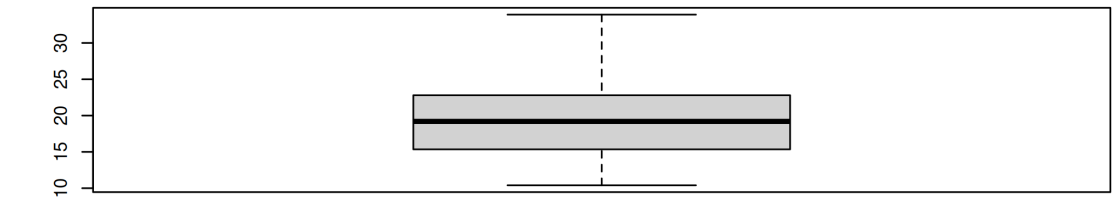
```r
# store the current parameter settings in init
init <- par(no.readonly=TRUE)

# specify that 3 graphs to be combined and filled by rows
par(mfrow = c(3, 1))

# specify the graphs to be combined
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)
```








```r
# restore the setting stored in init
par(init)
```

Column

mfcol combines plots filled by columns i.e it takes two arguments, the number of rows and number of columns and then starts filling the plots by columns. Below is the syntax for mfcol:

```
# mfcol syntax
par(mfcol = c(number of rows, number of columns))
```

Let us begin by combining 4 plots in 2 rows and 2 columns:

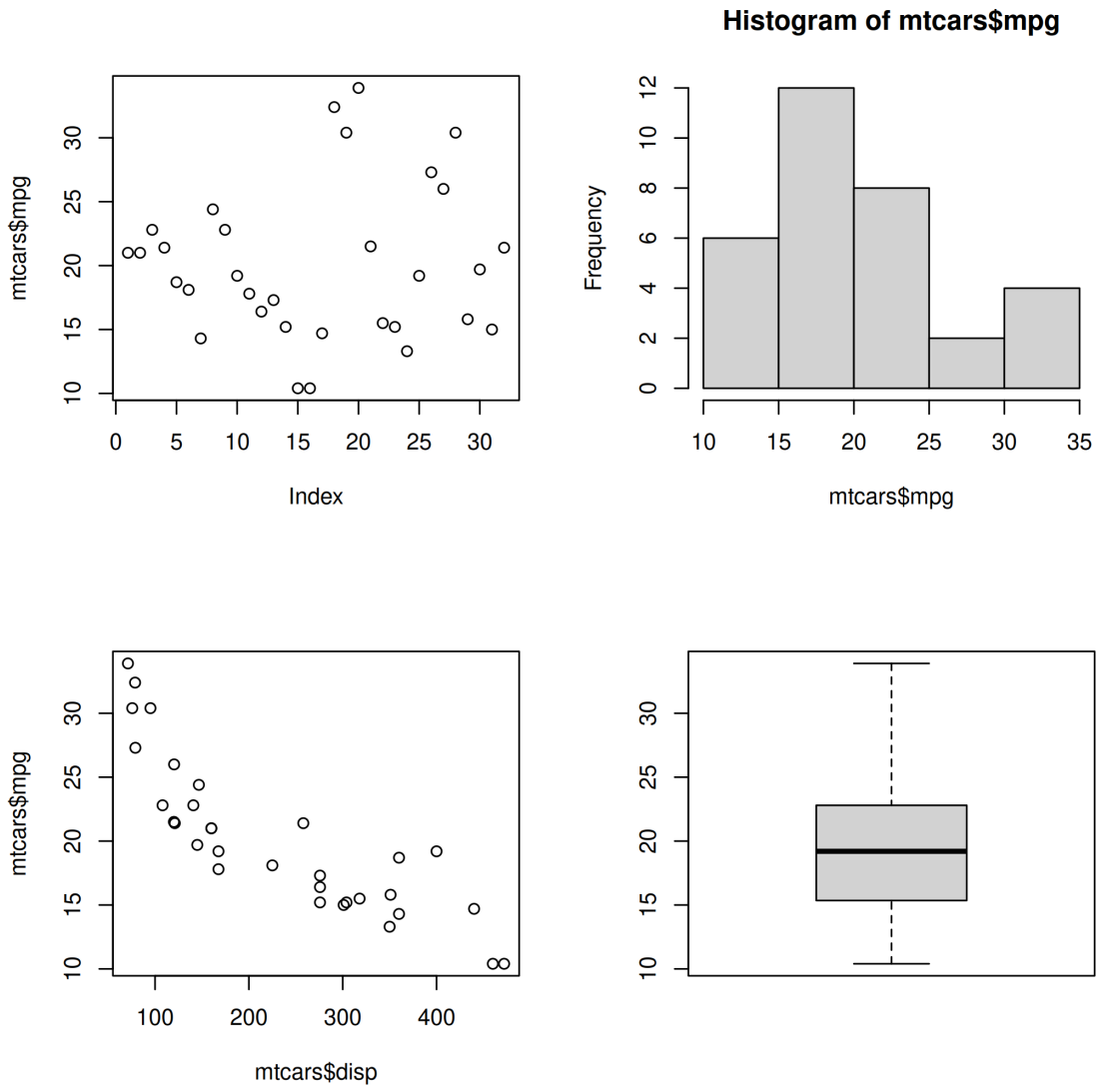
Case Study 6

Combine 4 plots in 2 rows and 2 columns, filled by columns.

```
# store the current parameter settings in init
init <- par(no.readonly=TRUE)

# specify that 4 graphs to be combined and filled by columns
par(mfcol = c(2, 2))

# specify the graphs to be combined
plot(mtcars$mpg)
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)
```




```
# restore the setting stored in init
par(init)
```

Special Cases

What happens if we specify lesser or more number of graphs? In the next two examples, we will specify lesser or more number of graphs than we ask the par() function to combine. Let us see what happens in such instances:

Case 1: Lesser number of graphs specified We will specify that 4 plots need to be combined in 2 rows and 2 columns but provide only 3 graphs.

Case 2: Extra graph specified We will specify that 4 plots need to be combined in 2 rows and 2 columns but specify 6 graphs instead of 4.

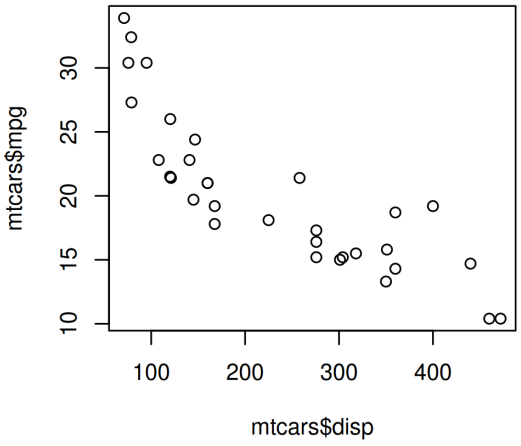
Case Study 7

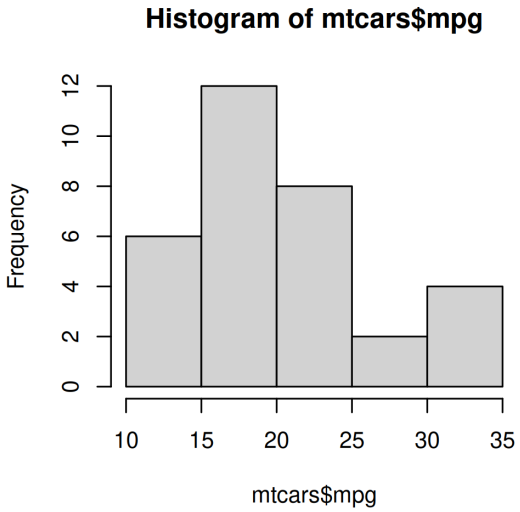
```
# store the current parameter settings in init
init <- par(no.readonly=TRUE)

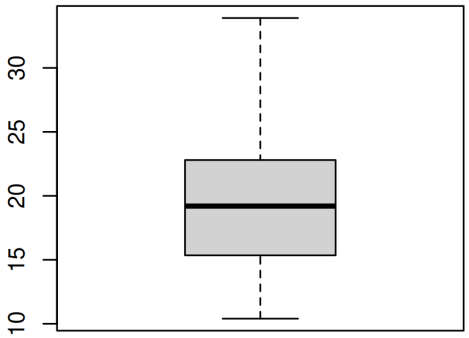
# specify that 4 graphs to be combined and filled by rows
par(mfrow = c(2, 2))

# specify the graphs to be combined
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)

# restore the setting stored in init
par(init)
```





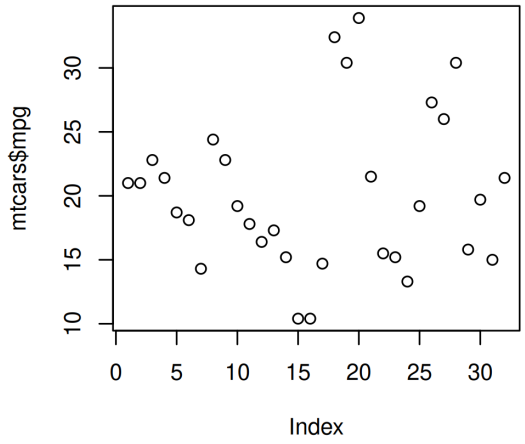


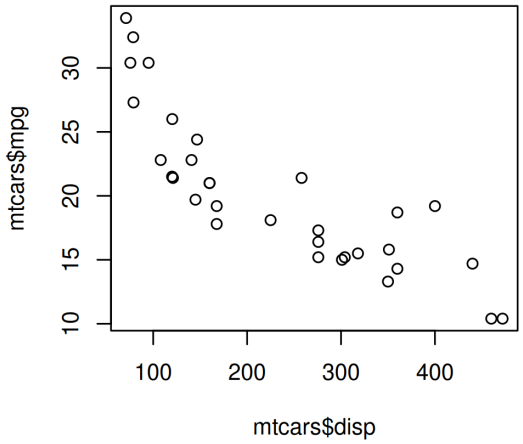
Case Study 8

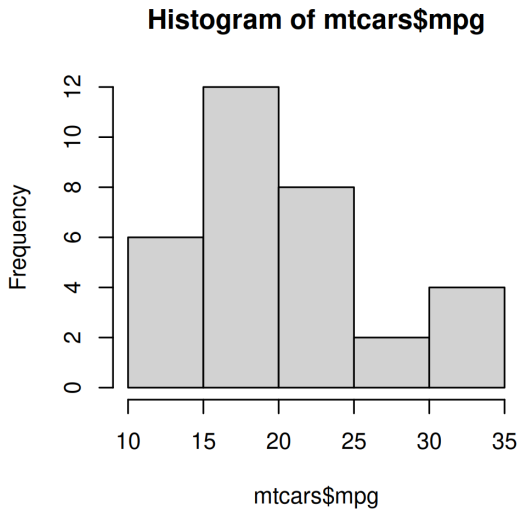
```
# store the current parameter settings in init
init <- par(no.readonly=TRUE)

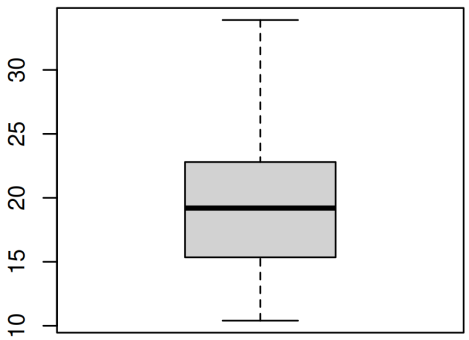
# specify that 4 graphs to be combined and filled by rows
par(mfrow = c(2, 2))

# specify the graphs to be combined
plot(mtcars$mpg)
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)
```



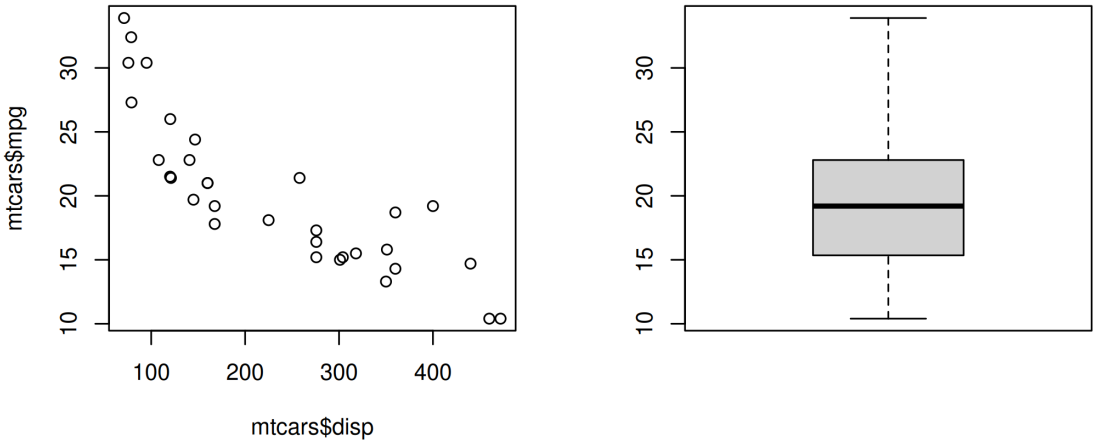






```
plot(mtcars$disp, mtcars$mpg)
boxplot(mtcars$mpg)

# restore the setting stored in init
par(init)
```



Layout

At the core of the layout() function is a matrix. We communicate the structure in which the plots must be combined using a matrix. As such, the layout function is more flexible compared to the par() function.

Option	Description	Value
matrix	matrix specifying location of plots	matrix
widths	width of columns	vector
heights	height of rows	vector

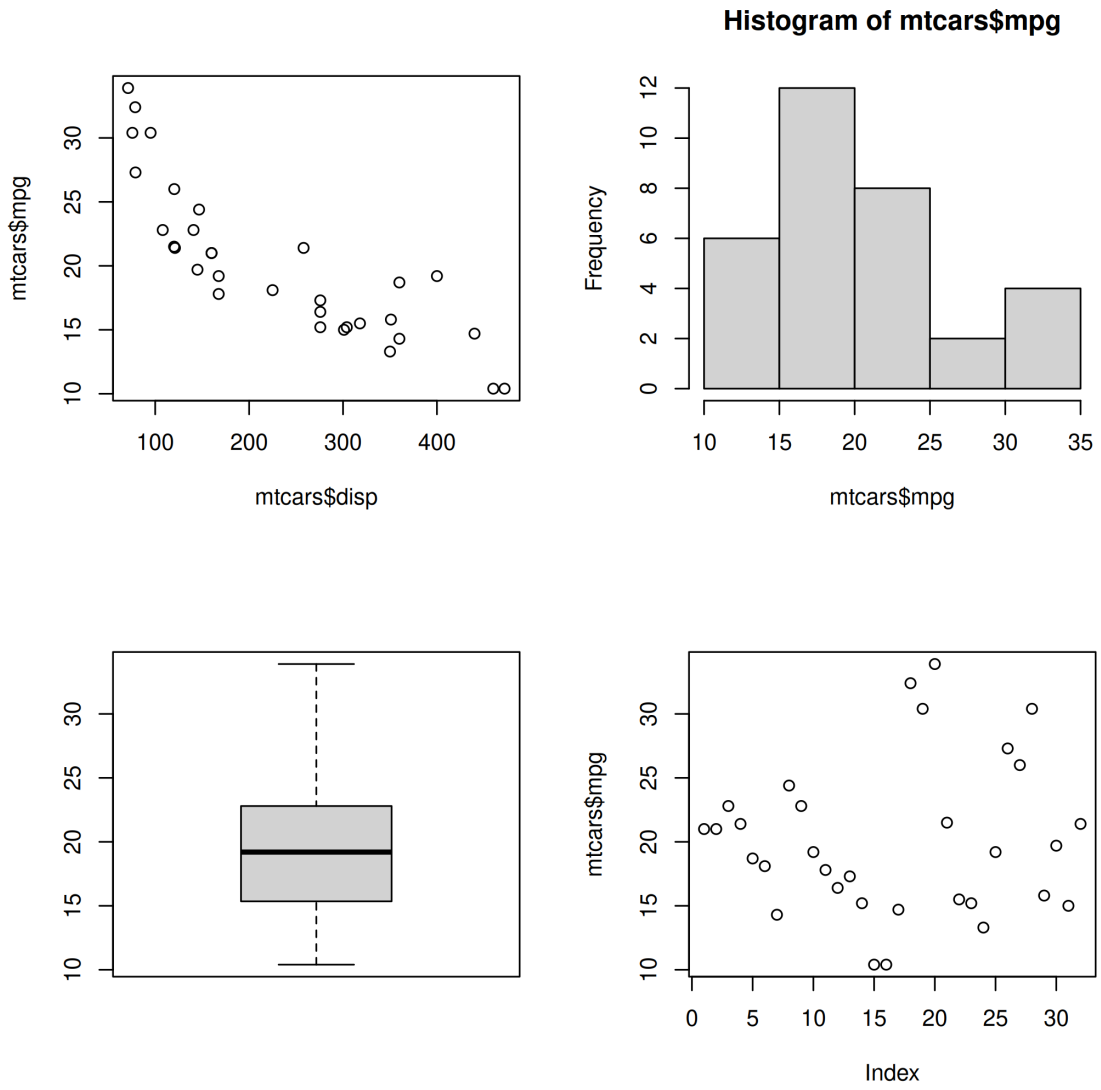
Let us begin by combining 4 plots in a 2 row/2 column structure. We do this by creating a layout using the matrix function.

Case Study 1

Combine 4 plots in 2 rows/2 columns filled by rows.

```
# specify the layout
# 4 plots to be combined in 2 row/ 2 columns and arranged by row
layout(matrix(c(1, 2, 3, 4), nrow = 2, byrow = TRUE))

# specify the 4 plots
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)
plot(mtcars$mpg)
```




```
layout(1)
```

Case Study 2

Combine 4 plots in 2 rows/2 columns filled by columns

To fill the plots by column, we specify byrow = FALSE in the matrix.

```
# specify the layout
# 4 plots to be combined in 2 row/ 2 columns and filled by columns
layout(matrix(c(1, 2, 3, 4), nrow = 2, byrow = FALSE))

# specify the 4 plots
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
```

```
boxplot(mtcars$mpg)
plot(mtcars$mpg)
layout(1)
```

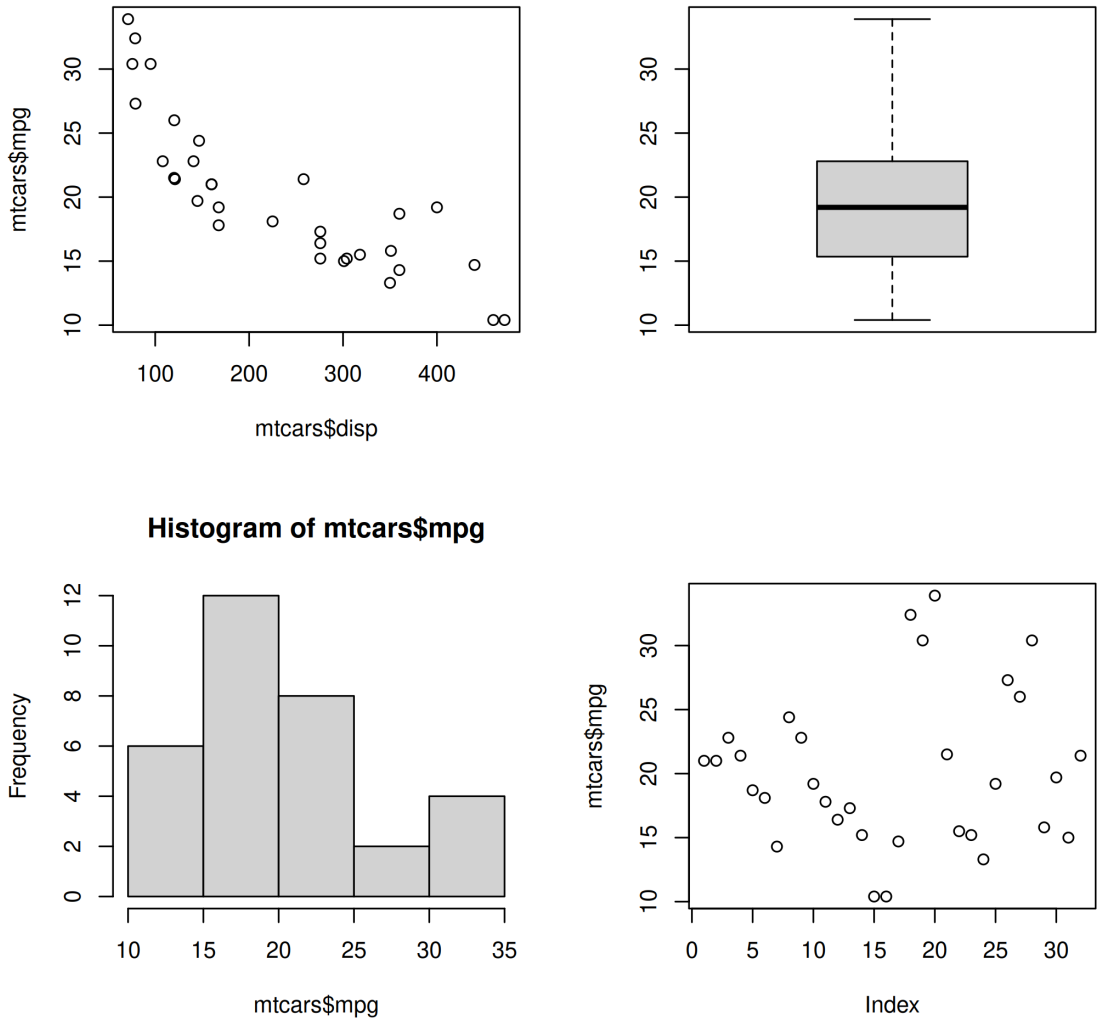


Figure 37: Ragged panels drawn with layout() and custom widths and heights

Case Study 3

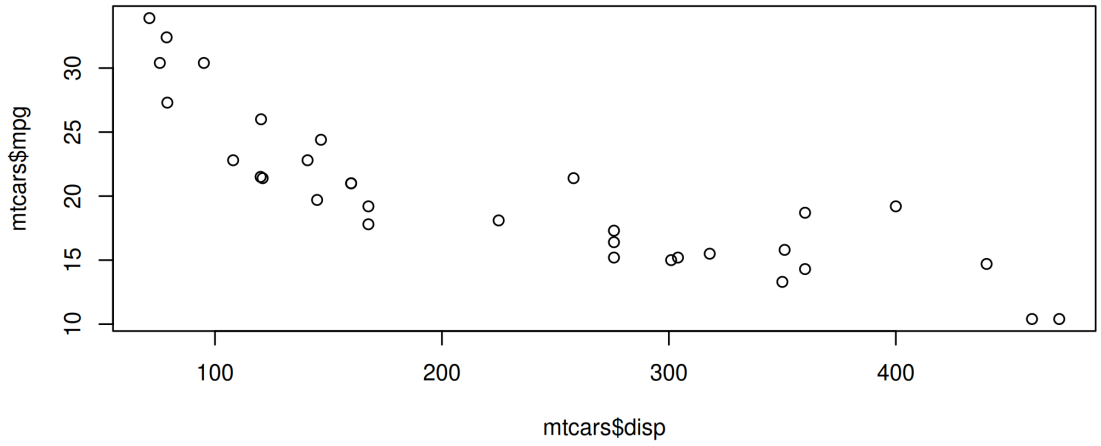
Combine 3 plots in 2 rows/2 columns filled by rows

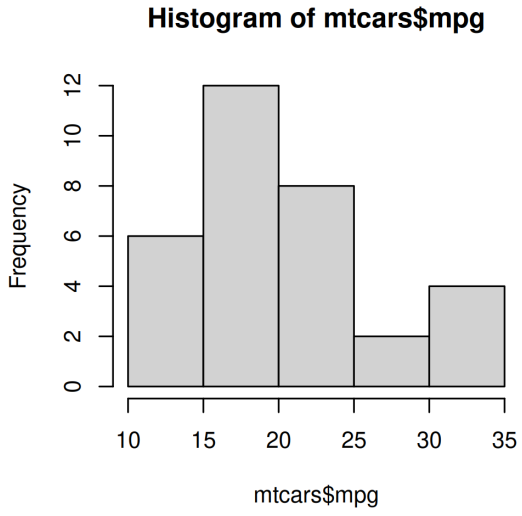
The magic of the layout() function begins here. We want to combine 3 plots and the first plot should occupy both the columns in row 1 and the next 2 plots should be in row 2. If you look at the matrix below, 1 is specified twice and since the matrix is filled by row, it will occupy both the columns in the first row. Similarly the first plot will occupy the entire first row. It will be crystal clear when you see the plot.

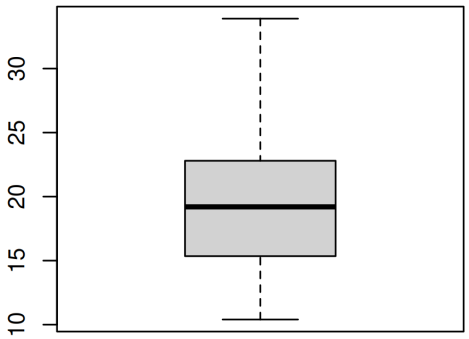
```r
# specify the matrix
matrix(c(1, 1, 2, 3), nrow = 2, byrow = TRUE)
##      [,1] [,2]
## [1,]    1    1
## [2,]    2    3

# 3 plots to be combined in 2 row/ 2 columns and arranged by row
layout(matrix(c(1, 1, 2, 3), nrow = 2, byrow = TRUE))

# specify the 3 plots
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)
```







```
layout(1)
```

Case Study 4

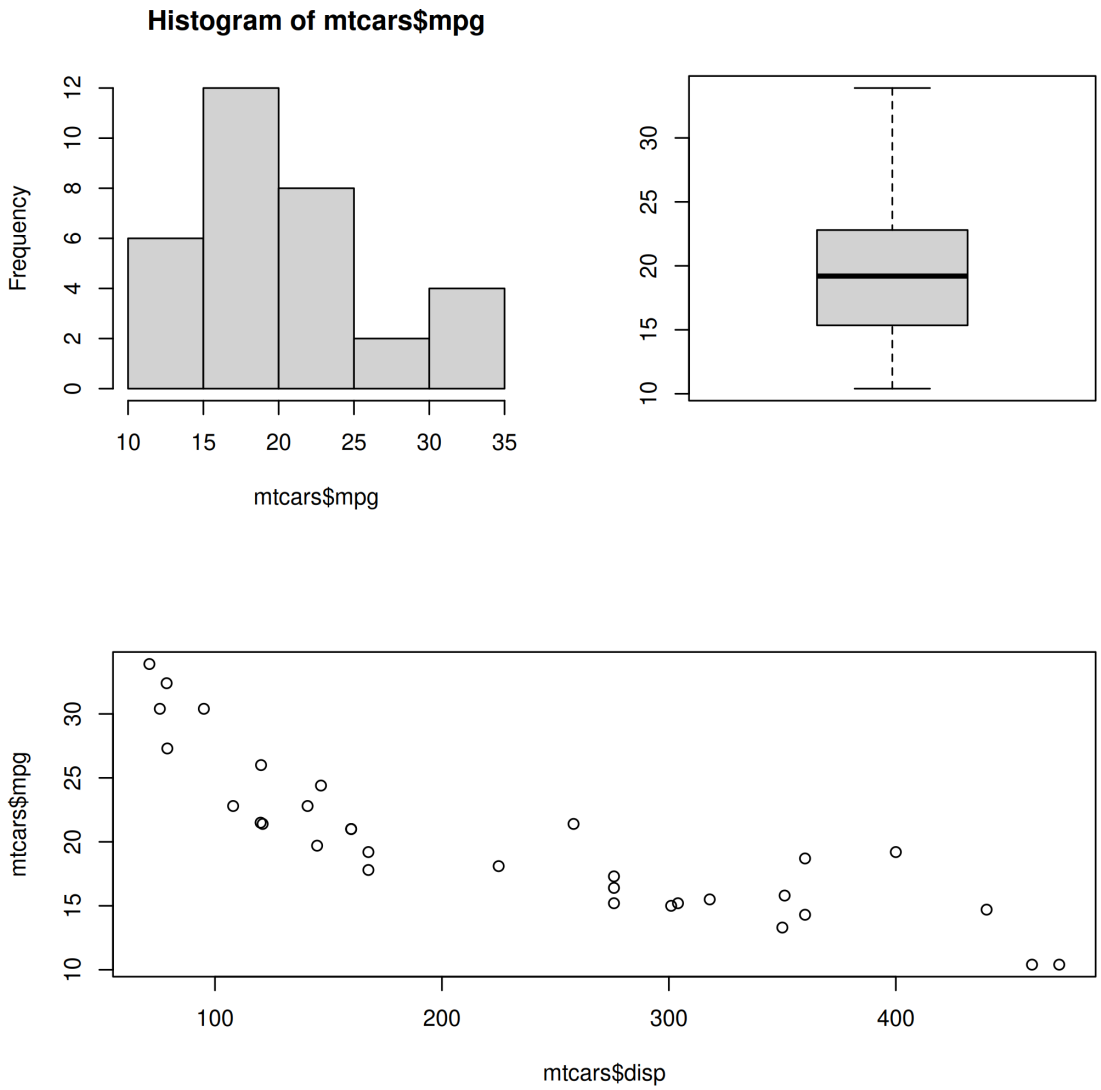
Combine 3 plots in 2 rows/2 columns filled by rows

The plots must be filled by rows and the third plot must occupy both the columns of the second row while the other two plots will be placed in the first row. The matrix would look like this:

```
# specify the matrix
matrix(c(1, 2, 3, 3), nrow = 2, byrow = TRUE)
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    3

# 3 plots to be combined in 2 row/ 2 columns and arranged by row
layout(matrix(c(1, 2, 3, 3), nrow = 2, byrow = TRUE))

# specify the 3 plots
hist(mtcars$mpg)
boxplot(mtcars$mpg)
plot(mtcars$disp, mtcars$mpg)
```




```
layout(1)
```

Case Study 5

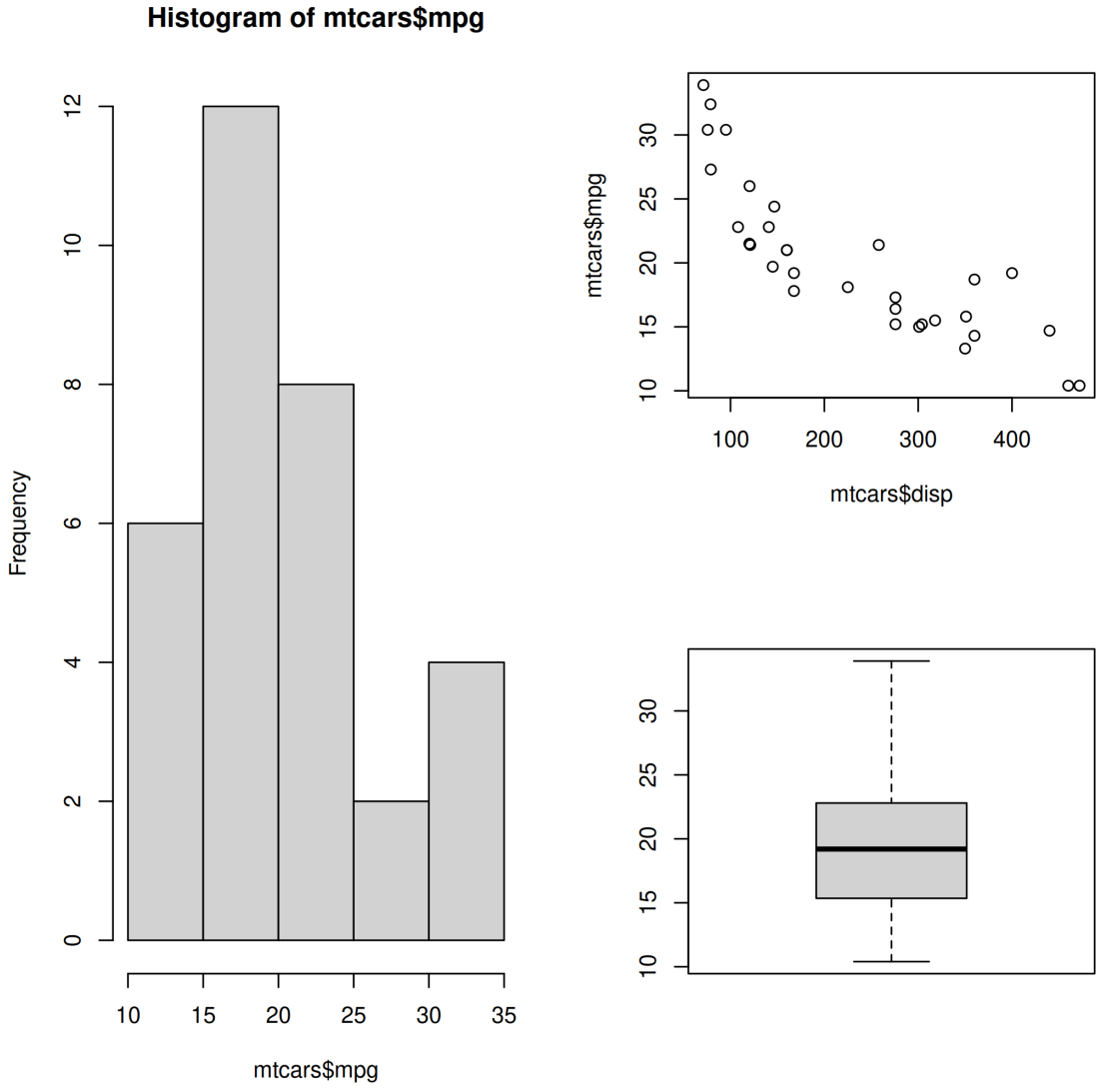
Combine 3 plots in 2 rows/2 columns filled by columns

The plots must be filled by columns and the first plot must occupy both the rows of the first column while the other two plots will be placed in the second column in two rows. The matrix would look like this:

```
# specify the matrix
matrix(c(1, 1, 2, 3), nrow = 2, byrow = FALSE)
##      [,1] [,2]
## [1,]    1    2
## [2,]    1    3
```

```r
# 3 plots to be combined in 2 row/ 2 columns and arranged by columns
layout(matrix(c(1, 1, 2, 3), nrow = 2, byrow = FALSE))

# specify the 3 plots
hist(mtcars$mpg)
plot(mtcars$disp, mtcars$mpg)
boxplot(mtcars$mpg)
```




```r
layout(1)
```

Case Study 6

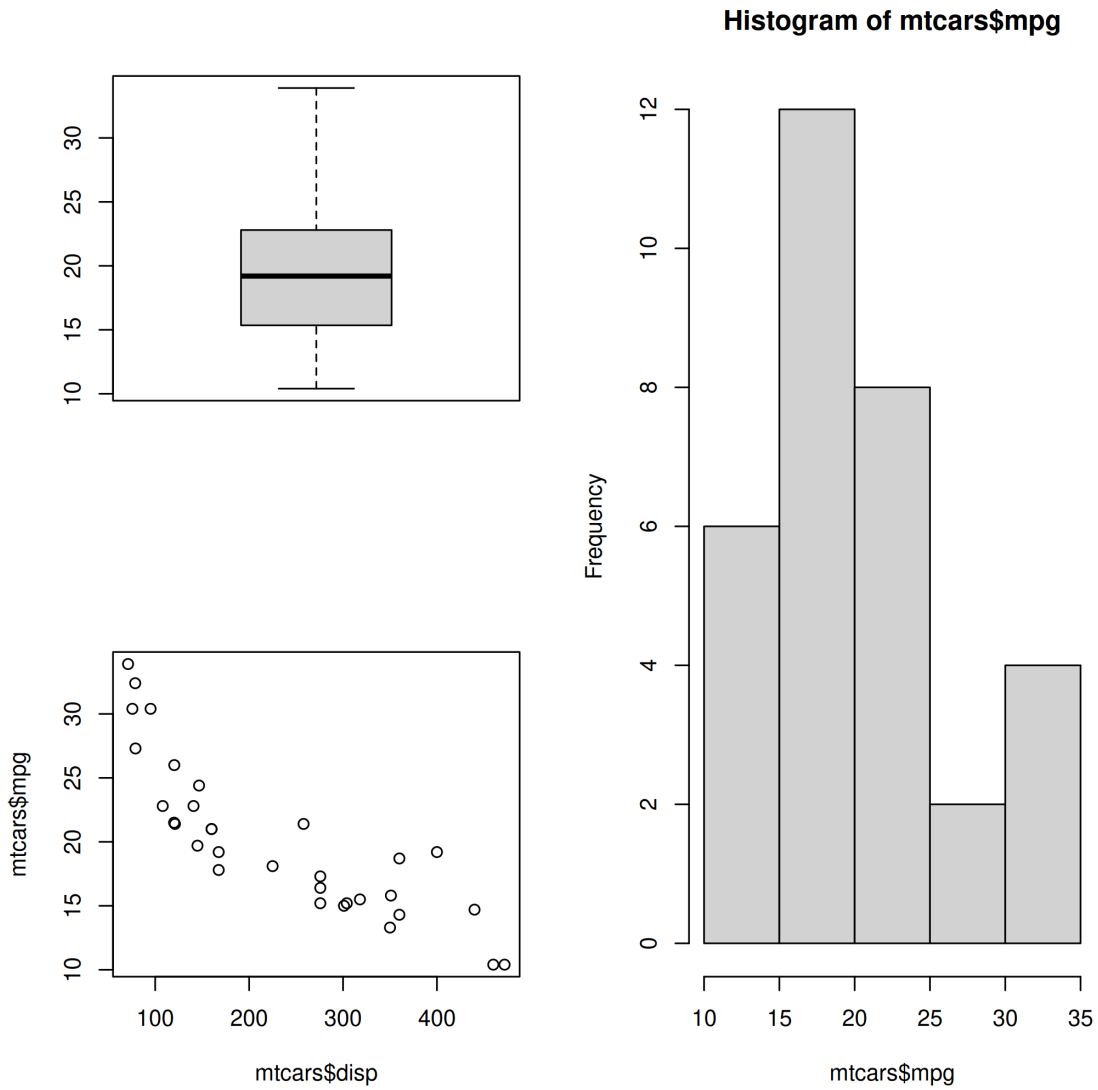
Combine 3 plots in 2 rows/2 columns filled by columns

The plots must be filled by columns and the third plot must occupy both the rows of the second column while the other two plots will be placed in the first column in two rows. The matrix would look like this:

```
# specify the matrix
matrix(c(1, 2, 3, 3), nrow = 2, byrow = FALSE)
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    3

# 3 plots to be combined in 2 row/ 2 columns and arranged by columns
layout(matrix(c(1, 2, 3, 3), nrow = 2, byrow = FALSE))

# specify the 3 plots
boxplot(mtcars$mpg)
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
```




```
layout(1)
```

In all the layouts created so far, we have kept the size of the rows and columns equal. What if you want to modify the width and height of the columns and rows? The widths and heights arguments in the layout() function address the above mentioned issue. Let us check them out one by one: The widths argument is used for specifying the width of the columns. Based on the number of columns in the layout, you can specify the width of each column. Let us look at some examples.

Case Study 7

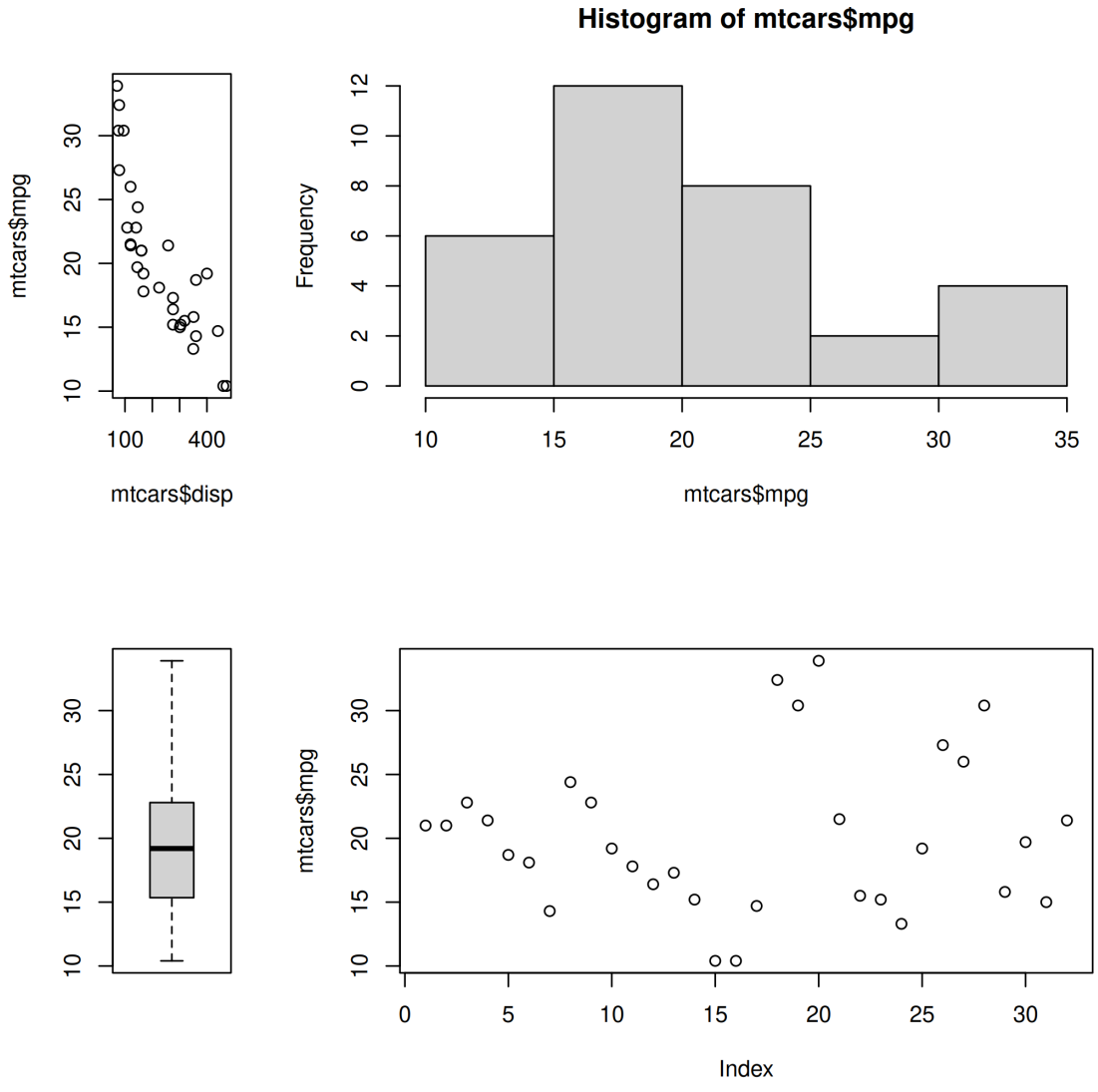
Width of the 2nd column is three times the width of the 1st column

```
# specify the matrix
matrix(c(1, 2, 3, 4), nrow = 2, byrow = TRUE)
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
```

```
# 4 plots to be combined in 2 row/ 2 columns and arranged by rows
layout(matrix(c(1, 2, 3, 4), nrow = 2, byrow = TRUE), widths = c(1, 3))

# specify the plots
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)
plot(mtcars$mpg)
```




```
layout(1)
```

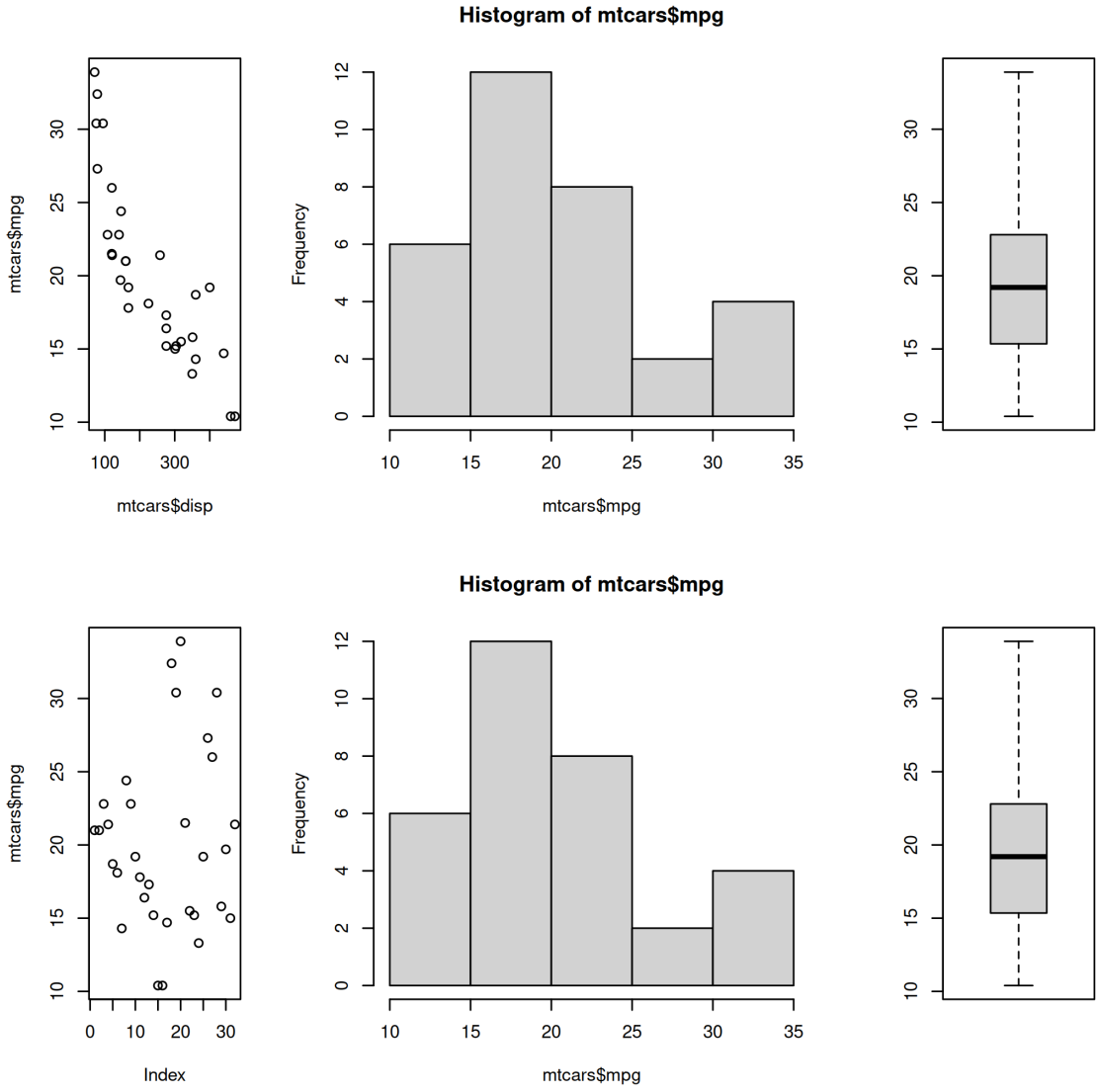
Case Study 8

Width of the 2nd column is twice that of the first and last column

```
# specify the matrix
matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, byrow = TRUE)
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6

# 6 plots to be combined in 2 row/ 3 columns and filled by rows
layout(matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, byrow = TRUE), widths = c(1, 2, 1))

# specify the plots
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)
plot(mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)
```




```
layout(1)
```

The heights arguments is used to modify the height of the rows and based on the number of rows specified in the layout, we can specify the height of each row.

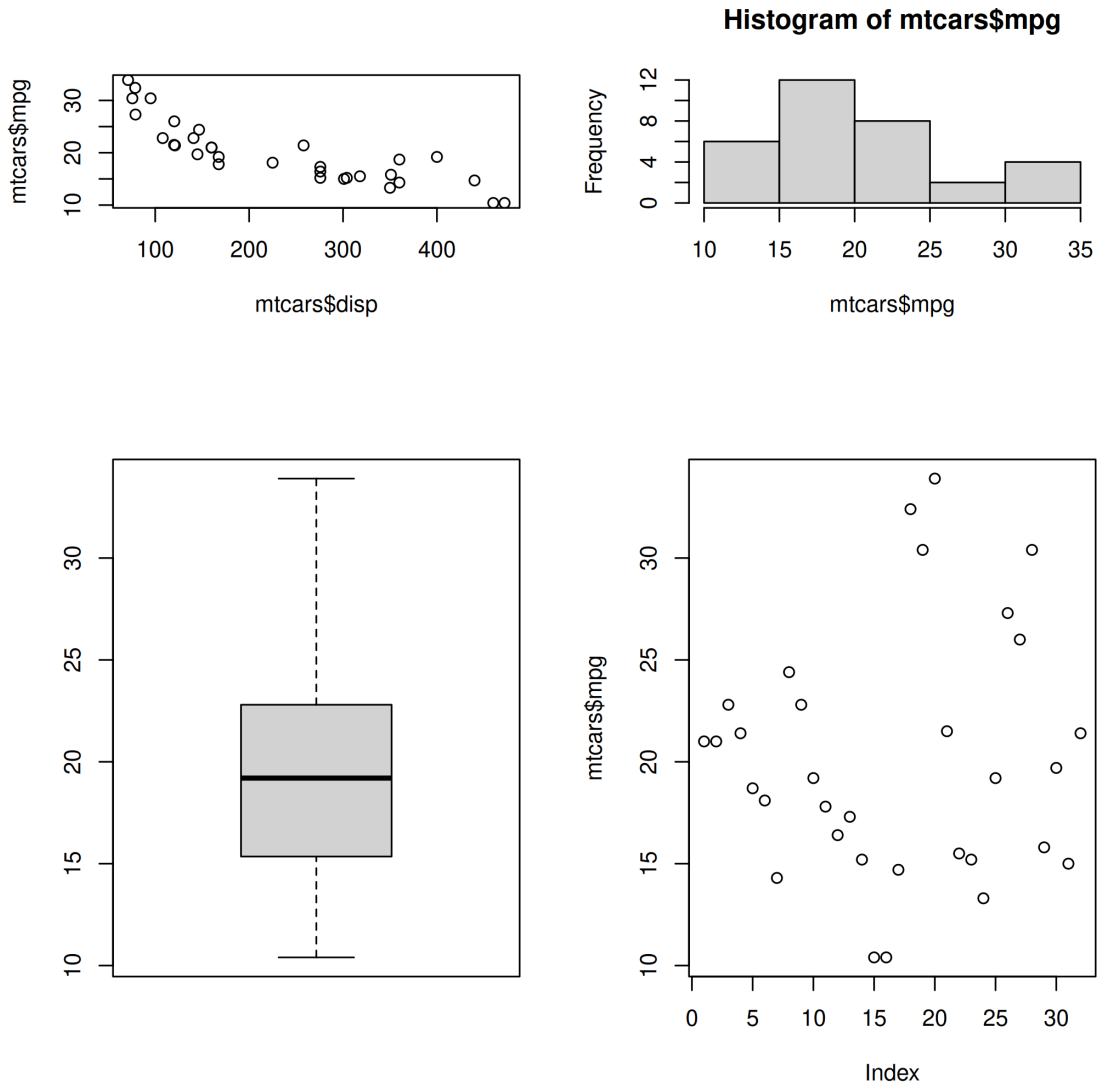
Case Study 9

Height of the 2nd row is twice that of the first row

```
# 4 plots to be combined in 2 row/ 2 columns and filled by rows
layout(matrix(c(1, 2, 3, 4), nrow = 2, byrow = TRUE), heights= c(1, 2))

# specify the 4 plots
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
```

```r
boxplot(mtcars$mpg)
plot(mtcars$mpg)
```




```r
layout(1)
```

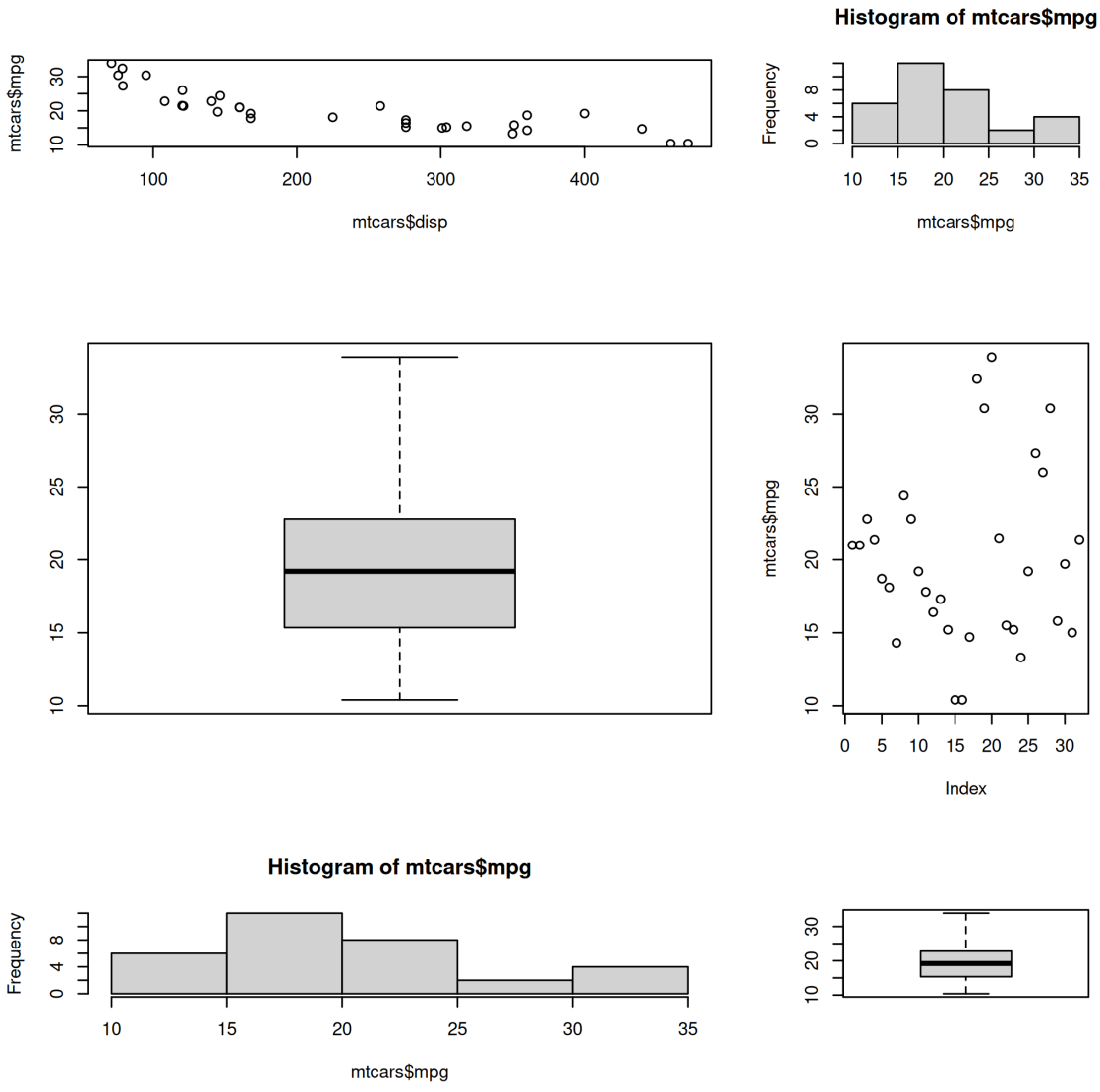
Putting it all together

Before we end this section, let us combine plots using both the widths and heights option

```r
# specify the matrix
matrix(c(1, 2, 3, 4, 5, 6), nrow = 3, byrow = TRUE)
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
## [3,]    5    6
```

```r
# 6 plots to be combined in 3 row/ 2 columns and arranged by rows
layout(matrix(c(1, 2, 3, 4, 5, 6), nrow = 3, byrow = TRUE), heights= c(1, 2, 1),
widths = c(2, 1))

# specify the 6 plots
plot(mtcars$disp, mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)
plot(mtcars$mpg)
hist(mtcars$mpg)
boxplot(mtcars$mpg)
```




```r
layout(1)
```

Production Export & Graphical Devices

Introduction

Everything so far has drawn to a screen. That is fine while you are exploring, and it is worthless the moment someone asks for a figure at a specific size, in a specific format, reproducibly, on a machine that is not yours.

In this chapter, we will learn to

- write plots to files instead of windows, with `png()`, `pdf()` and `svg()`
- understand what a graphical device actually is, and why `dev.off()` is not optional
- generate a batch of figures from a loop, which is the real payoff
- control page layout precisely with `par()`
- replace the automatic axis with your own using `axis()`
- choose colourblind-safe palettes using only what ships with R

This is the chapter where plots stop being sketches and start being output.

Screenshots Are Not Export

The obvious way to get a plot out of R is to take a picture of the screen. It works right up until it does not: the window is the wrong size, the resolution depends on your display's zoom, the text is sized for pixels and goes blurry in print, and the result cannot be regenerated because it depends on what was on screen when you pressed the button.

A real export asks three questions up front — how big, what format, what resolution — and answers them the same way every time.

What a Graphical Device Is

In R, nothing is drawn "to R". Marks go to a **graphical device**, which is an object that knows where the bytes end up. R starts with a default device (a window on desktop R, or `Rplots.pdf` in a script) and `png()` and friends just open another one. That is why the same code produces a window interactively and a file under `Rscript`, with no change to the plotting calls.

Three functions let you see what is going on:

```
dev.cur()   # number of the active device
```

```
png
  2
```

```
dev.list()  # every open device
```

```
png
  2
```

```
dev.off()   # close the most recently opened one
```

```
null device
          1
```

Devices are a **stack**. Opening a new one makes it active; closing it hands control back to the one underneath. That stack is why forgetting `dev.off()` leaves you with a device you cannot write to — and why a script that never calls it dumps its plots into a file called `Rplots.pdf` you did not ask for.

> A useful habit: in any script or function that opens a device, pair it with `on.exit()`. That way the device closes even if the code errors halfway through, which is exactly when a stray device causes the most confusion. `dev.off()` returns the name and number of the device it closed, so you can also save the handle when you want to reopen it later.

Raster: png()

`png()` writes a bitmap. The important arguments are `width`, `height`, `units` and `res`.

```
png('chart.png', width = 800, height = 600)
plot(mtcars$disp, mtcars$mpg)
dev.off()
```

The trap here is `units`. **It defaults to "px", not inches.** So the call above produces an 800 *pixel* wide image, and `res` is ignored — resolution only applies when `units` is a physical unit.

```
# 6 inches wide at 300 dpi -> 1800 pixels, and res is honoured
png('chart-print.png', width = 6, height = 4, units = 'in', res = 300)
plot(mtcars$disp, mtcars$mpg)
dev.off()
```

Rule of thumb: if the figure is going into a document, use `units = 'in'` and `res = 300`. That is print-crisp. If it is going on a screen, `units = 'px'` at a size you already know is right is fine and simpler.

`pointsize` sets the default text size in points, which matters because the default is tuned for a 12-point-ish screen context. On a small, high-resolution export, raise it or the labels vanish.

Vector: pdf()

`pdf()` writes PostScript-based vector output. The marks are stored as instructions rather than pixels, so the file stays sharp at any zoom and prints at any size. This is the right default for publication.

```
pdf('chart.pdf', width = 6, height = 4)
plot(mtcars$disp, mtcars$mpg)
dev.off()
```

Unlike `png()`, `width` and `height` here are **inches** by default, so the same numbers produce a very different file. Vector output also means the file size stops tracking the resolution, because there is no resolution.

```
# onefile = FALSE puts each page in its own file, which is what you want when
# a batch job might fail on page 40 of 200
pdf('pages/page.pdf', width = 6, height = 4, onefile = FALSE)
```

```r
plot(mtcars$disp, mtcars$mpg)
dev.off()
```

Vector: svg()

`svg()` writes Scalable Vector Graphics, the format web pages and illustration tools read. It requires a cairo-enabled build of R, which is the default on current R but not guaranteed everywhere — particularly not on every headless Linux server.

```r
capabilities('cairo')   # must be TRUE
```

```r
svg('chart.svg', width = 6, height = 4)
plot(mtcars$disp, mtcars$mpg)
dev.off()
```

The honest way to use `svg()` in a script that has to run anywhere is to ask first:

```r
if (isTRUE(capabilities('cairo'))) {
  svg('chart.svg', width = 6, height = 4)
  plot(mtcars$disp, mtcars$mpg)
  dev.off()
} else {
  message('cairo unavailable; wrote PDF instead')
  pdf('chart.pdf', width = 6, height = 4)
  plot(mtcars$disp, mtcars$mpg)
  dev.off()
}
```

Comparing the three

```r
# Write all three from the same code, then compare the results.
outdir <- file.path(tempdir(), 'devices')
dir.create(outdir, showWarnings = FALSE)

png(file.path(outdir, 'a.png'), width = 1800, height = 1200, res = 300)
plot(mtcars$disp, mtcars$mpg, pch = 19, col = '#0072B2')
dev.off()

pdf(file.path(outdir, 'a.pdf'), width = 6, height = 4)
plot(mtcars$disp, mtcars$mpg, pch = 19, col = '#0072B2')
dev.off()

svg(file.path(outdir, 'a.svg'), width = 6, height = 4)
plot(mtcars$disp, mtcars$mpg, pch = 19, col = '#0072B2')
dev.off()

file.info(list.files(outdir, full.names = TRUE))['size']
```

Those file sizes are worth noticing. The PNG is measured in megabytes and the two vector files in kilobytes, for the same 6-by-4-inch figure. Vector output is not marginally better than

raster for print; it is a different order of magnitude, and it is why "save as PDF" is the standing advice for anything leaving your screen.

Batch Reports

The reason to care about devices at all is the batch case. If you are producing a figure per customer, per region, per model, you want a loop, not a hundred copies of the same code — and the loop has exactly one requirement: **the device must be opened and closed inside the loop body.**

```
outdir <- file.path(tempdir(), 'batch')
dir.create(outdir, showWarnings = FALSE)

groups <- unique(mtcars$cyl)
for (g in groups) {
  png(file.path(outdir, paste0('cyl-', g, '.png')),
      width = 1200, height = 800, res = 150)
  boxplot(mtcars$mpg[mtcars$cyl == g], main = paste(g, 'cylinders'),
          ylab = 'Miles per gallon')
  dev.off()   # inside the loop. Always.
}
```

Put the `dev.off()` outside the loop and you will open 3 devices, write 3 plots to whichever was active when each `png()` call was made — which is only the first — and leave two open at the end. The symptom is usually a folder of zero-byte files, or a folder containing one file that is wrong.

Two details separate a script that works from one that is trustworthy. First, `dir.create()` before the loop, not inside it. Second, print what you wrote:

```
for (g in groups) {
  f <- file.path(outdir, paste0('cyl-', g, '.png'))
  png(f, width = 1200, height = 800, res = 150)
  boxplot(mtcars$mpg[mtcars$cyl == g], main = paste(g, 'cylinders'))
  dev.off()
  cat(basename(f), file.info(f)$size, 'bytes\n')
}
```

A size of 0 in that log means the device was never closed. This is the cheapest possible check and it catches the most common batch bug there is.

Controlling the Page with par()

`par()` sets the parameters of the current device. The full set is large; these are the ones you will actually reach for.

mar and oma

`mar` sets the four margins in **lines of text**, in the order bottom, left, top, right. `oma` sets the same four *outside* the whole figure, and only matters when several panels are drawn together.

```
init <- par(no.readonly = TRUE)
par(mfrow = c(1, 2))
plot(mtcars$disp, mtcars$mpg, main = 'Default mar')
```

```r
par(mar = c(2, 2, 1, 1))
plot(mtcars$disp, mtcars$mpg, main = 'mar = c(2, 2, 1, 1)')
par(init)
```

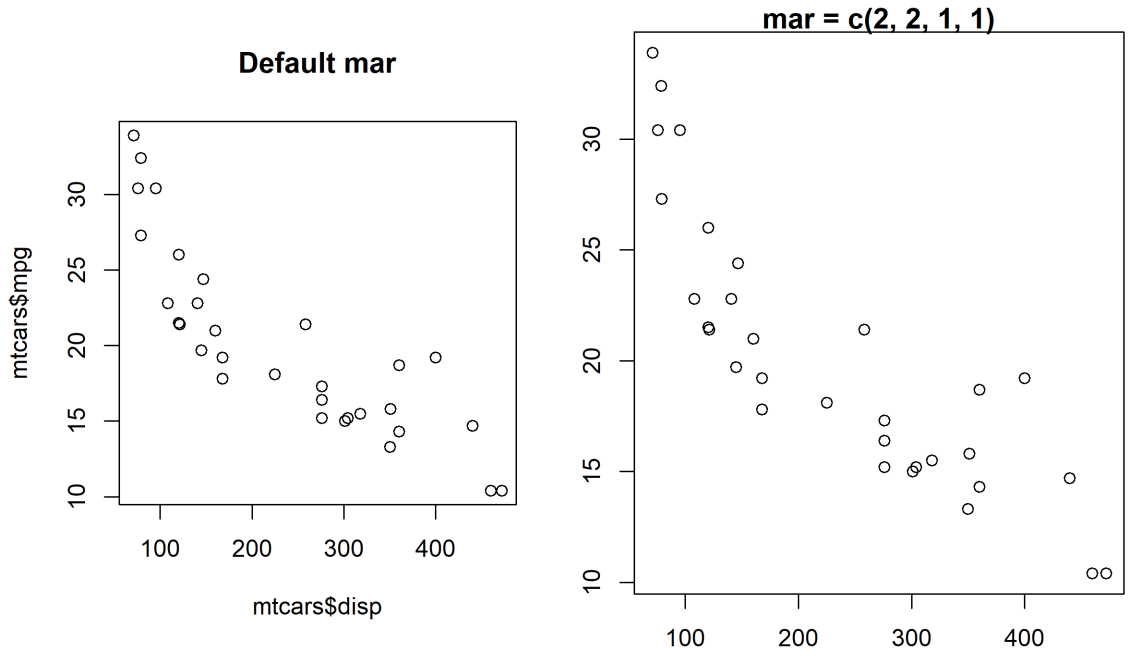


Figure 38: Effect of `par(mar)` on the space available to titles and labels

Because the units are lines of text, margins scale with `cex` and `pointsize`. A margin tuned on screen will be wrong in a high-resolution export unless you also raise `pointsize`. This is the single most common reason an exported figure looks cramped while the on-screen version looks fine.

mgp

`mgp` is three numbers: how many lines in from the edge the axis line sits, how many for the tick labels, and how many for the axis title. The default is `c(3, 1, 0)`.

```r
init <- par(no.readonly = TRUE)
par(mfrow = c(1, 2))
plot(mtcars$disp, mtcars$mpg, main = 'mgp = c(3, 1, 0)')
par(mgp = c(2, 0.7, 0))
plot(mtcars$disp, mtcars$mpg, main = 'mgp = c(2, 0.7, 0)')
par(init)
```

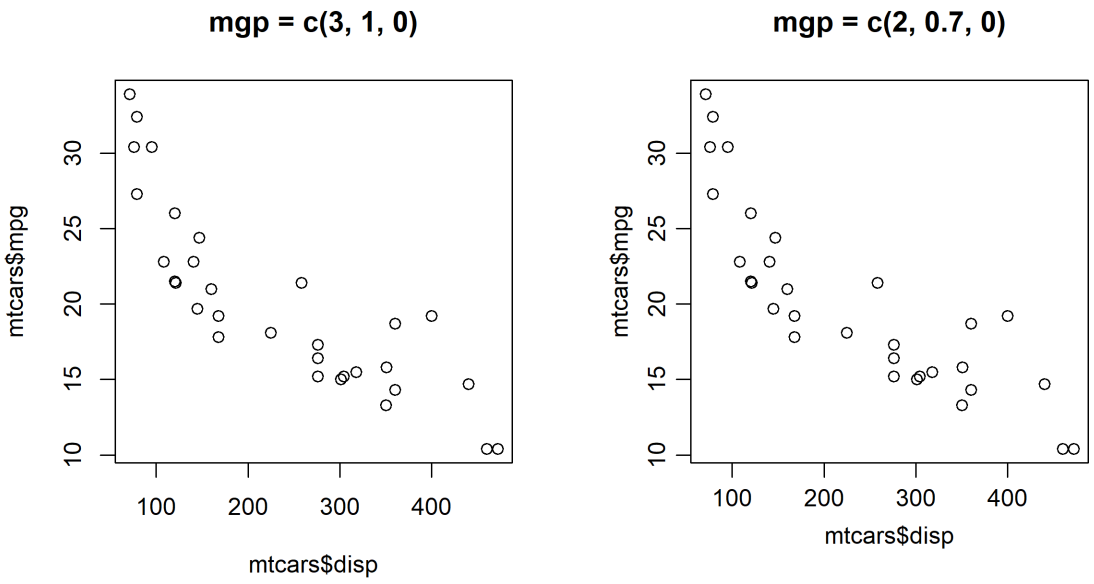


Figure 39: Effect of `par(mgp)` on axis label placement

las

`las` controls the orientation of tick labels: 0 (default) always parallel to their axis, 1 always horizontal, 2 always perpendicular, 3 always vertical. `las = 1` is worth setting on any chart with numeric ticks — it stops labels from being drawn slanted on the y axis.

```
init <- par(no.readonly = TRUE)
par(mfrow = c(1, 2))

par(las = 1)
plot(mtcars$disp, mtcars$mpg, main = 'las = 1 (horizontal)')

par(las = 0)
plot(mtcars$disp, mtcars$mpg, main = 'las = 0 (parallel to axis)')

par(init)
```

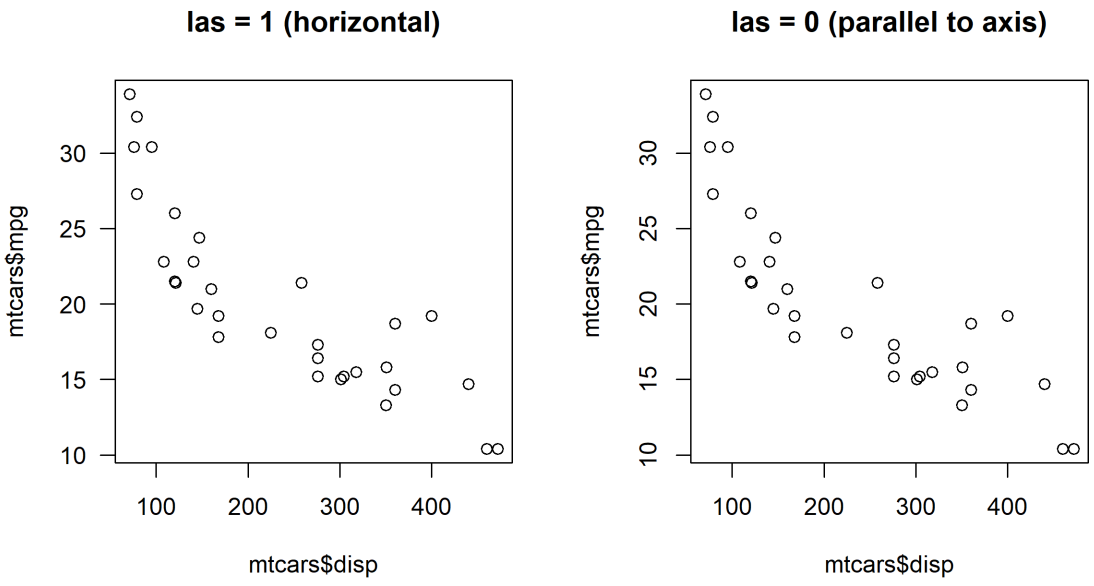


Figure 40: Effect of `par(las)` on the orientation of axis labels

bty, xaxs, tck

`bty` chooses the box drawn around the plot: `'o'` (default, all four sides), `'l'` (axes only — usually what you want), `'7'`, `'u'`, `'.'`, `'['`.

`xaxs` and `yaxs` default to `'r'`, which extends the axis range by 4% at each end so points do not sit on the frame. Set them to `'i'` for the exact data range.

```
init <- par(no.readonly = TRUE)
par(mfrow = c(1, 2))
plot(mtcars$disp, mtcars$mpg, main = "bty = 'o'", bty = 'o')
par(bty = 'l')
plot(mtcars$disp, mtcars$mpg, main = "bty = 'l'", bty = 'l')
par(init)
```

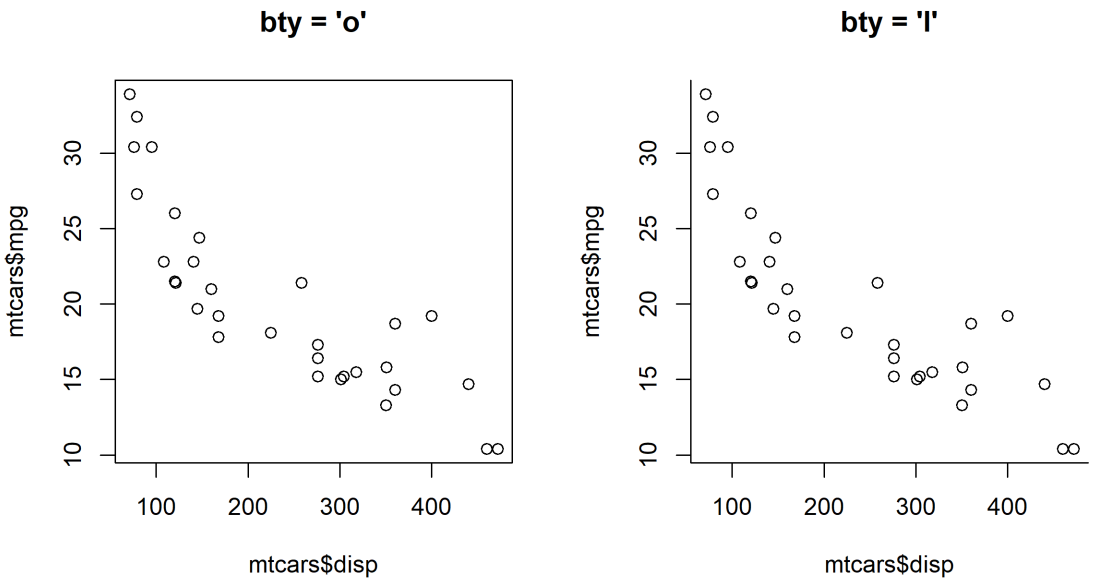


Figure 41: Effect of `par(bty)` on the box drawn around the plot

`tck` is tick length as a fraction of the axis interval. Positive draws ticks inside, negative outside, and 0 removes them.

```
init <- par(no.readonly = TRUE)
par(mfrow = c(1, 2))
plot(mtcars$disp, mtcars$mpg, main = 'tck = 0.02 (inside)', tck = 0.02)
par(tck = -0.02)
plot(mtcars$disp, mtcars$mpg, main = 'tck = -0.02 (outside)', tck = -0.02)
par(init)
```

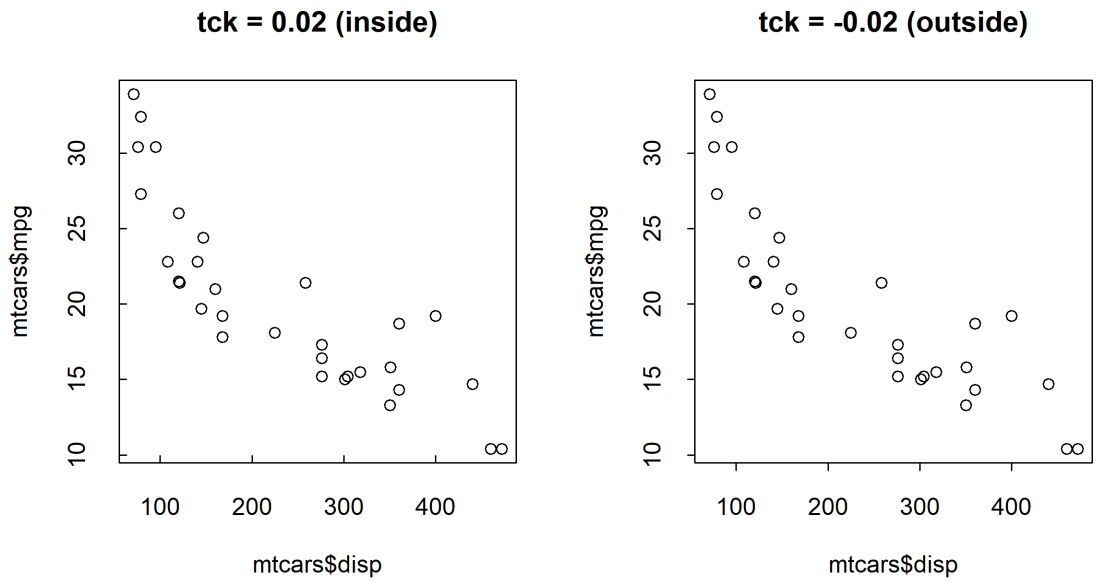


Figure 42: Effect of `par(tck)` on tick mark direction

`par()` is device state, so it persists until changed. Capture and restore it rather than hand-writing the old values back — `op <- par(mar = c(2, 2, 1, 1))` returns the previous values invisibly, and `par(op)` puts them back exactly. In a function, wrap that in `on.exit(par(op))` so an error cannot leave the device mangled for whatever runs next.

Custom Axes

The automatic axis picks ticks at "nice" numbers. Sometimes you need ticks where your data actually is — at quarters, at round dates, at the values that matter.

`axis()` draws an axis manually. Its arguments are the side (`1` bottom, `2` left, `3` top, `4` right), the positions, and the labels.

```
d <- c(1, 2, 3, 5, 8, 13)
plot(d, type = 'b', col = 'blue', xaxt = 'n', xlab = 'Position',
     ylab = 'Value')
axis(1, at = seq_along(d), labels = LETTERS[seq_along(d)])
```

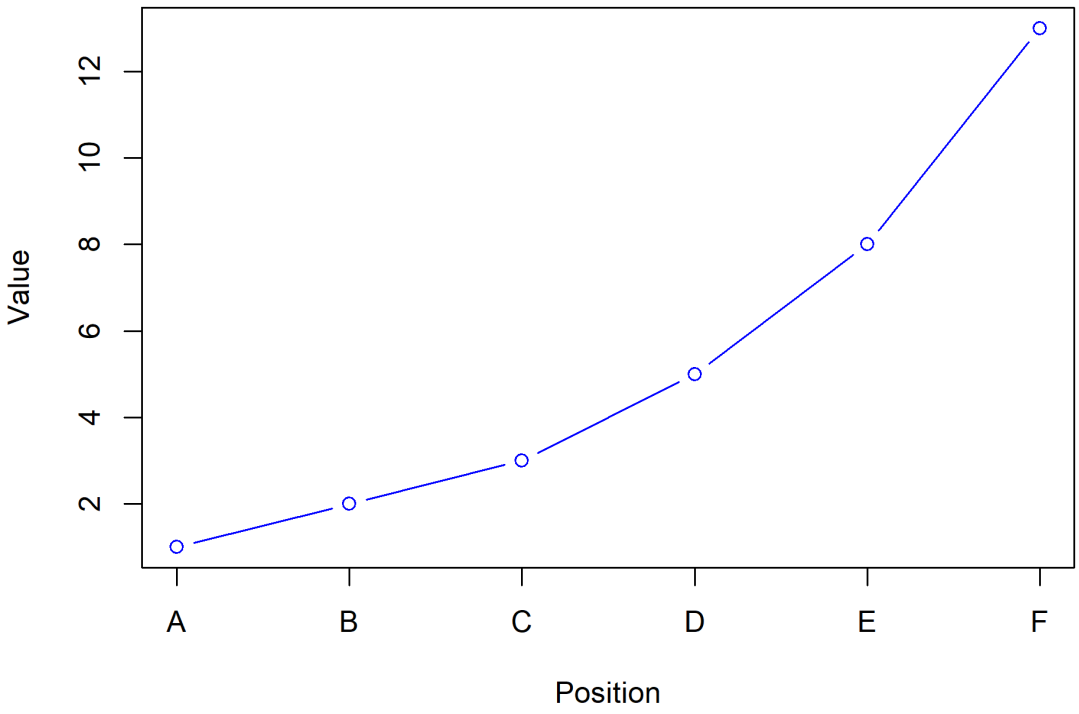


Figure 43: Replacing the automatic x axis with `axis()` and custom labels

`xaxt = 'n'` removes the automatic axis first. Without it you draw two axes over each other, which is a common and confusing bug.

To blank the labels but keep the ticks, ask for them explicitly:

```
d <- c(1, 2, 3, 5, 8, 13)
plot(d, type = 'b', col = 'blue', xlab = 'Position', ylab = 'Value')
axis(1, at = seq_along(d), labels = FALSE)
axis(4)
```

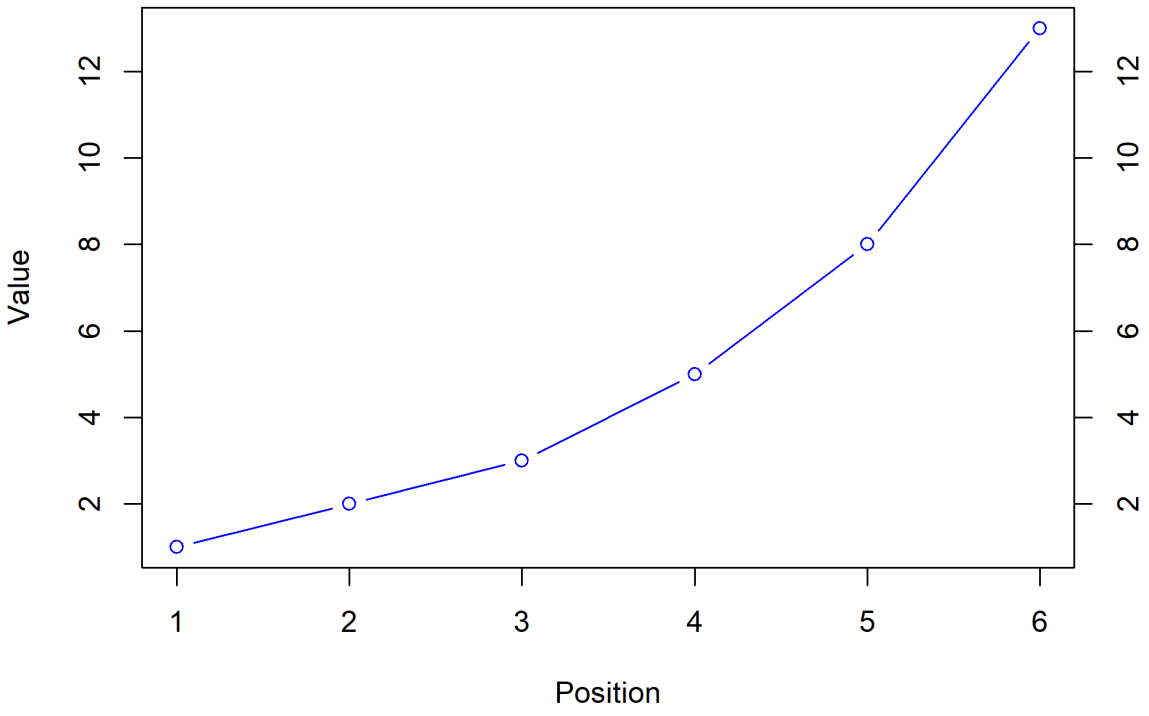


Figure 44: Labelless ticks on the bottom axis, and a second axis on the right

`axis(4)` adds a second axis on the right — the standard way to show a different unit on the same plot.

Dates work because axis positions are numeric and the labels are text:

```
sales <- c(120, 145, 138, 160, 155, 171, 180)
when <- as.Date('2026-01-01') + 0:6 * 30
plot(when, sales, type = 'b', col = '#0072B2', pch = 19,
     xaxt = 'n', xlab = '', ylab = 'Sales')
axis(1, at = as.numeric(when),
     labels = format(when, '%b %d'), las = 1)
```

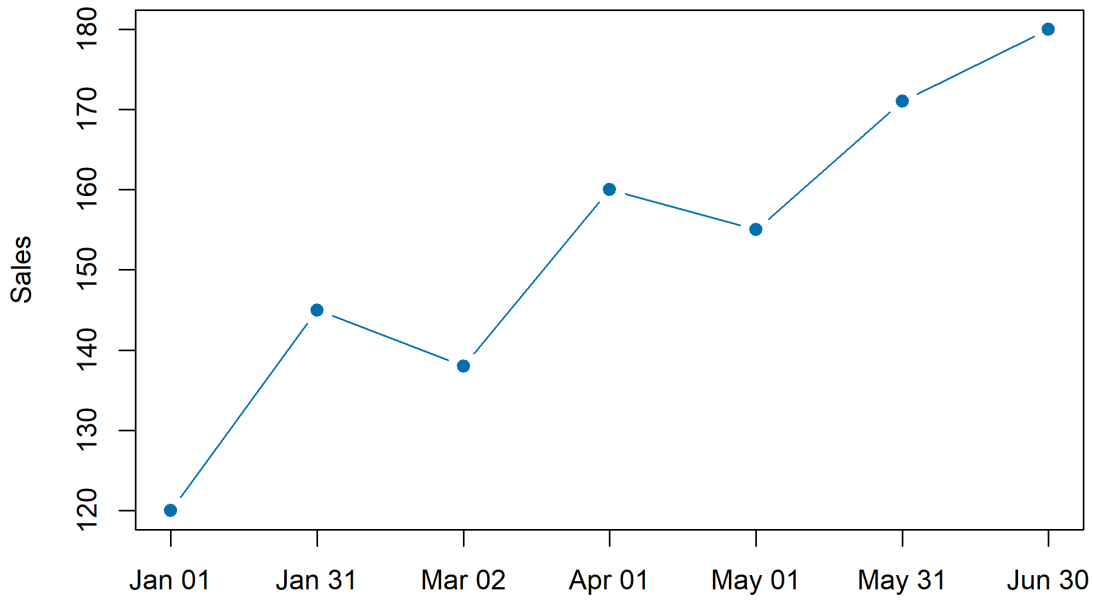


Figure 45: A date axis drawn with `axis()`, formatted from numeric positions

`las = 1` there is doing real work: with six month-start dates the default would slant them into each other.

Colourblind-Safe Palettes

Colour is where accessibility quietly fails, and the fix used to require a package. It no longer does — R 4.1 and later ship palettes designed to be distinguishable under the common forms of colour vision deficiency.

```
palette.pals()
```

That lists every built-in palette by name. The one to reach for is `"Okabe-Ito"`, a colourblind-safe qualitative palette:

```
palette.colors(palette = 'Okabe-Ito')
```

Two things to notice. The values come back as hex strings **printed by R**, and you should use them exactly as printed. Hand-typing a hex code from memory or from a blog post is how palettes get subtly wrong. Second, the first colour is black, so index with `[-1]` or keep it deliberately — a plot whose first series is black tends to look like an axis.

```
pal <- palette.colors(palette = 'Okabe-Ito')[-1]

plot(mtcars$disp, mtcars$mpg, type = 'n',
```

```
     xlab = 'Displacement', ylab = 'Miles per gallon')
points(mtcars$disp, mtcars$mpg, pch = 19, col = pal[1])
legend('topleft', legend = '4 cyl', pch = 19, col = pal[1], bty = 'n')
```

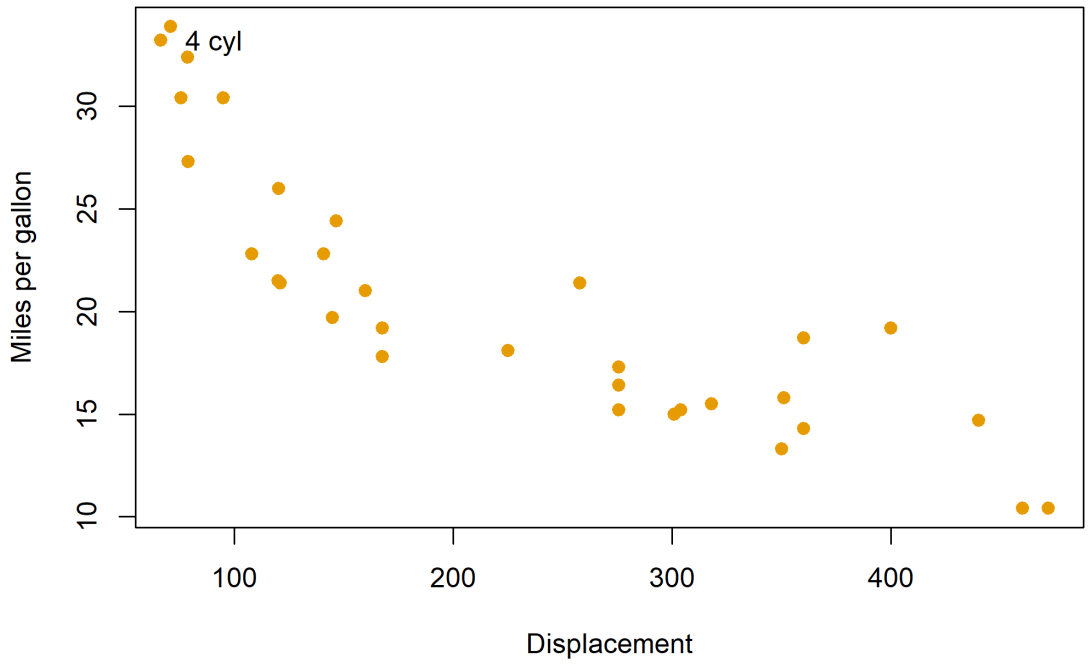


Figure 46: A single series drawn in the first colour of the Okabe-Ito palette

Colours are **recycled** if you ask for more than the palette holds, and recycling is not a good idea — the fifth and first series would be identical. `palette.colors()` also accepts `alpha`, which is the easy way to get a translucent fill without hand-converting colours:

```
palette.colors(palette = 'Okabe-Ito', alpha = 0.5)
```

Checking for Greyscale

The other accessibility test is printing. A figure that relies on colour alone often collapses when photocopied, and the check costs three lines — because luminance is a fixed weighted sum of the RGB channels.

```
pal <- palette.colors(palette = 'Okabe-Ito')[-1]
rgb <- t(grDevices::col2rgb(pal)) / 255
# Rec. 601 luma weights. Perceptual lightness is not the same as the mean of
# the three channels, which is why the naive average looks wrong.
luma <- rgb %*% c(0.299, 0.587, 0.114)
grey <- grDevices::rgb(luma, luma, luma)

init <- par(no.readonly = TRUE)
```

```r
par(mfrow = c(1, 2), mar = c(4.5, 4, 3, 1))
plot(mtcars$disp, mtcars$mpg, type = 'n',
     xlab = 'Displacement', ylab = 'mpg', main = 'Colour')
points(mtcars$disp, mtcars$mpg, pch = 19, col = pal[1:3])
plot(mtcars$disp, mtcars$mpg, type = 'n',
     xlab = 'Displacement', ylab = 'mpg', main = 'Same palette, greyscale')
points(mtcars$disp, mtcars$mpg, pch = 19, col = grey[1:3])
par(init)
```

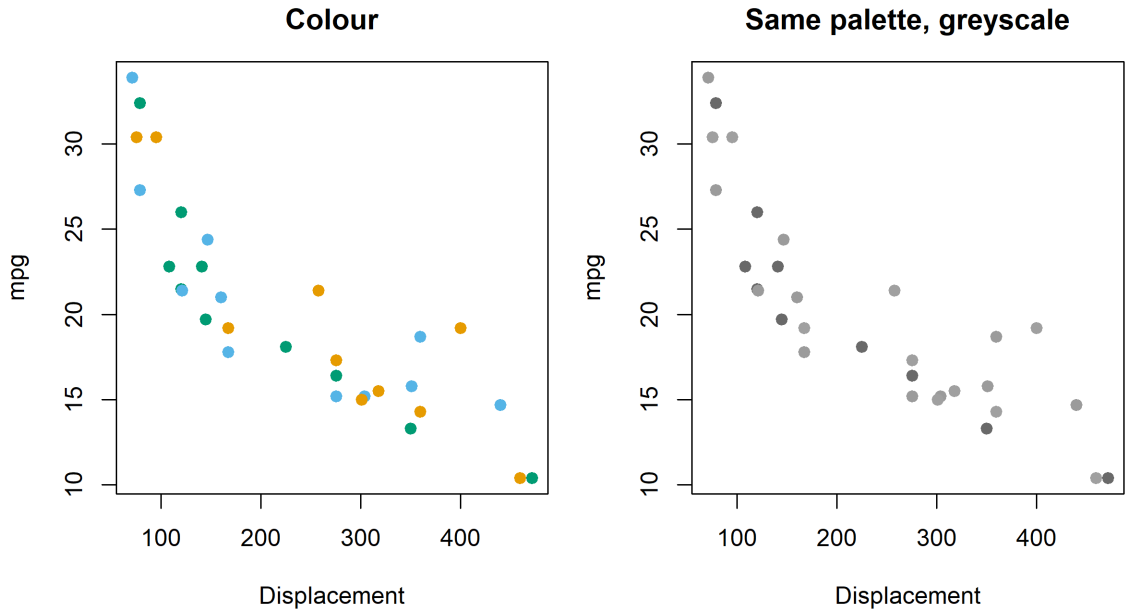


Figure 47: The same three-colour scatterplot in colour and converted to greyscale

If a greyscale version of your figure has three series that are hard to tell apart, colour was doing work it should not have been asked to do alone. Add a second channel — `pch`, `lty`, or direct labels.

Exercises

Work through these before moving on. The solutions are collapsed so you can try first; expand them when you are stuck or when you want to compare approaches.

Exercise 1

Export the `mtcars` scatterplot as a 6-by-4-inch PNG at 300 dpi, with margins tight enough that the axis labels are not clipped. Print the resulting file size to confirm it is non-zero.

Solution

```r
outdir <- file.path(tempdir(), 'ex1')
dir.create(outdir, showWarnings = FALSE)

f <- file.path(outdir, 'mtcars.png')
png(f, width = 6, height = 4, units = 'in', res = 300)
```

```
# mar is in lines of text, so it has to be set inside the device whose
# pointsize it should be interpreted against.
op <- par(mar = c(4, 4, 2.5, 1))
plot(mtcars$disp, mtcars$mpg, pch = 19, col = '#0072B2',
     xlab = 'Displacement', ylab = 'Miles per gallon')
par(op)
dev.off()
```

```
png
  2
```

```
cat(basename(f), file.info(f)$size, 'bytes\n')
```

```
mtcars.png 17892 bytes
```

The size print is not decoration. A `dev.off()` left out of this script produces a zero-byte file and no error at all.

Exercise 2

Draw the `mpg` against `disp` scatterplot with a y axis whose ticks sit only at the integers from 10 to 40, and no automatic y axis at all.

Solution `{r #| label: fig-ex2 #| fig-cap: "Exercise 2 solution: a custom y axis drawn with `axis(2, at =)`" #| fig-alt: "The scatterplot of displacement against miles per gallon, with the automatic y axis suppressed and replaced by ticks at 10, 20, 30 and 40 only." plot(mtcars$disp, mtcars$mpg, pch = 19, col = '#0072B2', yaxt = 'n', xlab = 'Displacement', ylab = 'Miles per gallon') axis(2, at = seq(10, 40, by = 10))`

`yaxt = 'n'` is the y-axis equivalent of `xaxt = 'n'`. Omit it and the automatic axis is drawn first and then drawn over, which reads as slightly bold, doubled tick labels.

Exercise 3

Write a three-page PDF, one plot per page, using a loop over the three cylinder groups in `mtcars`, and confirm the file exists afterwards.

Solution

```
outdir <- file.path(tempdir(), 'ex3')
dir.create(outdir, showWarnings = FALSE)

f <- file.path(outdir, 'by-cyl.pdf')
pdf(f, width = 6, height = 4)

op <- par(mar = c(4, 4, 3, 1))
for (g in sort(unique(mtcars$cyl))) {
  plot(mtcars$disp[mtcars$cyl == g],
       mtcars$mpg[mtcars$cyl == g],
       pch = 19, col = '#0072B2', xlim = range(mtcars$disp),
       main = paste(g, 'cylinders'), xlab = 'Displacement',
       ylab = 'Miles per gallon')
```

```
  plot.new()   # advances to the next page
}
par(op)
dev.off()
```

```
png
  2
```

```
file.exists(f)
```

```
[1] TRUE
```

Note what `pdf()` does *not* do: it does not start a new page per plot. In base graphics every plot overwrites the last, so `plot.new()` is how you advance the page — and it is the PDF analogue of opening a new device inside the loop, for the same reason.

The Full Set

Everything in this chapter is summarised on the one-page `par()` and devices cheatsheet. It is a static artifact rather than a chapter, so it is worth keeping open next to whatever you are building.

That closes the book. Chapter 1 made the case for staying in base graphics; this chapter made the plots production-ready. The middle chapters taught the arguments one at a time, and that is the pattern to keep — change one thing, look at what happened.

References

Everything this book depends on, in one place. The primary sources for base graphics are the R manuals that ship with the software — they are free, they are current, and `?graphics` is more accurate than any blog post.

Base R graphics

R Core Team (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. Available at https://www.R-project.org/.

Chambers, J. M. (1993). *Statistical Models in S*. Chapman & Hall, London.

Chambers, J. M. (2008). *Software for Data Analysis: R and S-PLUS*. Springer, New York.

Chambers, J. M. (2018). *Scripting R: Analyzing and Programming with the R Language*. O'Reilly Media, Sebastopol, CA.

Murrell, P. (2017). *R Graphics*, 3rd edition. Chapman & Hall/CRC, Boca Raton, FL. The definitive reference for base graphics specifically. `?graphics` and the `R Graphics` help pages are abridged versions of it.

Venables, W. N. and Ripley, B. D. (2002). *Modern Applied Statistics with S*. Springer, New York. The `S` ancestor of base graphics, and still the clearest account of why the design is shaped the way it is.

The grammar of graphics, and the alternative

Wilkinson, L. (2005). *The Grammar of Graphics*. Springer, New York.

Wickham, H. and Grolemund, G. (2017). *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O'Reilly Media, Sebastopol, CA. Available at https://r4ds.hadley.nz/.

Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis*, 3rd edition. Springer, New York.

Chang, W. (2024). *R Graphics Cookbook*, 2nd edition. O'Reilly Media, Sebastopol, CA. Available at https://r-graphics.org/. Recipe-first, and covers base and ggplot2 side by side.

Colour and accessibility

Okabe, M. and Ito, K. (2008). "Color Universal Design (CUD): How to make figures and presentations that are friendly to Colorblind people." In *Proceedings of the 2008 Symposium on Universal Design and Accessibility*, Tokyo, Japan.

Masataka, D. (2020). *Color Design: Methods, Tools, and Applications*. Apress, Singapore. Background on perceptual colour spaces, lightness and contrast that Chapter 12's greyscale check relies on.

The Okabe-Ito palette used in Chapter 12 is available in R 4.1 and later without any additional package, via `palette.colors(palette = 'Okabe-Ito')`.

Datasets used in this book

Most figures are drawn from datasets that ship with R, and the two that do not are vendored in this repository so the book builds offline. Full provenance:

- `AirPassengers` — Box & Jenkins (1975) monthly airline passengers, 1949–1960. Distributed with R from the `datasets` package.
- `mtcars` — extracted from the 1974 *Motor Trend* US magazine. Distributed with R from the `datasets` package.
- `iris` — Anderson, E. (1935). "The irises of the Gaspé Peninsula." *Bulletin of the American Iris Society*, 59, 17–50. Distributed with R from the `datasets` package.
- `ToothGrowth` — the growth of 60 guinea pigs, distributed with R from the `datasets` package.

- `hsb2.csv` — High School and Beyond survey, 200 observations, originally from the UCLA Institute for Digital Research and Education. Vendored as `data/hsb2.csv`. Used illustratively; not redistributable as an authoritative source, and no claim is made about the sampling frame.
- `brics-gdp-2010-14.csv` — a frozen vintage of annual GDP growth for the BRICS nations, 2010–2014, hand-entered as a teaching dataset. It differs slightly from the World Bank's current revised series and should be treated as illustrative, **not** as a citable statistic. Where the numbers matter, use the World Bank directly.

Reproducing the measurements in Chapter 1

Chapter 1's claims are measured, not asserted. Each figure in that chapter is drawn from a CSV committed to this repository, and each CSV is produced by a script in `scripts/`. None of them runs when the book is built, which is deliberate: a book arguing for zero dependencies cannot quietly require two of them to make its case.

Claim	Script	Output
Draw time and peak memory, base versus ggplot2, 10k and 100k points	`scripts/bench.R`	`data/bench-base-vs-ggplot2.csv`
Wall clock to start a plotting session, by engine	`scripts/startup.R`	`data/startup-latency.csv`
Declared non-base CRAN dependencies, per package	`scripts/cran-deps.R`	`data/cran-deps.csv`
Published container image sizes, R versus R + tidyverse	`scripts/docker-sizes.R`	`data/docker-sizes.csv`

Run any of them with `Rscript scripts/<name>.R`. Each stamps the date, R version and platform it ran under into its output, so a stale row is visibly stale. Absolute timings are machine-specific and should be read as an order of magnitude; the shape of each result travels across machines far better than the individual numbers do.

`scripts/bench.R` requires `bench` and `ggplot2`, which the book itself never needs. `scripts/docker-sizes.R` and `scripts/cran-deps.R` read published metadata over HTTP, so their numbers reflect the state of the world on their `retrieved_on` date.

Further reading in the R ecosystem

The author's own CRAN packages are adjacent to this book's subject matter rather than part of it. Their plotting layers use ggplot2; Chapter 1 discusses that honestly, and Appendix A maps the two systems onto each other.

- `olsrr` — companion functions for ordinary least squares, including diagnostic plots.
- `rfm` — recency, frequency and monetary analysis, with cohort and segment plots.
- `blorr` — logistic regression diagnostics and plots.
- `descriptr` — frequency tables, cross-tabulations and summary statistics, each with a plot.

A note on the sources for this chapter

The author packages above were checked by reading their source on **2026-10-04**, not inferred from dependency metadata. Dependency metadata records what a package *needs*, not what it *draws with*, and drawing that distinction incorrectly is exactly the error Chapter 1 warns about.

About the Author

Aravind Hebbali is the founder of Rsquared Academy. He earned his Masters in Economics from Madras School of Economics. As an active R user, he has authored several R packages such as

- olsrr
- rfm
- descriptr
- blorr
- xplorerr

In 2015, he founded Rsquared Academy, a free and open source education initiative with focus on data science and analytics. Apart from self paced online courses, Rsquared Academy offers customized learning modules for corporates and universities.

You can find him on GitHub.

Base R ↔ ggplot2: A Rosetta Stone

Introduction

Appendix A is a lookup table, not a chapter. You do not read it; you keep it open next to whichever system you are working in and use it to move a plot across.

Every row maps one base R construct to its nearest ggplot2 equivalent, and the last column is the part that matters: **when to stop translating and switch.**

Two things this appendix is not. It is not a ggplot2 tutorial — there are better ones, and the ggplot2 documentation is excellent. And it is not an argument for either system. Chapter 1 made that argument; this is the translation layer.

> **This book does not use ggplot2.** No example here is evaluated, and no figure on this page requires it. The ggplot2 side appears as code you can read and run yourself, but the book is built without the dependency and stays that way. Only the base R column is rendered.

The one-sentence version

ggplot2 is a **declarative** system: you describe what the data means and let the package decide how to draw it. Base graphics is a **direct** system: you issue drawing commands and the marks appear.

Everything below is a consequence of that difference. Declarative systems infer grouping, transform scales and build facets for you. Direct systems do none of it until you write it.

Marks and coordinates

The most-used arguments translate almost one to one, because they describe marks rather than structure.

Base R	ggplot2	When to switch
`pch = 19`	`shape = 19`	Never. Same idea.
`cex = 1.4`	`size = 3`	Never — but note `size` is not a linear multiple of `cex`; it is in millimetres, so you must retune.
`col = 'red'`	`colour = 'red'`	Never.
`bg = 'yellow'`	`fill = 'yellow'`	Never. `bg` only applies to shapes 21–25, which are already filled.
`lwd = 2`	`linewidth = 1`	Never — same non-linearity as `cex`.
`lty = 2`	`linetype = 'dashed'`	Never, though ggplot2 also accepts the numeric codes.
`type = 'p'`	`geom_point()`	Switch when you start mapping a variable to mark properties.
`type = 'l'`	`geom_line()`	Never.
`type = 'b'`, `'o'`	`geom_line() + geom_point()`	Never — two layers, which is the ggplot2 idiom.
`type = 'h'`	`geom_histogram()`	Switch. ggplot2 bins separately from the plot.
`type = 'n'`	`ggplot()` + a layer only	Never, but `type = 'n'` is *how* you set up axes for manual layering.
`plot(x, y)`	`ggplot(df, aes(x, y)) + geom_point()`	Switch as soon as the plot has a third variable.
`points(x, y)`	`+ geom_point(data = ...)`	Stay in base for overlays onto an existing plot.
`lines(x, y)`	`+ geom_line(data = ...)`	Stay in base for overlays.
`text(x, y, labels)`	`geom_text(aes(label = ...))`	Switch when labels are data-driven rather than hand-placed.
`mtext(s, side)`	`labs(title=, caption=)`	Never for titles; `side` has no ggplot2 equivalent.

Base R	ggplot2	When to switch
`title(main, sub)`	`labs(title, subtitle)`	Never.
`abline(a, b)`	`geom_abline(intercept=, slope=)`	Never.
`segments()`, `arrows()`	`geom_segment()`, `geom_arrow()`	Never. `geom_segment` is the easy one to miss.
`polygon()`	`geom_polygon()`	Never.

The scale trap

`cex = 2` is not `size = 2`. `cex` is a multiplier on the device's default text size; ggplot2's `size` is an absolute diameter in millimetres. If you carry a plot across and change nothing else, the marks will be the wrong size, and the usual response — nudging the number until it looks right — makes the code meaningless to the next reader.

Titles, labels and ranges

Base R	ggplot2	When to switch
`main = 'x'`	`labs(title = 'x')`	Never.
`sub = 'x'`	`labs(subtitle = 'x')`	Never.
`xlab`, `ylab`	`labs(x = , y = )`	Never.
`xlim`, `ylim`	`coord_cartesian(xlim=, ylim=)`	**Always.** See below.
`log = 'x'`	`scale_x_log10()`	Switch. `plot(log = '')` rescales in place with no warning; `scale_*` transforms the data and labels the axis for you.
`asp = 1`	`coord_fixed()`	Never.

`xlim` is not `coord_cartesian(xlim =)`

This is the single most damaging translation error, so it is worth being precise.

```
plot(mtcars$disp, mtcars$mpg, xlim = c(100, 200))   # DROPS points outside the
range
ggplot(mtcars, aes(disp, mpg)) + geom_point() +
  coord_cartesian(xlim = c(100, 200))               # zooms, keeps every point
```

The base version **removes** observations outside the limits before drawing. The ggplot2 version **zooms** and keeps them, ready to be revealed if the limits move. The base idiom is faster and the ggplot2 one is almost always what you meant. If you are unsure which you have, count your points either way.

Grouping and the language difference

Base graphics has no concept of a grouping variable. You make groups by subsetting and looping. ggplot2 has one, and it is the reason people switch.

Base R	ggplot2	When to switch
`col = as.numeric(factor(g))`	`aes(colour = g)`	Switch. ggplot2 recycles a palette; base gives you `1..k` integers that look like nonsense colours.
`pch = as.numeric(factor(g))`	`aes(shape = g)`	Switch.
`legend('topright', legend, pch, col)`	automatic, via mapped aesthetics	Switch. This is the big one — base legends are written by hand.
`text(..., labels = g)` on points	`geom_text(aes(label = g))`	Stay in base for a handful of direct labels.
`for (g in g) { plot(d[g,]); }`	`facet_wrap(~ g)`	Switch once there are more than about three panels.
`par(mfrow = c(2, 2))` + `plot()`	`facet_wrap(~ g, nrow = 2)`	Switch, but see the caveat below.
`layout(matrix(...))`	`facet_grid()` / `patchwork`	Switch. Ragged panels are the one thing base `layout()` does that ggplot2 does not do well without a second package.

Colour is where the difference bites hardest, and it is worth showing what the base version actually costs:

```
# The base R idiom. Correct, and nobody writes it twice.
g <- factor(mtcars$cyl)
# as.numeric(g) gives the factor CODES 1..k, not the labels. Hand them to
# palette() to get colours, because R will happily accept the raw integers
# and draw palette entries 1, 2, 3 as if you had meant them.
cols <- palette()[as.numeric(g)]

plot(mtcars$disp, mtcars$mpg, col = cols, pch = 19,
     xlab = 'Displacement', ylab = 'Miles per gallon')
legend('topright', legend = levels(g), pch = 19,
       col = palette()[1:3], bty = 'n')
```

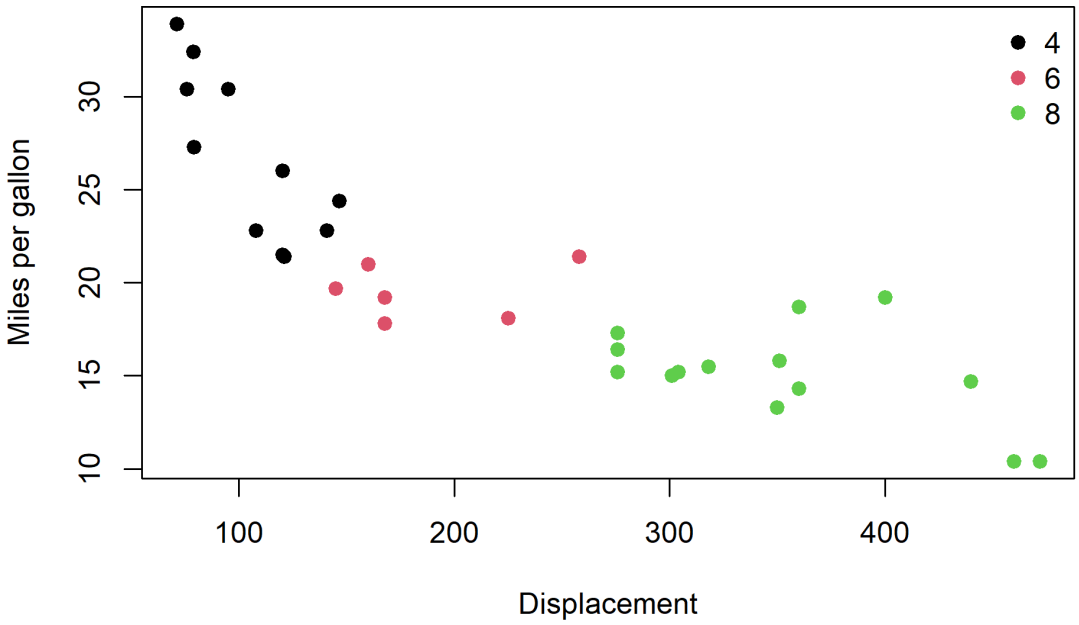


Figure 48: Mapping a factor to colour in base R: recycle the palette, then build the legend by hand

Three things had to happen by hand that ggplot2 would have done for you: recycle a palette across the groups, label the groups from the factor levels, and build a legend whose colours match the plot. Worse, the code above is one careless edit away from silently wrong — change `as.numeric(g)` to `as.numeric(levels(g))` and the legend no longer matches the points, with no warning from R.

The same plot in ggplot2 is one `aes(colour = cyl)` mapping and a free legend, but it needs ggplot2. That trade is Chapter 1's argument in miniature.

High-level functions

These are the ones that do the most work per line in base R, and most have no ggplot2 counterpart because ggplot2 builds them from layers.

Base R	ggplot2	When to switch
`barplot(table(g))`	`geom_bar()` on counts	Stay in base for a quick categorical count — `table()` is one call.
`barplot(h, beside = TRUE)`	`geom_bar(position = 'dodge')`	Never.
`barplot(m, beside = FALSE)`	`geom_bar(position = 'stack')`	Never.

Base R	ggplot2	When to switch
`barplot(..., horiz = TRUE)`	`coord_flip()`	Never.
`boxplot(y ~ g)`	`geom_boxplot(aes(x = g, y = y))`	Stay in base; `boxplot()` with a formula is excellent.
`boxplot(..., notch = TRUE)`	`geom_boxplot(notch = TRUE)`	Never.
`hist(x, breaks = 20)`	`geom_histogram(bins = 20)`	**Careful.** `breaks` and `bins` are not the same parameter.
`pairs(df)`	`pairs()` in `GGally`, or `geom_bin2d`	Stay in base — `pairs()` has no base-R equivalent in ggplot2 at all.
`dotchart(table(g))`	`geom_dotplot()`	Stay in base; no ggplot2 function does this cleanly.
`stripchart(y ~ g)`	`geom_jitter()` or `geom_dotplot()`	Never.
`mosaicplot(table(a, b))`	`geom_mosaic()` (a **different package**)	Stay in base. The ggplot2 version is not in ggplot2.
`rug(x)`	`geom_rug()`	Never.
`curve(f, from, to)`	`geom_function()`	Never.
`matplot(m)`	no equivalent	Stay in base.
`image(m)`	`geom_raster()`	Stay in base unless you need a ggplot2 theme.

`hist(x, breaks = 20)` sets 20 *breakpoints*, which R expands into some number of *bins* — and not always 20. `geom_histogram(bins = 20)` sets 20 bins exactly. A histogram that changes height when you move it between systems is this, not a bug.

Devices, layout and everything else

This is where the systems have nothing in common, so the honest answer is that these do not translate.

Base R	ggplot2	When to switch
`png('a.png', res = 300)`	`ggsave('a.png', dpi = 300)`	Either. `ggsave()` also needs a device for non-standard formats.
`pdf('a.pdf')`	`ggsave('a.pdf')`	Either.

Base R	ggplot2	When to switch
`svg('a.svg')`	`svglite::svglite()`	Stay in base — this needs another package in ggplot2.
`dev.off()`	automatic	ggplot2. One fewer thing to forget.
`par(mar=, oma=, mgp=)`	`theme(plot.margin, ...)`	Stay in base. `theme()` has ~80 elements and this book has 11.
`par(las = 1)`	`theme(axis.text.x = element_text(angle = 0))`	Stay in base; far less typing.
`par(bty = 'l')`	`theme(panel.border = element_blank(), panel.grid = element_blank())`	Stay in base.
`palette.colors('Okabe-Ito')`	`scale_color_OkabeIto()` (a **different package**)	Either. Base gives it to you with no dependency.
`xlab`, `main`, legend styling	`theme()`, `guides()`	Stay in base for a single figure.
`expression(beta[1] * x)`	`parse()`, or `labs()` with `parse = TRUE`	Never.
`suppressWarnings()`	`suppressWarnings()`	Same.

When to switch

The rule from Chapter 1, restated as a checklist. Switch when **most** of these are true:

- the plot has a variable mapped to colour, shape, size or linetype
- it needs facets, and there are more than about three panels
- you are on your third or fourth iteration and still changing the chart
- someone else will maintain it and needs to read it without learning base R
- it is going into a report where the theme must match a house style

Stay in base when:

- the plot is written once and drawn many times
- it has to run somewhere you do not control — a CRAN check, a locked-down server, a minimal container
- it must be reproducible with `install.packages()` not appearing anywhere
- you want to place one specific mark at one specific coordinate
- the plot *is* the thing you are building, and the analysis is incidental

Mixed codebases are normal and fine. Plenty of working reports draw 95% of their figures in base graphics and reach for ggplot2 once, for the one chart where faceting carries the argument. The mistake is picking one system by default and never reconsidering — not mixing them.

What does not translate

For completeness, so you do not waste time looking:

- **`par()`** is entirely base R. There is no ggplot2 equivalent, because everything it does is a global device setting that ggplot2 replaces with `theme()` applied per plot. If your base R code is heavy on `par()`, that is a signal it wants `theme()`, not a sign that `par()` has a translation.
- **The graphics device stack** is base R. ggplot2 removes it from your vocabulary entirely, which is genuinely one of its advantages.
- **`layout()`** ragged panels have no first-class ggplot2 equivalent.
- **`type = 'n'`** layering has no ggplot2 equivalent, and does not need one — it is the direct-mode idiom that ggplot2's layers replace.

Where to go next

- The **R Graphics Cookbook** by Winston Chang (`r-graphics.org`) is recipe-first and covers both systems side by side. It is the right book when you know what you want to draw and need the code.
- The **ggplot2 book** by Wickham and Grolemund is the right book when you want to understand the system rather than copy from it.
- **`?graphics`**, `?par`, and `?palette` in your own R installation remain the most accurate and most current reference for base graphics, and they ship with the software.
